```
            isActive = value;
            gameObject.SetActive(isActive);
        }
        get => isActive;
    }
}

private GameObject poolObject; // 오브젝트 풀에서 관리하는 게임오브젝트 프리팹
private List<PoolItem> poolItemList; // 관리하는 모든 오브젝트를 저장하는 리스트

private readonly int increaseCount = 5; // 오브젝트가 부족할 때
                                        Instantiate()로 추가할 오브젝트 개수

public int MaxCount { private set; get; } // 현재 리스트에 등록된 오브젝트 개수
public int ActiveCount { private set; get; } // 현재 활성 상태인 오브젝트 개수

// 오브젝트를 임시로 보관할 위치
private Vector3 tempPosition =
    new Vector3(float.MaxValue, float.MaxValue, float.MaxValue);

public MemoryPool(GameObject poolObject)
{
    MaxCount = 0;
    ActiveCount = 0;
    this.poolObject = poolObject;

    poolItemList = new List<PoolItem>();

    InstantiateObjects();
}

/// <summary>
/// increaseCount 단위로 오브젝트를 생성한다.
/// </summary>
public void InstantiateObjects()
{
    MaxCount += increaseCount;

    for ( int i = 0; i < increaseCount; ++ i )
    {
        PoolItem poolItem = new PoolItem();

        poolItem.gameObject = GameObject.Instantiate(poolObject);
        poolItem.gameObject.transform.position = tempPosition;
```

```
58:             poolItem.IsActive = false;
59:
60:             poolItemList.Add(poolItem);
61:         }
62:     }
63:
64:     /// <summary>
65:     /// 현재 관리 중인(활성화/비활성화) 모든 오브젝트를 삭제한다.
66:     /// </summary>
67:     public void DestroyObjects()
68:     {
69:         if ( poolItemList == null ) return;
70:
71:         int count = poolItemList.Count;
72:         for ( int i = 0; i < count; ++ i )
73:         {
74:             GameObject.Destroy(poolItemList[i].gameObject);
75:         }
76:
77:         poolItemList.Clear();
78:     }
79:
80:     /// <summary>
81:     /// poolItemList에 저장된 오브젝트를 활성화해서 사용한다.
82:     /// 모든 오브젝트가 사용 중이면 InstantiateObjects()로 추가 생성한다.
83:     /// </summary>
84:     public GameObject ActivatePoolItem(Vector3 position)
85:     {
86:         if ( poolItemList == null ) return null;
87:
88:         // 생성해서 관리하는 모든 오브젝트와 현재 활성화 상태인 오브젝트 개수 비교
89:         // 모든 오브젝트가 활성화 상태이면 새로운 오브젝트 필요
90:         if ( MaxCount == ActiveCount )
91:         {
92:             InstantiateObjects();
93:         }
94:
95:         int count = poolItemList.Count;
96:         for ( int i = 0; i < count; ++ i )
97:         {
98:             PoolItem poolItem = poolItemList[i];
99:
100:            if ( poolItem.IsActive == false )
101:            {
```

```
102:                ActiveCount ++;
103:
104:                poolItem.gameObject.transform.position = position;
105:                poolItem.IsActive = true;
106:
107:                return poolItem.gameObject;
108:            }
109:        }
110:
111:        return null;
112:    }
113:
114:    /// <summary>
115:    /// 사용이 끝난 오브젝트를 비활성화 상태로 설정한다.
116:    /// </summary>
117:    public void DeactivatePoolItem(GameObject removeObject)
118:    {
119:        if ( poolItemList == null || removeObject == null ) return;
120:
121:        int count = poolItemList.Count;
122:        for ( int i = 0; i < count; ++ i )
123:        {
124:            PoolItem poolItem = poolItemList[i];
125:
126:            if ( poolItem.gameObject == removeObject )
127:            {
128:                ActiveCount --;
129:
130:                poolItem.IsActive = false;
131:                poolItem.gameObject.transform.position = tempPosition;
132:
133:                return;
134:            }
135:        }
136:    }
137:
138:    /// <summary>
139:    /// 게임에 사용 중인 모든 오브젝트를 비활성화 상태로 설정한다.
140:    /// </summary>
141:    public void DeactivateAllPoolItems()
142:    {
143:        if ( poolItemList == null ) return;
144:
145:        int count = poolItemList.Count;
```

```
146:        for ( int i = 0; i < count; ++ i )
147:        {
148:            PoolItem poolItem = poolItemList[i];
149:
150:            if ( poolItem.gameObject != null && poolItem.IsActive == true )
151:            {
152:                poolItem.IsActive = false;
153:                poolItem.gameObject.transform.position = tempPosition;
154:            }
155:        }
156:
157:        ActiveCount = 0;
158:    }
159: }
```

06~21줄 메모리 풀로 관리하는 하나의 오브젝트 단위를 나타내는 PoolItem 클래스를 정의합니다. PoolItem 클래스에는 화면에 출력되는 실제 게임오브젝트 정보를 저장하는 gameObject, gameObject의 활성화/비활성화 상태 정보를 나타내는 isActive 변수를 선언합니다. 또한 외부에서 접근해 isActive값과 실제 게임오브젝트의 활성 여부를 제어하는 IsActive 속성을 정의합니다.

34~43줄 메모리를 할당할 때 호출하는 MemoryPool() 생성자에서 변수를 초기화하고 InstantiateObjects() 메서드를 호출해 최초 increaseCount개의 오브젝트를 미리 생성합니다.

48~62줄 오브젝트를 increaseCount개만큼 생성하는 InstantiateObjects() 메서드는 increaseCount개만큼 오브젝트를 추가로 생성하므로 increaseCount를 MaxCount에 더해줍니다. increaseCount 횟수만큼 반복문을 호출해 PoolItem 인스턴스를 선언하고 메모리를 할당합니다. Instantiate()로 오브젝트를 생성하고 poolItem.gameObject에 방금 생성한 오브젝트 정보를 저장합니다. poolItem.gameObject의 위치를 임시 위치 tempPosition으로 설정하고 필요할 때 활성화해서 사용하도록 비활성화합니다. poolItem을 poolItemList에 저장해 관리합니다.

67~78줄 현재 관리 중인 모든 오브젝트를 삭제하는 DestroyObjects() 메서드는 poolItemList 리스트가 비었으면 현재 관리 중인 오브젝트가 없다는 뜻이므로 반환하고 메서드를 끝냅니다. Destroy() 메서드를 호출해 poolItemList에 저장된 모든 오브젝트를 삭제합니다. poolItemList.Clear() 메서드를 호출해 리스트를 초기화합니다.

84~112줄 poolItemList에 비활성화 상태인 오브젝트 중 하나를 선택해 활성화하는 ActivatePoolItem() 메서드는 poolItemList 리스트가 비었으면 현재 관리 중인 오브젝트가 없다는 뜻이므로 null을 반환합니다. 현재 poolItemList에 있는 모든 오브젝트를 사용 중이면 InstantiateObjects() 메서드를 호출해 오브젝트를 추가로 생성합니다. poolItemList에 있는 오브젝트 중 비활성(IsActive == false) 상태인 오브젝트를 활성화하고 위치를 설정한 후 해당 오브젝트를 제어할 수 있도록 오브젝트 정보를 반환합니다.

117~136줄 사용이 끝난 오브젝트를 비활성화하는 DeactivatePoolItem() 메서드는 poolItemList 리스트가 null이거나 비활성화하려는 오브젝트(removeObject)가 null이면 반환하고 메서드를 종료합니다. poolItemList 리스트 요소를 차례대로 탐색해 매개변수로 받아온 removeObject와 동일한 오브젝트를 찾아 해당 요소를 비활성화합니다.

141~158줄 활성화 상태인 모든 리스트 요소를 비활성화하는 DeactivateAllPoolItems() 메서드는 poolItemList 리스트가 null이면 반환하고 메서드를 끝냅니다. poolItemList 리스트 요소를 차례대로 탐색해 현재 활성화 상태인 오브젝트를 모두 비활성 상태로 설정합니다.

02 다음으로, 보석에 적용할 페이드 효과를 준비합니다. Assets 〉 Scripts 〉 Common 폴더에 페이드 효과 재생을 제어하는 FadeEffect.cs 스크립트를 생성합니다.

Assets/Scripts/Common/FadeEffect.cs

코드 09-14 페이드 효과 재생 제어

```
01: using System.Collections;
02: using UnityEngine;
03: using UnityEngine.Events;
04:
05: public static class FadeEffect
06: {
07:     public static IEnumerator Fade(SpriteRenderer target, float start, float end,
            float fadeTime=1f, UnityAction action=null)
08:     {
09:         if ( target == null ) yield break;
10:
11:         float percent = 0;
12:
13:         while ( percent < 1 )
14:         {
```

```
15:             percent += Time.deltaTime / fadeTime;
16:
17:             Color color = target.color;
18:             color.a = Mathf.Lerp(start, end, percent);
19:             target.color = color;
20:
21:             yield return null;
22:         }
23:
24:         action?.Invoke();
25:     }
26: }
```

05줄 페이드 효과는 게임 전반에 걸쳐 범용으로 사용하므로 정적 클래스와 메서드를 정의해 인스턴스 변수를 생성하지 않고 바로 호출할 수 있도록 합니다.

07줄 모든 Fade() 메서드는 SpriteRenderer, Image 등 알파값을 변경할 대상, 알파의 시작값 start와 종료값 end, 페이드 효과를 재생하는 시간인 fadeTime 매개변수를 사용합니다. 또한 페이드 효과 재생이 끝난 이후 원하는 메서드를 호출하도록 5번째 매개변수에 UnityAction형 변수를 선언합니다.

09줄 페이드 효과를 재생할 대상인 target이 없으면 yield break;로 코루틴 메서드를 끝냅니다.

13~22줄 while() 반복문 내부에서 페이드 효과를 재생합니다.

15줄 percent 값은 Time.deltaTime / fadeTime만큼 값이 증가하므로 0.0에서 1.0까지 fadeTime 시간 동안 재생합니다.

17~19줄 현재 색상 정보인 target.color를 color 변수에 저장하고 Mathf.Lerp() 메서드를 호출해 색상 알파값인 color.a를 start부터 end까지 변경합니다. 그리고 변경된 색상 정보를 target.color에 저장합니다.

24줄 페이드 효과 재생이 끝나면 action에 메서드가 등록되었는지 null 검사를 진행하고 null이 아니면 해당 메서드를 실행합니다.

SpriteRenderer가 아닌 다른 컴포넌트의 페이드 효과를 재생하고 싶으면 07~25줄 Fade() 메서드를 복사/붙여 넣기 한 후 첫 번째 매개변수 SpriteRenderer형만 원하는 컴포넌트 자료형으로 변경합니다.

Do it! 실습 보석 수집 효과 관리하고 움직임 제어하기

01 Assets 〉 Scripts 〉 Common 폴더에 보석 수집 효과를 관리하는 GemCollector.cs 스크립트를 생성합니다.

Assets/Scripts/Common/GemCollector.cs

코드 09-15 보석 획득 효과 활성화/비활성화

```
using UnityEngine;
using UnityEngine.Events;

public class GemCollector : MonoBehaviour
{
    [SerializeField]
    private GameObject gemEffectPrefab;
    [SerializeField]
    private RectTransform uiElement;       // 보석이 이동할 목표 위치(Gem Icon UI)
    [SerializeField]
    private UnityEvent onGemCollectEvent; // 보석을 얻을 때 호출할 메서드 등록
    private MemoryPool memoryPool;         // 보석을 생성하고 관리할 메모리 풀

    private void Awake()
    {
        memoryPool = new MemoryPool(gemEffectPrefab);
    }

    public void SpawnGemEffect(Vector2 point, int count=5)
    {
        for ( int i = 0; i < count; ++ i )
        {
            GameObject gem = memoryPool.ActivatePoolItem(point);
            gem.GetComponent<GemCollectEffect>().Setup(this, uiElement);
        }
    }

    public void OnGemCollect(GameObject gem)
    {
        onGemCollectEvent?.Invoke(); // 보석을 얻을 때 이벤트에 등록된 메서드 호출
        memoryPool.DeactivatePoolItem(gem); // 매개변수로 받은 gem 오브젝트 비활성화
    }
}
```

10~11줄 보석 수집 효과 재생이 끝나면 개수를 늘리는 등 다양한 행위를 할 수 있습니다. 이런 작업을 수행할 때 GemCollector.cs 스크립트를 수정하지 않도록 **UnityEvent**형인 **onGemCollectEvent** 변수를 선언하고 보석 수집 효과 재생이 끝났을 때 호출하고 싶은 메서드를 Inspector View에서 등록합니다.

12줄 보석 수집 효과는 적을 물리칠 때마다 생성/삭제를 반복하므로 오브젝트를 활성화/비활성화로 관리하는 **MemoryPool** 변수를 선언합니다.

14~17줄 **Awake()** 메서드에서 생성/관리할 프리팹 정보를 매개변수로 하여 메모리를 할당합니다.

19~26줄 적을 물리쳤을 때 **SpawnGemEffect()** 메서드를 호출해 **point** 위치에 **count**개만큼 보석 수집 효과를 생성합니다.

28~32줄 보석 수집 효과가 UI 위치까지 이동했을 때 호출하는 **OnGemCollect()** 메서드는 **onGemCollectEvent**에 등록된 메서드를 실행하고 매개변수로 받은 보석 수집 효과를 비활성화합니다.

02 같은 폴더에 보석 수집 효과를 제어하는 GemCollectEffect.cs 스크립트를 생성합니다.

Assets/Scripts/Common/GemCollectEffect.cs

코드 09-16 보석 수집 효과 이동 제어

```
01: using UnityEngine;
02:
03: public class GemCollectEffect : MonoBehaviour
04: {
05:     private Camera mainCamera;
06:     private SpriteRenderer spriteRenderer;
07:     private GemCollector gemCollector;
08:     private RectTransform uiElement; // 보석이 이동할 목표 UI
09:     private Vector3 start, end, point1;
10:     private float percent, duration;
11:     private readonly float minDuration = 0.5f, maxDuration = 2f, r = 3f;
12:
13:     private void Awake()
14:     {
15:         mainCamera = Camera.main;
16:         spriteRenderer = GetComponent<SpriteRenderer>();
17:     }
18:
```

```
19:     public void Setup(GemCollector gemCollector, RectTransform uiElement)
20:     {
21:         this.gemCollector = gemCollector;
22:         this.uiElement = uiElement;
23:         percent = 0f;
24:         start = transform.position;
25:         duration = Random.Range(minDuration, maxDuration);
26:         point1 = Utils.GetNewPoint(start, Random.Range(0, 360), r);
27:
28:         // duration 시간 동안 서서히 사라짐
29:         StartCoroutine(FadeEffect.Fade(spriteRenderer, 1, 0, duration));
30:     }
31:
32:     private void Update()
33:     {
34:         if ( percent >= 1f )
35:         {
36:             gemCollector.OnGemCollect(gameObject);
37:             return;
38:         }
39:
40:         UpdateEndPoint();
41:         percent += Time.deltaTime / duration;
42:         transform.position = Utils.QuadraticCurve(start, point1, end, percent);
43:     }
44:
45:     private void UpdateEndPoint()
46:     {
47:         Vector3 screenPoint = RectTransformUtility.WorldToScreenPoint(
                null, uiElement.position);
48:
49:         if ( RectTransformUtility.ScreenPointToWorldPointInRectangle(
50:             uiElement, screenPoint, mainCamera, out Vector3 worldPoint) )
                    end = worldPoint;
51:     }
52: }
```

보석 수집 효과는 생성한 이후 활성화/비활성화로 계속 사용하므로 **Awake()** 메서드는 오브젝트가 생성될 때 1회만 호출하고 **Setup()** 메서드는 오브젝트를 활성화할 때마다 호출해 위치, 재생 시간 등의 변수를 매번 새롭게 초기화합니다.

32~43줄 보석 수집 효과가 활성화되면 duration 시간 동안 percent가 0.0에서 1.0이 되고 1.0이 되면 gemCollector.OnGemCollect() 메서드를 호출해 보석 수집 효과를 비활성화합니다. 플레이어가 이동하면 카메라의 위치가 바뀌므로 현재 위치 기준으로 목표 UI 위치를 갱신하고자 UpdateEndPoint() 메서드를 호출하고 Utils.QuadraticCurve() 메서드를 호출해 보석 수집 효과가 2차 베지어 곡선 모양으로 움직이도록 합니다.

45~51줄 UpdateEndPoint() 메서드는 WorldToScreenPoint() 메서드를 호출해 UI의 월드 좌표 uiElement.position을 화면 좌표인 screenPoint로 변환합니다. ScreenPointToWorldPointInRectangle() 메서드를 호출해 화면 좌표인 screenPoint를 월드 좌표인 worldPoint로 변환하고 end에 저장합니다.

알아 두면 좋아요! 코드에서 사용한 유니티 메서드

```
Vector3 screenPoint = RectTransformUtility.WorldToScreenPoint(
    Camera cam,
    Vector3 worldPoint);
```

오브젝트의 월드 좌표(worldPoint)를 화면 좌표(screenPoint)로 변환합니다.
첫 번째 매개변수에 등록하는 카메라는 Canvas 컴포넌트의 Render Mode가 [Screen Space - Overlay]일 때는 기본 카메라 자동 설정인 null로, [Screen Space - Camera]나 [World Space]일 때는 Canvas 컴포넌트에 등록된 카메라와 동일한 카메라를 매개변수로 설정합니다.

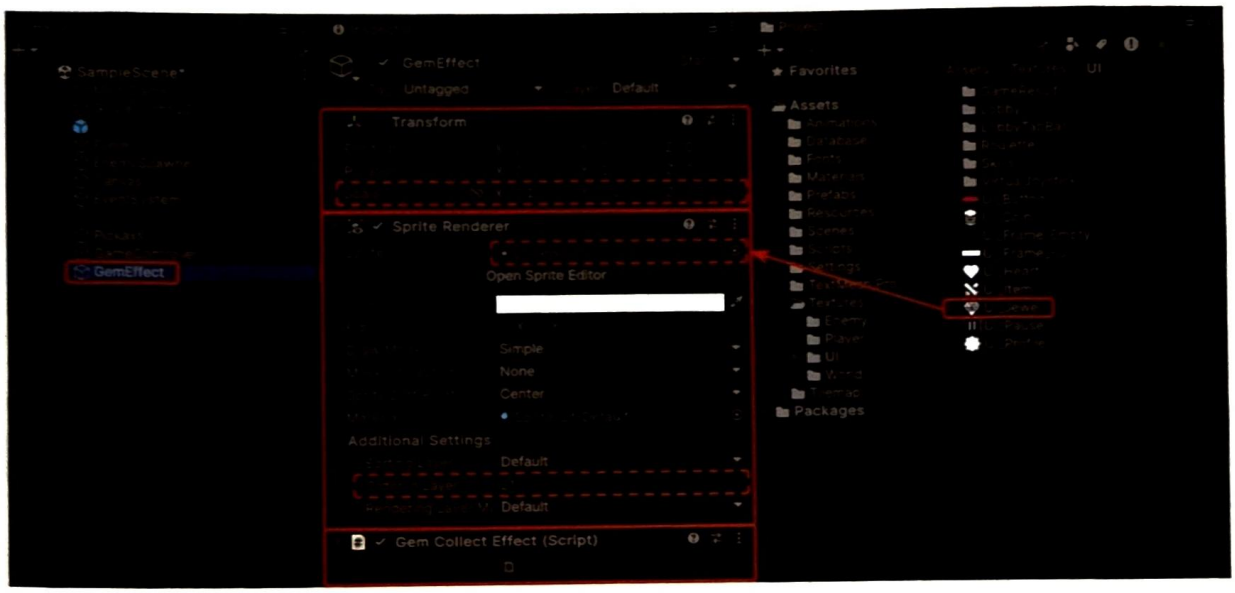
```
bool RectTransformUtility.ScreenPointToWorldPointInRectangle(
    RectTransform rect,
    Vector2 screenPoint,
    Camera cam,
    out Vector3 worldPoint);
```

화면 좌표(screenPoint)를 월드 좌표(worldPoint)로 변환합니다.
RectTransform rect는 중심이 되는 Rect Transform 정보, Vector2 screenPoint는 화면 좌표, Camera cam은 현재 화면을 출력하는 카메라, out Vector3 worldPoint는 rect와 screenPoint를 기준으로 연산한 월드 좌표입니다.

UI 좌표(position)와 2D/3D 오브젝트 좌표(position)가 다르므로 UI 월드 좌표를 기준으로 화면 좌표를 구하고 구한 화면 좌표를 기준으로 2D/3D 오브젝트가 이동할 월드 좌표를 구합니다.

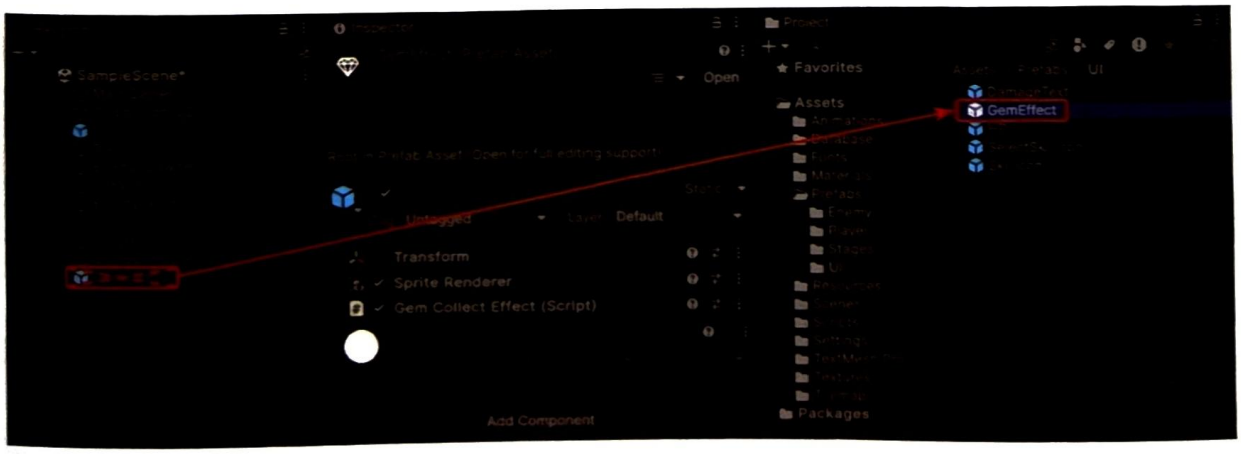
Do it! 실습 보석 효과와 UI 출력하기

01 [GameObject → 2D Object → Sprites → Square] 메뉴로 보석 효과 오브젝트로 사용할 2D 사각형 오브젝트를 생성하고 이름을 'GemEffect'로 변경합니다. Transform 컴포넌트의 Scale을 (0.2, 0.2, 0.2)로 설정합니다. Sprite Renderer 컴포넌트의 Sprite에 UI_Jewel.png 이미지를 등록하고 Order in Layer를 '21'로 설정합니다. 마지막으로 Gem Collect Effect 컴포넌트를 추가합니다.



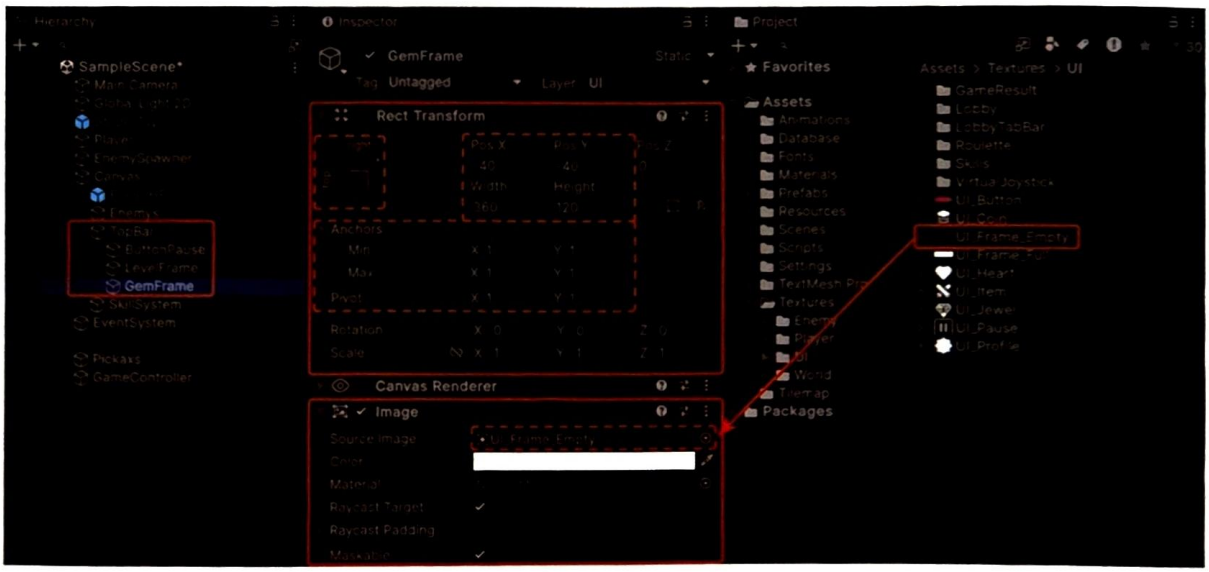
보석 효과 오브젝트 생성하고 설정하기

02 GemEffect 오브젝트를 Project View의 Assets 〉 Prefabs 〉 UI 폴더로 끌어서 놓아 프리팹을 생성하고 Hierarchy View에 있는 GemEffect 오브젝트는 삭제합니다.



GemEffect 오브젝트 프리팹 생성하기

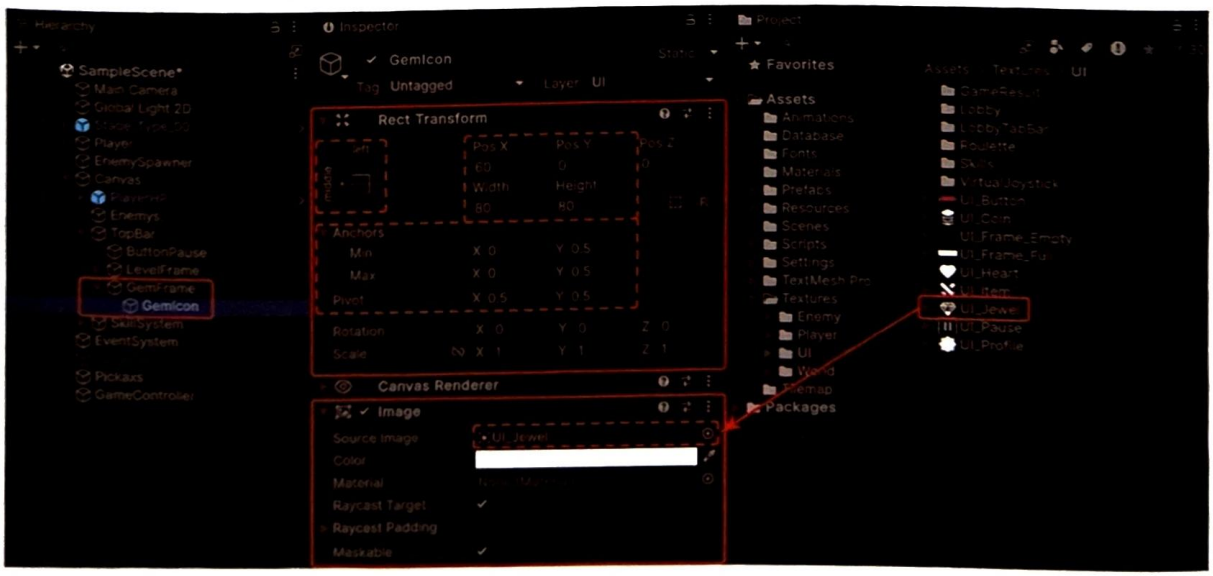
03 수집한 보석 정보를 UI에 출력하도록 [GameObject → UI → Image] 메뉴로 보석 UI를 관리하는 Image UI를 생성하고 이름을 'GemFrame'으로 변경한 후 TopBar 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 top | right로, Rect는 (-40, -40, 360, 120)으로 설정합니다. 그리고 Image 컴포넌트의 Source Image에는 UI_Frame_Empty.png 이미지를 등록합니다.



보석 UI를 관리하는 Image UI 생성하고 설정하기

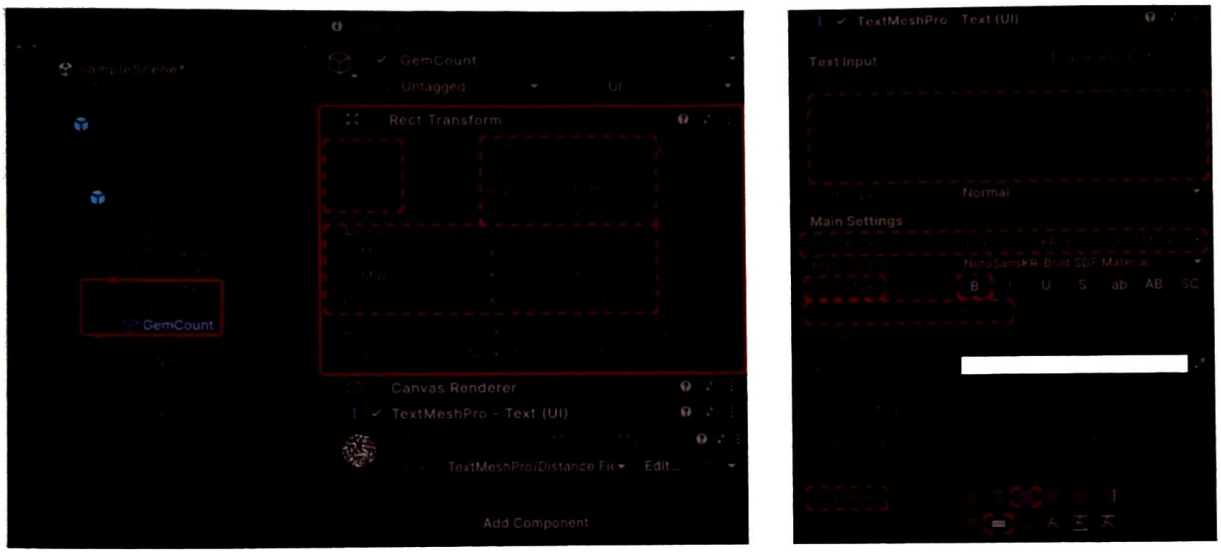
04 [GameObject → UI → Image] 메뉴로 보석 아이콘을 출력하는 Image UI를 생성하고 이름을 'GemIcon'으로 변경한 후 GemFrame 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | left로, Pivot은 보석 효과가 아이콘 가운데 위치까지 이동하도록 (0.5, 0.5)로 설정합니다. Rect는 (60, 0, 80, 80)으로 설정하고 Image 컴포넌트의 Source Image에 UI_Jewel.png 이미지를 등록합니다.



보석 아이콘을 출력하는 Image UI 생성하고 설정하기

05 [GameObject → UI → Text - TextMeshPro] 메뉴로 보석 개수를 출력하는 Text UI를 생성하고 이름을 'GemCount'로 변경한 후 GemFrame 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (110, 20, 20, 20)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 '0'을 입력하고, Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Style은 굵게 [B]로, Font Size는 '60'으로, Alignment는 Right | Middle로 설정합니다.



보석 개수를 출력하는 Text UI 생성하고 설정하기

06 보석 정보를 저장하고 보석을 수집할 때 호출하는 `AddGem()` 메서드를 정의하고자 PlayerBase.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 09-17 보석 정보 저장하고 수집 처리

```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
        (... 생략 ...)
18:     public float AccumulationExp { get; set; } = 0f;
19:     // 현재 스테이지에서 획득한 보석 개수
20:     public int GEM { get; private set; } = 0;
        (... 생략 ...)
107:    public void AddGEM()
```

추가

추가

```
108:    {
109:        GEM ++;
110:    }
111:}
```

07 텍스트 UI에 보석 개수를 출력하도록 UIPlayerData.cs 스크립트를 수정합니다.

Assets/Scripts/UI/UIPlayerData.cs

코드 09-18 플레이어가 게임에서 수집한 보석 개수 출력

```
    (... 생략 ...)
05: public class UIPlayerData : MonoBehaviour
06: {
07:     [SerializeField]
08:     private TextMeshProUGUI textLevel;
09:     [SerializeField]
10:     private Image fillGaugeEXP;
11:     [SerializeField]                                  추가
12:     private TextMeshProUGUI textGEMCount;
13:     [SerializeField]
14:     private PlayerBase entity;
        (... 생략 ...)
27:     public void UpdateGEM()                           추가
28:     {
29:         textGEMCount.text = entity.GEM.ToString();
30:     }
31: }
```

08 TopBar 오브젝트 UI Player Data 컴포넌트의 Text GEM Count 변수에 GemCount 오브젝트를 등록합니다. TopBar 오브젝트에 Gem Collector 컴포넌트를 추가하고 Gem Effect Prefab 변수에는 GemEffect 프리팹을, UI Element 변수에는 GemIcon 오브젝트를 등록합니다.

보석 효과가 UI에 도착하면 플레이어가 지닌 보석 개수를 늘리고 UI 정보를 갱신하도록 On Gem Collect Event ()에 메서드 2개를 등록합니다. Player 오브젝트의 Player Base 컴포넌트 AddGEM() 메서드와 TopBar 오브젝트의 UI Player Data 컴포넌트 UpdateGEM() 메서드를 각각 등록합니다.

TopBar 오브젝트에 Gem Collector 컴포넌트 추가하고 설정하기

09 `GemCollector` 정보를 `EnemyBase.Initialize()` 매개변수로 전달하도록 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 09-19 EnemyBase.Initialize() 매개변수로 GemCollector 정보 전달

```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
07:     [SerializeField]
08:     private Tilemap tilemap;
09:     [SerializeField]
10:     private GameObject[] enemyPrefabs;
11:     [SerializeField]
12:     private Transform parentTransform;
13:     [SerializeField]                                    추가
14:     private GemCollector gemCollector;
        (... 생략 ...)
25:     private void Awake()
26:     {
            (... 생략 ...)
32:         // 임의의 타일에 적 10개 생성
33:         for ( int i = 0; i < enemyCount; ++ i )
34:         {
                (... 생략 ...)
39:             clone.GetComponent<EnemyBase>().Initialize(this, parentTransform,
                    gemCollector);                          수정
```

```
40:             clone.GetComponent<EnemyFSM>().Setup(target);
41:
42:             Enemies.Add(clone.GetComponent<EntityBase>());
43:         }
44:     }
    (... 생략 ...)
76: }
```

10 적을 물리쳤을 때 임의의 개수만큼 보석을 생성하도록 EnemyBase.cs 스크립트를 수정합니다.

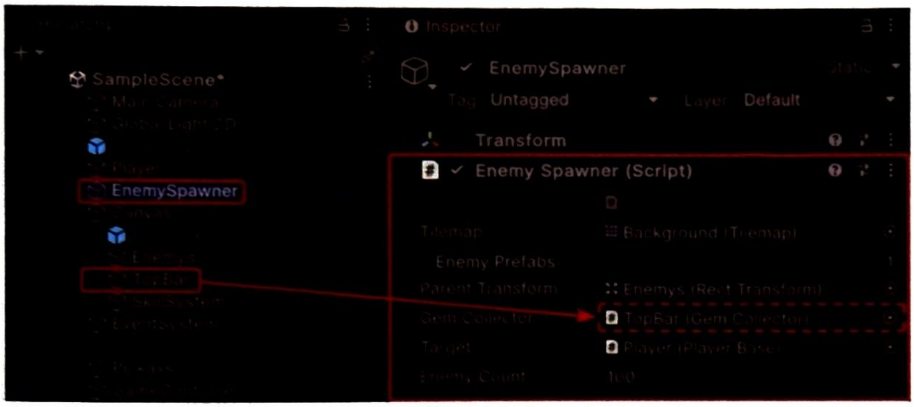
Assets/Scripts/Enemy/EnemyBase.cs

코드 09-20 적을 물리쳤을 때 보석 생성

```
01: using UnityEngine;
02:
03: public class EnemyBase : EntityBase
04: {
05:     [SerializeField]
06:     private Transform hudPoint;
07:     [SerializeField]
08:     private GameObject uiPrefab;
09:     [SerializeField]                                    추가
10:     private int gemMin = 5, gemMax = 21;
11:
12:     private EnemySpawner enemySpawner;
13:     private GemCollector gemCollector;                  추가
    (... 생략 ...)
28:     public void Initialize(EnemySpawner enemySpawner, Transform parent,
            GemCollector gemCollector)                      수정
29:     {
30:         this.enemySpawner = enemySpawner;
31:         this.gemCollector = gemCollector;               추가
            (... 생략 ...)
37:     }
38:
39:     protected override void OnDie()
40:     {
41:         // 임의의 개수(gemMin ~ gemMax-1)만큼 보석 생성    추가
42:         gemCollector.SpawnGemEffect(transform.position,
                Random.Range(gemMin, gemMax));
```

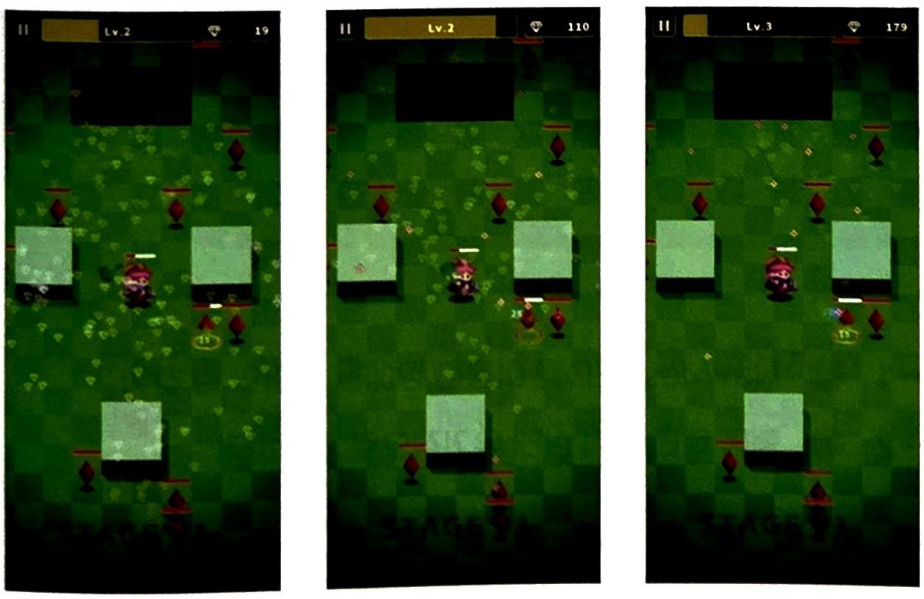
```
43:         // 적은 레벨업 하지 않으므로 적 경험치 스탯만큼 플레이어 경험치 증가
44:         (Target as PlayerBase).AccumulationExp += Stats.CurrentExp.Value;
45:         // 적 본인(this) 사망 처리
46:         enemySpawner.Deactivate(this);
47:     }
48: }
```

11 EnemySpawner 오브젝트 Enemy Spawner 컴포넌트의 Gem Collector 변수에 TopBar 오브젝트를 등록합니다.



EnemySpawner 오브젝트의 Enemy Spawner 컴포넌트 변수 설정하기

12 플레이 버튼 ▶을 눌러 게임을 실행하면 적을 물리쳤을 때 임의의 개수만큼 보석 효과를 재생하고, 효과가 오른쪽 위 보석 개수 UI까지 이동하면 플레이어가 지닌 보석 개수가 1씩 늘어납니다.



보석을 이용한 재화 시스템 결과 화면

10장

똑똑한 적 AI – 행동 트리와 스마트 경로 탐색하기

이번 장에서는 적의 경로 탐색과 적 AI를 구현합니다. 먼저 NavMesh2D를 이용해 경로 데이터를 저장하고 장애물을 피해 목표 위치까지 이동하는 적을 만듭니다. 이와 함께 행동 트리로 대기, 순찰, 배회, 추적, 공격 등을 구현하고 상황에 따라 적이 행동을 선택하도록 합니다.

학습 목표

1. NavMesh2D를 활용하여 장애물을 회피하는 경로 탐색 시스템을 구현할 수 있다.
2. 행동 트리를 기반으로 한 적 AI 상태 전이 시스템을 설계할 수 있다.

10-1 NavMesh2D를 이용한 장애물 회피와 길 찾기

10-2 행동 트리로 적 상태 유기적으로 제어하기

10-1

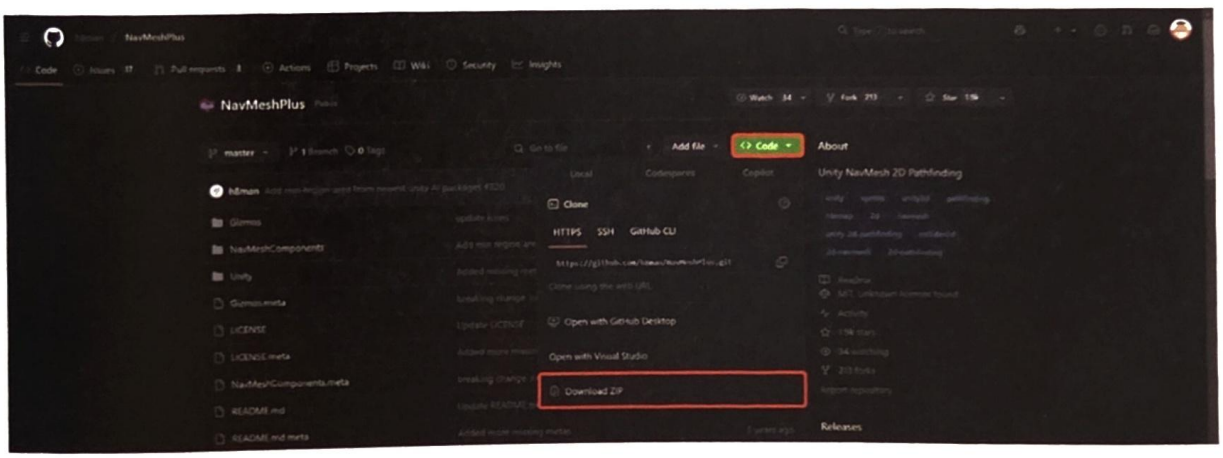
NavMesh2D를 이용한 장애물 회피와 길 찾기

이 절에서는 적의 경로 탐색을 구현합니다. NavMesh2D를 이용해 월드의 경로 데이터를 저장하고, 이 데이터로 장애물을 피해 목표 위치까지 이동하는 적을 만듭니다.

NavMesh2D 에셋 가져오기

유니티는 특정 위치까지 장애물을 피해 이동할 수 있도록 경로 탐색 시스템인 AI Navigation을 지원합니다. 하지만 이 기능은 3D에서만 사용할 수 있으므로 2D에서 해당 기능을 사용하려면 AI Navigation과 연동할 수 있도록 만든 별도의 에셋을 사용해야 합니다.

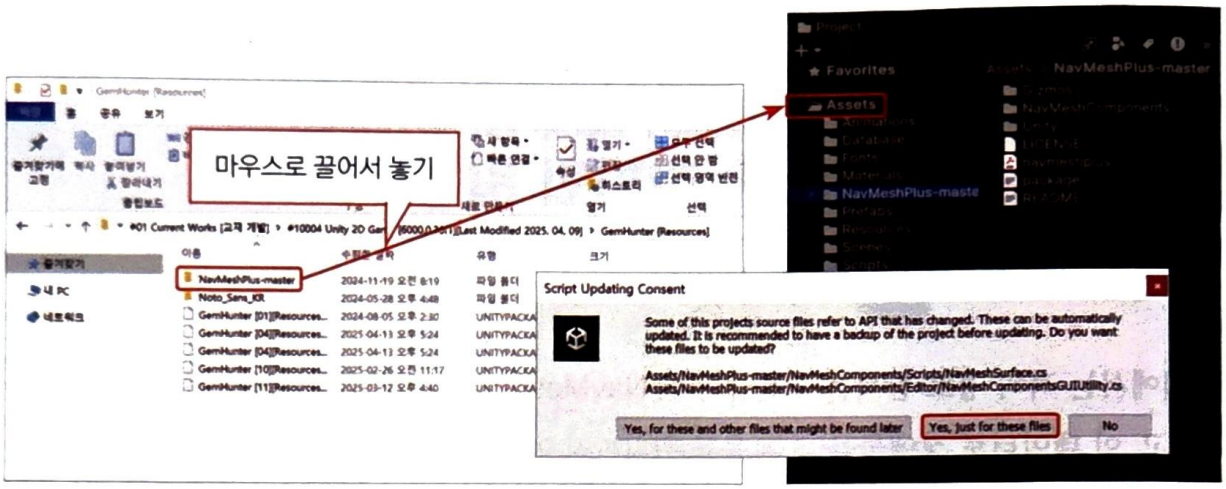
NavMeshPlus 깃허브(https://github.com/h8man/NavMeshPlus.git)에 방문하여 [Code → Download ZIP] 메뉴로 에셋을 내려받습니다.



NavMesh2D에 필요한 에셋 내려받기

내려받은 에셋의 압축을 풀고 NavMeshPlus-master 폴더를 Project View의 원하는 위치로 끌어서 놓아 가져옵니다. 이때 스크립트 업데이트에 동의하는지를 묻는 창이 뜨면 지금 추가하는 파일에만 적용하도록 〈Yes, just for these files〉 버튼을 선택합니다.

프로젝트에서 여러 가지 외부 파일(에셋)을 사용할 때 업데이트 후 올바르게 작동하지 않기도 하므로 보통은 이번처럼 〈Yes, just for these files〉를 선택합니다.



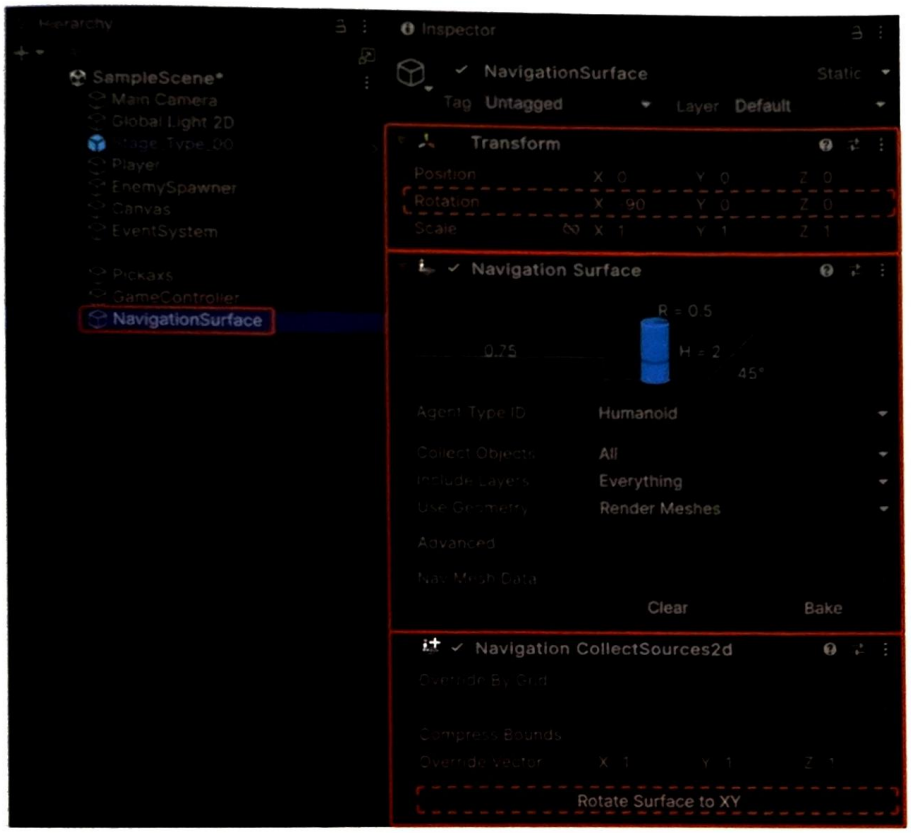
현재 프로젝트로 NavMeshPlus-master 가져오기

Do it! 실습 NavMesh2D로 경로 데이터 생성하기

NavMesh2D 에셋을 가져왔다면 먼저 이를 이용하여 적이 움직일 수 있는 경로 데이터를 생성하고 저장해야 합니다.

01 현재 월드에 배치된 타일맵 오브젝트를 기준으로 경로 데이터를 생성하고자 [Game Object → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'NavigationSurface'로 변경합니다. NavigationSurface 오브젝트에는 Navigation Surface와 Navigation Collect Sources2d 컴포넌트를 추가합니다.

유니티 내장 AI는 3D용으로 xz 평면을 기준으로 경로를 생성하는데, 일반 2D 게임은 xy 평면을 기준으로 사용하므로 Navigation CollectSources2d 컴포넌트의 〈Rotate Surface to XY〉 버튼을 눌러 축을 변경합니다. 이 버튼을 누르면 NavigationSurface 오브젝트의 회전이 (0, 0, 0)에서 (-90, 0, 0)으로 바뀝니다.

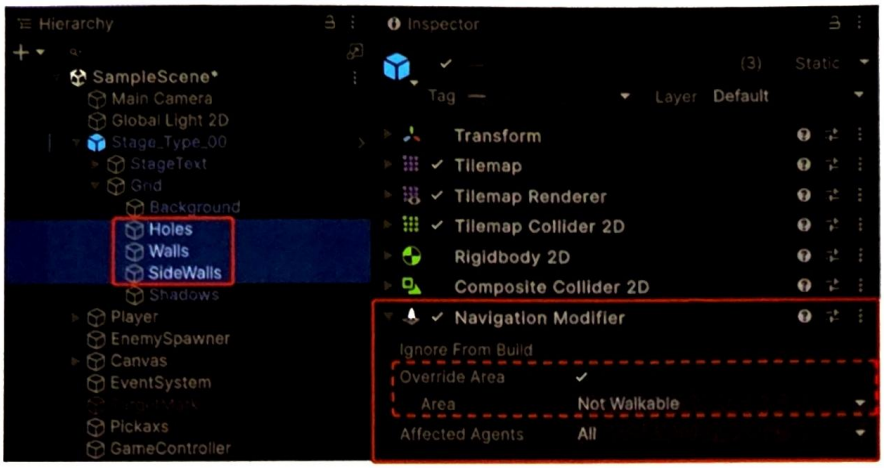


NavigationSurface 오브젝트에 Navigation Surface, Navigation CollectSources2d 컴포넌트 추가하기

02 타일맵 오브젝트마다 이동 가능, 이동 불가능과 같은 구역을 설정할 수 있습니다. Background 타일맵 오브젝트는 바닥을 표현한 타일이기에 이동할 수 있으므로 Navigation Modifier 컴포넌트를 추가한 후 Override Area는 true로, Area는 [Walkable]로 설정합니다.

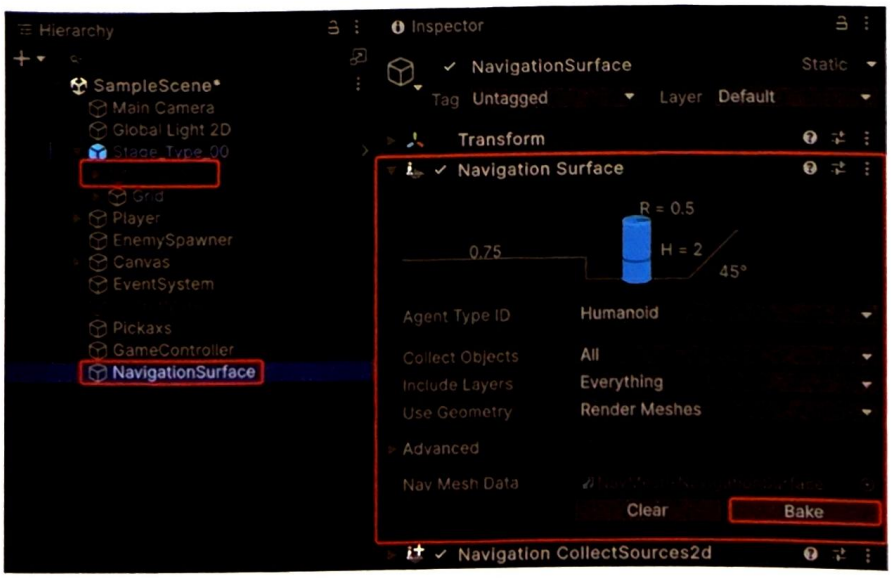
Background 타일맵을 이동할 수 있는 구역으로 설정하기

03 Holes, Walls, SideWalls 타일맵 오브젝트에 Navigation Modifier 컴포넌트를 각각 추가한 후 Override Area는 true로, Area는 [Not Walkable]로 설정합니다.



Holes, Walls, SideWalls 타일맵을 이동할 수 없는 구역으로 설정하기

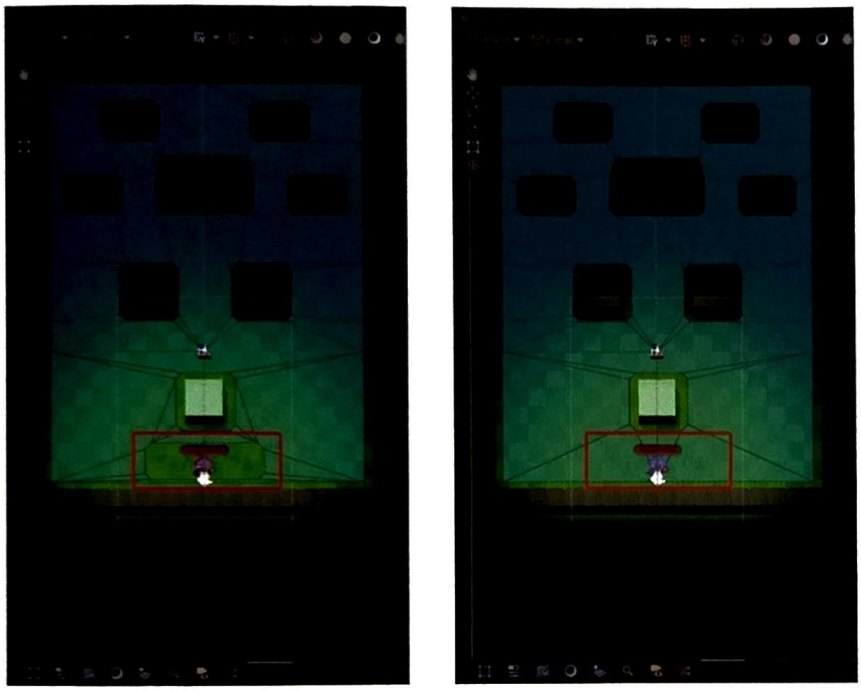
04 경로로 사용할 월드 설정이 끝났으면 경로 데이터를 생성합니다. 현재 스테이지 텍스트를 출력하는 StageText 오브젝트도 경로에 포함될 수 있으므로 StageText 오브젝트를 비활성화하고, NavigationSurface 오브젝트의 Navigation Surface 컴포넌트에 있는 〈Bake〉 버튼을 눌러 경로 데이터를 생성합니다. 맵 정보가 변경된다면 〈Bake〉 버튼으로 경로 데이터를 다시 저장해야 합니다. 경로 데이터 생성이 끝나면 StageText 오브젝트는 다시 활성화합니다.



Navigation Surface 컴포넌트의 〈Bake〉 버튼으로 이동 경로 설정하기

3D Text도 경로에 포함될 수 있으므로 비활성화하거나 Background 타일맵 오브젝트처럼 Navigation Modifier 컴포넌트를 추가해서 이동할 수 있는 구역으로 설정합니다.

05 NavigationSurface 오브젝트를 더블클릭하고 Scene View를 보면 이동할 수 있는 구역과 그렇지 못한 구역 정보가 표시됩니다. 3D Text를 활성화한 상태로 경로 데이터를 생성하면 왼쪽 그림처럼 텍스트 위치가 이동할 수 없는 구역으로 처리됩니다.



경로 데이터를 생성한 결과 화면

Do it! 실습 경로 데이터로 적 이동 제어하기

01 오브젝트가 생성한 경로 데이터를 이용해 이동하려면 유니티에서 제공하는 NavMesh Agent 클래스를 사용해야 합니다. 이 클래스를 이용해 적 오브젝트 이동을 제어하고자 EnemyFSM.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyFSM.cs

코드 10-01 NavMeshAgent를 이용한 적 이동 제어

```
01: using System.Collections;
02: using UnityEngine;
03: using UnityEngine.AI;
04:
05: public enum EnemyState { None = -1, Attack, }
06:
07: public class EnemyFSM : MonoBehaviour
08: {
```

추가

```
    (... 생략 ...)
14:     private EnemyBase owner;
15:     private NavMeshAgent navMeshAgent; // 적 이동 경로 설정과 이동 제어
16:     private EnemyState enemyState;
17:
18:     private void Awake()
19:     {
20:         owner = GetComponent<EnemyBase>();
21:         navMeshAgent = GetComponent<NavMeshAgent>();
22:
23:         navMeshAgent.updateRotation = false;
24:         navMeshAgent.updateUpAxis = false;
25:
26:         ChangeState(EnemyState.Attack);
27:     }
28:
29:     public void Setup(EntityBase target)
30:     {
31:         owner.target = target;
32:         // [Debug Test] 게임이 시작된 순간의 플레이어 위치로 이동
33:         navMeshAgent.SetDestination(target.MiddlePoint);
34:     }
    (... 생략 ...)
65: }
```

추가

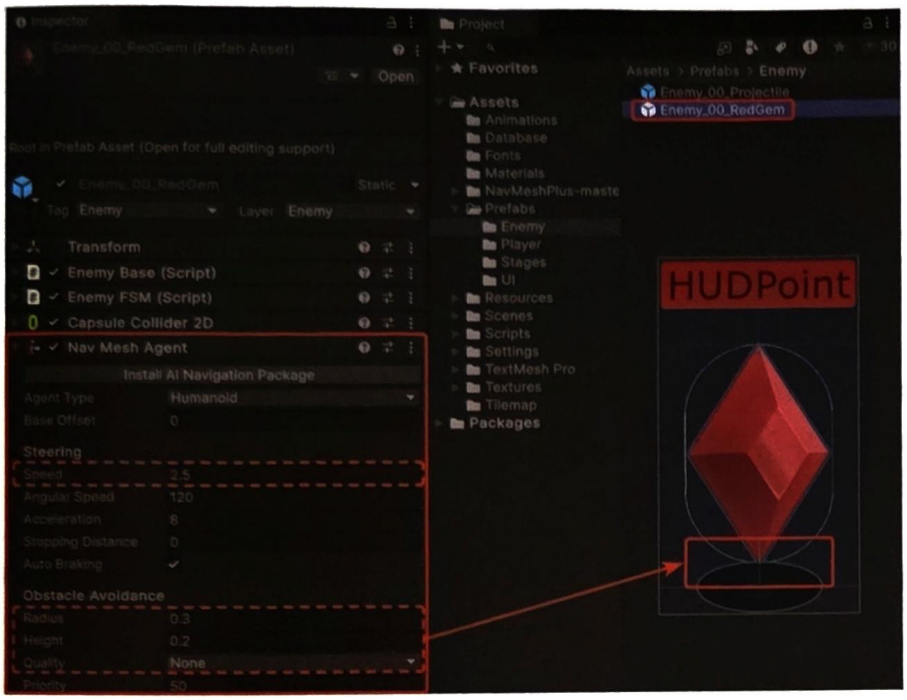
추가

추가

23~24줄 유니티에서 제공하는 NavMeshAgent는 기본적으로 3D용으로 제작된 경로 탐색 AI입니다. 자동 회전을 끄지 않으면 3D인 xz 평면 기준으로 에이전트가 회전하므로 자동 회전 옵션을 비활성화합니다.

32~33줄 목표 위치를 플레이어 위치로 설정하면 게임을 시작하는 동시에 모든 적이 플레이어의 처음 위치를 목표로 하여 이동합니다.

02 Enemy_00_RedGem 프리팹에 Nav Mesh Agent 컴포넌트를 추가합니다. 적의 이동 속도 Speed는 '2.5'로, 경로 이동에 사용하는 충돌 반지름 Radius는 '0.3'으로, 높이 Height는 '0.2'로 설정합니다. 이동하는 적끼리 겹칠 수 있도록 Quality는 [None]으로 설정합니다.



Enemy_00_RedGem 프리팹에 Nav Mesh Agent 컴포넌트 추가하고 설정하기

03 플레이 버튼 ▶ 을 눌러 게임을 실행하면 적이 장애물을 피해 게임을 실행한 순간의 플레이어 위치로 이동합니다.

10-2

행동 트리로 적 상태 유기적으로 제어하기

이 절에서는 적 AI를 구현합니다. 유니티가 제공하는 행동 트리로 대기 Idle, 순찰 Patrol, 배회 Wander, 추적 Chase, 공격 Attack 행동을 구현하고 상황에 따라 행동을 선택하도록 만듭니다.

행동 트리 살펴보기

행동 트리란?

행동 트리 Behavior Tree는 AI 캐릭터의 행동을 제어하거나 상태를 관리할 때 사용합니다. 행동 트리를 이용하면 게임을 개발할 때 비선형적이고 복잡한 AI 의사결정을 간단하게 모듈화하여 구현할 수 있습니다. 유니티 6부터는 이러한 행동 트리를 기본으로 제공하고, 노드 기반으로 관리할 수 있습니다.

노드란?

노드 Node는 에이전트가 지정한 대로 동작할 수 있도록 그래프의 흐름을 정의하는 **행동 그래프** Behavior Graph의 일부입니다. 유니티 행동 트리가 제공하는 노드로는 Events, Action, Flow(Modifier, Sequencing, Join) 등이 있습니다.

- Action 노드: 실제 행동을 수행하는 역할을 합니다. 일반적으로 트리 가장 아래쪽에 있으며 에이전트가 해야 할 구체적인 작업을 정의합니다.
- Modifier 노드: 트리의 흐름이나 실행 조건을 수정하는 역할을 합니다. 특정 조건을 확인하거나 실행 결과를 반환하기도 하며, 단일 자식 노드만 가질 수 있습니다.
- Sequencing 노드: 해당 노드가 아닌 자식 노드를 특정 조건에 따라 실행(순차 실행, 동시 실행 등)합니다. 모든 자식 노드가 성공해야 자신도 성공 상태를 반환할 수 있습니다.
- Join 노드: 여러 노드에서 발생한 결과를 결합해 행동을 결정하는 데 사용합니다. Parallel 노드, Sequence 노드와 함께 사용할 때가 흔합니다.

노드에는 Running, Waiting, Succeeded, Failed, Uninitialized 등 5가지 상태가 있습니다.

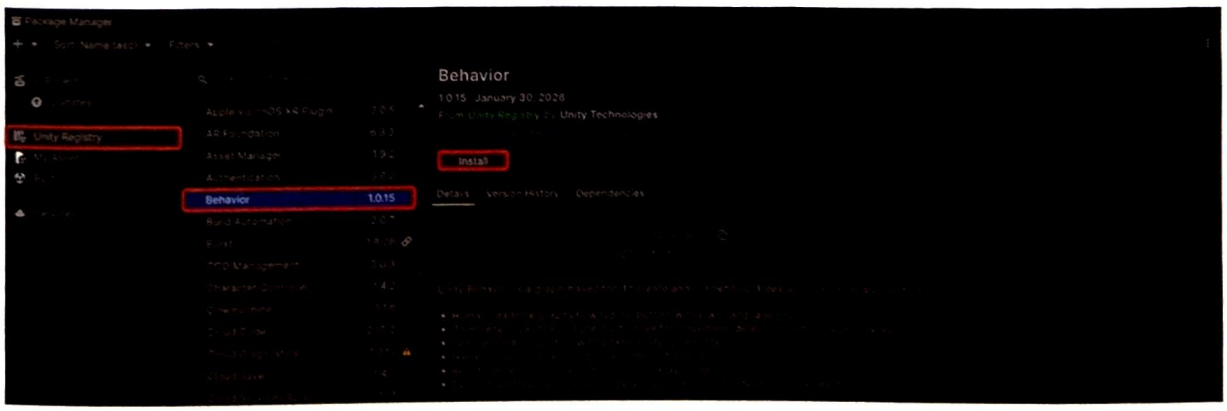
5가지 노드 상태

노드 상태	설명
Running	노드가 현재 실행되고 있습니다.
Waiting	노드가 하나 이상의 자식 브랜치를 기다립니다.
Succeeded	노드가 성공했습니다.
Failed	노드가 실패했습니다.
Uninitialized	노드가 아직 실행을 시작하지 않았습니다.

Do it! 실습 순찰하는 적 AI 구현하기

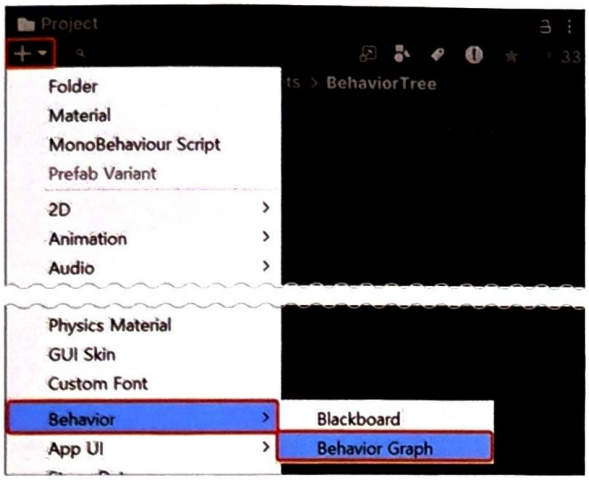
이 책에서는 제자리에 가만히 있는 대기 Idle, 지정된 경로를 차례대로 이동하는 순찰 Patrol, 임의의 위치로 이동하는 배회 Wander, 목표로 설정한 대상을 쫓아가는 추적 Chase, 목표를 공격하는 공격 Attack 등 적 행동 5개를 구현하고 적 종류에 따라 행동을 제어합니다.

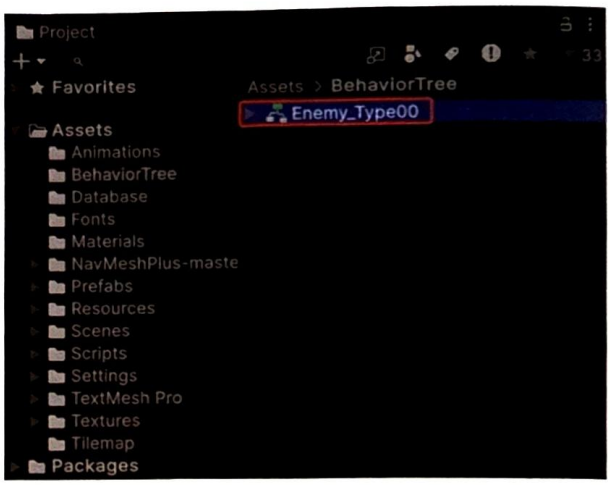
01 먼저 지정된 경로를 차례대로 이동하는 순찰 행동을 구현합니다. [Window → Package Management → Package Manager] 메뉴로 Package Manager View를 연 다음, Unity Registry 탭에서 Behavior 에셋을 검색하고 〈Install〉 버튼을 눌러 설치합니다.



Unity Registry로 Behavior 에셋 설치하기

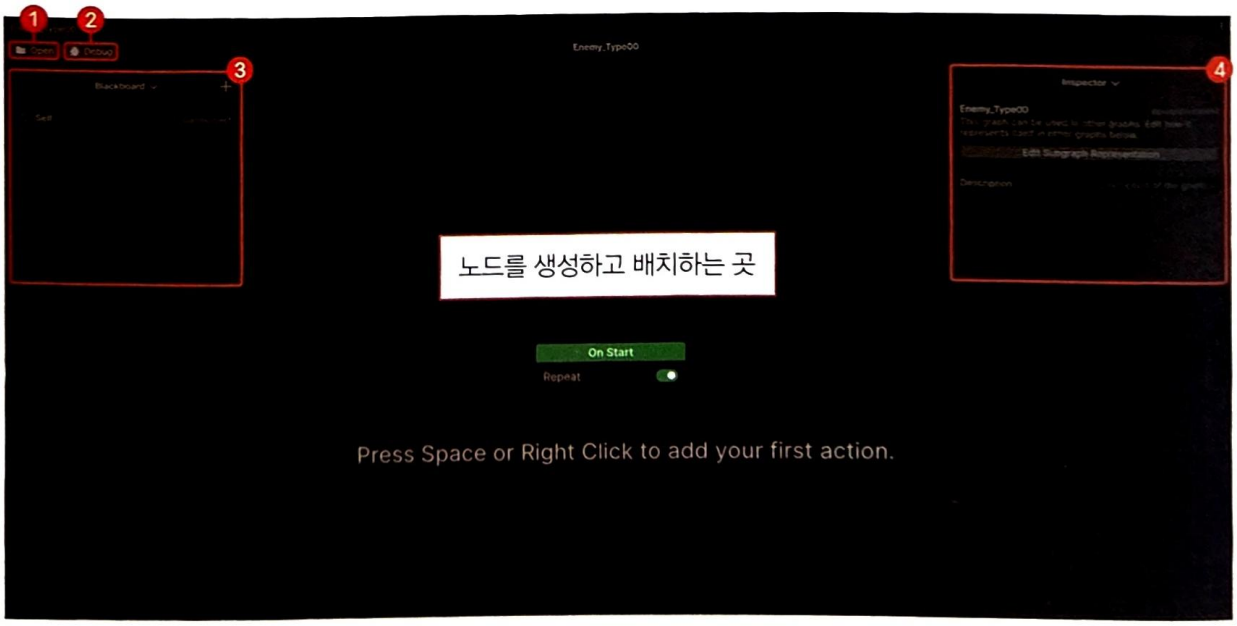
02 Behavior 에셋을 설치하는 작업이 끝나면 적의 행동 AI를 구현하고자 Project View의 + ▾ 버튼을 클릭해 [Behavior → Behavior Graph] 메뉴로 행동 그래프를 생성하고 이름을 'Enemy_Type00'으로 변경합니다. 생성한 행동 그래프는 Assets 〉 BehaviorTree 폴더에 저장합니다.





적 행동 AI를 구현하는 행동 그래프 생성하고 설정하기

03 Enemy_Type00을 더블클릭하면 Behavior Graph View가 열립니다. 지금은 행동 그래프가 Enemy_Type00 에셋 하나뿐이지만 여러 개라면 ❶ 왼쪽 위 〈Open〉 버튼을 눌러 다른 행동 그래프를 선택할 수 있습니다. ❷ 〈Debug〉 버튼은 플레이 모드에서 선택한 에이전트의 동작을 실시간으로 관찰할 때 사용합니다. ❸ 왼쪽 Blackboard는 행동 그래프에서 사용하고 외부에서 접근할 수 있는 변수 목록입니다. 자기 자신을 나타내는 Self 변수가 기본으로 생성됩니다. Blackboard에 선언한 변수는 Inspector View에 출력되고 외부에서 코드를 이용해 Set/Get 할 수 있습니다. ❹ 오른쪽 Inspector는 선택한 노드의 자세한 정보를 출력합니다. 그 외 빈 곳은 노드를 생성하고 배치할 수 있는 곳입니다.



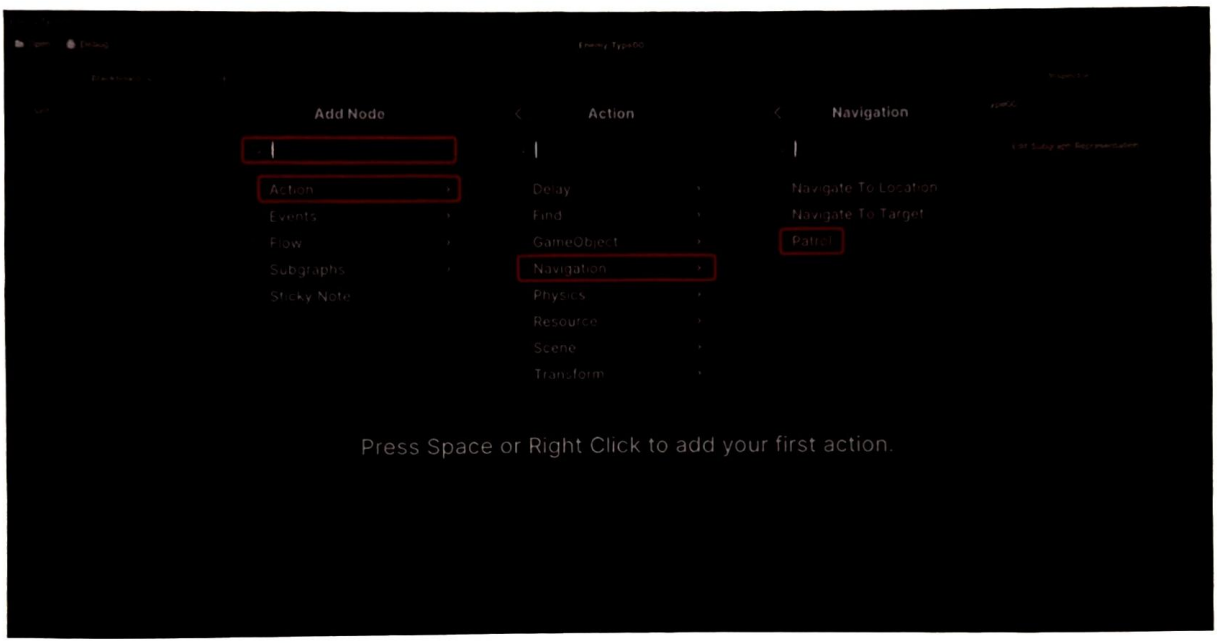
Enemy_Type00 Behavior Graph View

04 AI Navigation과 Behavior 모두 유니티에서 제공하므로 액션 노드에서 목표 위치로 이동하는 Navigate To Location, 목표 오브젝트로 이동하는 Navigate To Target, 순찰하는 Patrol 행동을 기본으로 제공하고, AI Navigation에 있는 NavMeshAgent와 자동으로 연동해 작동합니다.

에이전트 오브젝트에 NavMeshAgent 컴포넌트가 없으면 Transform을 이용해 이동합니다. 또한 AI Navigation은 xz 평면 기준인 3D용이므로 2D 게임을 제작할 때는 순찰 이외의 행동을 별도로 만들어서 사용하는 것이 좋습니다.

Behavior Graph View의 빈 곳을 마우스 오른쪽 버튼으로 클릭하고 [Add... → Action → Navigation → Patrol] 메뉴로 순찰 액션 노드를 생성합니다.

● 경로를 따라 선택하지 않고 'Patrol'을 검색해서 생성해도 됩니다.

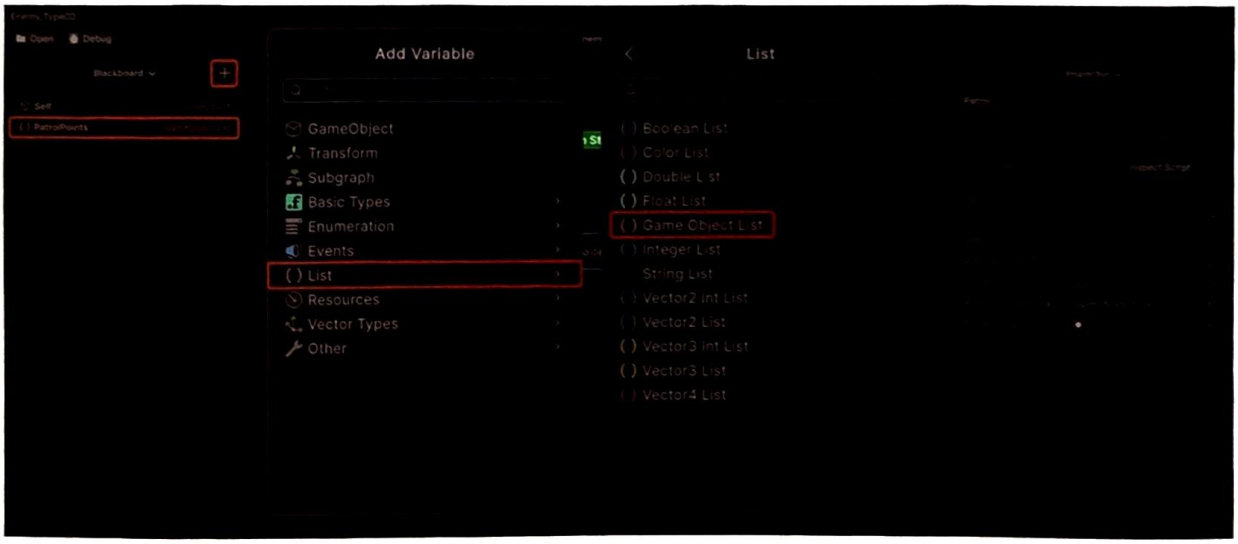


지정된 경로를 순찰하는 Patrol 노드 생성하기

05 행동 그래프의 노드는 스토리 Story 설정에 따라 노드에 설명과 등록할 수 있는 변수 일부가 출력되고, 노드를 선택하면 Inspector에 노드 설명과 설정할 수 있는 변수 목록이 출력됩니다. 〈Inspect Script〉 버튼을 누르면 해당 노드의 스크립트 정보를 볼 수 있습니다.

Agent가 Waypoints 경로를 따라 순찰하는 액션 노드

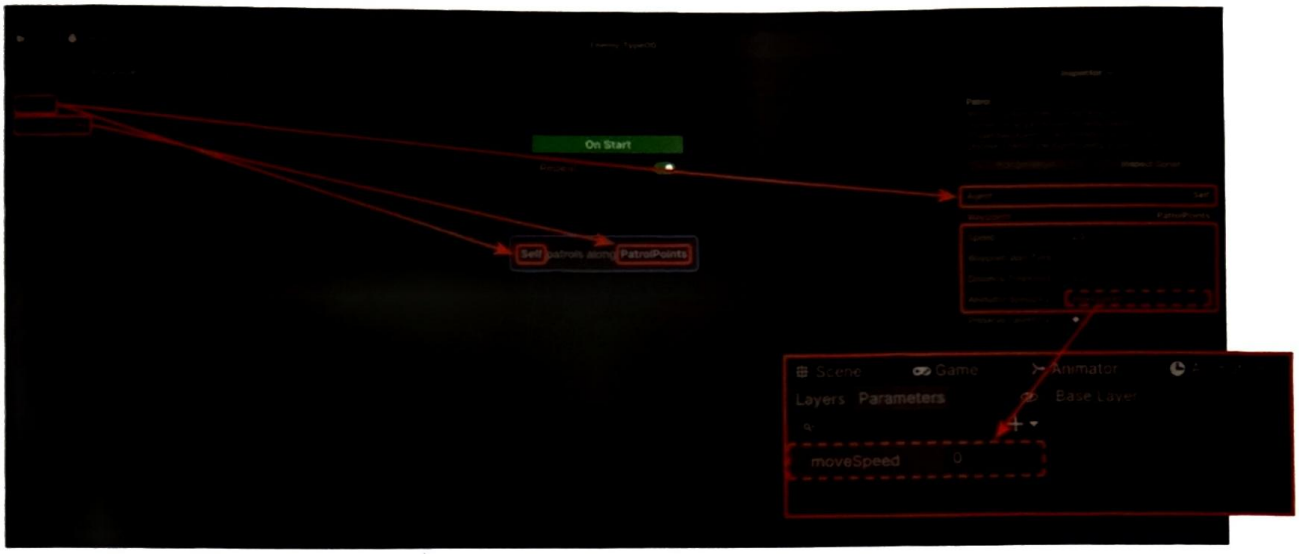
06 Blackboard의 ✚ 버튼을 클릭하고 [List → Game Object List] 메뉴로 게임오브젝트 리스트 변수를 선언합니다. 변수를 마우스 오른쪽 버튼으로 클릭하고 [Rename] 메뉴를 선택하여 이름을 'PatrolPoints'로 변경합니다.



순찰 경로로 사용할 Game Object List 변수 선언하기

07 순찰을 실행할 대상인 Agent, 순찰 경로인 Waypoints와 그 외 변수를 설정합니다. 순찰을 실행할 대상과 순찰 경로처럼 노드 정보를 표현하는 스토리의 변수를 설정하거나 노드를 선택해 출력되는 Inspector View에서 변수를 설정하는 것은 동일합니다.

적인 Enemy_00_RedGem 오브젝트가 순찰 행동을 할 때 Animator View에 설정한 파라미터가 실제 변경되므로 Enemy_00_RedGem 프리팹을 활성화한 후 Animator View에 Float Parameters를 생성하고 순찰 행동의 Animator Speed Param과 같은 이름인 'moveSpeed'로 설정합니다.



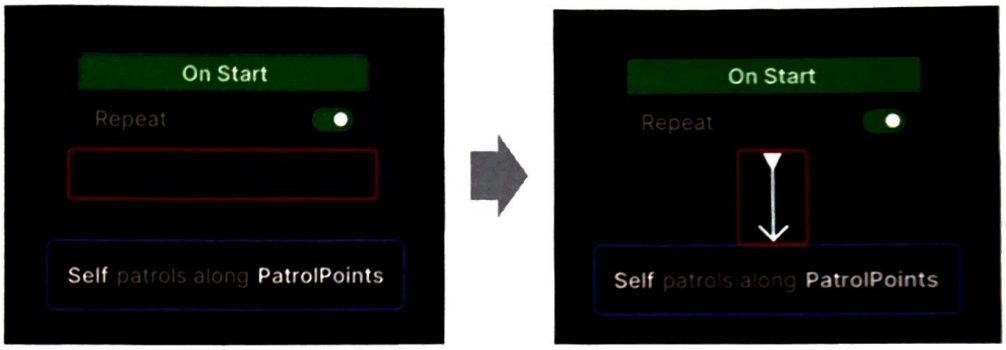
순찰(Patrol) 노드의 변수 설정하기

이 노드는 지정한 경로를 따라 순찰하는 액션 노드입니다.

순찰(Patrol) 액션 노드 변수 설명

변수	설명
Agent	순찰을 수행하는 대상(일반적으로 자기 자신을 사용)입니다.
Waypoints	순찰 경로입니다.
Speed	이동 속도입니다.
Waypoint Wait Time	각 경로에 도착했을 때 대기 시간입니다.
Distance Threshold	에이전트와 도착 위치의 거리가 해당 값 이하이면 도착으로 판단합니다.
Animator Speed Param	해당 에이전트가 Animator에서 사용하는 float 파라미터 변수입니다. (에이전트가 순찰하거나 대기할 때 대기/이동 애니메이션 재생도 연동해서 설정)

08 마우스를 이용하여 루트 노드Root Node인 On Start 노드를 Patrol 노드로 연결합니다. On Start 노드를 Repeat로 설정하고 On Start와 Patrol 노드를 연결하면 Patrol 노드를 반복 실행합니다.



Patrol 노드를 On Start 노드의 자식으로 설정하기

09 행동 그래프의 Blackboard에 선언한 순찰 경로를 설정하고자 EnemyFSM.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyFSM.cs

코드 10-02 적 순찰 경로 PatrolPoints 설정

```
01: using System.Collections;
02: using System.Linq;                                          추가
03: using UnityEngine;
04: using UnityEngine.AI;
05: using Unity.Behavior;                                       추가
06:
07: public enum EnemyState { None = -1, Attack, }
08:
09: public class EnemyFSM : MonoBehaviour
10: {
        (... 생략 ...)
17:     private NavMeshAgent navMeshAgent;          // 적 이동 경로 설정과 이동 제어
18:     private BehaviorGraphAgent behaviorAgent; // 적 행동 제어          추가
19:     private EnemyState enemyState;
20:
21:     private void Awake()
22:     {
23:         owner = GetComponent<EnemyBase>();
24:         navMeshAgent = GetComponent<NavMeshAgent>();
25:         behaviorAgent = GetComponent<BehaviorGraphAgent>();          추가
            (... 생략 ...)
31:     }
32:
33:     public void Setup(EntityBase target, GameObject[] wayPoints)          수정
34:     {
35:         owner.Target = target;
```

```
36:         // [Debug Test] 게임이 시작된 순간의 플레이어 위치로 이동
37:         navMeshAgent.SetDestination(target.transform.position);
38:         behaviorAgent.SetVariableValue("PatrolPoints", wayPoints.ToList());
39:     }
        (... 생략 ...)
70: }
```

삭제

추가

행동 그래프에서 사용하는 변수 중 Inspector View나 다른 클래스처럼 외부에서 접근할 수 있는 변수는 Blackboard에 선언합니다. 이렇게 선언한 변수는 Set 또는 Get 메서드를 호출해 값을 설정하거나 확인할 수 있습니다.

38줄 Blackboard에 선언한 `PatrolPoints` 변수에 `wayPoints` 리스트 변수를 저장합니다.

알아 두면 좋아요! 코드에서 사용한 유니티 메서드

```
bool BehaviorGraphAgent.SetVariableValue<T>(string variableName, <T> value);
```

행동 그래프의 Blackboard에 선언한 variableName 변숫값을 value로 설정합니다.

```
List<GameObject> GameObject[].ToList()
```

배열 데이터를 리스트로 변경합니다.

10 순찰 경로로 사용할 `WayPointData`를 설정하고 생성하는 적에게 전달하고자 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

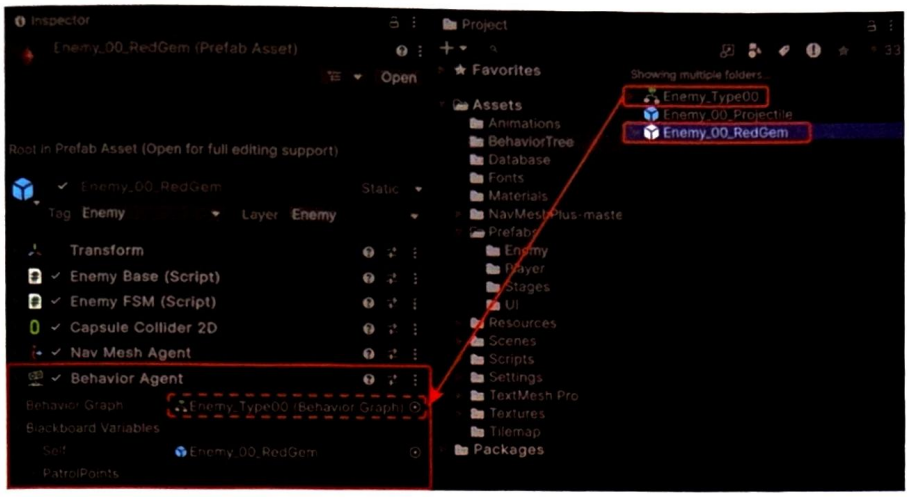
코드 10-03 적을 생성할 때 순찰 경로 정보 전달

```
    (... 생략 ...)
05: public class EnemySpawner : MonoBehaviour
06: {
        (... 생략 ...)
23:     public static List<EntityBase> Enemies { get; private set; } =
            new List<EntityBase>();
```

```
24:
25:     [System.Serializable]                                    추가
26:     private struct WayPointData
27:     {
28:         public GameObject[] wayPoints;
29:     }
30:     [SerializeField]
31:     private WayPointData[] wayPointData;
32:
33:     private void Awake()
34:     {
            (... 생략 ...)
40:         // 임의의 타일에 적 10개 생성
41:         for ( int i = 0; i < enemyCount; ++ i )
42:         {
43:             int type = Random.Range(0, enemyPrefabs.Length);
44:             int index = Random.Range(0, possibleTiles.Count);
45:             int wayIndex = Random.Range(0, wayPointData.Length);   추가
46:
47:             GameObject clone = Instantiate(enemyPrefabs[type], possibleTiles[index],
                    Quaternion.identity, transform);
48:             clone.GetComponent<EnemyBase>().Initialize(this, parentTransform,
                    gemCollector);
49:             clone.GetComponent<EnemyFSM>().Setup(
                    target, wayPointData[wayIndex].wayPoints);          수정
50:
51:             Enemies.Add(clone.GetComponent<EntityBase>());
52:         }
53:     }
        (... 생략 ...)
85: }
```

11 행동 그래프에 작성한 노드를 실행하고 Blackboard에 선언된 변수에 접근할 수 있도록 **BehaviorAgent** 클래스를 제공합니다. Enemy_00_RedGem 프리팹에 Behavior Agent 컴포넌트를 추가하고 행동 그래프 변수에 앞서 만든 Enemy_Type00을 등록합니다.

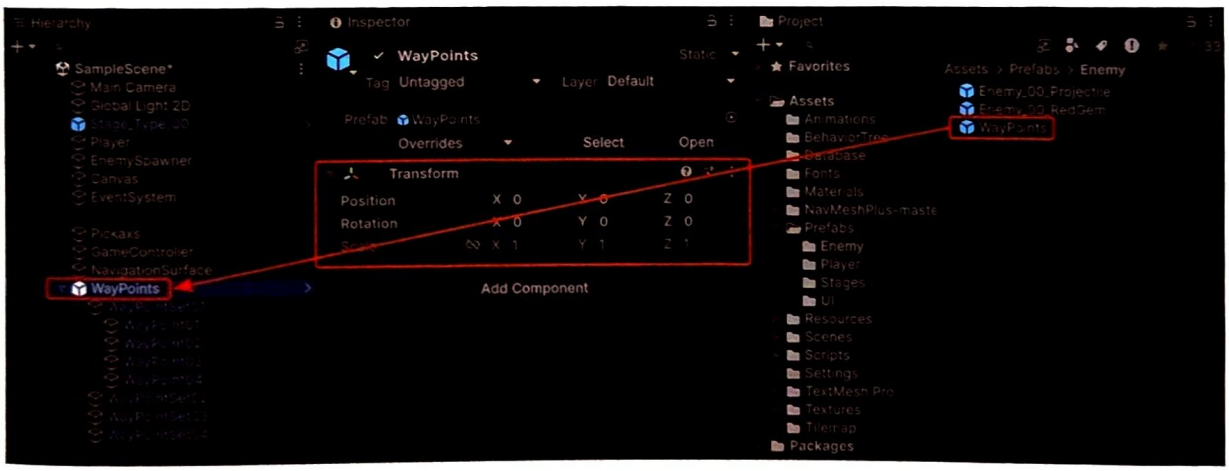
Enemy_Type00에 선언한 Blackboard 변수를 출력하는데, Self는 자동으로 설정되고 순찰 경로인 PatrolPoints는 EnemySpawner에서 적을 생성할 때 전달합니다.



Enemy_00_RedGem 프리팹에 Behavior Agent 컴포넌트 추가하고 설정하기

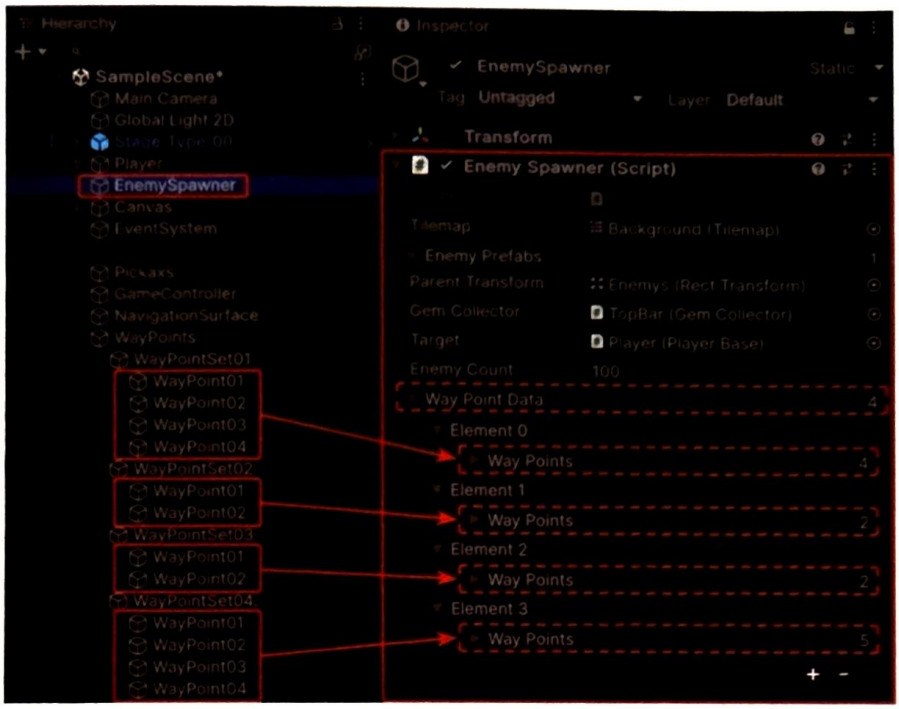
12 적 순찰 경로로 사용할 WayPoints 프리팹을 Hierarchy View로 끌어서 놓습니다. 현재 순찰 경로 4개를 생성했으며 각 적은 이 중 하나를 임의로 선택해서 순찰합니다. 맵 정보가 달라지거나 새로운 맵을 만든다면 순찰 경로도 새롭게 생성해야 합니다.

WayPoints 프리팹은 자료실에서 내려받은 패키지 안에 있습니다.



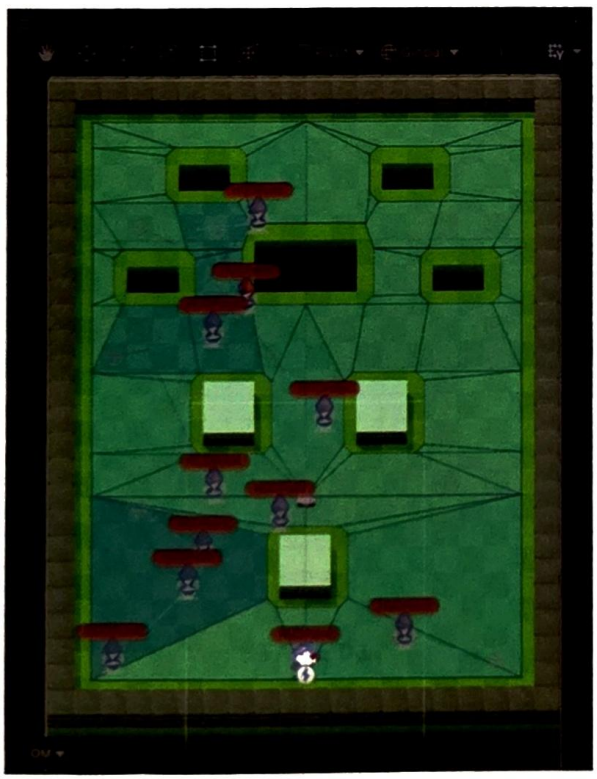
적 순찰 경로로 사용할 WayPoints 프리팹을 Hierarchy View로 끌어서 놓기

13 EnemySpawner 오브젝트의 Enemy Spawner 컴포넌트에 있는 Way Point Data 배열 크기를 '4'로 설정하고, Way Points 배열 변수에 각각 순찰 경로를 등록합니다.



Enemy Spawner 컴포넌트의 Way Point Data 배열에 적이 이동할 수 있는 모든 순찰 경로 등록하기

14 플레이 버튼 ▶을 눌러 게임을 실행하면 적마다 별도의 순찰 경로가 설정되고 지정한 순찰 경로를 따라 이동합니다.



적 AI를 구현한 결과 화면

Do it! 실습 열거형으로 대기와 순찰 행동 관리하기

01 행동 그래프의 Blackboard에 선언한 순찰 경로를 설정하고자 EnemyFSM.cs 스크립트를 수정합니다.

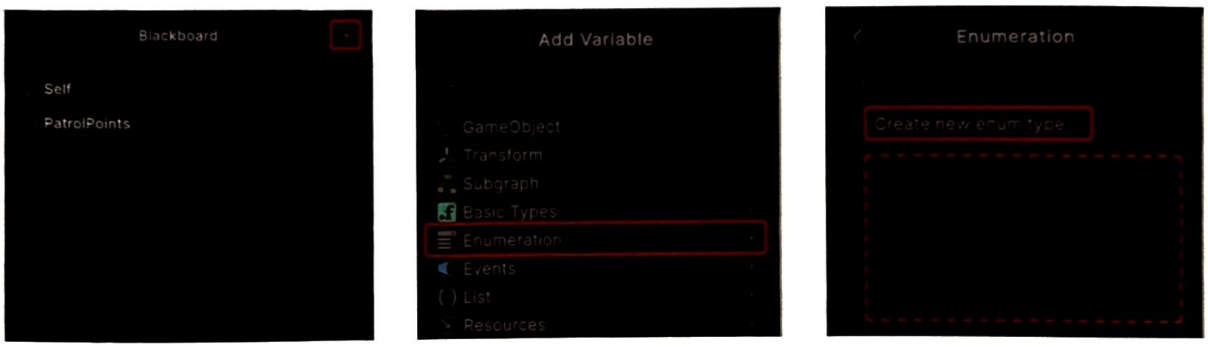
Assets/Scripts/Enemy/EnemyFSM.cs

코드 10-04 적 순찰 경로 PatrolPoints 설정

```
01: //using System.Collections;                                    ← 삭제
02: using System.Linq;
03: using UnityEngine;
04: using UnityEngine.AI;
05: using Unity.Behavior;
06:
07: //public enum EnemyState { None = -1, Attack, }                ← 삭제
08:
09: public class EnemyFSM : MonoBehaviour
10: {
11:     /*[SerializeField]
12:     private GameObject projectilePrefab;
13:     [SerializeField]                                           ← 삭제
14:     private Transform projectileSpawnPoint;*/
15:
16:     private EnemyBase owner;
17:     private NavMeshAgent navMeshAgent;        // 적 이동 경로 설정과 이동 제어
18:     private BehaviorGraphAgent behaviorAgent; // 적 행동 제어
19:     //private EnemyState enemyState;                           ← 삭제
20:
21:     private void Awake()
22:     {
            (... 생략 ...)
30:         //ChangeState(EnemyState.Attack);                     ← 삭제
31:     }
        (... 생략 ...)
39:     /*public void ChangeState(EnemyState newState)            ← 삭제
40:     {
            (... 생략 ...)
52:     }
53:
54:     private IEnumerator Attack()
55:     {
            (... 생략 ...)
67:     }*/
68: }
```

적의 상태를 변경하고 상태에 따라 행동을 수행하는 AI는 행동 트리가 제어하므로 관련 코드를 모두 삭제합니다. 또한 무기에 따라 다른 공격을 할 수 있도록 코드를 별도로 작성하고자 공격 코드도 모두 삭제합니다.

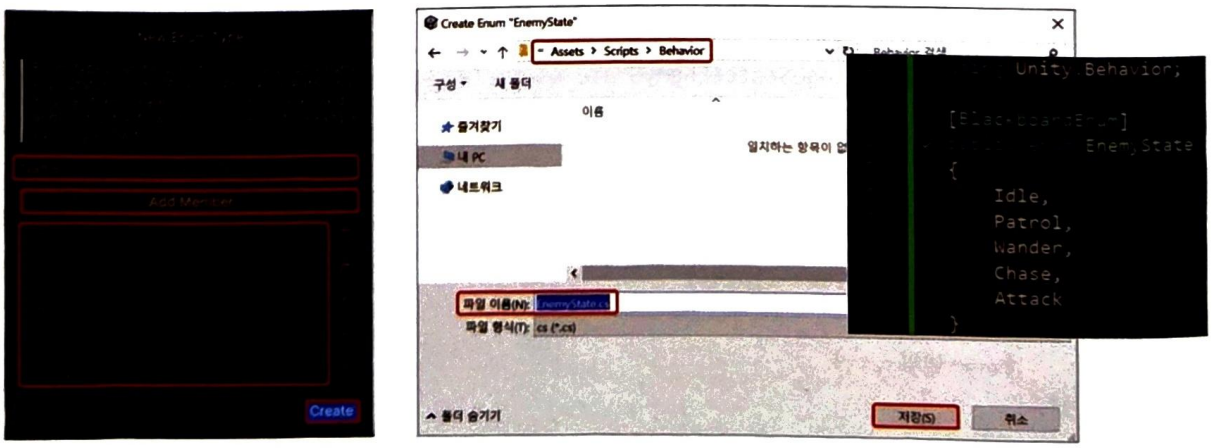
02 적의 상태는 열거형으로 관리하며 행동 그래프에서 사용하는 열거형은 Blackboard에서 새로운 열거형을 생성하거나 코드에 작성한 열거형 앞에 **[BlackboardEnum]** 어트리뷰트를 작성해도 됩니다. Enemy_Type00 에셋을 더블클릭해서 Behavior Graph View를 열고 Blackboard의 ✚ 버튼을 클릭하여 [Enumeration → Create new enum type...] 메뉴로 열거형을 생성합니다.



적 행동을 제어할 때 사용할 열거형 변수 선언하기

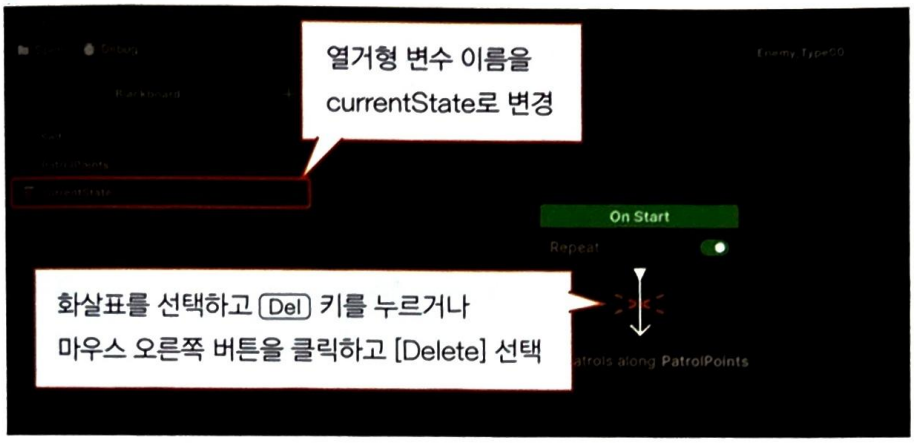
코드에서 열거형을 미리 추가하면 Enumeration 목록에 출력되므로 다음엔 변수를 쉽게 추가할 수 있습니다.

03 열거형 이름은 'EnemyState'로 설정하고 〈Add Member〉 버튼으로 상태 5개를 추가합니다. 5개 상태의 이름을 'Idle', 'Patrol', 'Wander', 'Chase', 'Attack'으로 설정하고 〈Create〉 버튼을 눌러 열거형을 스크립트로 저장합니다. 〈저장〉 버튼을 눌러 열거형이 작성된 스크립트를 생성(Assets/Scripts/Behavior/EnemyState.cs)합니다.



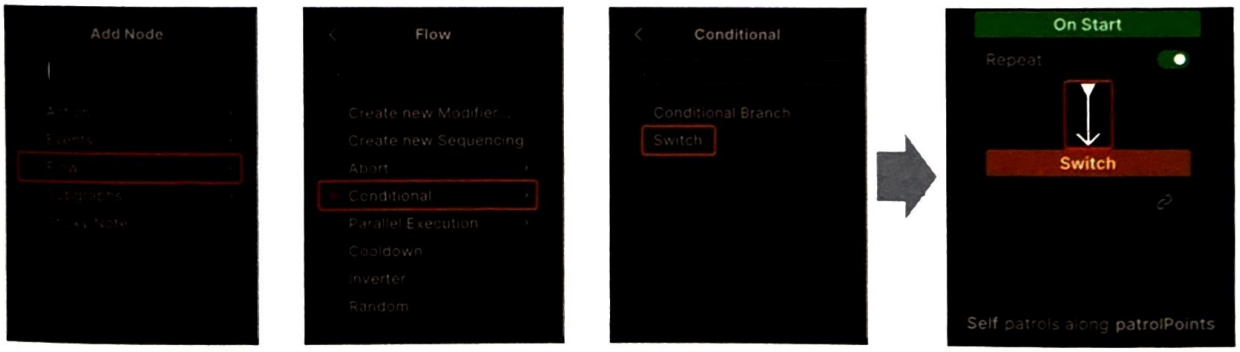
Blackboard에서 사용하는 열거형 생성하고 설정하기

04 생성한 열거형 변수의 이름을 'currentState'로 변경합니다. Switch 노드를 이용하여 현재 상태인 currentState에 따라 대기나 순찰 등의 행동을 수행하므로 On Start에서 Patrol 노드의 연결은 해제합니다.



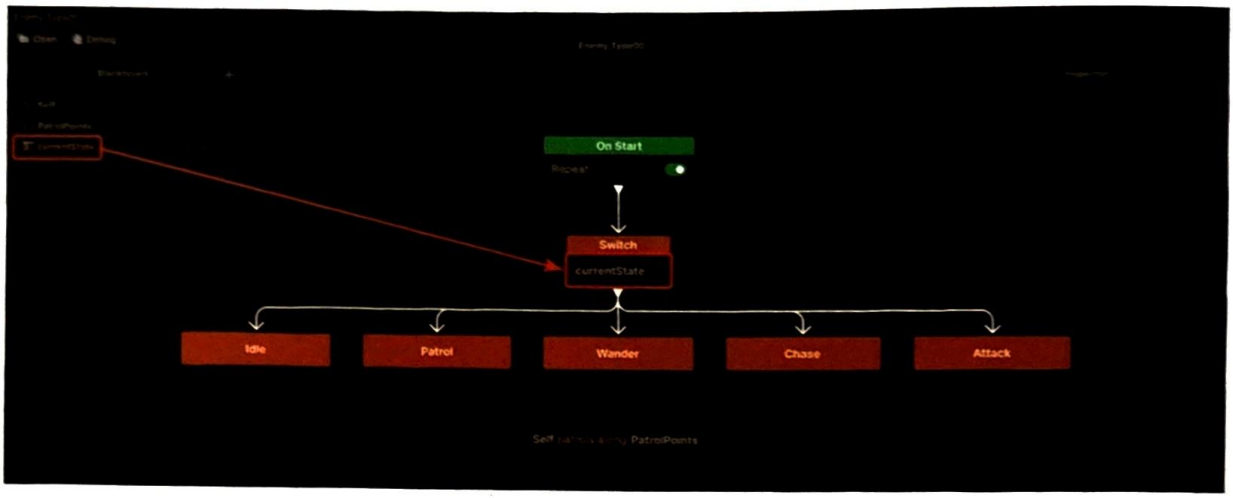
On Start와 Patrol 노드 연결 해제하기

05 상태에 따라 행동을 수행할 수 있도록 마우스 오른쪽 버튼을 눌러 [Add... → Flow → Conditional → Switch] 메뉴로 Switch 노드를 생성하고 On Start를 Switch 노드에 연결합니다.



상태에 따라 행동을 수행할 수 있도록 Switch 노드 생성하고 설정하기

06 Switch 노드에서 사용할 열거형 변수로 currentState를 등록합니다. 해당 열거형에 선언된 열거형 변수 노드가 Switch에서 자동 연결되어 생성됩니다. EnemyState 열거형에 정의한 Idle, Patrol, Wander, Chase, Attack 상태 노드가 Switch 노드의 자식 노드로 자동 생성됩니다.



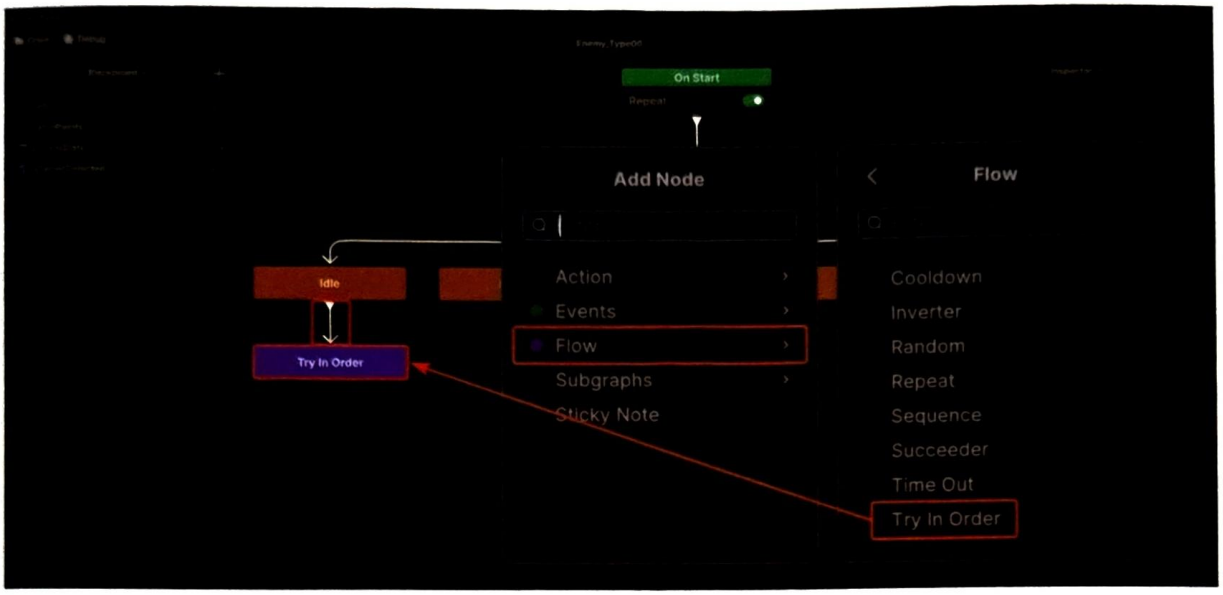
Switch 노드에서 사용하는 열거형 설정하기(버전에 따라 상태의 배치가 다르기도 함)

특정 노드를 마우스 오른쪽 버튼으로 눌러 [Align → All Children] 메뉴를 선택하면 해당 노드의 자식을 자동으로 정렬할 수 있습니다.

07 Blackboard의 ▪ 버튼을 눌러 [Basic Types → Boolean] 메뉴로 적 행동반경 내에 목표가 있는지를 저장하는 Boolean 변수를 선언하고, 이름을 'isTargetDetected'로 변경합니다.

적 행동반경 내에 목표가 있는지를 저장하는 Boolean 변수 선언하기

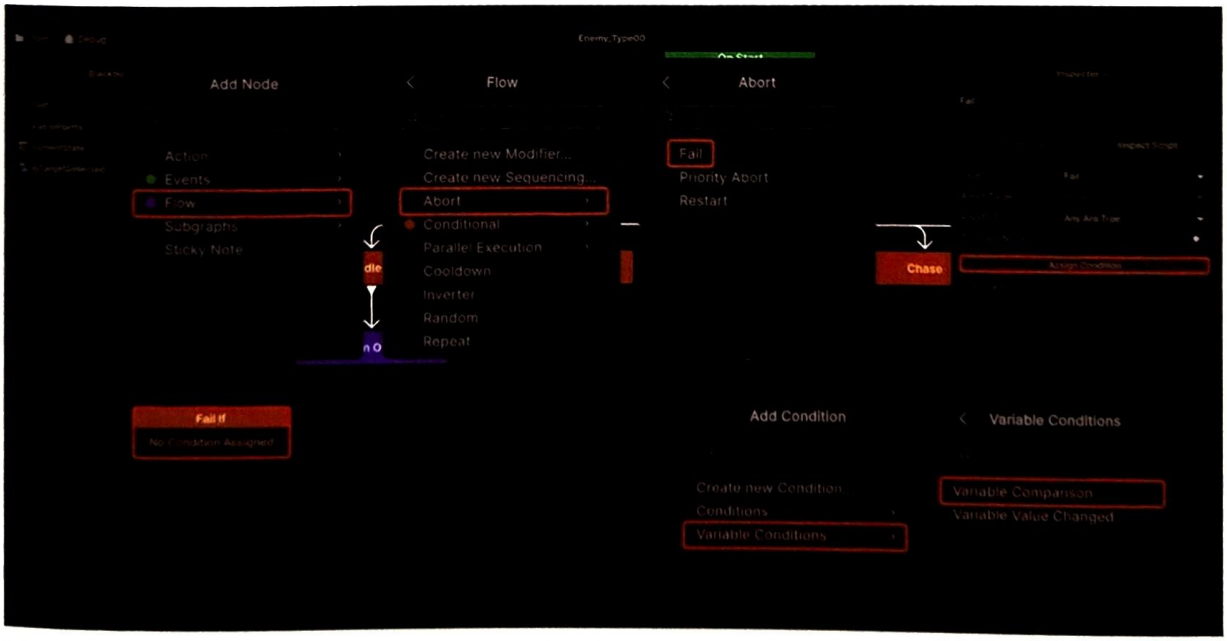
08 먼저 대기인 Idle 상태일 때 수행할 노드를 설정하는데, 대기 상태에서는 반경 내에 목표가 없으면 순찰 또는 배회 상태로, 목표가 있으면 추적 상태로 변경합니다. 성공할 때까지 분기를 순서대로 실행하는 Try In Order 노드(빈 곳에서 마우스 오른쪽 버튼을 눌러 [Add... → Flow → Try In Order] 메뉴 선택)를 생성하고, Idle 노드에서 Try In Order 노드로 연결합니다.



Idle 노드에서 이어지는 Try In Order 노드 생성하고 설정하기

- Try In Order 노드는 성공할 때까지 분기를 순서대로 실행합니다. n개의 자식을 가질 수 있으며 왼쪽부터 차례대로 실행해서 실패하면 다음 자식의 분기를 실행합니다.

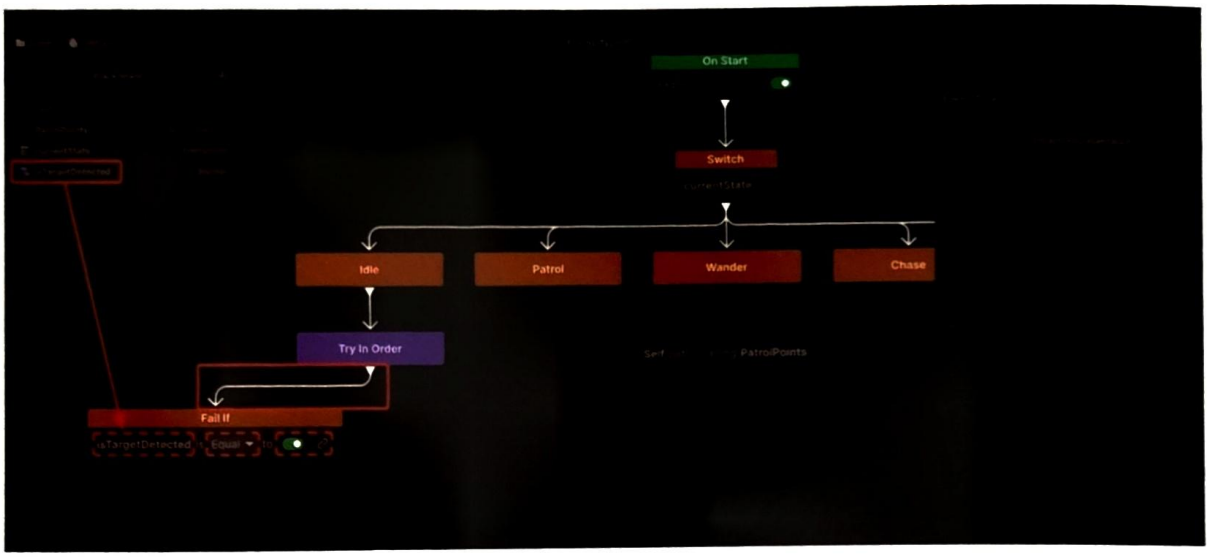
09 빈 곳에서 마우스 오른쪽 버튼을 눌러 [Add... → Flow → Abort → Fail] 메뉴로 분기를 실행 또는 중단하는 Fail If 노드를 생성합니다. 노드의 조건을 설정하고자 Fail If 노드를 선택하고 Inspector에서 〈Assign Condition〉 버튼을 클릭하여 [Variable Conditions → Variable Comparison] 메뉴로 변수 비교 조건을 Fail If 노드에 추가합니다.



분기를 실행 또는 중단하는 Fail If 노드 생성하고 설정하기

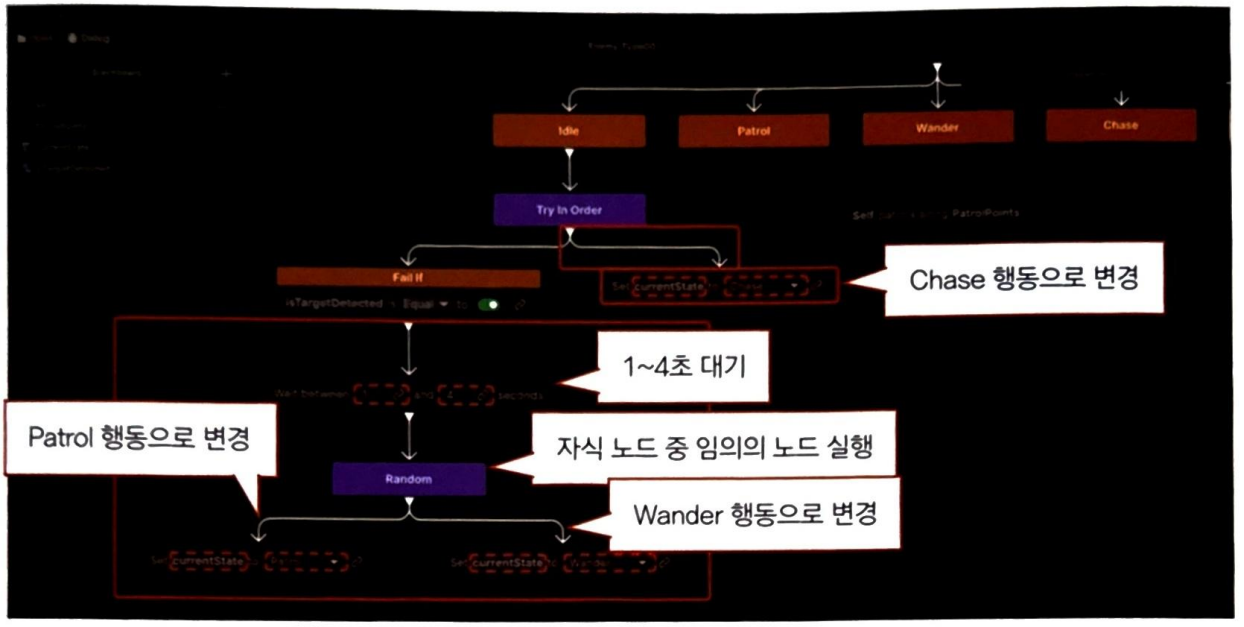
- Fail If 노드는 지정한 조건이 참(true)이면 해당 분기를 중단합니다. 원하는 조건을 〈Assign Condition〉 버튼으로 등록할 수 있으며 1개 이상 설정할 수 있습니다.

10 Fail If 노드의 Variable에는 isTargetDetected 변수를 설정하고 이 값이 true이면 현재 분기를 중단하도록 합니다. 즉, 목표로 설정하는 플레이어가 적 반경 내에 있으면 isTargetDetected가 true이므로 현재 분기를 중지하고 Try In Order 노드에 연결된 다른 분기를 실행합니다.



Fail If 노드 조건(Condition) 설정하기

11 [Add... → Action → Delay → Wait (Range)] 메뉴로 Wait (Range) 노드를 생성하고 Min = 1, Max = 4로 설정한 후 Fail If 노드에서 이어지도록 연결합니다. [Flow → Random] 메뉴로 자식 노드 중에서 임의의 노드를 실행하는 Random 노드를 생성하고 Wait (Range) 노드에서 이어지도록 연결합니다. [Action → Blackboard → Set Variable Value] 메뉴로 Set Variable Value 노드를 3개 생성하고 Variable은 currentState로 설정하고 Value가 [Patrol], [Wander]인 노드는 [Random] 노드에서 이어지도록 연결합니다. Value가 [Chase]인 노드는 Try In Order 노드에서 이어지도록 연결합니다.



반경 내에 목표가 없으면 순찰/배회 상태, 있으면 추적 상태로 변경하기

이번 단계에서 사용한 노드

노드	설명
Wait (Random) (Action Node)	min~max 시간 동안 대기한 후 다음 분기를 실행합니다.
Random (Flow Node) [Sequencing]	자식으로 연결된 노드 중에서 임의의 자식 노드를 선택해서 실행합니다.
Set Variable Value (Action Node)	Blackboard에 있는 변수를 특정한 값으로 설정할 때 사용합니다. Variable 변수뿐만 아니라 Value에도 Blackboard 변수를 등록할 수 있습니다.

앞 과정의 대기 행동을 스크립트로 구현한다면 다음처럼 코드로 작성해야 하고 상태를 변경하는 ChangeState()와 같은 코드도 추가해야 합니다.

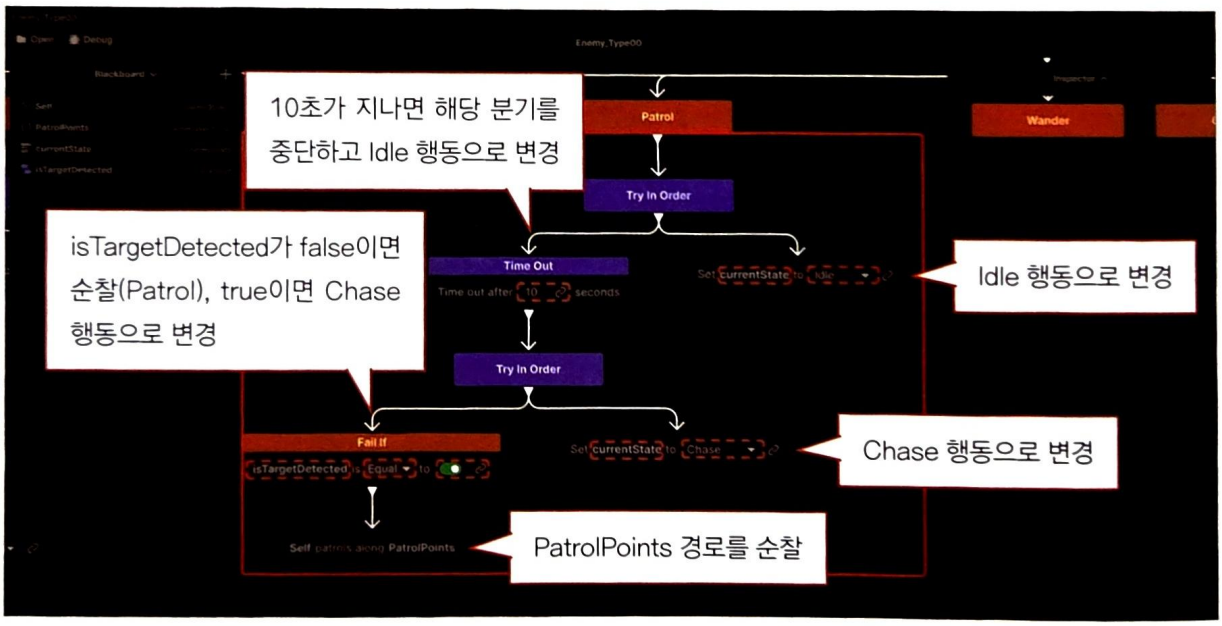
```
IEnumerator Idle()
{
    float time = Random.Range(2f, 5f);
    float currentTime = Time.time;

    while ( Time.time - currentTime < time )
    {
        yield return null;
    }

    if ( Random.value < 0.5f )  ChangeState(EnemyState.Wander);
    else                        ChangeState(EnemyState.Patrol);
}
```

스크립트로 대기 행동을 구현한 예

12 순찰Patrol 상태에서는 10초가 지나면 대기Idle 상태로 변경하고 반경 내에 목표가 있으면 추적Chase 상태로 변경합니다. 그렇지 않을 때는 순찰 노드를 실행해 경로를 순찰합니다.

[Flow → Try In Order] 메뉴로 Try In Order 노드를 생성하고 Patrol 노드에서 이어지도록 연결합니다. 10초가 지나면 대기 상태로 변경하도록 [Flow → Time Out] 메뉴로 Time Out 노드와 [Action → Blackboard → Set Variable Value] 메뉴로 Set Variable Value 노드를 생성하고 Try In Order 노드에서 이어지도록 연결합니다. Set Variable Value 노드의 Variable은 currentState로, Value는 [Idle]로 설정합니다.

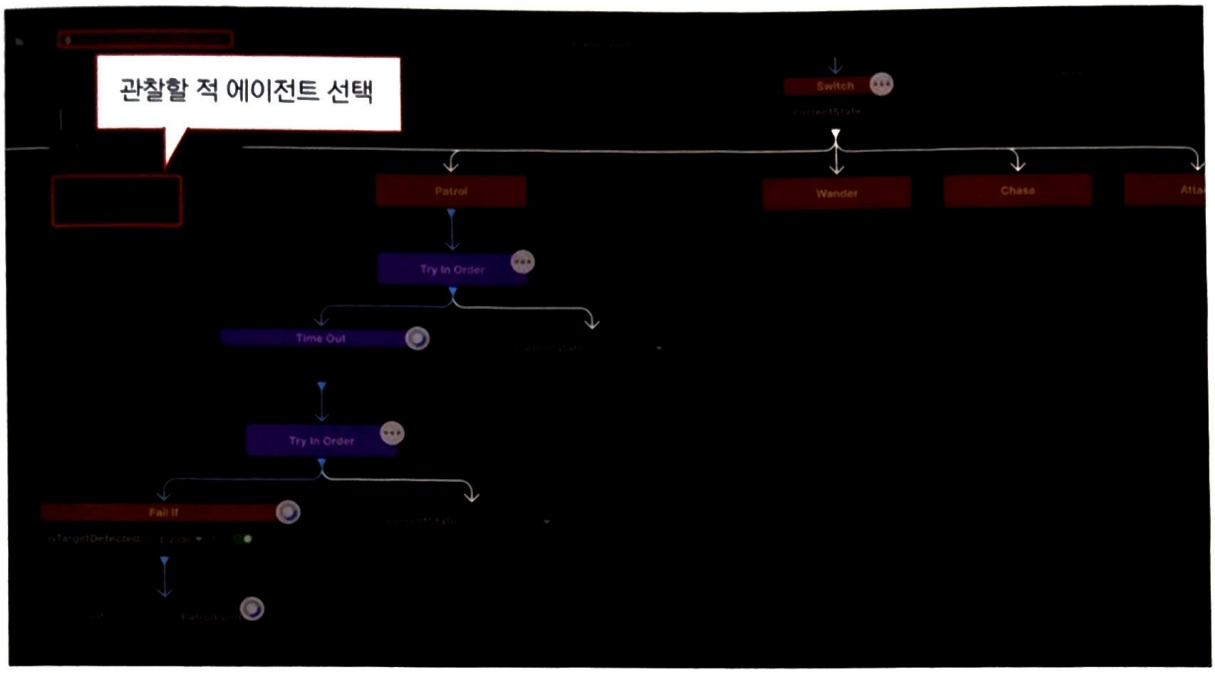
[Flow → Try In Order] 메뉴로 Try In Order 노드를 생성하고 Time Out 노드에서 이어지도록 연결합니다. Fail If 노드의 조건이 대기 상태에 설정한 노드와 동일하므로 대기(Idle)에 있는 Fail If 노드를 복사한 후 Try In Order 노드에서 이어지도록 연결합니다. Set Variable Value 노드를 생성하고 Variable은 currentState로, Value는 [Chase]로 설정한 후 Try In Order 노드에서 이어지도록 연결합니다. 그리고 Fail If 노드에서 앞서 만든 Patrol 노드로 이어지도록 연결합니다.



Patrol 행동 설정: 10초가 지나면 대기, 반경 내에 목표가 있으면 추적 상태로 변경하기

Time Out (Action Node)은 duration 시간이 지나면 해당 분기를 중단하는 노드입니다.

13 플레이 버튼 ▶ 을 눌러 게임을 실행하면 모든 적은 대기 상태로 시작하고 임의의 시간 이후 배회 또는 순찰 행동을 합니다. 현재 배회 상태는 설정하지 않았으므로 배회 상태를 선택한 적은 모두 멈추고 순찰 상태를 선택한 적은 자신의 순찰 경로를 따라 이동합니다.

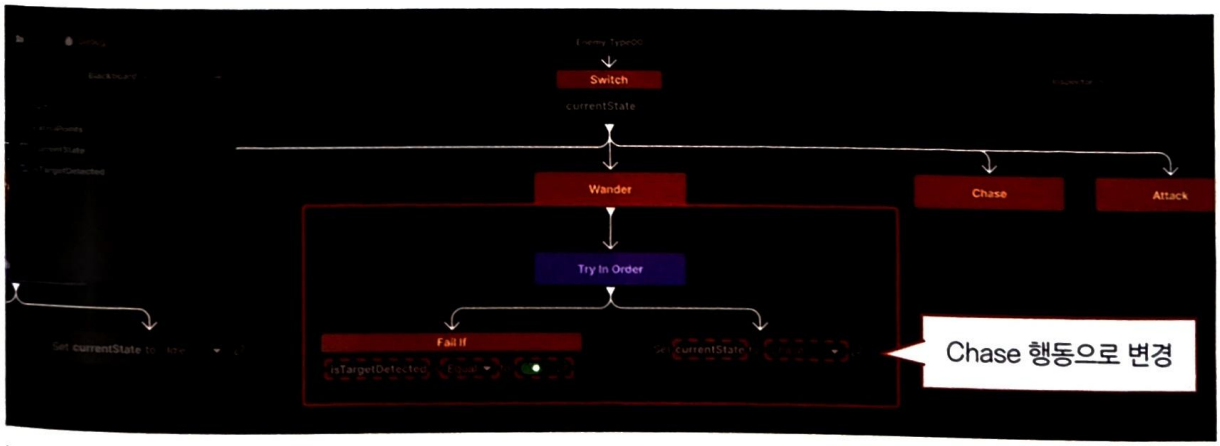


대기(Idle), 순찰(Patrol) 상태의 결과 화면

Do it! 실습 배회하는 적 구현하기

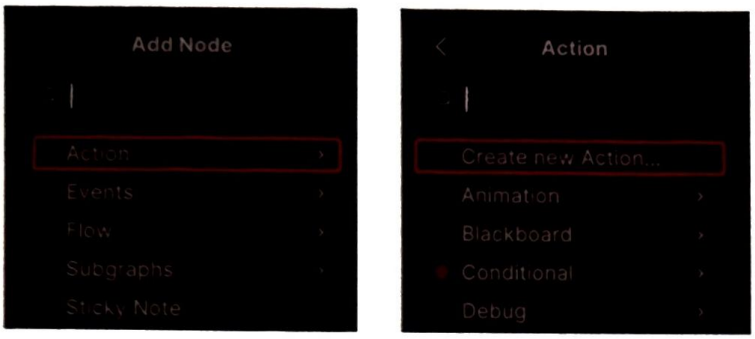
배회 상태에서는 반경 내에 목표가 없으면 배회 행동을 하고 목표가 있으면 추적 상태로 변경합니다.

01 [Flow → Try In Order] 메뉴로 Try In Order 노드를 생성하고 Wander 노드를 연결합니다. Patrol 노드의 Fail If 노드를 복사한 후 Try In Order 노드를 연결합니다. [Action → Blackboard → Set Variable Value] 메뉴로 Set Variable Value 노드를 생성하고 Variable은 currentState로, Value는 [Chase]로 설정한 후 Try In Order 노드를 연결합니다.



Wander 행동 설정: 반경 내에 목표가 없으면 배회 행동, 있으면 추적 상태로 변경하기

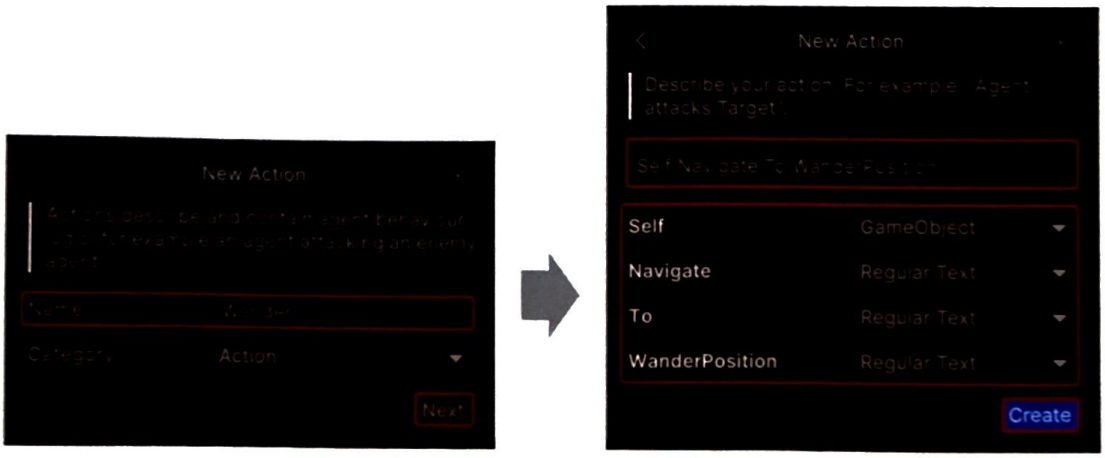
02 유니티에서 제공하는 노드 중에서 사용할 노드가 없다면 직접 만들 수 있습니다. 빈 곳을 마우스 오른쪽 버튼으로 클릭하고 [Add... → Action → Create New Action...] 메뉴로 새로운 액션 노드를 생성합니다.



새로운 액션 노드 생성하기

03 노드 이름을 'Wander'로 설정하고 〈Next〉 버튼을 누릅니다. 현재 액션 노드를 생성하므로 Name에 설정한 Wander를 기준으로 WanderAction.cs 스크립트 파일이 생성되고 클래스명이 **WanderAction**으로 설정됩니다. Wander 노드 Story 입력 상자에는 'Self Navigate To WanderPosition'을 입력하고 Self만 [GameObject] 자료형으로, 나머지는 [Regular Text] 자료형으로 설정한 후 〈Create〉 버튼을 눌러 배회 Wander 액션 노드 스크립트를 생성(Assets/Scripts/Behavior/WanderAction.cs)합니다.

Story 입력 상자에 해당 노드 설명과 등록할 변수를 작성할 수 있고 단어 단위로 띄어 쓰며 단어마다 변수형을 설정합니다. Regular Text는 단순히 노드에 출력되는 텍스트이며 원하는 변수형을 선택하면 해당 자료형의 변수를 등록할 수 있습니다. Blackboard에 이미 선언된 변수와 동일한 단어를 입력하면 자동으로 해당 자료형으로 설정되므로 그대로 사용하거나 변경할 수 있습니다.



Wander 액션 노드의 Name과 Story 설정하기(행동 그래프에서 사용할 노드의 변수와 내용)

04 배회 액션 노드의 WanderAction.cs 스크립트를 수정합니다.

Assets/Scripts/Behavior/WanderAction.cs

코드 10-05 배회 액션 노드

```
using System;
using Unity.Behavior;
using UnityEngine;
using Action = Unity.Behavior.Action;
using Unity.Properties;
using UnityEngine.AI;                                   // 추가

[Serializable, GeneratePropertyBag]
[NodeDescription(name: "Wander", story: "[Self] Navigate To WanderPosition",
    category: "Action", id: "feba2423f15ba2149a91deb41ad2eaf8")]
public partial class WanderAction : Action
{
    [SerializeReference] public BlackboardVariable<GameObject> Self;

    private NavMeshAgent agent;                         // 추가
    private Vector3 wanderPosition;
    private float currentWanderTime = 0f;
    private float maxWanderTime = 5f;

    protected override Status OnStart()
    {
        int jitterMin = 0;    // 최소 각도                 // 추가
        int jitterMax = 360; // 최대 각도
        // 현재 위치를 원점으로 하는 원의 반지름
        float wanderRadius = UnityEngine.Random.Range(2.5f, 6f);
        // 선택 각도(wanderJitterMin ~ wanderJitterMax)
        int wanderJitter = UnityEngine.Random.Range(jitterMin, jitterMax);

        // 목표 위치 = 자신(Self) 위치 + 각도(wanderJitter)에 해당하는
           반지름(wanderRadius) 크기의 원의 둘레 위치
        wanderPosition = Self.Value.transform.position +
            Utils.GetPositionFromAngle(wanderRadius, wanderJitter);
        agent = Self.Value.GetComponent<NavMeshAgent>();
        agent.SetDestination(wanderPosition);
        currentWanderTime = Time.time;

        return Status.Running;
    }

```

```
37:     protected override Status OnUpdate()
38:     {
39:         if ( (wanderPosition - Self.Value.transform.position).sqrMagnitude < 0.1f
40:             || Time.time - currentWanderTime > maxWanderTime )
41:         {
42:             return Status.Success;
43:         }
44:
45:         return Status.Running;
46:     }
47:
48:     /*protected override void OnEnd()
49:     {
50:     }*/
51: }
```

추가

삭제

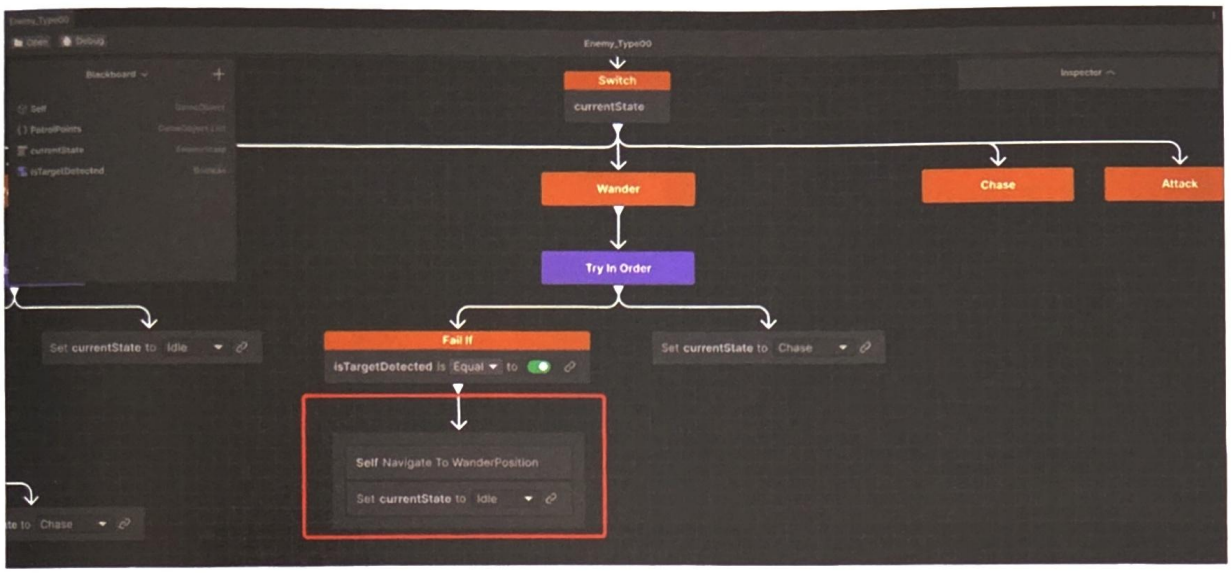
12줄 [SerializeReference] 어트리뷰트를 사용하고 변수 자료형을 BlackboardVariable<>로 선언하면 행동 그래프의 Inspector에 변수를 표시할 수 있습니다.

39~43줄 배회 목표 위치에 도달했거나 배회하기 행동을 5초 동안 수행했다면 Status.Success를 반환하고 다음 분기를 실행합니다.

액션 노드 재정의 메서드

메서드	설명
OnStart()	해당 노드를 실행할 때 1회 호출됩니다.
OnUpdate()	해당 노드를 실행하는 동안 프레임마다 호출됩니다.
OnEnd()	해당 노드의 실행이 종료될 때 1회 호출됩니다.

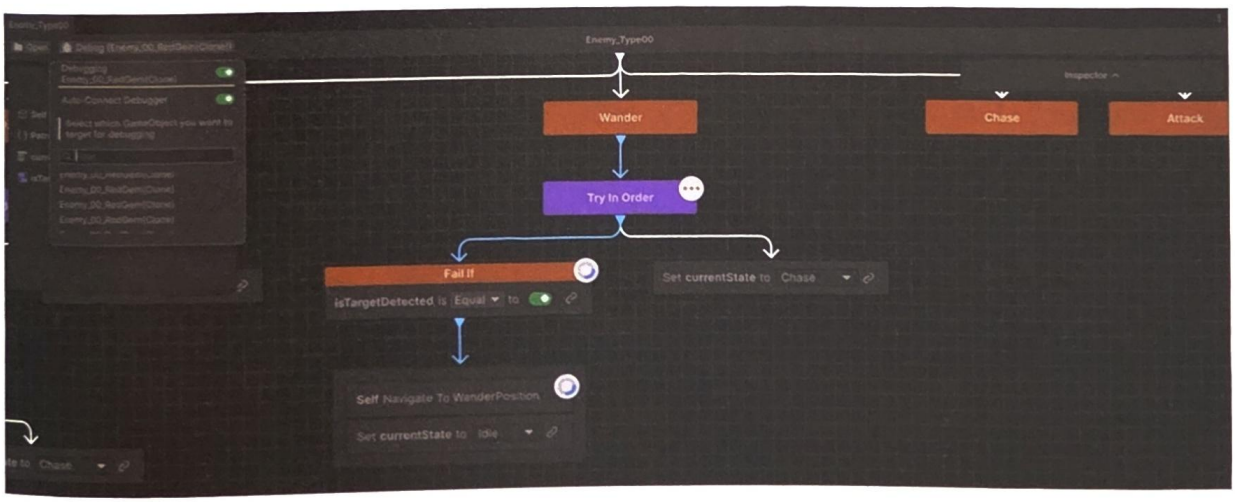
05 Wander 상태일 때 적이 배회하도록 앞서 만든 Wander 노드를 Fail If 노드와 연결합니다. Wander 노드가 Success를 반환하면 이어서 실행할 Set Variable Value 노드를 생성하고 Variable은 currentState로, Value는 [Idle]로 설정합니다. 그리고 이 노드를 끌어서 놓아 그룹화합니다.



Wander, Set Variable Value 노드 등록하기

Wander → Set Variable Value처럼 화살표로 연결한 것과 결과는 같습니다. 실행하는 액션을 한 그룹으로 나타내고 싶을 때 노드를 다른 노드로 드래그해서 그룹으로 설정합니다.

06 플레이 버튼 ▶을 눌러 게임을 실행하면 배회 Wander 상태가 되었을 때 배회 행동을 실행합니다. 배회 행동이 Running(O)에서 Success(V)로 바뀌면 아래에 그룹으로 연결된 currentState = Idle 노드를 실행합니다.

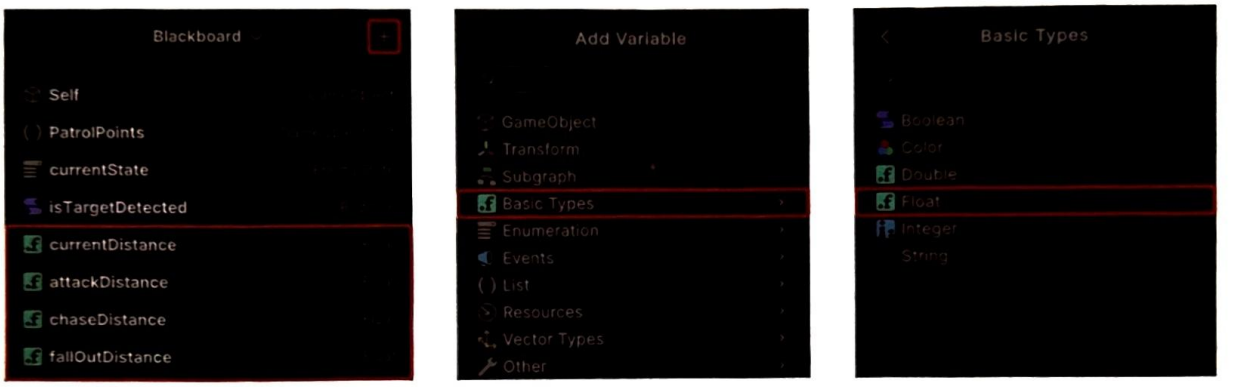


배회 행동 결과 화면

Do it! 실습 추적하는 적 구현하기

01 추적 Chase 상태에서는 목표가 추적 범위에 있으면 추적 행동을 수행하고 추적 범위를 벗어나면 대기 Idle 상태로, 공격할 수 있는 거리로 들어가면 공격 Attack 상태로 변경합니다. 추적 상태뿐만 아니라 다른 상태로 변경할 때도 목표와 거리가 기준이므로 Blackboard의 ✚ 버튼을 눌러 [Basic Types → Float] 메뉴로 Float 변수 4개를 선언합니다.

그리고 목표와의 현재 거리를 나타내는 'currentDistance', 공격 행동으로 전환하는 거리 'attackDistance', 추적 행동으로 전환하는 거리 'chaseDistance', 추적을 포기하고 다른 행동으로 전환하는 거리 'fallOutDistance'로 각각 이름을 변경합니다.

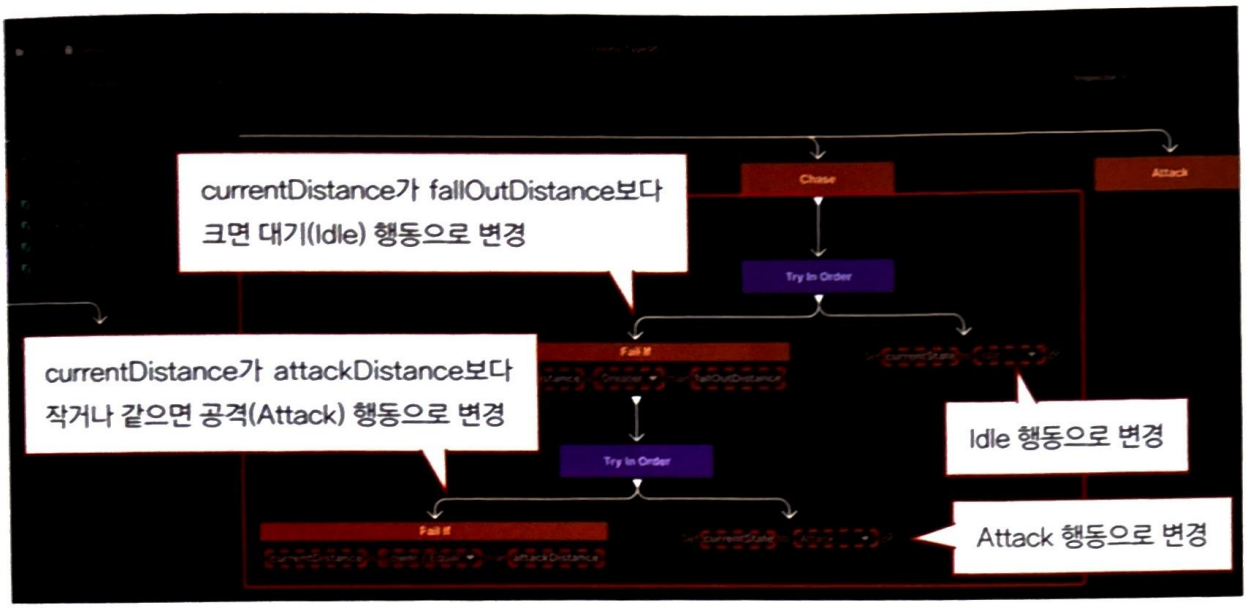


목표와의 거리 계산에 사용할 Float 변수 선언하기

02 [Flow → Try In Order] 메뉴로 Try In Order 노드를 생성하고 Chase 노드를 연결합니다. Wander 노드의 Fail If 노드를 복사한 후 Try In Order 노드를 연결합니다.

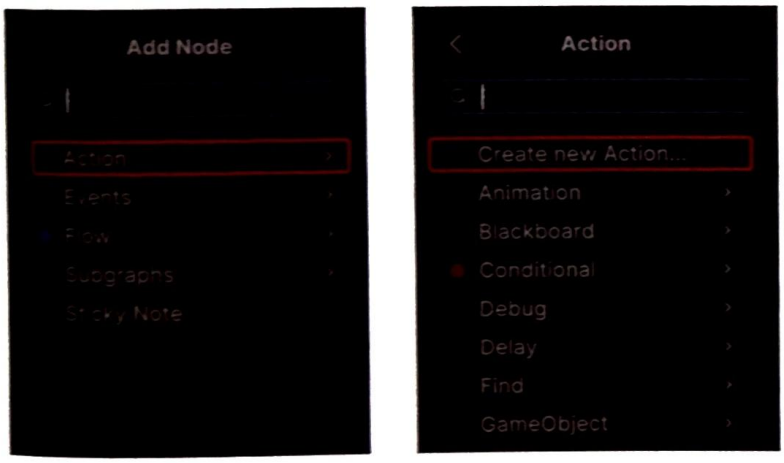
Fail If 노드의 Variable은 currentDistance로, Value는 fallOutDistance로 변경한 후 조건을 [Greater]로 설정합니다. currentDistance가 fallOutDistance보다 크면 대기 상태로 변경하고자 [Action → Blackboard → Set Variable Value] 메뉴로 Set Variable Value 노드를 생성하고 Variable은 currentState, Value는 [Idle]로 설정한 후 Try In Order 노드를 연결합니다.

Fail If에서 이어지는 Try In Order 노드를 하나 더 생성하고 Fail If와 Set Variable Value 노드를 연결합니다. Fail If 노드의 Variable은 currentDistance로, Value는 attackDistance로, 조건은 [Lower/Equal]로 설정합니다. Set Variable Value 노드의 Variable은 currentState로, Value는 [Attack]으로 설정합니다.



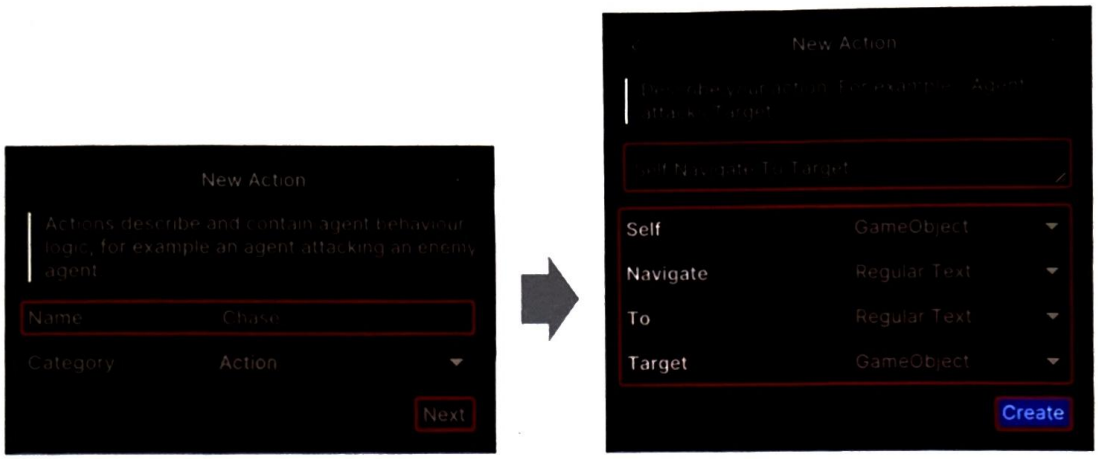
Chase 행동 설정: 적과 목표와의 거리에 따라 대기나 공격 상태로 변경하거나 목표 추격하기

03 빈 곳을 마우스 오른쪽 버튼으로 클릭하고 [Add... → Action → Create New Action...] 메뉴로 새로운 액션 노드를 생성합니다.



새로운 액션 노드 생성하기

04 노드 이름을 'Chase'로 설정하고 〈Next〉 버튼을 누릅니다. Chase 노드의 Story 입력 상자에는 'Self Navigate To Target'을 입력합니다. Self, Target은 [GameObject] 자료형으로, 나머지는 [Regular Text] 자료형으로 설정한 후 〈Create〉 버튼을 눌러 추적 액션 노드 스크립트를 생성(Assets/Scripts/Behavior/ChaseAction.cs)합니다.



Chase 노드의 Name과 Story 설정하기(행동 그래프에서 사용할 노드의 변수와 내용)

05 추적 액션 노드 ChaseAction.cs 스크립트를 수정합니다.

Assets/Scripts/Behavior/ChaseAction.cs

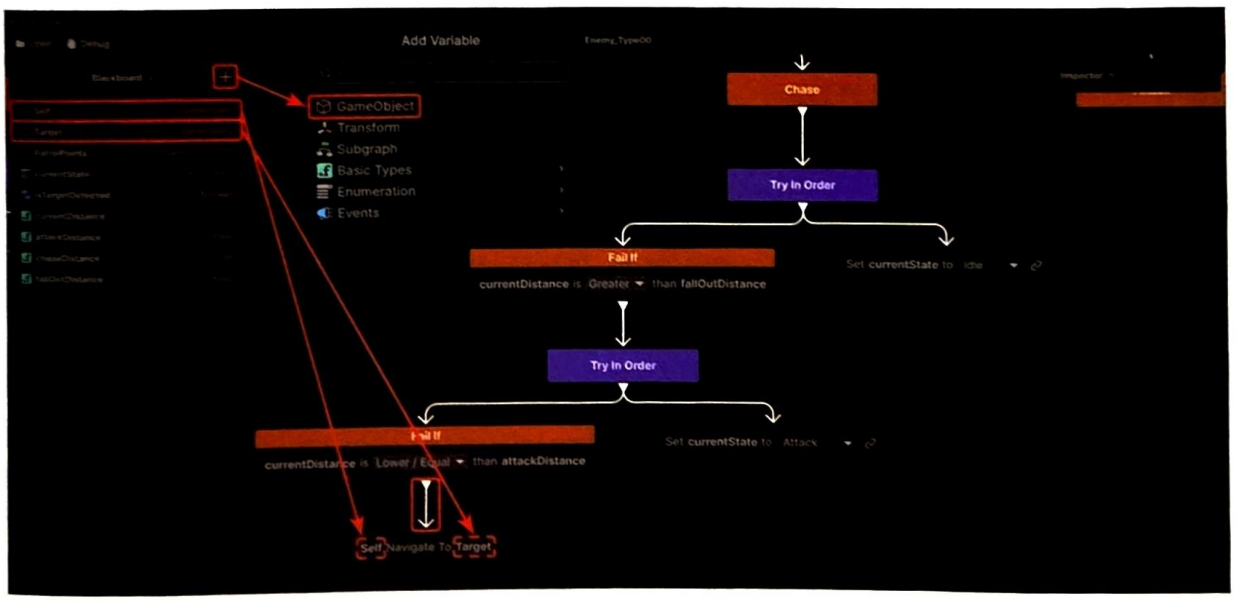
코드 10-06 추적 액션 노드

```
01: using System;
02: using Unity.Behavior;
03: using UnityEngine;
04: using Action = Unity.Behavior.Action;
05: using Unity.Properties;
06: using UnityEngine.AI;                                    추가
07:
08: [Serializable, GeneratePropertyBag]
09: [NodeDescription(name: "Chase", story: "[Self] Navigate To [Target]",
        category: "Action", id: "c957dd19f317cac57cc23659314cc90c")]
10: public partial class ChaseAction : Action
11: {
12:     [SerializeReference] public BlackboardVariable<GameObject> Self;
13:     [SerializeReference] public BlackboardVariable<GameObject> Target;
14:
15:     private NavMeshAgent agent;                            추가
16:     private EntityBase target;
17:
18:     protected override Status OnStart()
19:     {
20:         agent = Self.Value.GetComponent<NavMeshAgent>();   추가
21:         target = Target.Value.GetComponent<EntityBase>();
22:
23:         agent.speed = 5f;
24:         agent.SetDestination(target.MiddlePoint);
```

```
25:
26:         return Status.Running;
27:     }
28:
29:     /*protected override Status OnUpdate()
30:     {
31:         return Status.Success;
32:     }
33:
34:     protected override void OnEnd()
35:     {
36:     }*/
37: }
```

삭제

06 Chase 상태일 때 적이 목표를 추적하도록 Chase 노드를 등록하고 Fail If 노드를 연결합니다. Blackboard의 ✚ 버튼을 눌러 [GameObject] 메뉴로 GameObject 변수를 선언하고 이름을 'Target'으로 변경합니다. 이 Target 변수를 Chase 노드의 Target에 등록합니다.

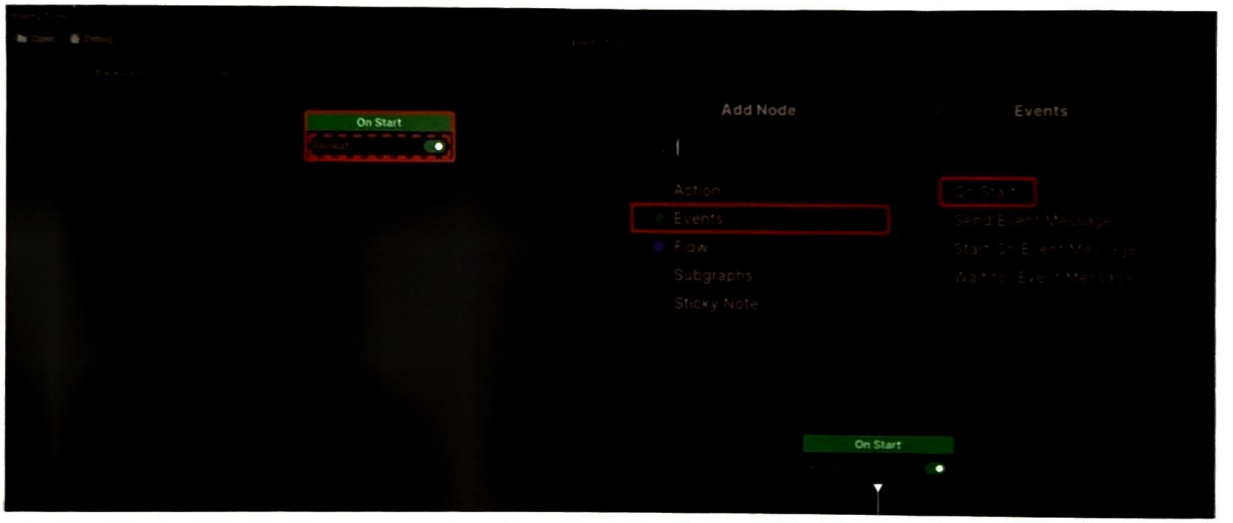


Chase 노드 등록하기

Do it! 실습 행동을 바꾸는 데 필요한 거리 계산하기

목표와의 거리를 계산하고 거리에 따라 isTargetDetected를 true 또는 false로 설정하는 로직은 목표의 현재 상태인 currentState와 관계없이 작동해야 하므로 별도 로직으로 처리합니다.

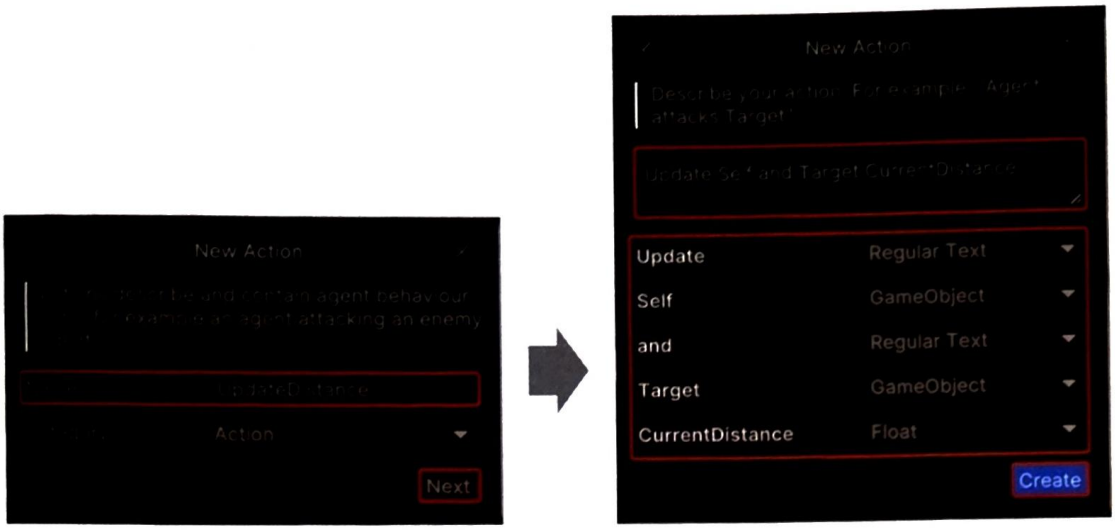
01 [Events → On Start] 메뉴로 On Start 노드를 생성하고 Repeat로 설정합니다.



거리를 계산하는 루트 노드 On Start 만들기

02 Behavior Graph View의 빈 곳을 마우스 오른쪽 버튼으로 클릭하고 [Add → Action → Create New Action...] 메뉴로 새로운 노드를 생성합니다. 노드 이름을 'UpdateDistance'로 변경하고 〈Next〉 버튼을 누릅니다.

거리 계산 노드의 Story 입력 상자에는 'Update Self and Target CurrentDistance'를 입력하고, Self와 Target은 [GameObject] 자료형으로, CurrentDistance는 [Float] 자료형으로 설정한 후 〈Create〉 버튼을 눌러 스크립트를 생성(Assets/Scripts/Behavior/UpdateDistanceAction.cs)합니다.



거리를 계산하는 액션 노드 UpdateDistance 생성하고 설정하기

03 거리를 계산하는 액션 노드 UpdateDistanceAction.cs 스크립트를 수정합니다.

Assets/Scripts/Behavior/UpdateDistanceAction.cs

코드 10-07 거리를 계산하는 액션 노드

```
    (... 생략 ...)
07: [Serializable, GeneratePropertyBag]
08: [NodeDescription(name: "UpdateDistance", story: "Update [Self] and [Target]
      [CurrentDistance]", category: "Action", id: "3c9a06a64c58a776850e299112510b01")]
09: public partial class UpdateDistanceAction : Action
10: {
11:     [SerializeReference] public BlackboardVariable<GameObject> Self;
12:     [SerializeReference] public BlackboardVariable<GameObject> Target;
13:     [SerializeReference] public BlackboardVariable<float> CurrentDistance;
14:
15:     private EntityBase self;
16:     private EntityBase target;
17:
18:     protected override Status OnStart()
19:     {
20:         self = Self.Value.GetComponent<EntityBase>();
21:         target = Target.Value.GetComponent<EntityBase>();
22:
23:         return Status.Running;
24:     }
25:
26:     protected override Status OnUpdate()
27:     {
```

추가

추가

07 추적할 수 있는 거리인지 판단하도록 CheckTargetDetectCondition.cs 스크립트를 수정합니다.

Assets/Scripts/Behavior/CheckTargetDetectCondition.cs

코드 10-08 추적할 수 있는 거리인지 판단하는 조건

```
01: using System;
02: using Unity.Behavior;
03: using UnityEngine;
04:
05: [Serializable, Unity.Properties.GeneratePropertyBag]
06: [Condition(name: "CheckTargetDetect",
        story: "Compare values of [CurrentDistance] and [ChaseDistance]",
        category: "Conditions", id: "be692720f1423151935deb31c792089c")]
07: public partial class CheckTargetDetectCondition : Condition
08: {
09:     [SerializeReference] public BlackboardVariable<float> CurrentDistance;
10:     [SerializeReference] public BlackboardVariable<float> ChaseDistance;
11:
12:     public override bool IsTrue()
13:     {
14:         if ( CurrentDistance.Value <= ChaseDistance.Value )
15:         {
16:             return true;
17:         }
18:
19:         return false;
20:     }
21:
22:     /*public override void OnStart()
23:     {
24:     }
25:
26:     public override void OnEnd()
27:     {
28:     }*/
29: }
```

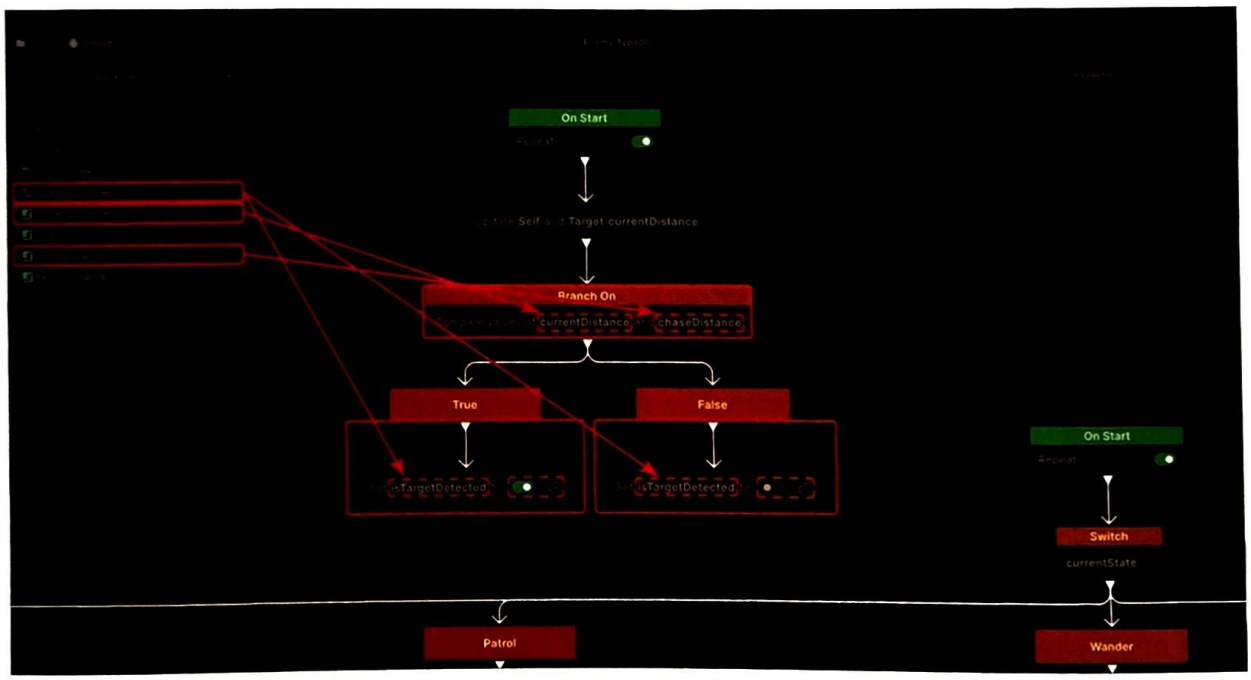
추가

삭제

코드로도 조건 Condition을 만들 수 있다는 것을 보이고자 만든 것으로, 지금처럼 두 수를 비교하는 조건은 〈Assign Condition〉 버튼을 눌러 [Variable Conditions → Variable Comparison] 메뉴를 사용해도 됩니다.

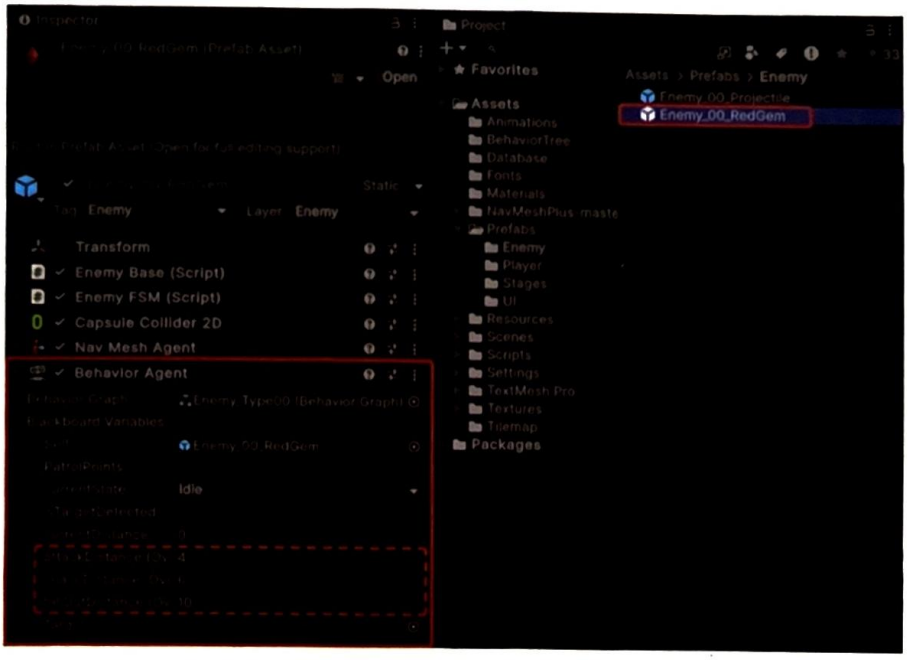
08 Branch on 노드에는 UpdateDistance 노드를 연결합니다. Branch on 노드에 currentDistance, chaseDistance 변수를 등록합니다. [Action → Blackboard] 메뉴에서 Set Variable Value 노드를 2개 생성하고 Variable은 isTargetDetected를, Value는 true와 false를 각각 설정합니다. Branch 노드가 true를 반환하면 isTargetDetected가 true로 설정되도록 노드를 연결하고, false를 반환하면 isTargetDetected가 false로 설정되도록 노드를 연결합니다.

On Start부터 차례대로 적과 플레이어의 거리를 계산해 currentDistance에 저장하고, 이 값과 chaseDistance를 비교해 둘 사이의 거리 currentDistance가 추적할 수 있는 거리 chaseDistance보다 멀면 isTargetDetected는 false로, 가깝거나 같으면 true로 설정합니다.



currentDistance가 chaseDistance보다 작거나 같으면 isTargetDetected = true 크면 false

09 Enemy_00_RedGem 프리팹의 Behavior Agent 컴포넌트에 있는 attackDistance는 '4', chaseDistance는 '6', fallOutDistance는 '10'으로 설정합니다. 이렇게 적 종류에 따라 거리를 다르게 설정할 수 있습니다.



Enemy_00_RedGem 프리팹의 Behavior Agent 컴포넌트 변수 설정하기

10 Behavior Graph View의 Blackboard에 선언한 목표를 설정하고자 EnemyFSM.cs 스크립트를 수정합니다.

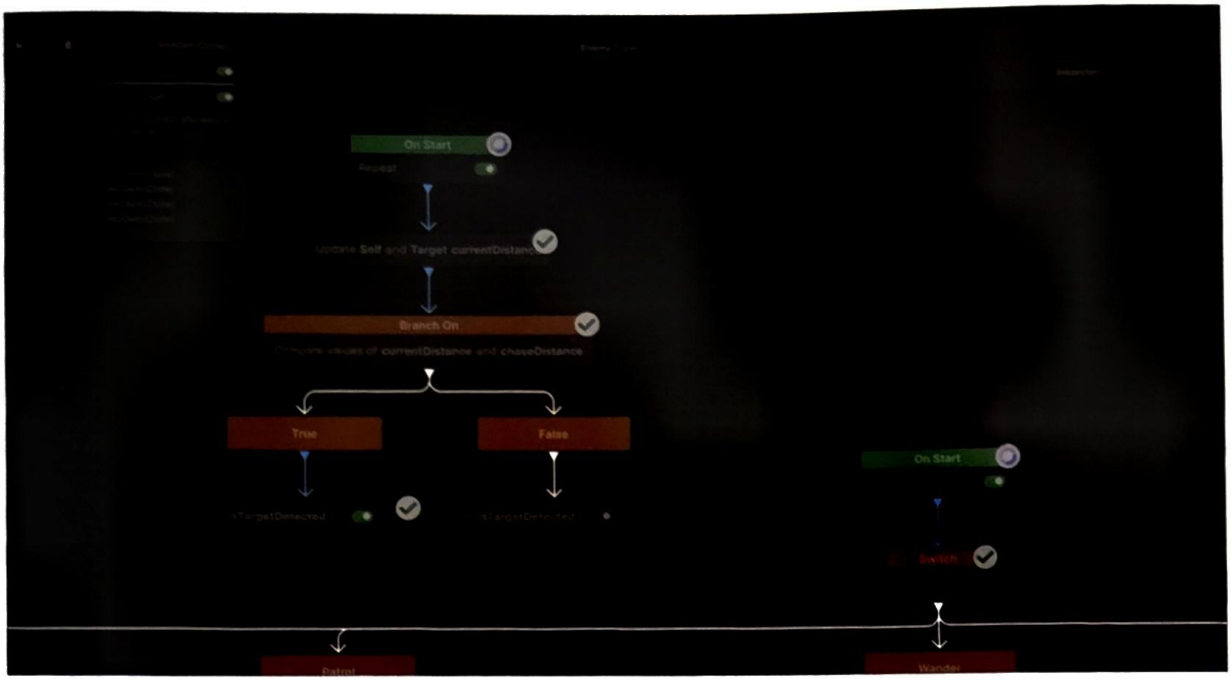
Assets/Scripts/Enemy/EnemyFSM.cs

코드 10-09 목표(Target) 설정

```
    (... 생략 ...)
06: public class EnemyFSM : MonoBehaviour
07: {
        (... 생략 ...)
22:     public void Setup(EntityBase target, GameObject[] wayPoints)
23:     {
24:         owner.Target = target;
25:         behaviorAgent.SetVariableValue("PatrolPoints", wayPoints.ToList());
26:         behaviorAgent.SetVariableValue("Target", target.gameObject);
27:     }
28: }
```

추가

11 플레이 버튼 ▶을 눌러 게임을 실행하면 현재 상태 currentState에 따라 행동을 정하는 로직과 함께 거리를 계산합니다. 이때 isTargetDetected를 true 또는 false로 설정하는 로직은 따로 동작합니다.



적과 목표의 거리 계산 결과 화면

Do it! 실습 다양한 무기를 이용한 적 공격 구현하기

무기를 장착한다는 개념으로 같은 적이라도 무기만 교체하면 다른 공격을 할 수 있도록 무기 클래스를 별도로 제작해서 사용합니다.

01 Assets 〉 Scripts 〉 Enemy 폴더에 모든 적 무기가 상속받는 WeaponBase.cs 스크립트를 생성합니다.

Assets/Scripts/Enemy/WeaponBase.cs

코드 10-10 모든 무기가 상속받는 기반 클래스

```
01: using UnityEngine;
02:
03: public abstract class WeaponBase : MonoBehaviour
04: {
05:     [SerializeField]
06:     protected GameObject projectilePrefab;
07:     [SerializeField]
08:     protected Transform projectileSpawnPoint;
09:
10:     private float currentCooldownTime = 0f;
11:     private float maxCooldownTime = 0f;
```

```
12:     private bool isSkillAvailable = true;
13:     protected float damage;
14:     protected EntityBase owner;
15:
16:     public void Setup(EntityBase owner)
17:     {
18:         this.owner = owner;
19:         damage = owner.Stats.GetStat(StatType.Damage).Value;
20:         maxCooldownTime = owner.Stats.GetStat(StatType.CooldownTime).Value;
21:     }
22:
23:     private void Update()
24:     {
25:         if ( isSkillAvailable == false && Time.time - currentCooldownTime >
                maxCooldownTime )
26:         {
27:             isSkillAvailable = true;
28:         }
29:     }
30:
31:     public void TryAttack()
32:     {
33:         if ( isSkillAvailable == true )
34:         {
35:             OnAttack();
36:             isSkillAvailable = false;
37:             currentCooldownTime = Time.time;
38:         }
39:     }
40:
41:     public abstract void OnAttack();
42: }
```

19~20줄 실제 무기를 구현하는 자식 클래스에서 활용할 수 있도록 적 EntityBase의 Stats에서 공격력과 쿨타임 정보를 받아서 멤버 변수 damage, maxCooldownTime에 저장합니다.

23~29줄 Update() 메서드에서 공격 가능 여부를 판단합니다.

31~39줄 TryAttack() 메서드는 현재 공격할 수 있는 상태인지 검사하고, 공격할 수 있는 상태이면 OnAttack() 메서드로 공격하고 변수를 초기화합니다. 이 메서드는 행동 그래프의 적 공격 행동에서 호출합니다.

41줄 OnAttack() 메서드는 실제 공격하는 방법으로, 자식 클래스에서 재정의합니다.

02 04-3절 EnemyFSM.cs 스크립트의 **Attack()** 메서드에 작성한 직선 발사체 하나 생성하는 무기를 구현하고자 Assets 〉 Scripts 〉 Enemy 폴더에 WeaponStraight.cs 스크립트를 생성합니다.

Assets/Scripts/Enemy/WeaponStraight.cs

코드 10-11 직선 발사체를 생성하는 무기

```
01: using UnityEngine;
02:
03: public class WeaponStraight : WeaponBase
04: {
05:     public override void OnAttack()
06:     {
07:         GameObject clone = GameObject.Instantiate(projectilePrefab,
                projectileSpawnPoint.position, Quaternion.identity);
08:         clone.GetComponent<EnemyProjectile>().Setup(owner.Target.MiddlePoint,
                damage);
09:     }
10: }
```

03 현재 장착한 무기 정보인 currentWeapon을 설정하고자 EnemyFSM.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemyFSM.cs

코드 10-12 현재 장착 중인 무기 설정

```
    (... 생략 ...)
06: public class EnemyFSM : MonoBehaviour
07: {
08:     private EnemyBase owner;
09:     private NavMeshAgent navMeshAgent;          // 적 이동 경로 설정과 이동 제어
10:     private BehaviorGraphAgent behaviorAgent; // 적 행동 제어
11:     private WeaponBase currentWeapon;           // 현재 활성화된 무기 ← 추가
12:
13:     private void Awake()
14:     {
15:         owner = GetComponent<EnemyBase>();
16:         navMeshAgent = GetComponent<NavMeshAgent>();
17:         behaviorAgent = GetComponent<BehaviorGraphAgent>();
18:         currentWeapon = GetComponent<WeaponBase>();  ← 추가
19:
```

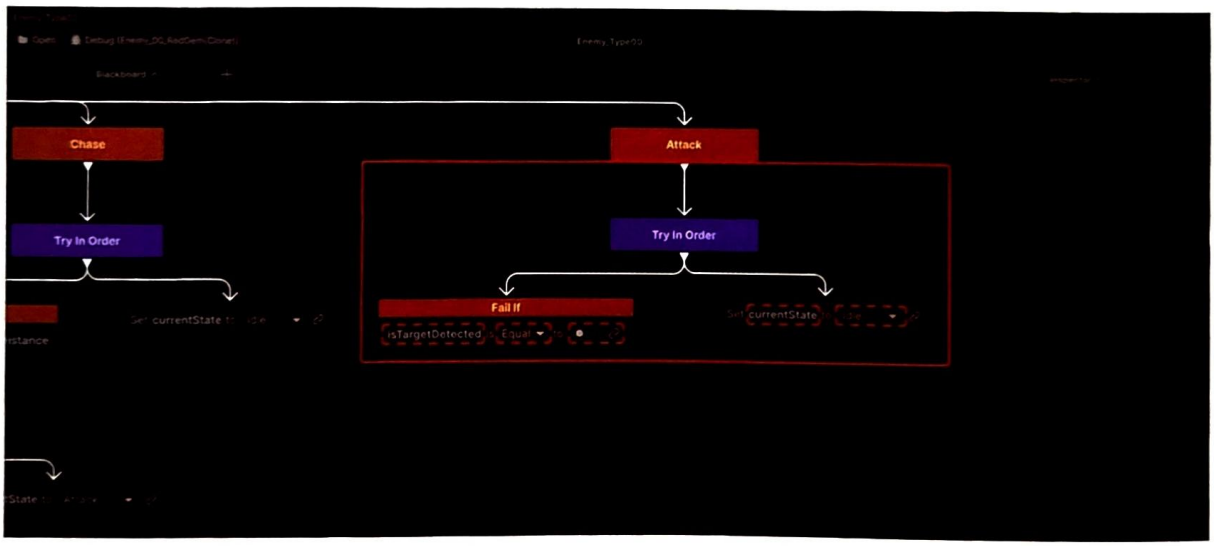
```
20:         navMeshAgent.updateRotation = false;
21:         navMeshAgent.updateUpAxis = false;
22:         currentWeapon.Setup(owner);
23:     }
        (... 생략 ...)
31: }
```

추가

04 공격 Attack 상태에서는 목표가 공격 범위에 있으면 현재 등록된 무기로 공격하고 범위를 벗어나면 대기 상태로 바꿉니다.

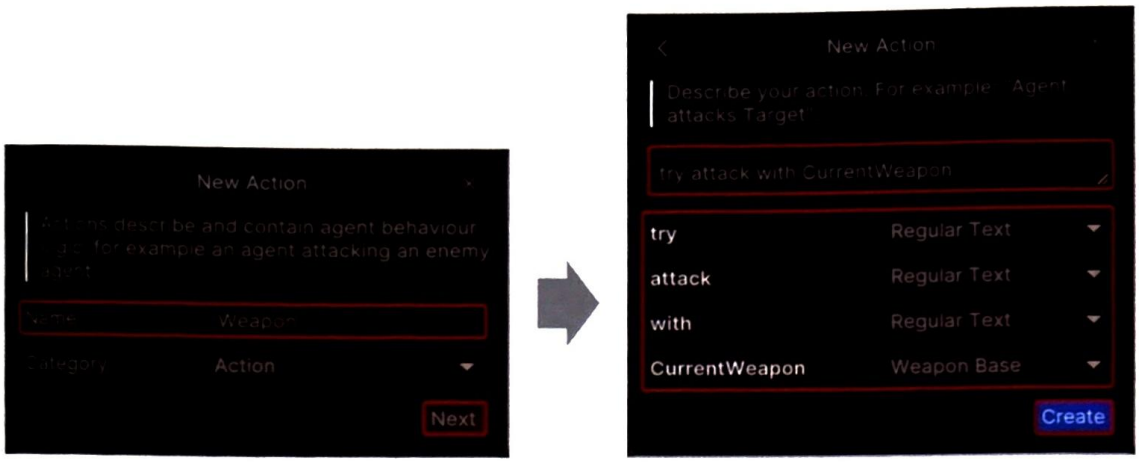
[Flow → Try In Order] 메뉴로 Try In Order 노드를 생성하고 Attack 노드를 연결합니다. Wander 노드의 Fail If 노드를 복사한 후 Try In Order 노드를 연결합니다. Fail If 노드의 조건을 false로 설정합니다.

[Action → Blackboard → Set Variable Value] 메뉴로 Set Variable Value 노드를 생성하고 Variable은 currentState로, Value는 [Idle]로 설정한 후 Try In Order 노드를 연결합니다.



공격(Attack) 행동 설정: 목표가 범위에 있으면 현재 등록된 무기로 공격

05 Behavior Graph View의 빈 곳을 마우스 오른쪽 버튼으로 클릭하고 [Add... → Action → Create New Action...] 메뉴로 새로운 노드를 생성합니다. 노드 이름을 'Weapon'으로 변경하고 〈Next〉 버튼을 누릅니다. Story 입력 상자에 'try attack with CurrentWeapon'을 입력한 다음, CurrentWeapon은 [Other → Components]에서 [Weapon Base] 자료형을 선택하고 〈Create〉 버튼을 눌러 스크립트를 생성(Assets/Scripts/Behavior/WeaponAction.cs)합니다.



현재 장착 중인 무기로 공격하는 행동(Weapon) 생성하고 설정하기

● Blackboard에 이름이 같은 변수가 없다면 자동으로 변경되지 않으므로 CurrentWeapon은 [Weapon Base] 자료형으로 직접 변경해야 합니다.

06 무기 공격 액션 노드인 WeaponAction.cs 스크립트를 수정합니다.

Assets/Scripts/Behavior/WeaponAction.cs

코드 10-13 무기 공격 액션 노드

```
    (... 생략 ...)
07: [Serializable, GeneratePropertyBag]
08: [NodeDescription(name: "Weapon", story: "try attack with [CurrentWeapon]",
        category: "Action", id: "0d968a6a7a0bae6af66dad92443774b1")]
09: public partial class WeaponAction : Action
10: {
11:     [SerializeReference] public BlackboardVariable<WeaponBase> CurrentWeapon;
12:
13:     /*protected override Status OnStart()
14:     {
15:         return Status.Running;
16:     }*/
17:
18:     protected override Status OnUpdate()
19:     {
20:         CurrentWeapon.Value.TryAttack();
21:
22:         return Status.Success;
23:     }
24:
```

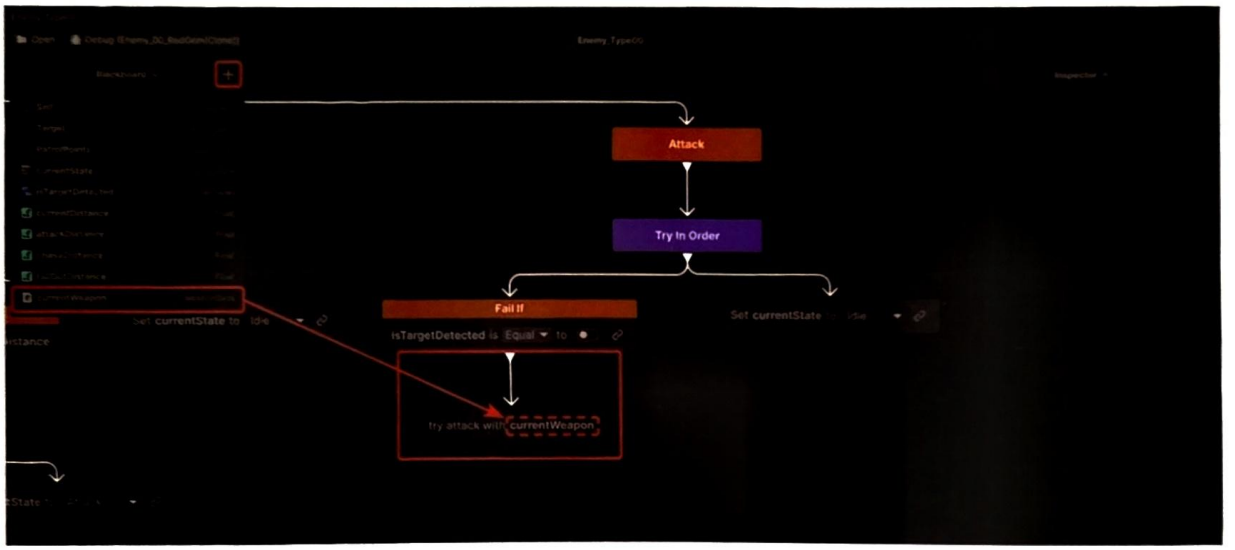
삭제

추가

```
25:     /*protected override void OnEnd()
26:     {
27:     }*/
28: }
```

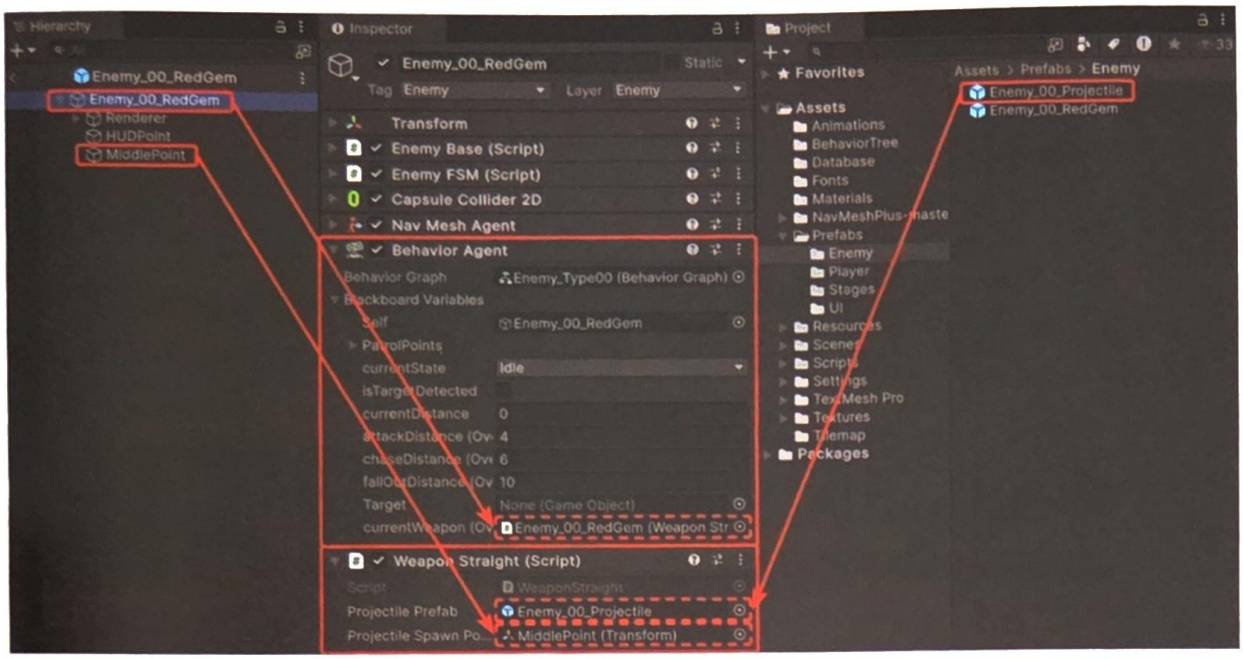
삭제

07 Blackboard의 ✚ 버튼을 눌러 [Other → Components → Weapon Base]를 추가하고 이름을 'currentWeapon'으로 변경합니다. Weapon 액션 노드에 Fail If 노드를 연결하고 currentWeapon 변수를 등록합니다.



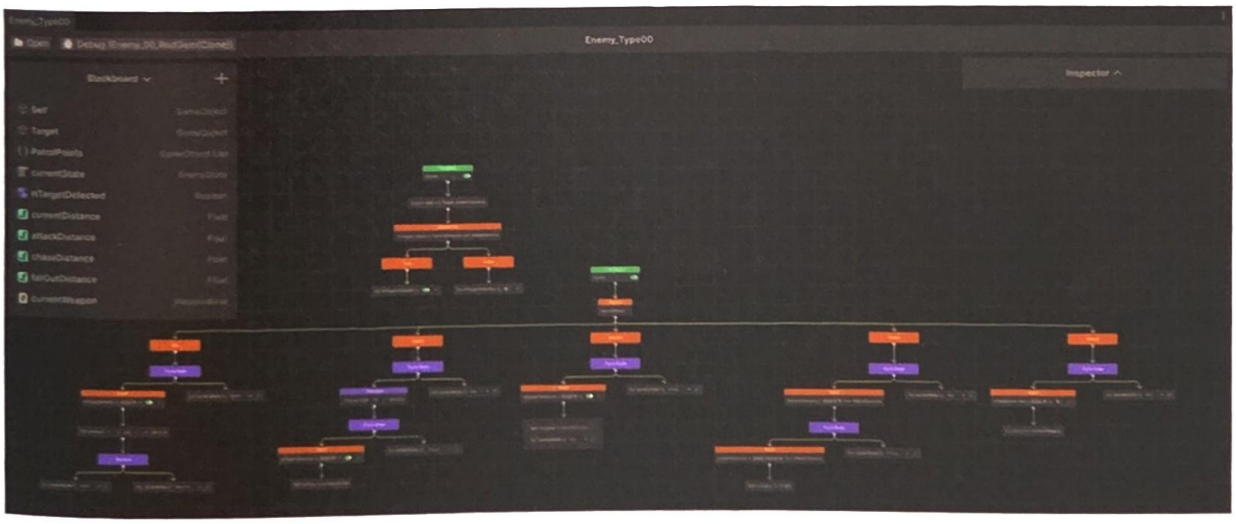
현재 장착 중인 무기(currentWeapon)로 공격하는 Weapon 액션 노드 등록하기

08 Enemy_00_RedGem 프리팹에 직선 발사체 공격을 하도록 Weapon Straight 컴포넌트를 추가합니다. Behavior Agent 컴포넌트의 currentWeapon에 Enemy_00_RedGem 오브젝트를 등록해 무기를 Weapon Straight로 설정합니다. Weapon Straight 컴포넌트의 Projectile Prefab에 Enemy_00_Projectile 프리팹을, Projectile Spawn Point에 MiddlePoint 오브젝트를 등록합니다.



Enemy_00_RedGem 프리팹에 컴포넌트 추가하고 설정하기

09 Enemy_Type00 행동 그래프에는 2개의 On Start 루트 노드가 있습니다. 첫 번째는 목표 상태에 따라 대기·순찰·배회·추적·공격 행동을 선택하고 두 번째는 목표와의 거리를 계산해서 chaseDistance보다 작거나 같으면 isTargetDetected를 true로, 크면 false로 설정하여 상태를 바꿉니다.



행동 트리(Behavior Tree)를 이용한 적 AI 구현 결과

Enemy_Type00 에셋을 Ctrl + D 로 복제하고 노드 일부를 삭제하거나 추가하면 대기나 순찰만 하는 적, 추적만 하는 적처럼 다양한 적 AI를 구현할 수 있습니다.

넷째마당

게임 완성! 씬과 스테이지 구성하기

이번 마당에서는 게임 플레이를 하나의 완성된 단위로 묶는 환경 구축을 다룹니다. 먼저 씬과 로비 씬을 추가하여 게임의 첫인상을 설계하고 비동기 로딩 시스템으로 끊김이 없는 화면 전환을 구현합니다. 또한 챕터와 스테이지를 구분하여 도전 과제를 부여하고 게임이 끝나도 데이터가 유지되도록 JSON 형식의 세이브 시스템을 구축하여 완성도를 높입니다.

11장 비동기 씬 로딩과 로비 시스템 갖추기

12장 챕터와 스테이지 관리하고 JSON 게임 데이터 저장하기

11장

비동기 씬 로딩과 로비 시스템 갖추기

이번 장에서는 씬을 추가하고 이를 전환할 때 로딩 화면을 출력해 봅니다. 먼저 타이틀 이미지와 로비 씬으로 이동하는 인트로 씬을 추가하고 플레이어가 가진 보석 정보와 챕터 UI를 출력하는 로비 씬을 만듭니다.

학습 목표

1. AsyncOperation을 활용한 비동기 씬 로딩 시스템을 구현할 수 있다.
2. 스와이프 기반 UI를 설계하여 모바일 친화적인 인터페이스를 구현할 수 있다.

11-1 비동기 로딩과 씬 구성하기

11-2 스와이프 UI로 챕터 선택하기

11-1

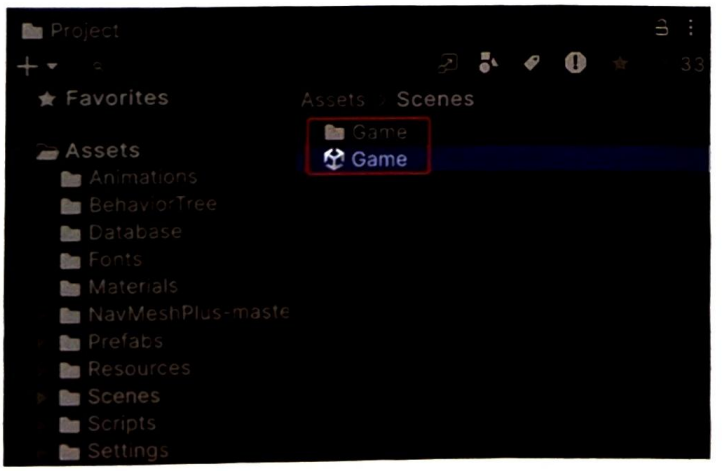
비동기 로딩과 씬 구성하기

이 절에서는 씬을 추가하고 로딩 화면을 출력합니다. 먼저 타이틀 이미지를 출력하고 아무 키나 누르면 로비 Lobby 씬으로 이동하는 인트로 Intro 씬을 추가합니다. 이어서 보석 등 플레이어의 재화 정보와 챕터 UI를 출력하는 로비 씬을 만들어 봅니다. 먼저 로딩 시스템부터 구현해 볼까요?

Do it! 실습 로딩 시스템 구현하기

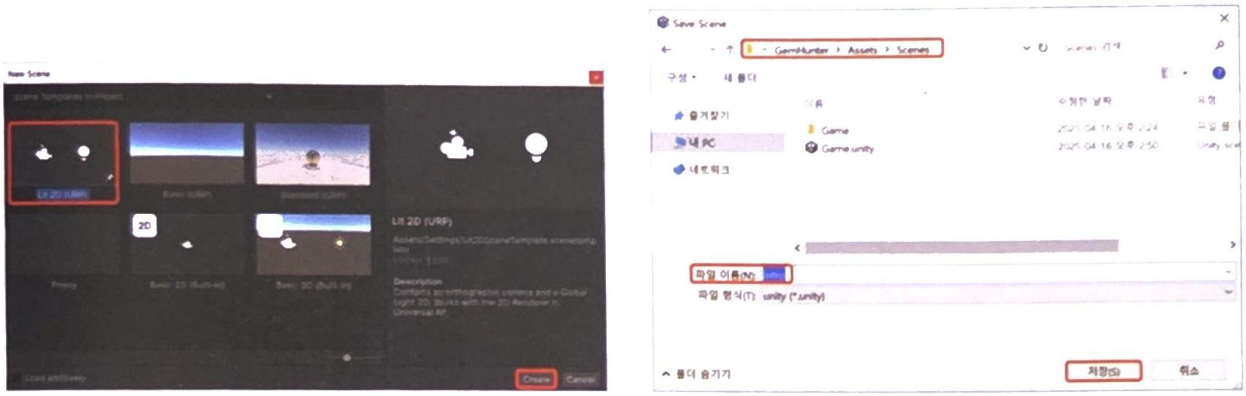
추가한 씬을 전환할 때는 로딩 화면을 표시하는데, 이때 로딩 진행 과정도 함께 나타내 봅니다. 이번 실습은 과정이 많으므로 꼼꼼히 확인하며 진행하세요.

01 이 책에서 개발하는 게임은 Intro, Lobby, Game 등 3개 씬으로 구성되며 각 씬을 불러오는 동안 로딩 화면을 출력합니다. 앞서 만든 SampleScene은 실제 게임을 플레이하는 씬이므로 씬 이름과 씬에서 사용 중인 NavMesh 데이터를 저장한 SampleScene 폴더 이름을 'Game'으로 변경합니다.



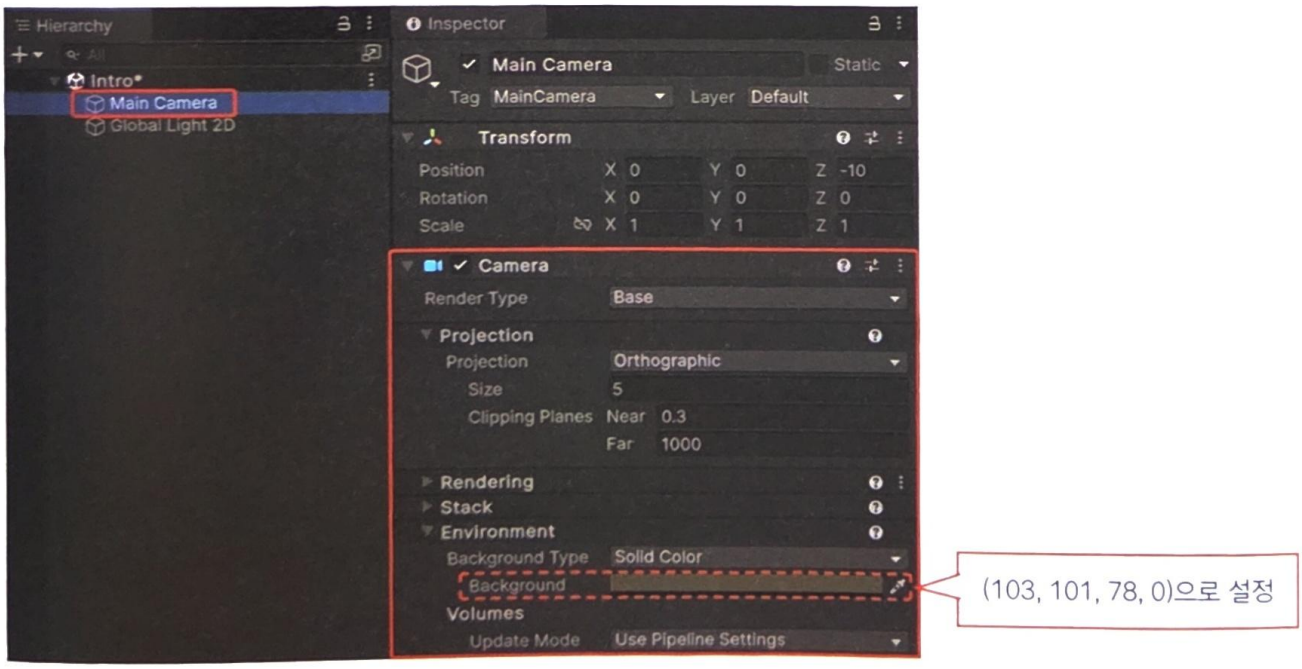
SampleScene 씬과 폴더 이름 변경하기

02 [File → New Scene] 메뉴를 열고 Lit 2D (URP) 씬 템플릿을 선택하고 나서 〈Create〉 버튼을 눌러 새로운 씬을 생성합니다. 생성한 씬은 [File → Save] 메뉴(단축키 Ctrl + S)로 Assets 〉 Scenes 폴더에 'Intro'라는 이름으로 저장합니다.



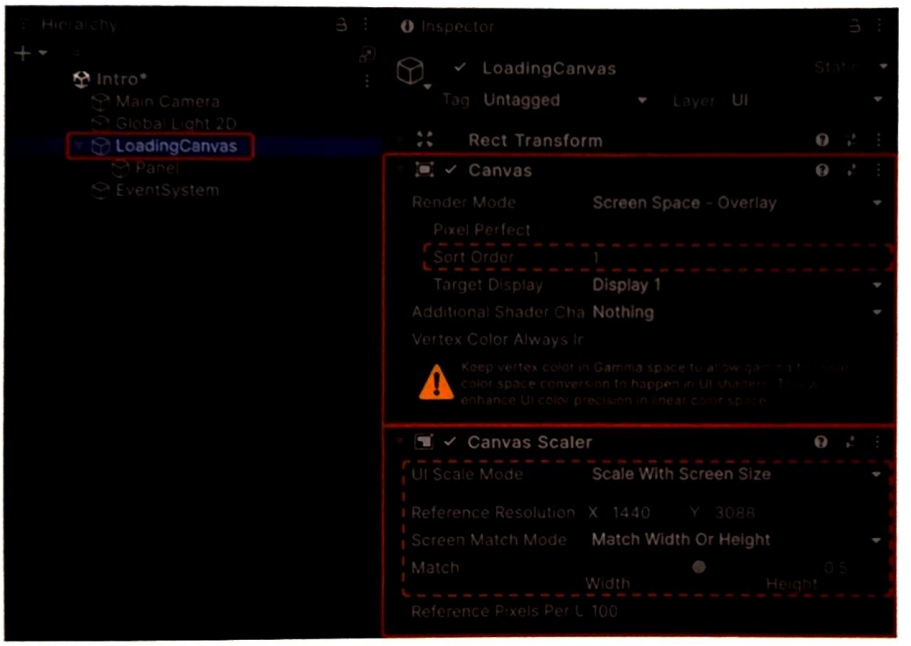
인트로 씬 생성하고 저장하기

03 제공하는 타이틀 이미지 색상과 어울리도록 인트로 씬에 있는 Main Camera 오브젝트의 Camera 컴포넌트에 있는 Environment 섹션에서 배경 색상 Background를 (103, 101, 78, 0)으로 설정합니다.



Camera 컴포넌트에서 배경 색상 설정하기

04 [GameObject → UI → Panel] 메뉴로 로딩 UI를 관리하는 Panel UI를 생성합니다. Panel 오브젝트와 함께 생성한 Canvas 오브젝트 이름을 'LoadingCanvas'로 변경하고 Canvas 컴포넌트의 Sort Order를 '1'로 설정합니다. Canvas Scaler 컴포넌트의 UI Scale Mode를 [Scale With Screen Size]로, Reference Resolution을 (1440, 3088)로, Match를 '0.5'로 설정합니다.

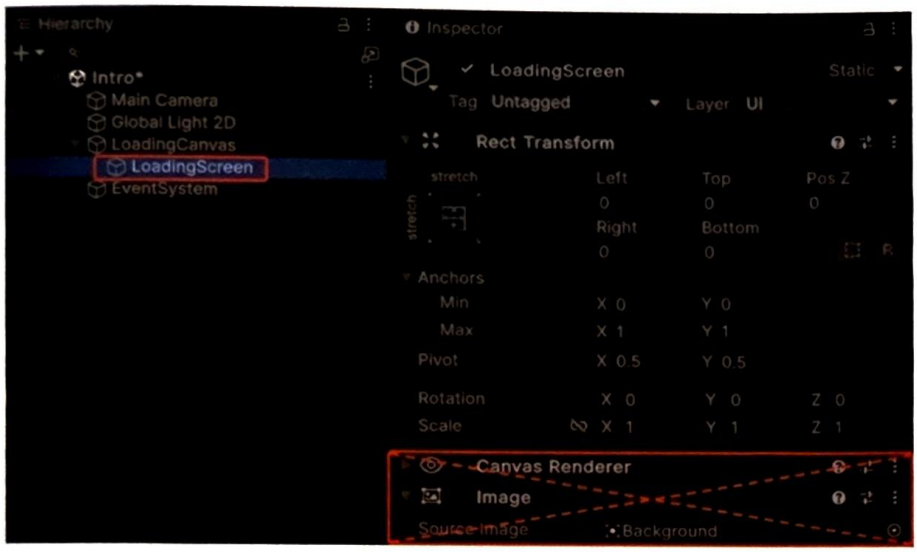


로딩 화면을 출력하는 Canvas 오브젝트 설정하기

알아 두면 좋아요! Canvas 오브젝트의 출력 순서를 정하려면?

UI를 배치하는 Canvas 오브젝트는 용도에 따라 여러 개 생성할 수 있는데, 이때 어떤 Canvas의 UI가 더 앞에 보일지를 설정할 수 있습니다. 로딩 화면은 화면 가장 앞에 두어야 하므로 Sort Order가 '0'인 다른 Canvas보다 앞에 그리도록 Sort Order를 '1'로 설정합니다. 즉, Sort Order가 클수록 앞에 그려집니다.

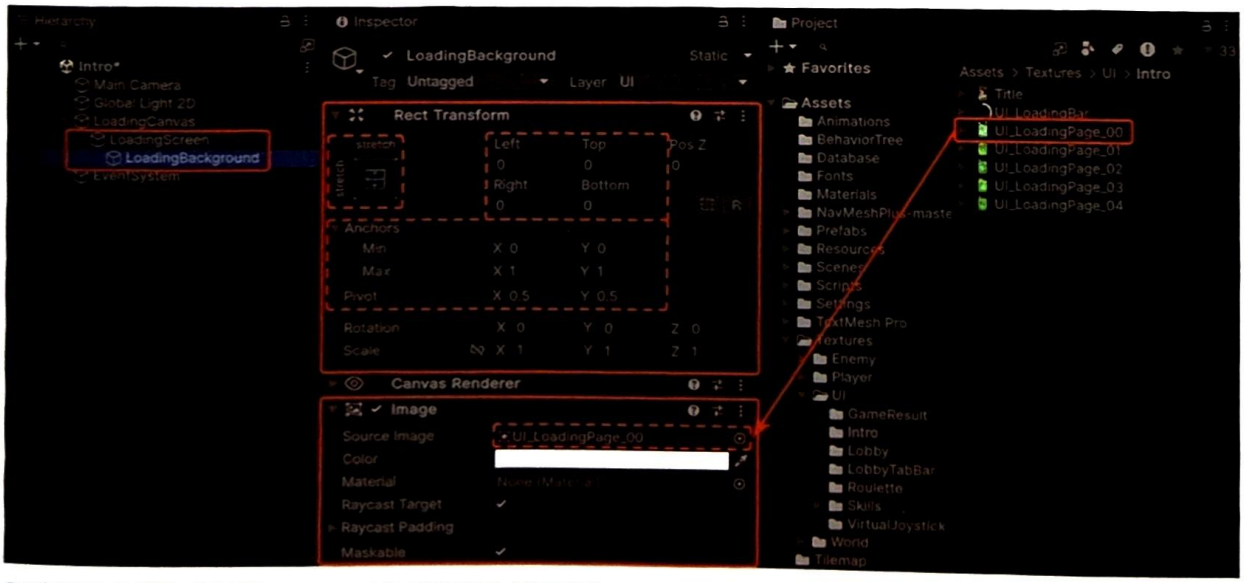
05 앞서 생성한 Panel 오브젝트의 이름을 'LoadingScreen'으로 변경합니다. Image와 Canvas Renderer 컴포넌트는 삭제하여 빈 오브젝트로 사용합니다.



로딩 UI를 관리하는 Panel UI 설정하기

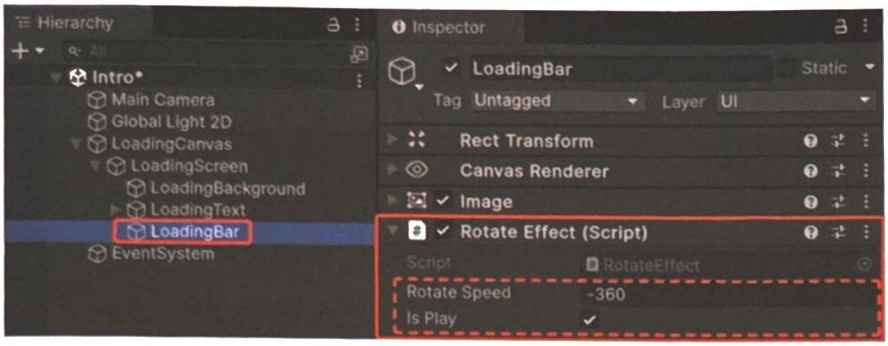
06 [GameObject → UI → Image] 메뉴로 로딩 배경 화면을 출력하는 Image UI를 생성하고 이름을 'LoadingBackground'로 변경한 후 LoadingScreen 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. Image 컴포넌트의 Source Image에 UI_LoadingPage_00.png 이미지를 등록합니다.



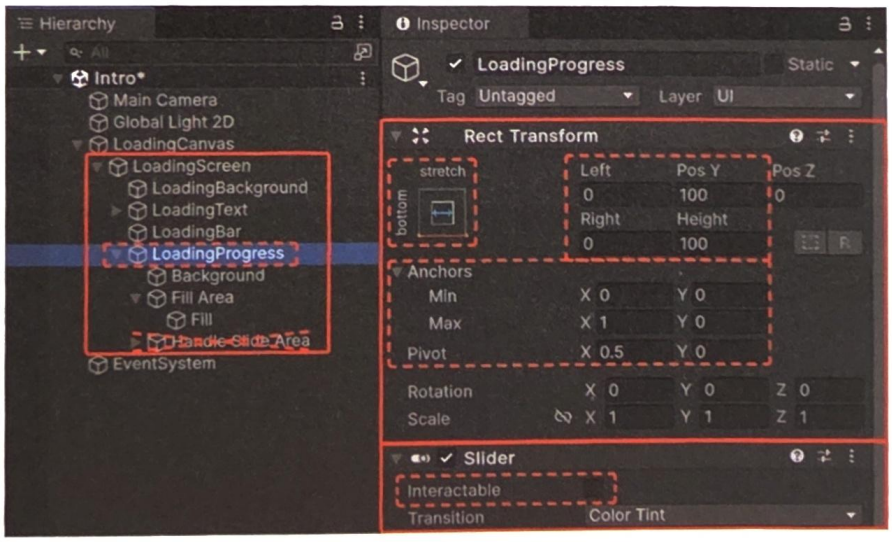
로딩 배경 화면을 출력하는 Image UI 생성하고 설정하기

11 LoadingBar가 빙글빙글 회전하도록 LoadingBar 오브젝트에 Rotate Effect 컴포넌트를 추가하고 Rotate Speed는 '-360'으로, Is Play에 체크 표시합니다.



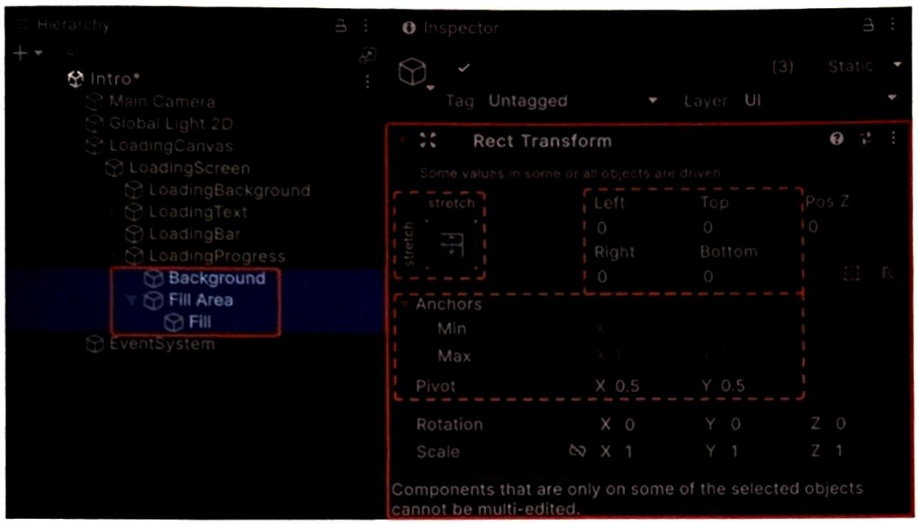
LoadingBar 오브젝트에 Rotate Effect 컴포넌트 추가하고 설정하기

12 [GameObject → UI → Slider] 메뉴로 로딩 진행도를 출력하는 Slider UI를 생성하고 이름을 'LoadingProgress'로 변경한 후 LoadingScreen 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 bottom | stretch로, Rect는 (0, 100, 0, 100)으로 설정합니다. Slider UI의 진행도는 코드를 이용해 제어하므로 상호 작용하지 않도록 Slider 컴포넌트의 Interactable은 비활성화하고 Handle Slide Area 오브젝트는 삭제합니다.



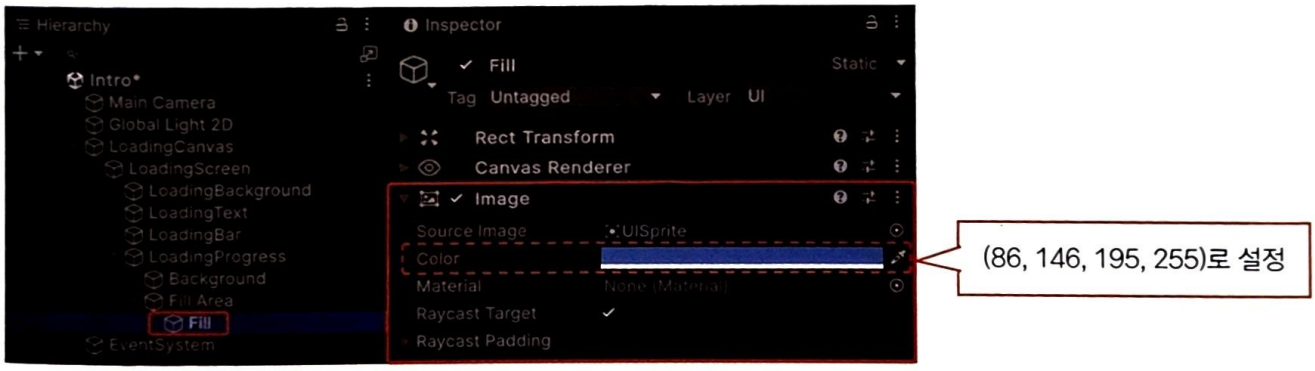
로딩 진행도를 출력하는 Slider UI 생성하고 설정하기

13 LoadingProgress 오브젝트의 자식으로 있는 Background, Fill Area, Fill 오브젝트의 Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다.



로딩 진행도 크기 설정하기

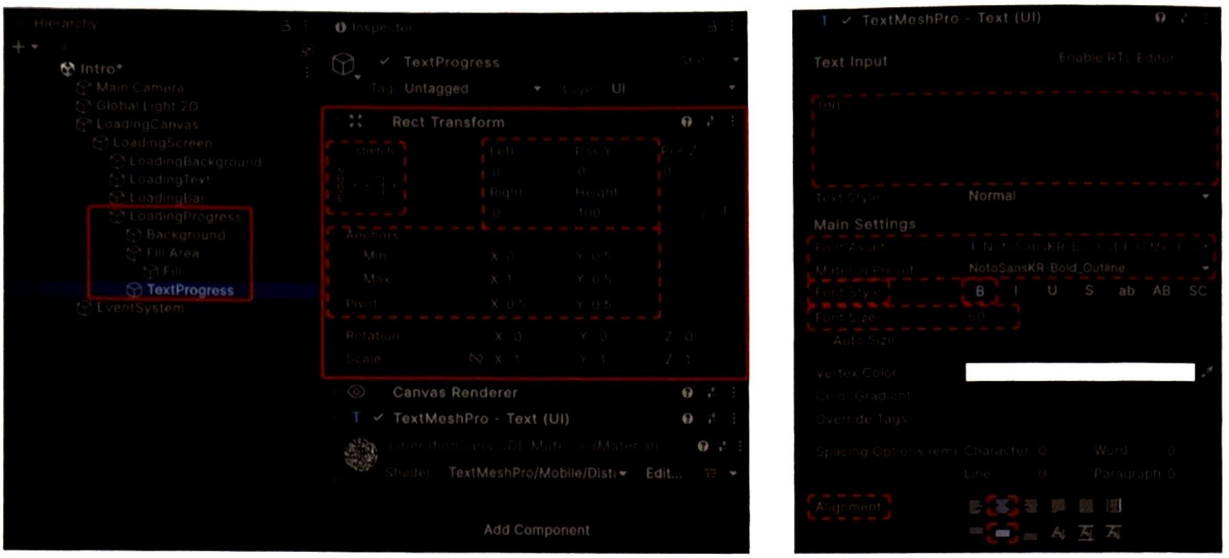
14 Fill 오브젝트의 Image 컴포넌트에서 Color를 (86, 146, 195, 255)로 설정합니다.



로딩 진행도 색상 설정하기

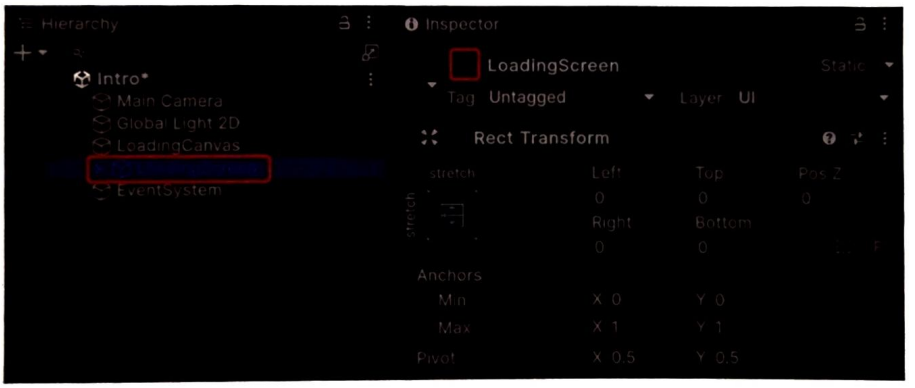
15 [GameObject → UI → Text - TextMeshPro] 메뉴로 로딩 진행도를 출력하는 Text UI를 생성하고 이름을 'TextProgress'로 변경한 후 LoadingProgress 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | stretch로, Rect는 (0, 0, 0, 100)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 '100%'를 입력합니다. Font Asset에 NotoSansKR-Bold SDF를 등록하고 Material Preset은 [NotoSansKR-Bold_Outline]으로 선택합니다. 마지막으로, Font Style은 굵게 [B]로, Font Size는 '60'으로, Alignment는 Center | Middle로 설정합니다.



로딩 진행도를 출력하는 Text UI 생성하고 설정하기

16 LoadingCanvas 오브젝트는 모든 씬에서 활성화하고 LoadingScreen 오브젝트는 로딩할 때만 활성화하도록 LoadingScreen 오브젝트를 비활성화합니다.



LoadingScreen 오브젝트 비활성화하기

17 Assets 〉 Scripts 〉 Common 폴더에 로딩 화면 출력과 씬 로드를 제어하는 SceneLoader.cs 스크립트를 생성합니다.

Assets/Scripts/Common/SceneLoader.cs

코드 11-01 로딩 화면 출력과 씬 로드

```
01: using System.Collections;
02: using UnityEngine;
03: using UnityEngine.UI;
04: using UnityEngine.SceneManagement;
05: using TMPro;
```

```
06:
07: public enum SceneNames { Intro = 0, Lobby, Game }
08:
09: public class SceneLoader : MonoBehaviour
10: {
11:     public static SceneLoader Instance { get; private set; }
12:
13:     [SerializeField]
14:     private GameObject loadingScreen;     // 로딩 화면
15:     [SerializeField]
16:     private Image loadingBackground;      // 로딩 화면에 출력할 배경 이미지
17:     [SerializeField]
18:     private Sprite[] loadingSprites;      // 배경 이미지 목록
19:     [SerializeField]
20:     private Slider loadingProgress;       // 로딩 진행도
21:     [SerializeField]
22:     private TextMeshProUGUI textProgress; // 로딩 진행도 텍스트
23:
24:     private void Awake()
25:     {
26:         if ( Instance != null && Instance != this )
27:         {
28:             Destroy(gameObject);
29:         }
30:         else
31:         {
32:             Instance = this;
33:             DontDestroyOnLoad(gameObject);
34:         }
35:     }
36:
37:     public void LoadScene(string name)
38:     {
39:         int index = Random.Range(0, loadingSprites.Length);
40:         loadingBackground.sprite = loadingSprites[index];
41:         loadingProgress.value = 0f;
42:         loadingScreen.SetActive(true);
43:
44:         StartCoroutine(LoadSceneAsync(name));
45:     }
46:
47:     public void LoadScene(SceneNames name)
48:     {
```

```
49:         LoadScene(name.ToString());
50:     }
51:
52:     private IEnumerator LoadSceneAsync(string name)
53:     {
54:         AsyncOperation asyncOperation = SceneManager.LoadSceneAsync(name);
55:
56:         // 비동기 작업(씬 불러오기)을 완료할 때까지 반복
57:         while ( asyncOperation.isDone == false )
58:         {
59:             // 비동기 작업 진행 상황(0.0~1.0)
60:             loadingProgress.value = asyncOperation.progress;
61:             textProgress.text =
                    $"{Mathf.RoundToInt(asyncOperation.progress * 100)}%";
62:
63:             yield return null;
64:         }
65:
66:         float changeDelay = 0.5f;
67:         yield return new WaitForSeconds(changeDelay);
68:
69:         loadingScreen.SetActive(false);
70:     }
71: }
```

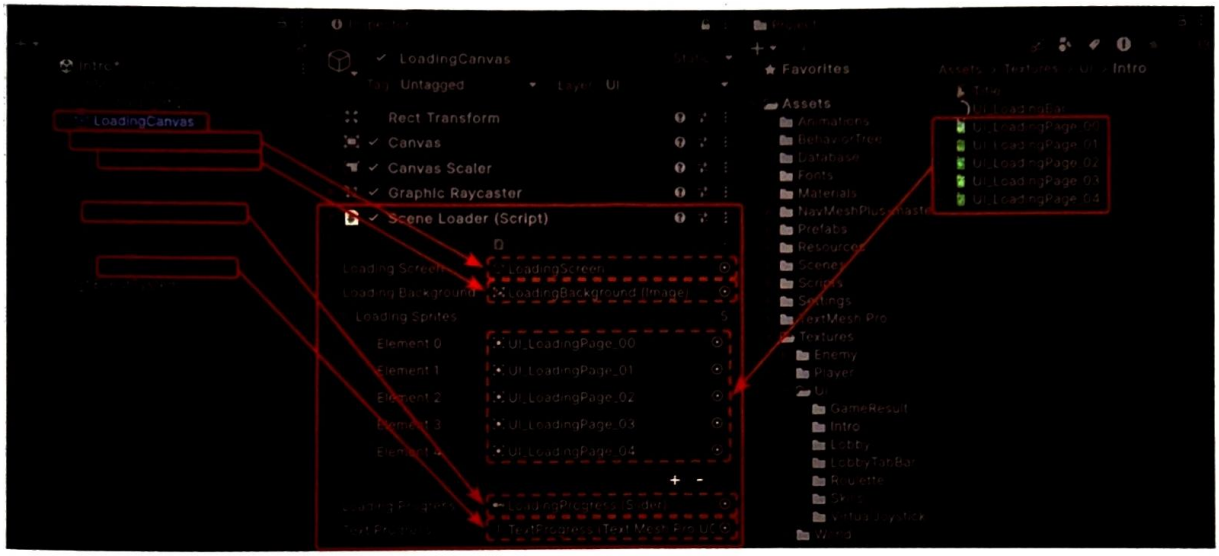
52~70줄 동기 방식인 LoadScene() 메서드와 다르게 비동기 방식으로 씬을 불러오는 동안 다른 작업을 수행할 수 있습니다. LoadSceneAsync() 메서드가 반환하는 AsyncOperation 객체로 비동기 작업 상태를 추적합니다.

SceneManager.LoadSceneAsync()가 반환하는 AsyncOperation 객체의 속성

속성	설명
isDone	비동기 작업이 완료인지, 진행 중인지를 반환합니다. (true: 작업 완료, false: 작업 진행 중)
progress	비동기 작업 진행 상황입니다. 0.0f~1.0f 사이 값으로, 작업이 끝나면 1.0을 반환합니다.
allowSceneActivation	씬이 준비되는 즉시 활성화할 것인지로, 기본값은 true입니다. false로 설정하면 로딩이 끝나더라도 씬을 활성화하지 않습니다.

18 LoadingCanvas 오브젝트에 Scene Loader 컴포넌트를 추가합니다. 이 컴포넌트의 Loading Screen 변수에는 LoadingScreen 오브젝트를, Loading Background 변수에는 LoadingBackground 오브젝트를 등록합니다.

Loading Sprites 배열 변수에는 로딩 화면에 출력할 UI_LoadingPage_00~04.png까지 이미지 5개를 등록합니다. 마지막으로, Loading Progress 변수에는 LoadingProgress 오브젝트를, Text Progress 변수에는 TextProgress 오브젝트를 등록합니다.



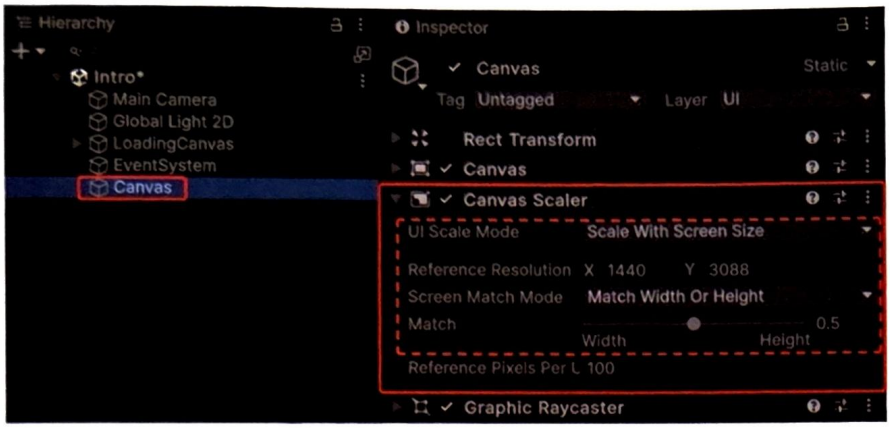
LoadingCanvas 오브젝트에 Scene Loader 컴포넌트 추가하고 설정하기

아직은 미완성이므로 실행 결과는 다음 실습까지 마친 후에 확인합니다.

Do it! 실습 인트로 씬 만들기

씬을 연결하는 로딩 화면을 만들었으므로 이번에는 게임을 시작하는 인트로 씬과 가진 보석을 표시하고 챕터를 고르는 로비 씬을 만듭니다.

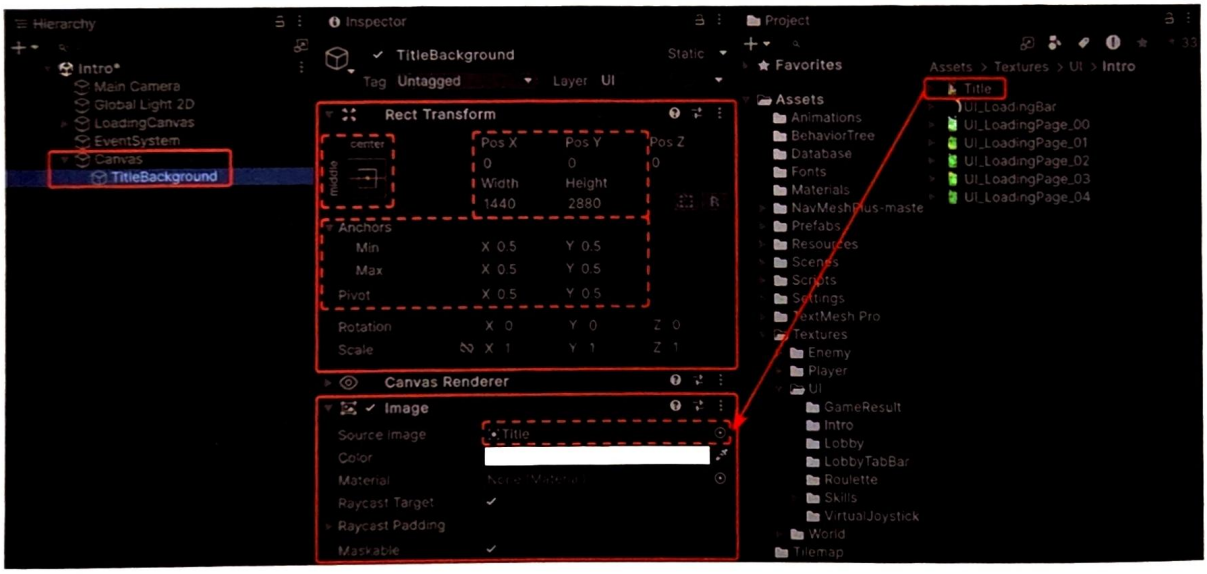
01 [GameObject → UI → Canvas] 메뉴로 인트로 씬을 구성할 Canvas를 생성합니다. Canvas 오브젝트의 Canvas Scaler 컴포넌트에서 UI Scale Mode는 [Scale With Screen Size]로, Reference Resolution은 (1440, 3088)로, Match는 '0.5'로 설정합니다.



인트로 씬을 출력하는 Canvas 오브젝트 생성하고 설정하기

02 [GameObject → UI → Image] 메뉴로 인트로 씬 타이틀 화면을 출력하는 Image UI를 생성하고 이름을 'TitleBackground'로 변경합니다.

Rect Transform 컴포넌트의 Rect는 (0, 0, 1440, 2880)으로 설정하고, Image 컴포넌트의 Source Image에 Title.png 이미지를 등록합니다.



인트로 씬 타이틀 화면을 출력하는 Image UI 생성하고 설정하기

03 [GameObject → UI → Text - TextMeshPro] 메뉴로 'PRESS ANY KEY' 텍스트를 출력할 Text UI를 생성하고 이름을 'PressAnyKey'로 변경합니다.

Rect Transform 컴포넌트의 Anchor Presets은 bottom | stretch로 선택하고 Rect는 (0, 200, 0, 100)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 'PRESS ANY KEY'를 입력하고, Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Style은 굵게 [B]로, Font Size는 '100'으로, Alignment는 Center | Middle로 지정합니다.

'PRESS ANY KEY' 텍스트를 출력하는 Text UI 생성하고 설정하기

04 TextMeshProUGUI의 페이드 효과를 재생하도록 FadeEffect.cs 스크립트를 수정합니다.

Assets/Scripts/Common/FadeEffect.cs

코드 11-02 TextMeshProUGUI의 페이드 효과 재생

```
01: using System.Collections;
02: using UnityEngine;
03: using UnityEngine.Events;
04: using TMPro;                                          추가
05:
06: public static class FadeEffect
07: {
        (... 생략 ...)
28:     public static IEnumerator Fade(TextMeshProUGUI target, float start, float end,   추가
            float fadeTime=1f, UnityAction action=null)
29:     {
30:         if ( target == null ) yield break;
31:
32:         float percent = 0;
33:
34:         while ( percent < 1 )
35:         {
36:             percent += Time.deltaTime / fadeTime;
37:
38:             Color color = target.color;
39:             color.a = Mathf.Lerp(start, end, percent);
40:             target.color = color;
```

```
41:
42:             yield return null;
43:         }
44:
45:         action?.Invoke();
46:     }
47: }
```

05 Utils.cs 스크립트를 수정하여 플랫폼별로 키를 눌렀는지 검사하는 **IsAnyInputDown()** 메서드를 정의합니다.

Assets/Scripts/Common/Utils.cs

코드 11-03 키를 눌렀는지 검사

```
01: using UnityEngine;
02: using UnityEngine.InputSystem;          ← 추가
03:
04: public static class Utils
05: {
        (... 생략 ...)
104:    public static bool IsAnyInputDown()  ← 추가
105:    {
106:        // 키보드 키 입력
107:        if ( Keyboard.current != null &&
                Keyboard.current.anyKey.wasPressedThisFrame ) return true;
108:
109:        // 마우스 왼쪽 버튼 입력
110:        if ( Mouse.current != null &&
                Mouse.current.leftButton.wasPressedThisFrame ) return true;
111:
112:        // 모바일 터치
113:        if ( Touchscreen.current != null &&
                Touchscreen.current.primaryTouch.press.wasPressedThisFrame ) return true;
114:
115:        return false;
116:    }
117: }
```

06 Assets 〉 Scripts 폴더 아래에 IntroScene 폴더를 추가하고 인트로 씬을 제어하는 IntroSceneController.cs 스크립트를 생성합니다.

Assets/Scripts/IntroScene/IntroSceneController.cs

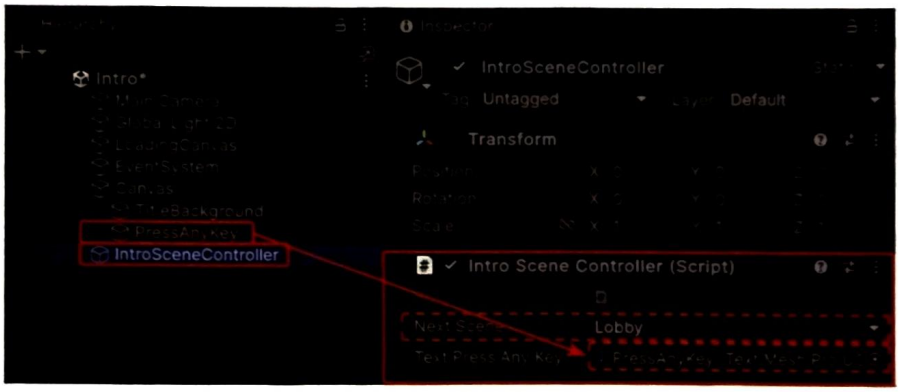
코드 11-04 인트로 씬 제어

```
01: using System.Collections;
02: using UnityEngine;
03: using TMPro;
04:
05: public class IntroSceneController : MonoBehaviour
06: {
07:     [SerializeField]
08:     private SceneNames nextScene;
09:     [SerializeField]
10:     private TextMeshProUGUI textPressAnyKey;
11:
12:     private IEnumerator Start()
13:     {
14:         while ( true )
15:         {
16:             yield return StartCoroutine(FadeEffect.Fade(textPressAnyKey, 1, 0));
17:
18:             yield return StartCoroutine(FadeEffect.Fade(textPressAnyKey, 0, 1));
19:         }
20:     }
21:
22:     private void Update()
23:     {
24:         if ( Utils.IsAnyInputDown() )
25:         {
26:             SceneLoader.Instance.LoadScene(nextScene);
27:         }
28:     }
29: }
```

12~20줄 인트로 씬은 타이틀 이미지와 'PRESS ANY KEY' 텍스트를 출력하는데, 이 텍스트는 Start() 코루틴 메서드에 작성한 FadeEffect.Fade() 메서드에 따라 알파값이 1.0에서 0.0으로, 0.0에서 1.0으로 같은 속도로 반복하며 깜박거리도록 설정합니다.

22~28줄 `Update()` 메서드의 `Utils.IsAnyInputDown()` 검사로 키보드, 마우스, 터치 중에서 입력이 있다면 `SceneLoader` 클래스의 `LoadScene()` 메서드를 호출해 로비 씬으로 이동합니다.

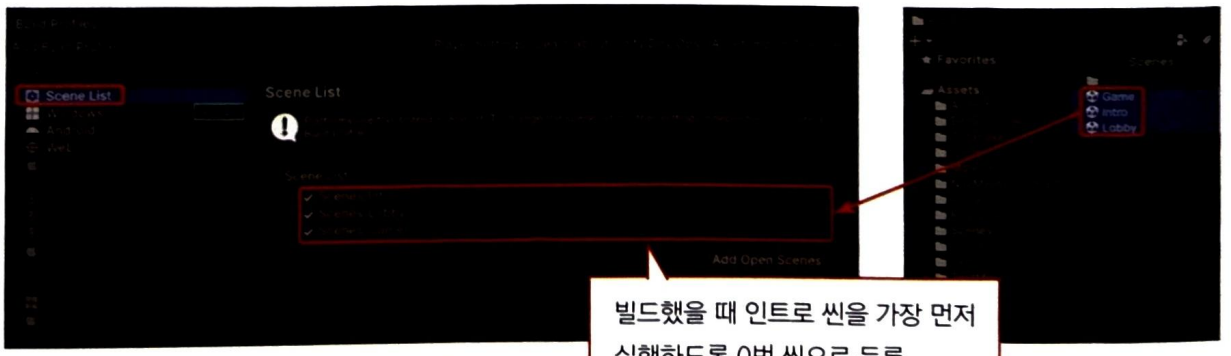
07 [GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'IntroScene Controller'로 변경한 후 Intro Scene Controller 컴포넌트를 추가합니다. 인트로 씬에서 이동할 다음 씬인 Next Scene은 [Lobby]로 설정하고 Text Press Any Key 변수에 Press AnyKey 오브젝트를 등록합니다.



빈 오브젝트에 Intro Scene Controller 컴포넌트 추가하고 설정하기

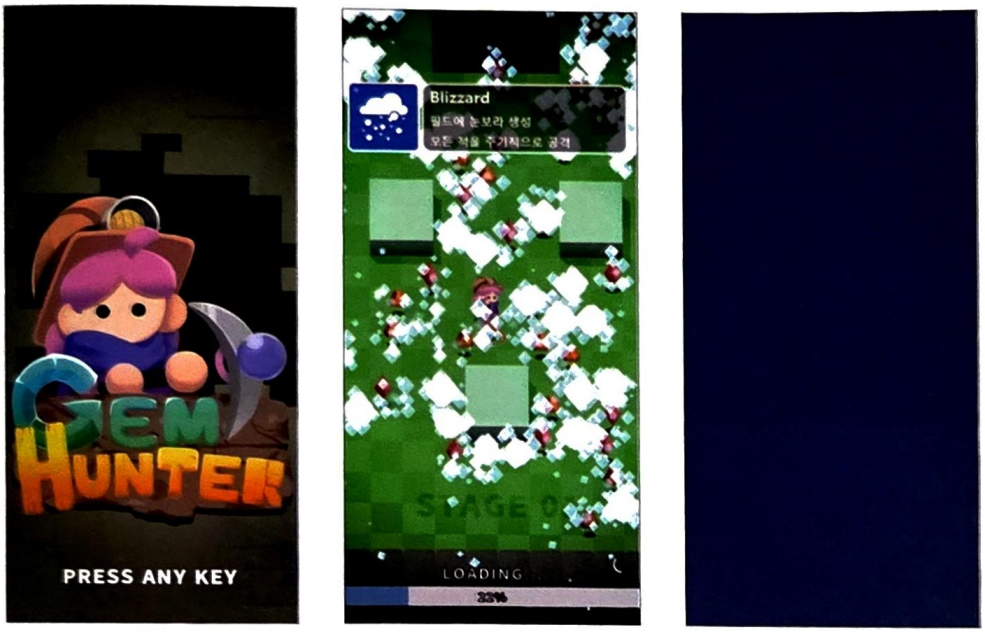
08 [File → New Scene] 메뉴를 클릭하고 Lit 2D (URP) 씬 템플릿을 선택한 후 〈Create〉 버튼을 눌러 새로운 씬을 생성합니다. [File → Save] 메뉴(단축키 Ctrl+S)로 생성한 씬을 Assets 〉 Scenes 폴더에 'Lobby'라는 이름으로 저장합니다.

[File → Build Profiles] 메뉴로 Build Profiles View를 활성화하고 왼쪽에서 [Scene List] 탭을 선택합니다. Scene List에 게임에서 사용할 씬을 Intro, Lobby, Game 순으로 등록합니다.



Build Profiles View의 Scene List에 씬 등록하기

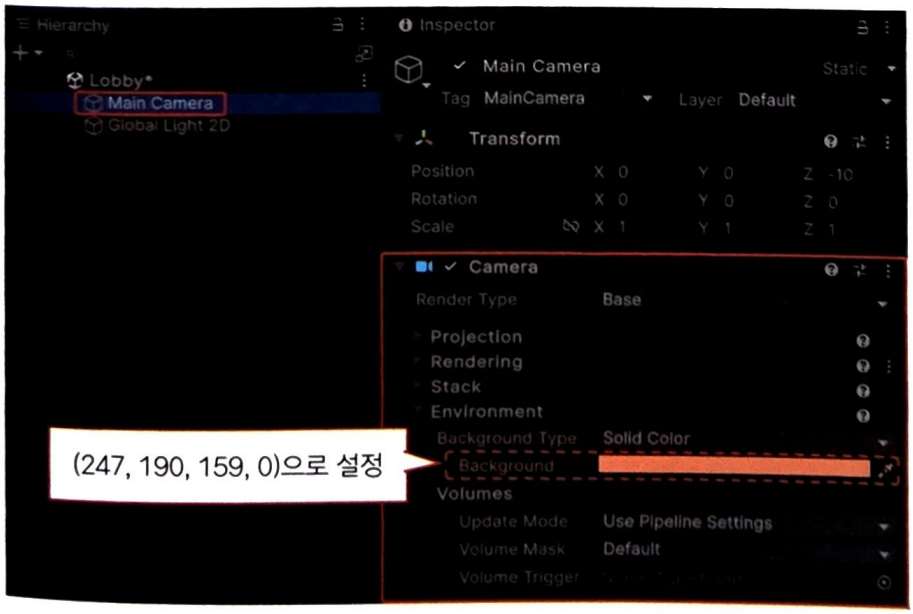
09 인트로 씬을 더블클릭해서 활성화하고 플레이 버튼 ▶ 을 눌러 게임을 실행하면 게임 타이틀 이미지와 깜빡거리는 'PRESS ANY KEY' 텍스트를 출력합니다. 아무 키나 누르면 로딩 화면을 출력하고, 로딩이 끝나면 기본 배경 색상의 로비 씬을 표시합니다.



인트로 씬, 로딩 씬과 잠시 후 만들 로비 씬

Do it! 실습 로비 씬 만들기

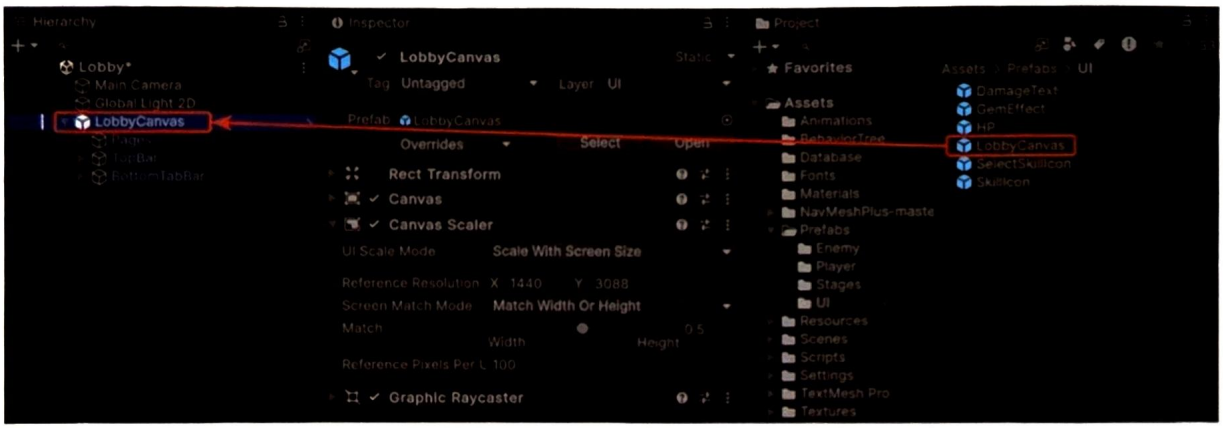
01 로비 씬을 더블클릭해서 활성화하고 Main Camera 오브젝트의 Camera 컴포넌트에서 배경색인 Background를 (247, 190, 159, 0)으로 수정합니다.



Camera 컴포넌트 배경색 설정하기

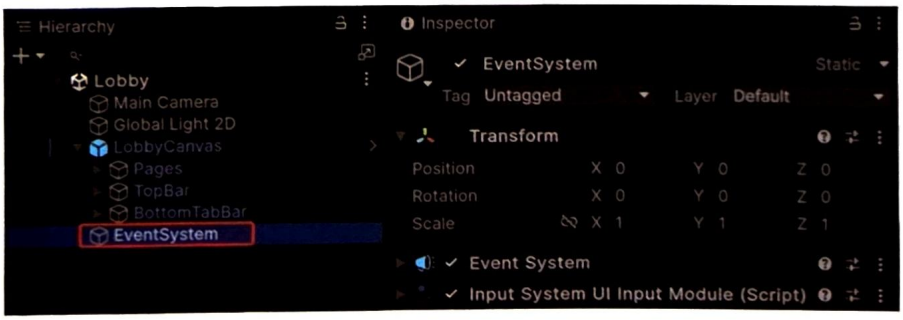
02 로비 씬에 출력할 UI는 미리 만들어 두었으므로 Assets 〉 Prefabs 〉 UI 폴더에 있는 LobbyCanvas 프리팹을 Hierarchy View로 끌어서 놓습니다.

LobbyCanvas 프리팹은 자료실에서 내려받은 패키지 안에 있습니다.



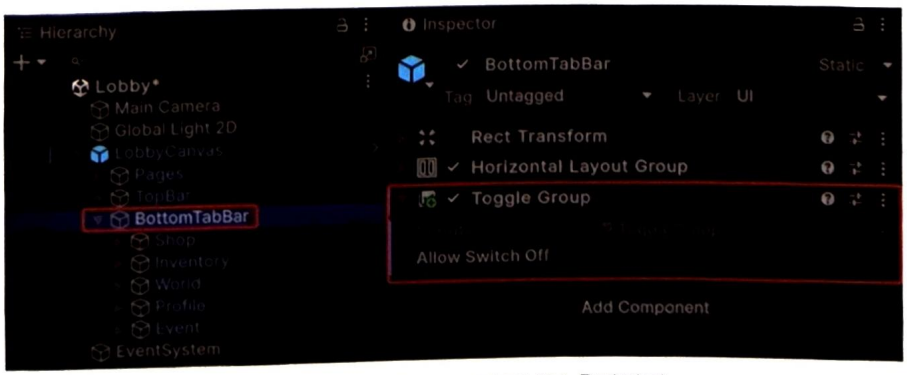
LobbyCanvas 프리팹을 Hierarchy View로 끌어서 놓기

03 [GameObject → UI → Event System] 메뉴로 UI 오브젝트의 상호작용 이벤트 처리에 사용할 EventSystem 오브젝트를 생성합니다.



EventSystem 오브젝트 생성하기

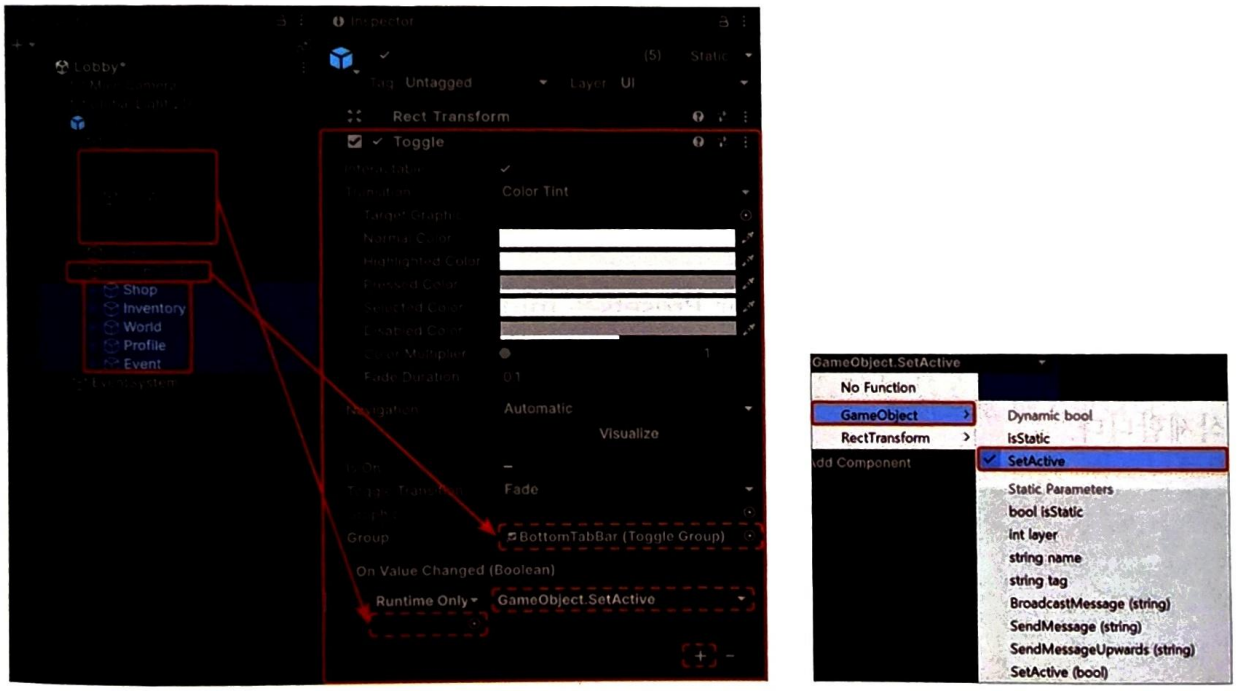
04 아래에 있는 Toggle UI를 그룹으로 관리해 동시에 하나의 Toggle UI만 활성화되도록 BottomTabBar 오브젝트에 Toggle Group 컴포넌트를 추가합니다.



BottomTabBar 오브젝트에 Toggle Group 컴포넌트 추가하기

05 Lobby 화면 아래에 있는 Toggle 오브젝트를 클릭하면 해당 Toggle과 연동된 페이지가 활성화되도록 설정합니다. 각 페이지는 Pages 오브젝트의 자식인 00_Shop, 01_Inventory, 02_World, 03_Profile, 04_Event 오브젝트로, 해당 오브젝트는 Rect Transform 컴포넌트만 있는 빈 오브젝트이고 해당 페이지에 출력할 UI를 자식으로 배치합니다.

BottomTabBar 오브젝트의 자식인 Shop, Inventory, World, Profile, Event 오브젝트의 Toggle 컴포넌트 Group 변수에 BottomTapBar 오브젝트를 등록하고 On Value Changed 이벤트에 요소를 추가합니다. Toggle 오브젝트와 이름이 같은 페이지 오브젝트를 등록하고 [GameObject → SetActive] 메뉴로 메서드를 등록합니다. 이때 SetActive 메서드는 Dynamic bool에서 선택해야 Toggle 컴포넌트의 Is On 설정값을 매개변수로 자동 전달합니다.



Toggle 오브젝트의 Toggle 컴포넌트에서 On Value Changed 이벤트 등록하기

06 플레이 버튼 ▶을 눌러 게임을 실행하고 로비 씬 아래의 탭바Tab Bar 아이콘을 클릭하면 해당 탭바와 연동된 페이지가 활성화됩니다. 세 번째 02_World 페이지는 11-2절에서 만들므로 아직은 빈 상태이고 그 외 페이지는 아이콘만 출력합니다.

탭바 아이콘을 출력한 모습

11-2

스와이프 UI로 챕터 선택하기

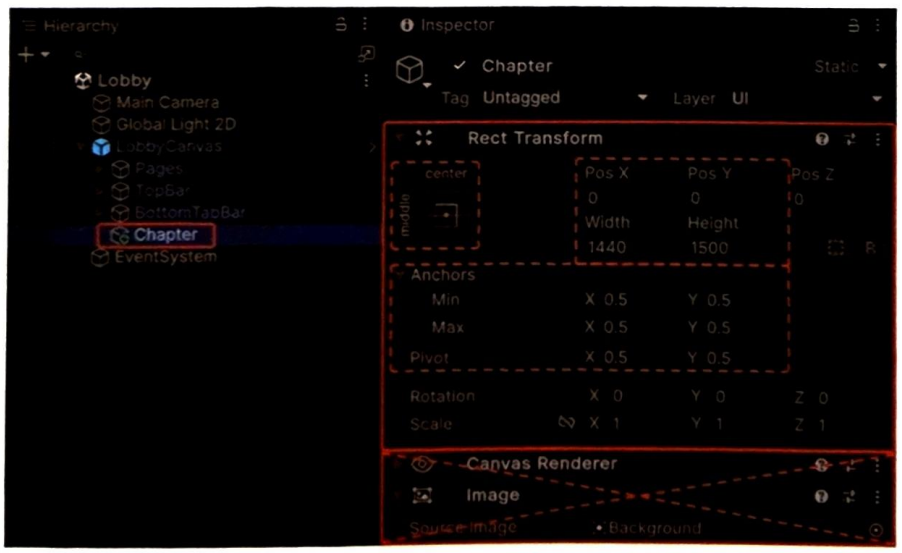
이 절에서는 챕터를 구성하고 스와이프로 활성화된 챕터를 선택하는 UI를 만듭니다.

Do it! 실습 챕터 데이터 설정하기

02_World 페이지에는 챕터 정보를 출력하고 활성화된 챕터에서 원하는 챕터를 선택해 게임을 실행하도록 구성합니다.

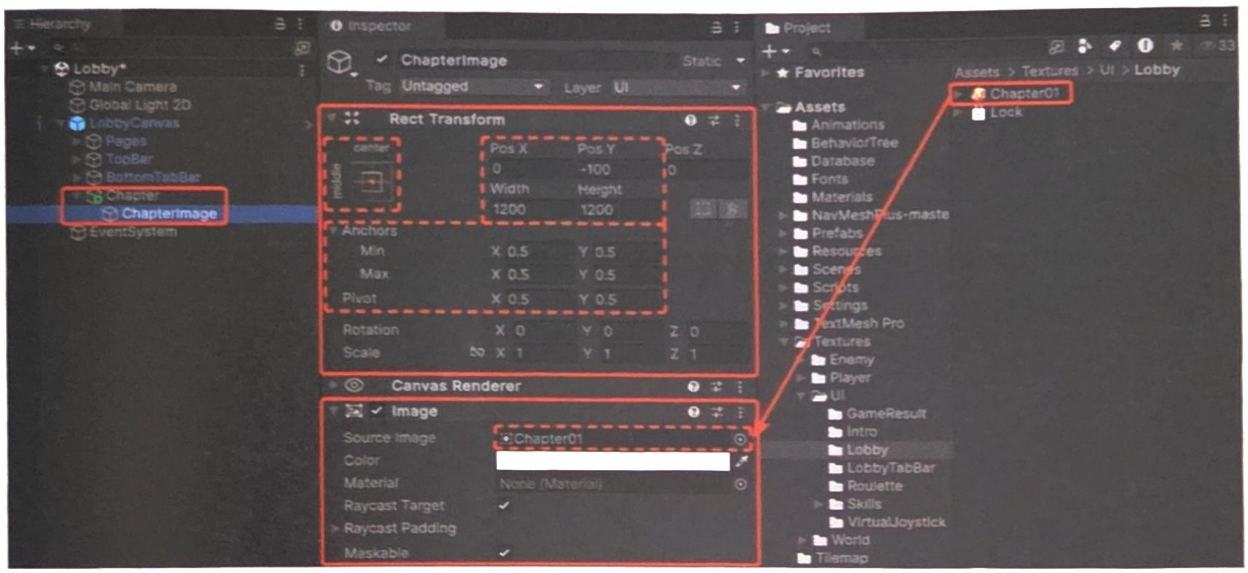
01 [GameObject → UI → Panel] 메뉴로 챕터 UI를 관리하는 Panel UI를 생성하고 이름을 'Chapter'로 변경합니다.

Rect Transform 컴포넌트의 Anchor Presets은 middle | center로, Rect는 (0, 0, 1440, 1500)으로 설정합니다. 화면에 출력하지는 않으므로 Image, Canvas Renderer 컴포넌트는 삭제합니다.



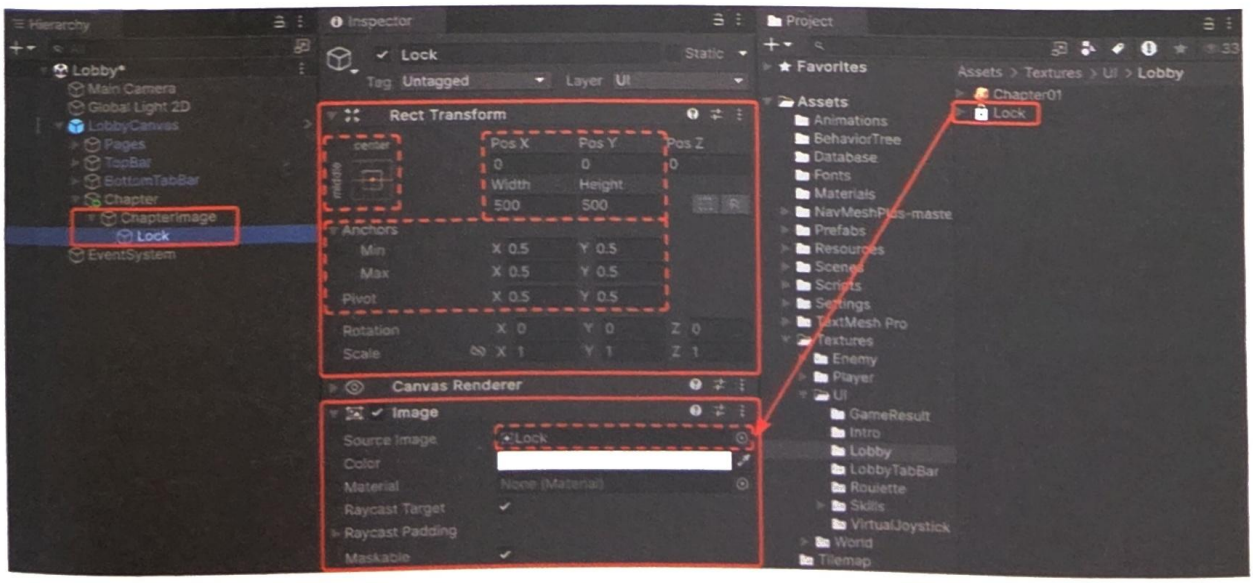
챕터 UI를 관리하는 Panel UI 생성하고 설정하기

02 [GameObject → UI → Image] 메뉴로 챕터 배경 이미지를 출력하는 Image UI를 생성하고 이름을 'ChapterImage'로 변경한 후 Chapter 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Rect는 (0, -100, 1200, 1200)으로 설정합니다. 그리고 Image 컴포넌트의 Source Image에 Chapter01.png 이미지를 등록합니다.



챕터 배경 이미지를 출력하는 Image UI 생성하고 설정하기

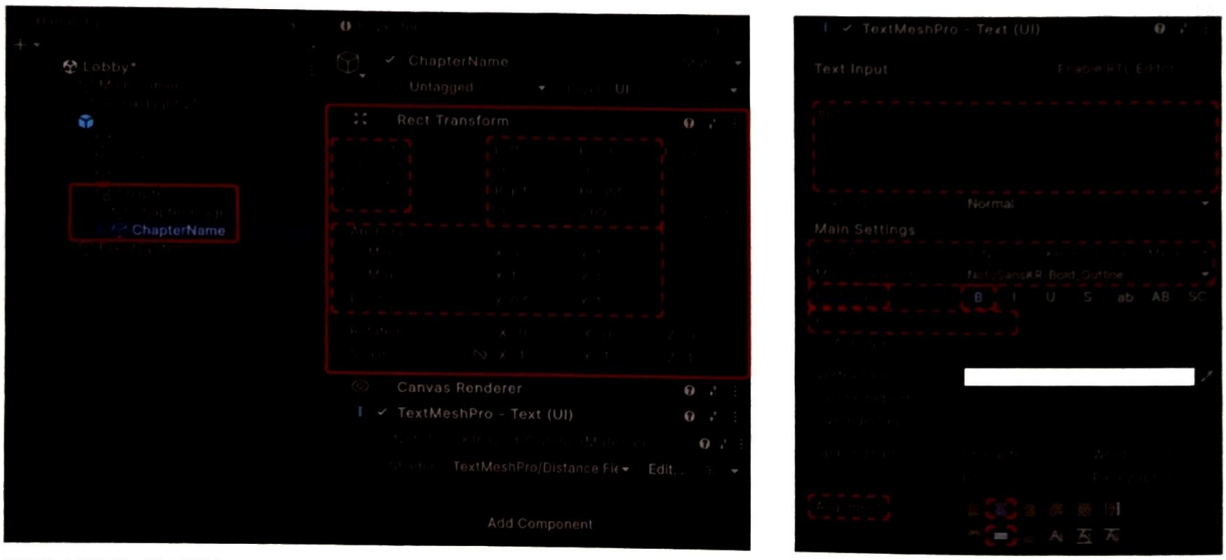
03 [GameObject → UI → Image] 메뉴로 챕터 잠금 이미지를 출력하는 Image UI를 생성하고 이름을 'Lock'으로 변경한 후 ChapterImage 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Rect는 (0, 0, 500, 500)으로 설정하고, Image 컴포넌트의 Source Image에 Lock.png 이미지를 등록합니다.



챕터 잠금 이미지를 출력하는 Image UI 생성하고 설정하기

04 [GameObject → UI → Text - TextMeshPro] 메뉴로 챕터 이름을 출력하는 Text UI를 생성하고 이름을 'ChapterName'으로 변경한 후 Chapter 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 top | stretch로, Rect는 (0, 0, 0, 200)으로 설정합니다.

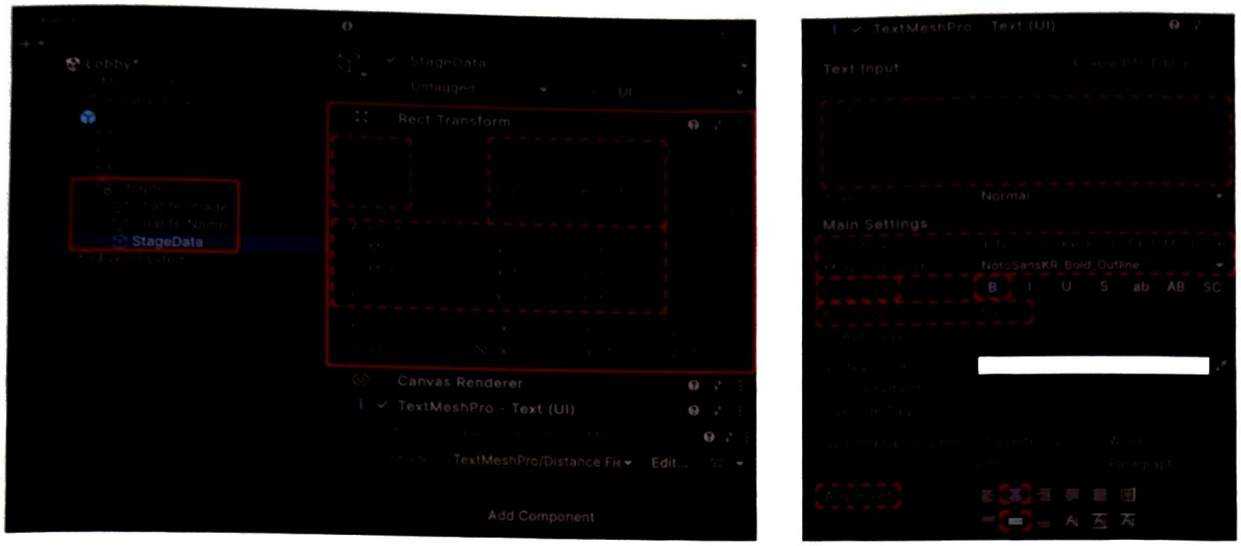
TextMeshPro - Text 컴포넌트의 Text Input Box에 '#01 탐욕의 광산'을 입력합니다. Font Asset에 NotoSansKR-Bold SDF를 등록하고 Material Preset은 [NotoSansKR-Bold_Outline]으로 선택합니다. 마지막으로, Font Style은 굵게 [B]로, Font Size는 '100'으로, Alignment는 Center | Middle로 설정합니다.



챕터 이름을 출력하는 Text UI 생성하고 설정하기

05 스테이지 정보를 출력하는 Text UI를 생성하고자 ChapterName 오브젝트를 Ctrl+D로 복제하고 이름을 'StageData'로 변경합니다.

Rect Transform 컴포넌트의 Rect는 (0, -150, 0, 200)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 '스테이지 10/50'을 입력하고 Font Size는 '60'으로 설정합니다.



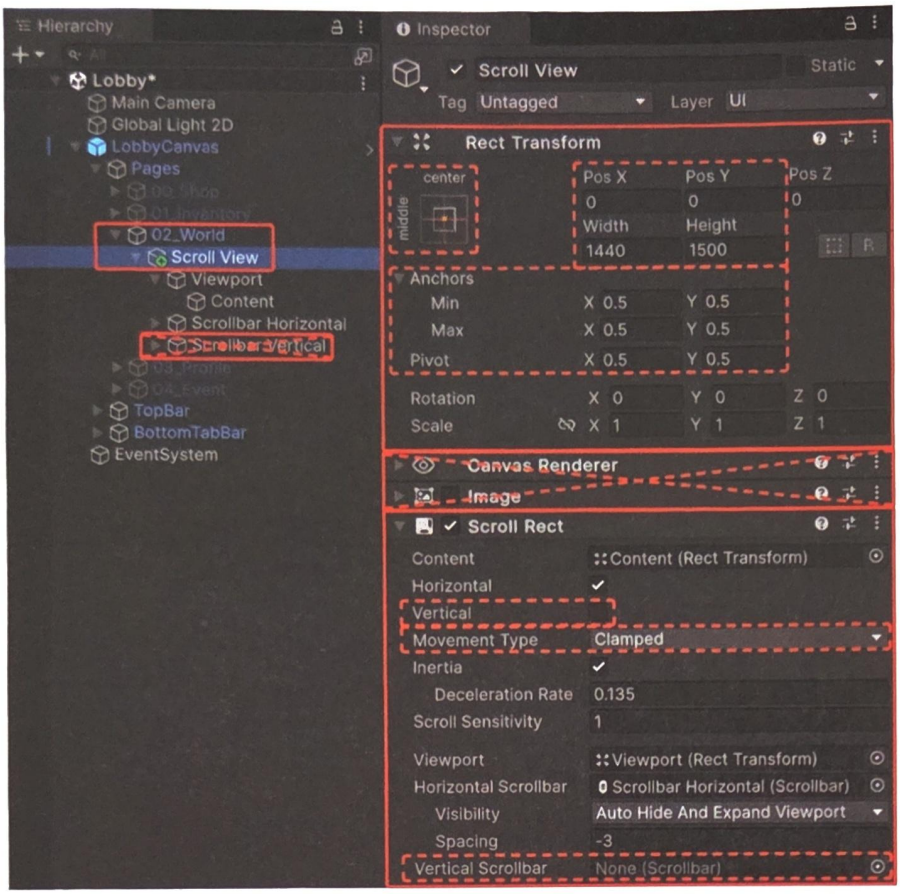
스테이지 정보를 출력하는 Text UI 생성하고 설정하기

06 Chapter 오브젝트를 Project View의 Assets 〉 Prefabs 〉 UI 폴더로 끌어서 놓아 프리팹을 생성하고, Hierarchy View에 있는 Chapter 오브젝트는 삭제합니다.

Chapter 오브젝트 프리팹 생성하기

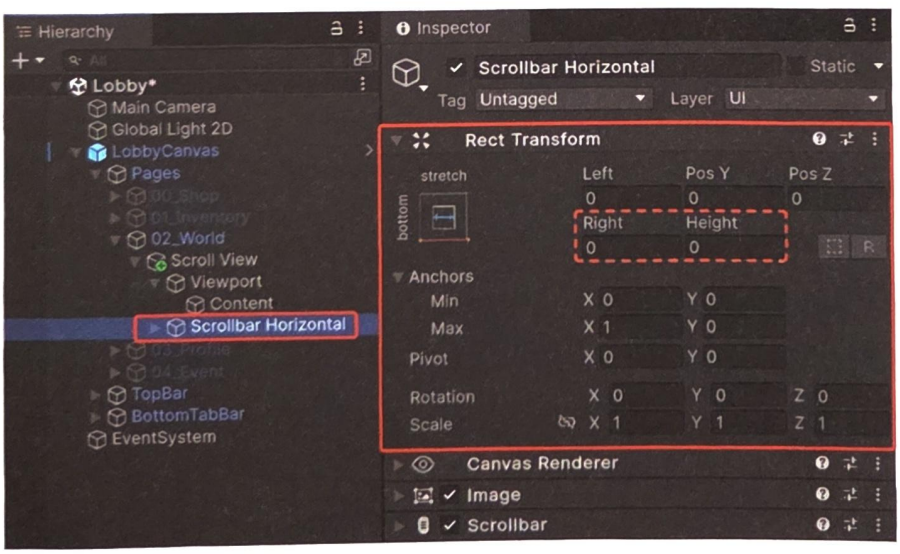
07 [GameObject → UI → Scroll View] 메뉴로 챕터 정보를 출력하는 Scroll View UI를 생성하고 02_World 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Rect를 (0, 0, 1440, 1500)으로 설정합니다. 화면에 출력하지 않으므로 Image, Canvas Renderer 컴포넌트는 삭제합니다. 가로축으로만 스크롤할 수 있도록 Scroll Rect 컴포넌트의 Vertical을 false로, Vertical Scrollbar를 [None]으로 설정합니다. Movement Type은 [Clamped]로 설정합니다. Scroll View 오브젝트의 자식인 Scrollbar Vertical 오브젝트는 삭제합니다.



챕터 정보를 출력하는 Scroll View UI 생성하고 설정하기

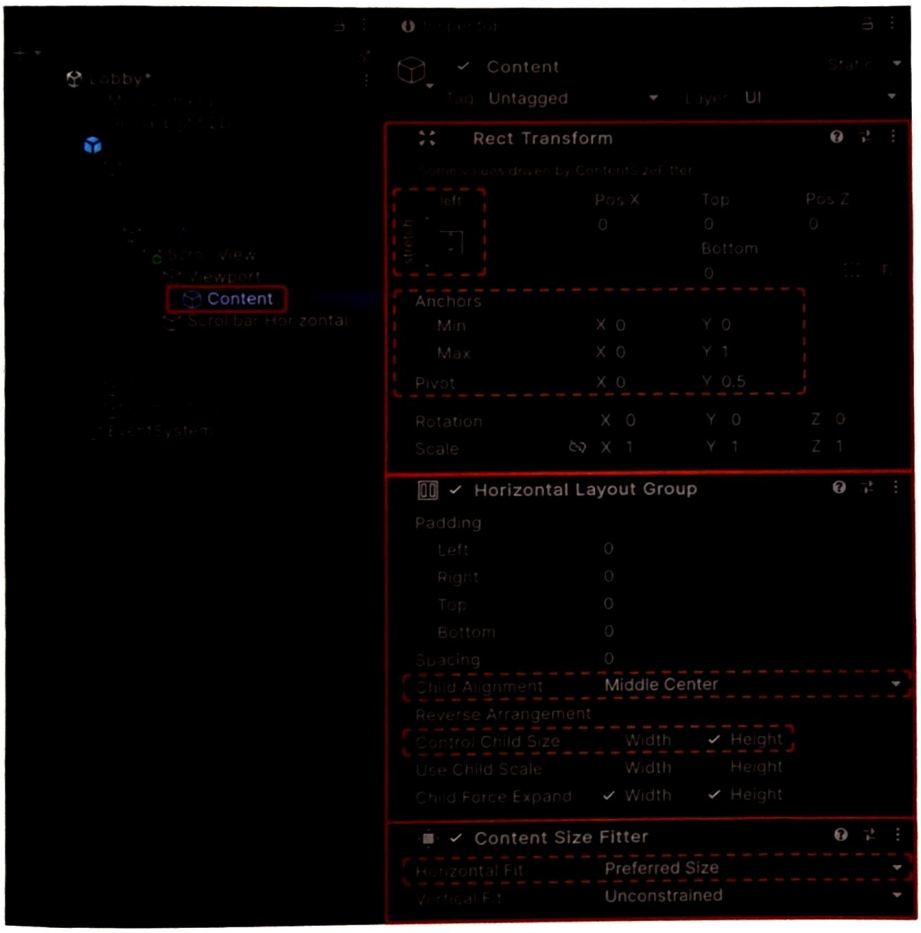
08 스크롤 기능만 사용하고 Scrollbar는 화면에서 숨기고자 Scrollbar Horizontal 오브젝트의 Rect Transform 컴포넌트에서 Right와 Height를 '0'으로 설정합니다.



Scrollbar Horizontal 오브젝트의 스크롤바가 보이지 않도록 Right와 Height를 '0'으로 설정하기

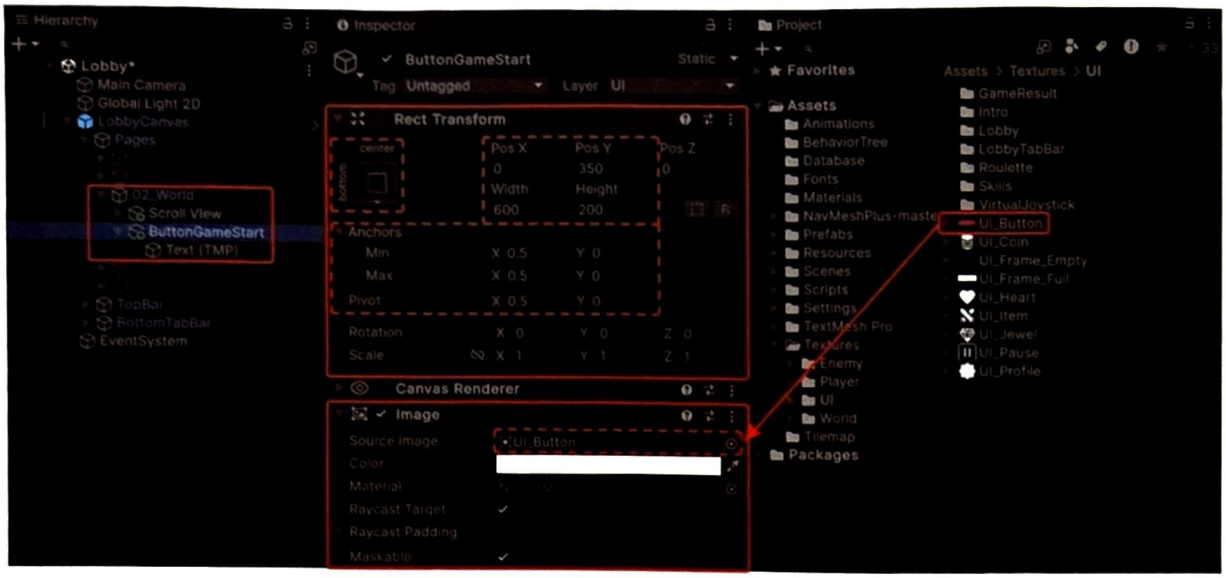
09 Content 오브젝트의 Rect Transform 컴포넌트에서 Anchor Presets을 stretch | left로 설정합니다. 자식으로 배치하는 오브젝트를 정렬하고자 Horizontal Layout Group 컴포넌트를, 자식 오브젝트의 크기에 따라 Content 오브젝트 크기를 설정하고자 Content Size Fitter 컴포넌트를 추가합니다.

Horizontal Layout Group 컴포넌트의 Child Alignment는 [Middle Center]로, Control Child Size의 Height는 true로 설정합니다. 이와 더불어 Content Size Fitter 컴포넌트의 Horizontal Fit을 [Preferred Size]로 선택합니다.



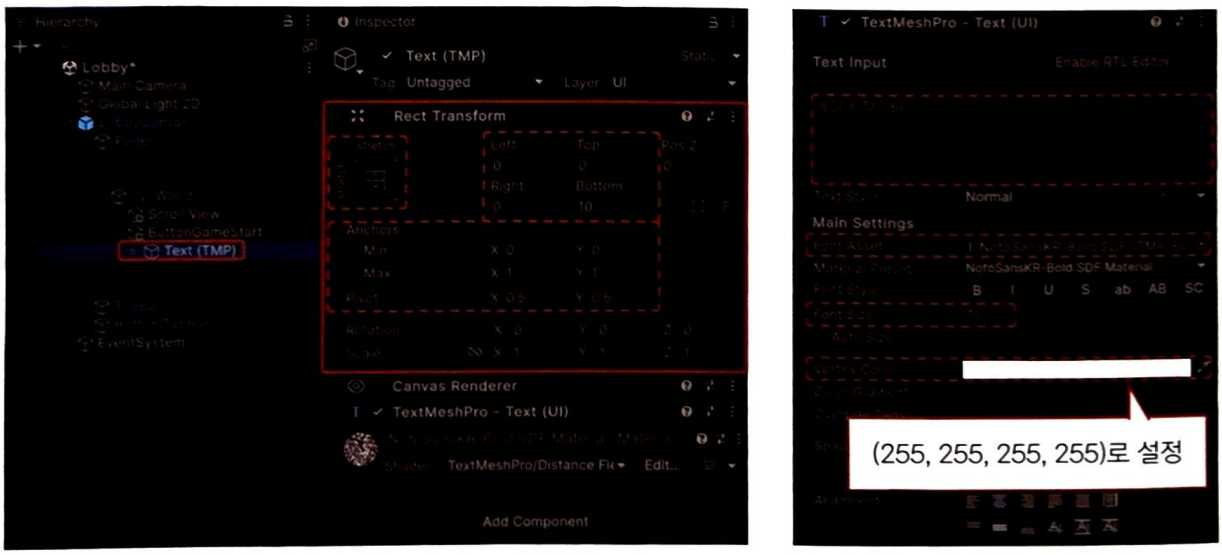
Content 오브젝트에 컴포넌트 추가하고 설정하기

10 [GameObject → UI → Button - TextMeshPro] 메뉴로 게임 시작 Button UI를 생성하고 이름을 'ButtonGameStart'로 변경한 후 02_World 오브젝트의 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 bottom | center로, Rect를 (0, 350, 600, 200)으로 설정합니다. Image 컴포넌트의 Source Image에 UI_Button.png 이미지를 등록합니다.



게임 시작 버튼 Button UI 생성하고 설정하기

11 ButtonGameStart 오브젝트의 자식인 Text (TMP) 오브젝트의 Rect Transform 컴포넌트에서 Rect를 (0, 0, 0, 10)으로 설정합니다. TextMeshPro - Text 컴포넌트의 Text Input Box에 '게임 시작 (-5)'를 입력하고, Font Asset에 NotoSansKR-Bold SDF를 등록합니다. Font Size는 '70'으로, Vertex Color는 (255, 255, 255, 255)로 설정합니다.



ButtonGameStart 오브젝트의 자식인 Text (TMP) 오브젝트 설정하기

12 Assets 〉 Scripts 〉 Database 폴더에 챕터 데이터를 저장하는 ChapterData.cs 스크립트를 생성합니다.

Assets/Scripts/Database/ChapterData.cs

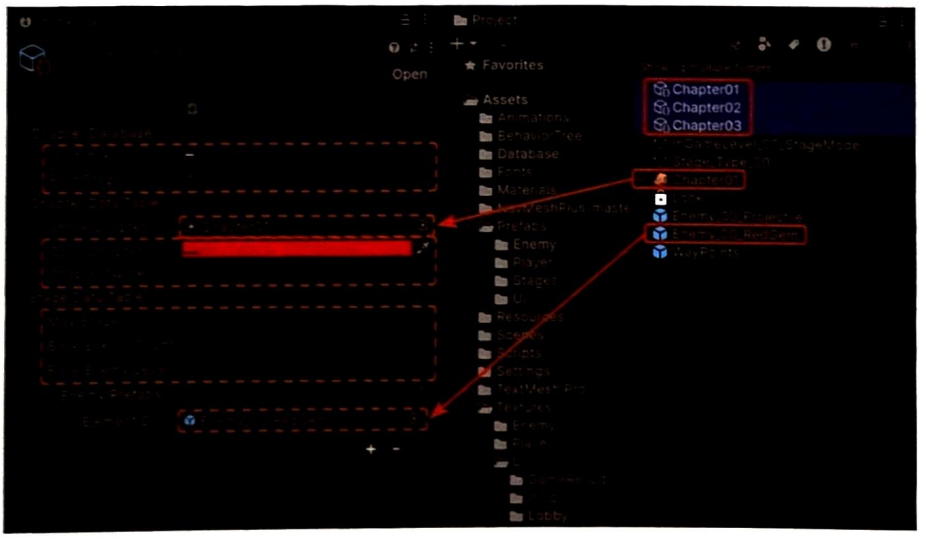
코드 11-05 챕터 데이터 저장 및 활용

```
01: using UnityEngine;
02:
03: [CreateAssetMenu]
04: public class ChapterData : ScriptableObject
05: {
06:     [SerializeField]
07:     private ChapterDatabase chapterDatabase;
08:     [SerializeField]
09:     private ChapterDataTable chapterDataTable;
10:     [SerializeField]
11:     private StageDataTable stageDataTable;
12:
13:     public ChapterDatabase ChapterDatabase => chapterDatabase;
14:     public ChapterDataTable ChapterDataTable => chapterDataTable;
15:     public StageDataTable StageDataTable => stageDataTable;
16: }
17:
18: [System.Serializable]
19: public struct ChapterDatabase
20: {
21:     public bool isUnlock; // 챕터 풀림 여부
22:     public int bestStage; // 최고 스테이지
23: }
24:
25: [System.Serializable]
26: public struct ChapterDataTable
27: {
28:     public Sprite spriteChapter; // 챕터 배경 이미지
29:     public Color colorChapter;   // Debug.. 현재는 챕터 이미지가 없어서 색상 변경
30:     public string chapterName;   // 챕터 이름
31: }
32:
33: [System.Serializable]
34: public struct StageDataTable
35: {
36:     public int maxStage;              // 최대 스테이지
37:     public int baseEnemyCount;        // 기본 적 숫자
38:     public int baseEnemyLevel;        // 기본 적 레벨
39:     public GameObject[] enemyPrefabs; // 등장하는 적 프리팹 배열
40: }
```

13 Project View의 ＋▾ 버튼을 클릭해 [Chapter Data] 메뉴로 Assets 〉 Database 폴더에 챕터 데이터를 저장하는 챕터 데이터 에셋을 3개 생성하고 이름을 각각 'Chapter01', 'Chapter02', 'Chapter03'으로 변경합니다. 그리고 다음 표를 참고하여 에셋 3개의 값을 설정합니다.

챕터 데이터 에셋 설정값

옵션	Chapter01	Chapter02	Chapter03
Is Unlock	true	false	false
Best Stage	1	1	1
Sprite Chapter	Chapter01.png	Chapter01.png	Chapter01.png
Color Chapter	(255, 255, 255, 255)	(255, 0, 0, 255)	(0, 0, 255, 255)
Chapter Name	탐욕의 광산	분노의 광산	탐욕의 광산 II
Max Stage	20	30	50
Bese Enemy Count	10	11	12
Base Enemy Level	1	2	3
배열 크기	1	1	1
Element 0	Enemy_00_RedGem	Enemy_00_RedGem	Enemy_00_RedGem



챕터 데이터를 저장하는 챕터 데이터 에셋 생성하고 설정하기

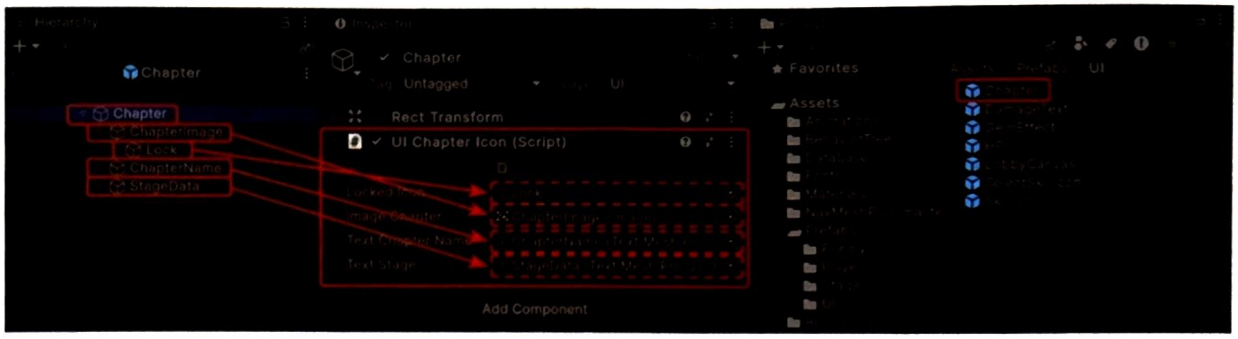
14 Assets 〉 Scripts 폴더 아래에 LobbyScene 폴더를 만들고, 이곳에 챕터 UI를 제어하는 UIChapterIcon.cs 스크립트를 생성합니다.

Assets/Scripts/LobbyScene/UIChapterIcon.cs

코드 11-06 챕터 UI 제어

```
01: using UnityEngine;
02: using UnityEngine.UI;
03: using TMPro;
04:
05: public class UIChapterIcon : MonoBehaviour
06: {
07:     [SerializeField]
08:     private GameObject lockedIcon;
09:     [SerializeField]
10:     private Image imageChapter;
11:     [SerializeField]
12:     private TextMeshProUGUI textChapterName;
13:     [SerializeField]
14:     private TextMeshProUGUI textStage;
15:
16:     public void Setup(int index, ChapterData chapterData)
17:     {
18:         lockedIcon.SetActive(!chapterData.ChapterDatabase.isUnlock);
19:
20:         imageChapter.sprite = chapterData.ChapterDataTable.spriteChapter;
21:         // Debug.. 챕터 이미지가 모두 있을 땐 색상 정보는 필요 없음
22:         imageChapter.color = chapterData.ChapterDataTable.colorChapter;
23:         textChapterName.text =
                $"#{index+1:D2} {chapterData.ChapterDataTable.chapterName}";
24:         textStage.text = $"스테이지 {chapterData.ChapterDatabase.bestStage}/
                {chapterData.StageDataTable.maxStage}";
25:     }
26: }
```

15 Chapter 프리팹을 더블클릭하고 UI Chapter Icon 컴포넌트를 추가한 후 Locked Icon 변수에는 Lock 오브젝트를, Image Chapter 변수에는 ChapterImage 오브젝트를, Text Chapter Name 변수에는 ChapterName 오브젝트를, Text Stage 변수에는 StageData 오브젝트를 각각 등록합니다.



Chapter 프리팹에 UI Chapter Icon 컴포넌트 추가하고 설정하기

16 Assets 〉 Scripts 〉 LobbyScene 폴더에 로비 씬을 제어하는 LobbySceneController.cs 스크립트를 생성합니다.

Assets/Scripts/LobbyScene/LobbySceneController.cs

코드 11-07 로비 씬 제어

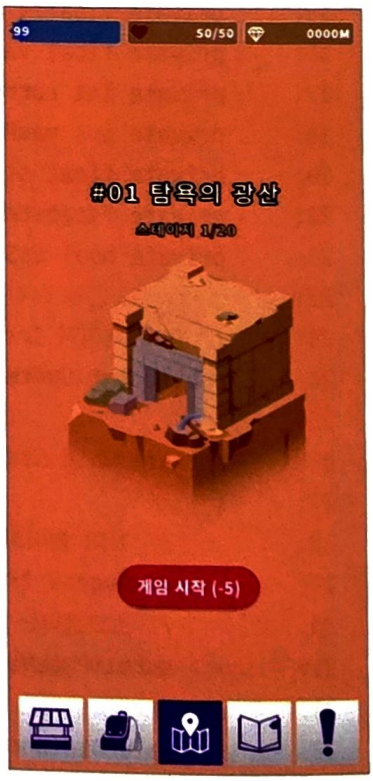
```
01: using UnityEngine;
02:
03: public class LobbySceneController : MonoBehaviour
04: {
05:     [SerializeField]
06:     private GameObject chapterIconPrefab;
07:     [SerializeField]
08:     private Transform parentContent;
09:     [SerializeField]
10:     private ChapterData[] allChapter;
11:
12:     private void Awake()
13:     {
14:         for ( int i = 0; i < allChapter.Length; ++ i )
15:         {
16:             GameObject icon = Instantiate(chapterIconPrefab, parentContent);
17:             icon.GetComponent<UIChapterIcon>().Setup(i, allChapter[i]);
18:         }
19:     }
20: }
```

17 Chapter 프리팹에서 나와 [GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'LobbySceneController'로 변경하고 나서 Lobby Scene Controller 컴포넌트를 추가합니다.

이 컴포넌트의 Chapter Icon Prefab 변수에는 Chapter 프리팹을, Parent Content 변수에는 Content 오브젝트를 등록합니다. All Chapter 배열 크기를 '3'으로 설정하고 Chapter01, 02, 03 에셋을 순서대로 등록합니다.

LobbySceneController 오브젝트에 Lobby Scene Controller 컴포넌트 추가하고 설정하기

18 플레이 버튼 ▶을 눌러 게임을 실행하면 로비 씬 02_World 페이지에 챕터 1 정보를 출력하고, 화면을 클릭해 좌우로 드래그하면 챕터 정보가 변경됩니다.



챕터 선택 화면

Do it! 실습 챕터 이동 스와이프 UI 만들기

01 챕터를 드래그했을 때 각 챕터 정보에 정확히 멈추고 현재 선택한 챕터 정보를 불러와 해당 챕터를 플레이할 수 있도록 설정합니다. 먼저 Assets 〉 Scripts 〉 Common 폴더에 SwipeUI.cs 스크립트를 생성합니다.

Assets/Scripts/Common/SwipeUI.cs

코드 11-08 Swipe UI 제어

```
01: using System.Collections;
02: using UnityEngine;
03: using UnityEngine.InputSystem;
04: using UnityEngine.UI;
05:
06: public class SwipeUI : MonoBehaviour
07: {
08:     [SerializeField]
09:     private Scrollbar scrollBar; // Scrollbar 위치를 바탕으로 현재 페이지 검사
10:     [SerializeField]
11:     private float swipeTime = 0.2f;      // 페이지가 스와이프되는 시간
12:     [SerializeField]
13:     private float swipeDistance = 50.0f; // 스와이프할 때 움직여야 하는 최소 거리
14:
15:     private float[] scrollPageValues;    // 각 페이지의 위칫값(0.0~1.0)
16:     private float valueDistance = 0;     // 각 페이지 사이의 거리
17:     private int currentPage = 0;         // 현재 페이지
18:     private int maxPage = 0;             // 최대 페이지
19:     private float startTouchX;           // 터치 시작 위치
20:     private float endTouchX;             // 터치 종료 위치
21:     private bool isSwipeMode = false;    // 현재 스와이프 중인지 체크
22:
23:     // 현재 페이지 Index 정보
24:     public int CurrentPage => currentPage;
25:
26:     private void Start()
27:     {
28:         // 최대 페이지 수
29:         maxPage = transform.childCount;
30:         // 스크롤되는 페이지의 각 value값을 저장하는 배열 메모리 할당
31:         scrollPageValues = new float[transform.childCount];
32:         // 스크롤되는 페이지 사이 거리
33:         valueDistance = 1f / (scrollPageValues.Length - 1f);
34:
```

```
35:         // 스크롤되는 페이지의 각 value 위치 설정 [0 <= value <= 1]
36:         for ( int i = 0; i < scrollPageValues.Length; ++ i )
37:         {
38:             scrollPageValues[i] = valueDistance * i;
39:         }
40:
41:         // 최초 시작할 때 0번 페이지를 볼 수 있도록 설정
42:         SetScrollBarValue(0);
43:     }
44:
45:     private void Update()
46:     {
47:         UpdateInput();
48:     }
49:
50:     public void SetScrollBarValue(int index)
51:     {
52:         currentPage = index;
53:         scrollBar.value = scrollPageValues[index];
54:     }
55:
56:     private void UpdateInput()
57:     {
58:         // 스와이프 중이면 터치 불가
59:         if ( isSwipeMode == true ) return;
60:
61:         #if UNITY_EDITOR
62:         if ( Mouse.current != null )
63:         {
64:             if ( Mouse.current.leftButton.wasPressedThisFrame )
65:             {
66:                 startTouchX = Mouse.current.position.ReadValue().x; // 클릭 시작점
67:             }
68:             else if ( Mouse.current.leftButton.wasReleasedThisFrame )
69:             {
70:                 endTouchX = Mouse.current.position.ReadValue().x;   // 클릭 종료점
71:                 UpdateSwipe();
72:             }
73:         }
74:         #elif UNITY_ANDROID
75:         if ( Touchscreen.current != null && Touchscreen.current.touches.Count > 0 )
76:         {
77:             var touch = Touchscreen.current.touches[0];
78:             if ( touch.press.wasPressedThisFrame )
```

```
79:             {
80:                 startTouchX = touch.position.ReadValue().x; // 시작점(방향 구분)
81:             }
82:             else if ( touch.press.wasReleasedThisFrame )
83:             {
84:                 endTouchX = touch.position.ReadValue().x;   // 종료점(방향 구분)
85:                 UpdateSwipe();
86:             }
87:         }
88:         #endif
89:     }
90:
91:     private void UpdateSwipe()
92:     {
93:         // 너무 짧은 거리를 움직였다면 스와이프하지 않음
94:         if ( Mathf.Abs(startTouchX-endTouchX) < swipeDistance )
95:         {
96:             // 원래 페이지로 스와이프해서 돌아감
97:             StartCoroutine(OnSwipeOneStep(currentPage));
98:             return;
99:         }
100:
101:        // 스와이프 방향
102:        bool isLeft = startTouchX < endTouchX;
103:        if ( isLeft == true ) // 이동 방향이 왼쪽일 때
104:        {
105:            if ( currentPage == 0 ) return; // 현재 페이지가 왼쪽 끝이면 종료
106:
107:            currentPage --; // 왼쪽으로 이동하고자 현재 페이지 1 감소
108:        }
109:        else // 이동 방향이 오른쪽일 때
110:        {
111:            if ( currentPage == maxPage - 1 ) return; // 페이지가 오른쪽 끝이면 종료
112:
113:            currentPage ++; // 오른쪽으로 이동하고자 현재 페이지 1 증가
114:        }
115:
116:        // currentIndex번째 페이지로 스와이프해서 이동
117:        StartCoroutine(OnSwipeOneStep(currentPage));
118:    }
119:
120:    /// <summary>
121:    /// 페이지를 한 장 옆으로 넘기는 스와이프 효과를 재생한다.
122:    /// </summary>
```

```
123:    private IEnumerator OnSwipeOneStep(int index)
124:    {
125:        float start = scrollBar.value;
126:        float percent = 0;
127:
128:        isSwipeMode = true;
129:
130:        while ( percent < 1 )
131:        {
132:            percent += Time.deltaTime / swipeTime;
133:
134:            scrollBar.value = Mathf.Lerp(start, scrollPageValues[index], percent);
135:
136:            yield return null;
137:        }
138:
139:        isSwipeMode = false;
140:    }
141:}
```

26~43줄 SwipeUI.cs 스크립트는 Content 오브젝트에 컴포넌트로 적용하고, 화면에 보이는 페이지는 Content 오브젝트 자식이므로 Start() 메서드 안에서 transform.childCount를 이용해 최대 페이지 수와 스크롤 페이지 사이의 거리, 위치 등을 설정합니다.

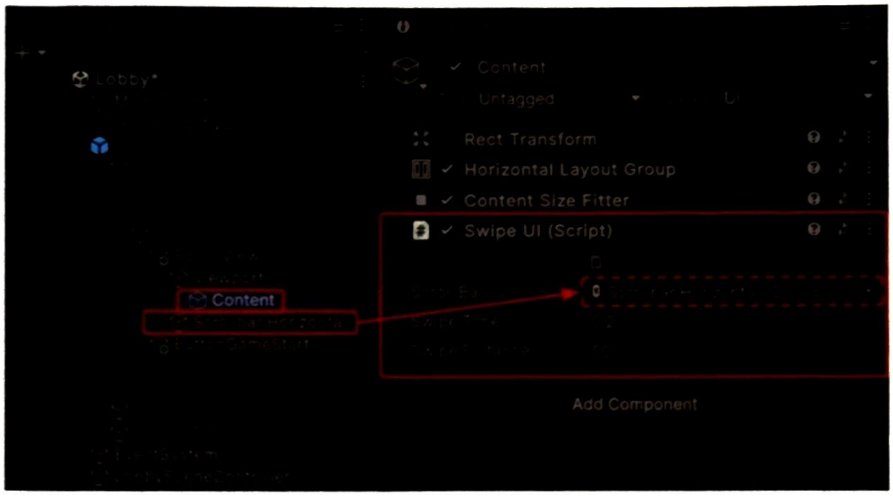
50~54줄 SetScrollBarValue() 메서드는 매개변수로 받은 index번째 페이지를 현재 페이지로 설정하고, scrollBar.value값을 설정합니다.

56~89줄 UpdateInput() 메서드는 전처리기를 이용해 현재 프로그램이 실행되는 환경이 유니티 에디터인지, 안드로이드인지를 구분하고, 마우스 클릭 또는 모바일 화면 터치 시작과 종료 위치를 계산합니다.

91~118줄 UpdateSwipe() 메서드에서 터치로 이동한 거리를 계산해서 너무 짧으면 원래 페이지로 돌아가고, 일정 거리 이상 이동했으면 왼쪽이나 오른쪽으로 페이지를 스크롤합니다.

123~140줄 실제 페이지를 스크롤하는 OnSwipeOneStep() 코루틴 메서드는 현재 스크롤 위치인 scrollBar.value부터 scrollPageValues[index]번째까지 swipeTime 동안 화면을 이동합니다. while() 반복문으로 화면을 스크롤하는 동안 isSwipeMode를 true로 설정해 추가로 조작할 수 없도록 합니다.

02 Content 오브젝트에 Swipe UI 컴포넌트를 추가하고 Scroll Bar 변수에 Scrollbar Horizontal 오브젝트를 등록합니다.



Content 오브젝트에 Swipe UI 컴포넌트 추가하고 설정하기

03 플레이 버튼을 눌렀을 때 호출하는 메서드를 정의하고자 LobbySceneController.cs 스크립트를 수정합니다.

Assets/Scripts/LobbyScene/LobbySceneController.cs

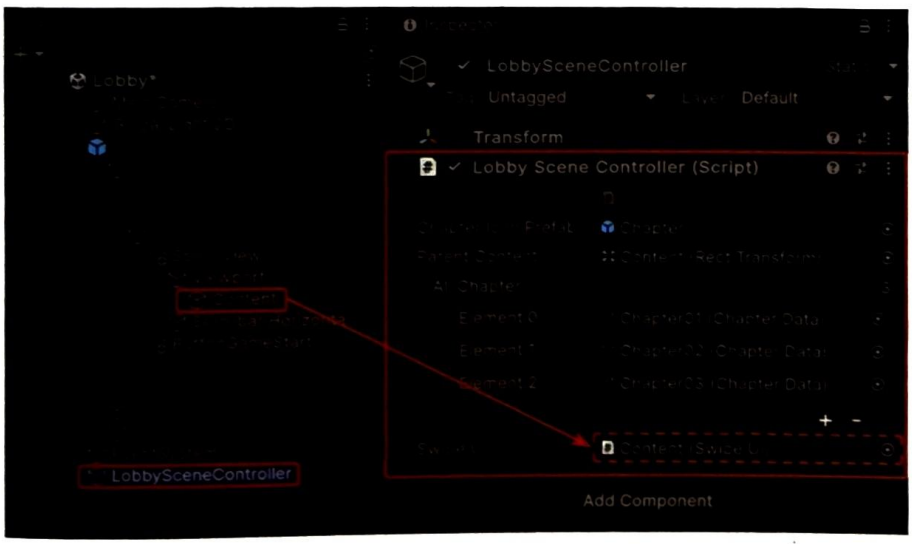
코드 11-09 로비 씬 제어

```
01: using UnityEngine;
02:
03: public class LobbySceneController : MonoBehaviour
04: {
05:     [SerializeField]
06:     private GameObject chapterIconPrefab;
07:     [SerializeField]
08:     private Transform parentContent;
09:     [SerializeField]
10:     private ChapterData[] allChapter;
11:     [SerializeField]                                    추가
12:     private SwipeUI swipeUI;
        (... 생략 ...)
23:     public void ButtonEvent_GameStart()                 추가
24:     {
25:         int index = swipeUI.CurrentPage;
26:
27:         if ( allChapter[index].ChapterDatabase.isUnlock == false )
28:         {
```

```
29:             Logger.Log("잠긴 챕터입니다.");
30:             return;
31:         }
32:
33:         Logger.Log($"챕터 {index+1} 시작");
34:
35:         SceneLoader.Instance.LoadScene(SceneNames.Game);
36:     }
37: }
```

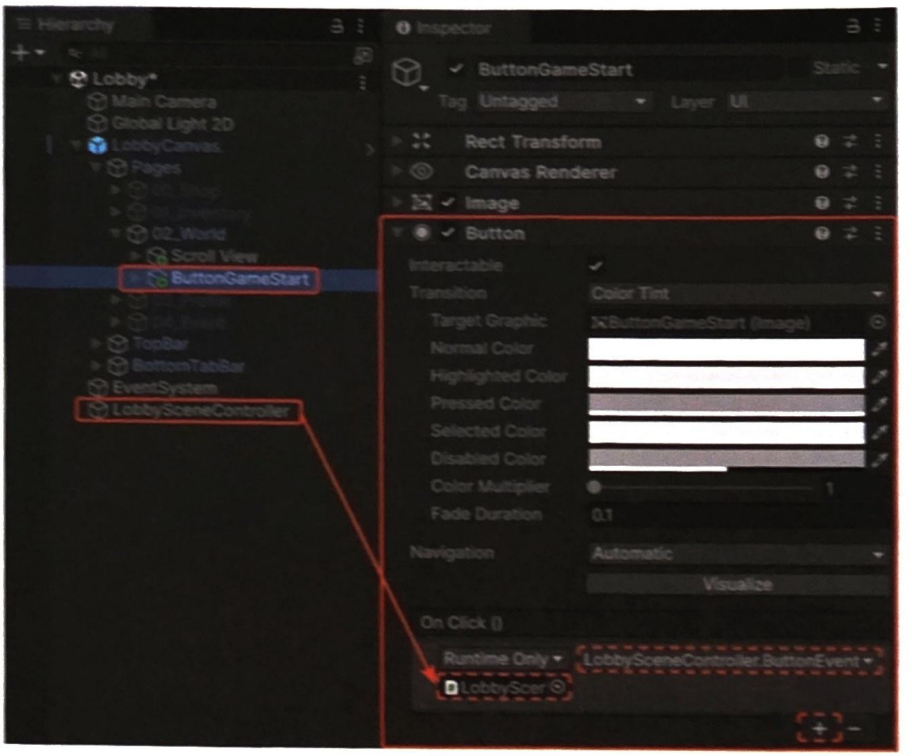
23~36줄 스와이프로 챕터를 선택하고 〈게임 시작 (-5)〉 버튼을 누르면 **ButtonEvent_GameStart()** 메서드를 호출합니다. **swipeUI.CurrentPage**에 저장된 현재 챕터 순서를 **index**에 저장하고 **index**번째 챕터가 잠겼는지 검사합니다. 챕터가 잠겼으면 로그를 출력하고 메서드를 종료합니다. 그렇지 않다면 챕터 시작 로그를 출력하고 게임 씬을 불러옵니다.

04 LobbySceneController 오브젝트 Lobby Scene Controller 컴포넌트의 Swipe UI 변수에 Content 오브젝트를 등록합니다.



LobbySceneController 오브젝트의 컴포넌트 변수 설정하기

05 ButtonGameStart 오브젝트의 Button 컴포넌트에 On Click () 이벤트 요소를 추가한 후 LobbySceneController 오브젝트의 LobbySceneController 컴포넌트 ButtonEvent_GameStart() 메서드를 등록합니다.



ButtonGameStart 오브젝트의 Button 컴포넌트에 On Click () 이벤트 등록하기

06 인트로 씬을 더블클릭하여 선택하고 플레이 버튼 ▶을 눌러 게임을 실행하면 로비 씬 02_World 페이지에 챕터 1 정보를 출력합니다. 스와이프 UI이므로 화면을 클릭해 좌우로 드래그하면 한 챕터씩 넘겨서 볼 수 있고 〈게임 시작 (-5)〉 버튼을 누르면 게임 씬을 로드해 게임을 진행합니다.

로비 씬 챕터 스와이프 화면 결과

12장

챕터와 스테이지 관리하고 JSON 게임 데이터 저장하기

이번 장에서는 챕터와 스테이지를 구현합니다. 현재 월드의 모든 적을 물리치면 다음 스테이지로 진행하고, 마지막 스테이지까지 클리어하면 '챕터 클리어' 문구를 출력합니다. 이와 함께 게임이 끝난 후에도 정보를 유지하도록 JSON 형식으로 저장하여 세이브 기능을 구현합니다.

학습 목표

1 스테이지 진행 흐름과 게임 루프를 설계할 수 있다.
2 JSON 직렬화를 활용하여 게임 데이터를 저장하고 복원할 수 있다.

12-1 스테이지 클리어 로직과 챕터 진행 시스템 만들기
12-2 JSON 직렬화로 게임 세이브와 로드 구현하기

12-1

스테이지 클리어 로직과 챕터 진행 시스템 만들기

이 절에서는 스테이지가 변할 때 적이 즉시 등장하면 의도치 않게 피격당할 수 있으므로 적이 등장할 타일에 반짝이는 효과를 추가하고 n초 후에 나타나도록 합니다. 이와 더불어 현재 월드의 모든 적을 물리치면 다음 스테이지로 진행하고 마지막 스테이지를 클리어하면 '챕터 클리어' 문구를 출력해 봅시다.

Do it! 실습 적 등장 위치 알리기

지금은 게임 씬이 시작되면 적이 바로 등장하는데, 이렇게 다음 스테이지로 넘어갈 때 예비 시간 없이 적이 등장한다면 당황스러울 겁니다. 이에 적이 등장할 위치를 먼저 알리고 일정한 시간이 지난 후에 적이 등장하도록 설정해 봅니다.

01 Assets 〉 Scripts 〉 Enemy 폴더에 적 등장 위치를 알려 줄 타일 반짝임을 제어하는 SpawnTile.cs 스크립트를 생성합니다.

Assets/Scripts/Enemy/SpawnTile.cs

코드 12-01 타일 반짝임 효과 제어

```
01: using System.Collections;
02: using UnityEngine;
03:
04: public class SpawnTile : MonoBehaviour
05: {
06:     private SpriteRenderer spriteRenderer;
07:
08:     private void Awake()
09:     {
10:         spriteRenderer = GetComponent<SpriteRenderer>();
11:     }
12:
```

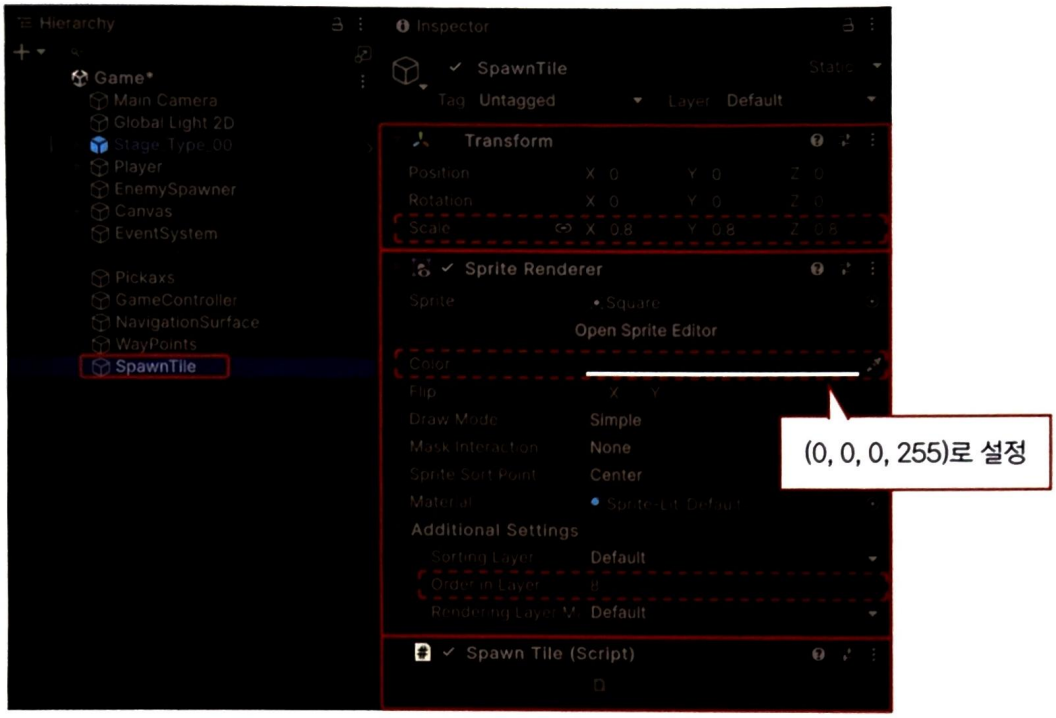
```
13:     private void OnEnable()
14:     {
15:         StartCoroutine(nameof(FadeLoop));
16:     }
17:
18:     private void OnDisable()
19:     {
20:         StopCoroutine(nameof(FadeLoop));
21:     }
22:
23:     private IEnumerator FadeLoop()
24:     {
25:         while ( true )
26:         {
27:             yield return FadeEffect.Fade(spriteRenderer, 1, 0, 0.5f);
28:
29:             yield return FadeEffect.Fade(spriteRenderer, 0, 1, 0.5f);
30:         }
31:     }
32: }
```

적 등장 위치를 알려 주는 타일은 생성 이후 활성화/비활성화해서 재활용합니다.

13~16줄 오브젝트를 활성화할 때 OnEnable() 메서드를 호출해 페이드 효과를 재생합니다.

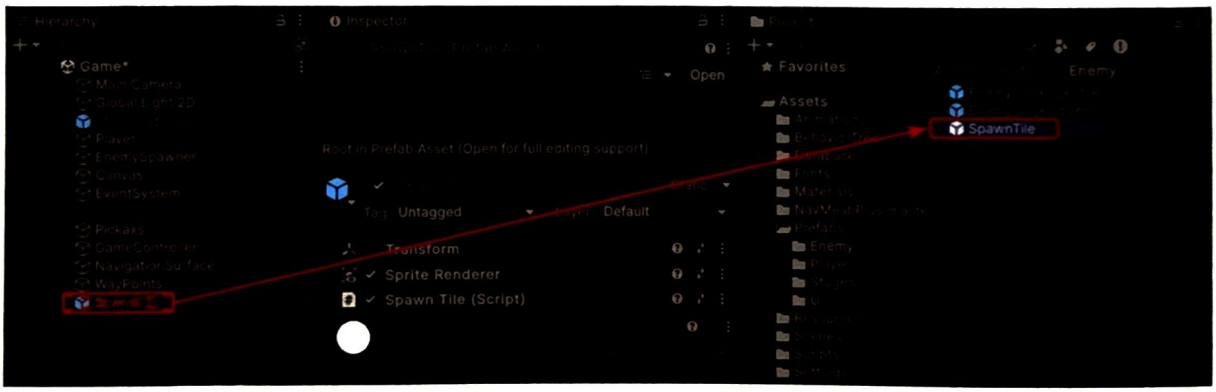
18~21줄 오브젝트를 비활성화할 때 OnDisable() 메서드를 호출해 페이드 효과 재생을 끝냅니다.

02 [GameObject → 2D Object → Sprites → Square] 메뉴로 적이 생성될 위치에 출력하는 타일 오브젝트를 생성하고 이름을 'SpawnTile'로 변경한 후 Transform 컴포넌트의 Scale을 (0.8, 0.8, 0.8)로 설정합니다. Sprite Renderer 컴포넌트의 Color는 (0, 0, 0, 255)로, Order in Layer는 '8'로 설정하고 Spawn Tile 컴포넌트를 추가합니다.



적 생성 위치에 출력하는 타일 오브젝트 생성하고 설정하기

03 SpawnTile 오브젝트를 Project View의 Assets 〉 Prefabs 〉 Enemy 폴더로 끌어서 놓아 프리팹을 생성하고 Hierarchy View에 있는 SpawnTile 오브젝트는 삭제합니다.



SpawnTile 오브젝트 프리팹 생성하기

04 적이 생성될 위치에 타일을 먼저 생성한 뒤 2초 후에 적을 생성하도록 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 12-02 적 생성 위치에 타일 생성

```
01: using System.Collections;                                   추가
02: using System.Collections.Generic;
03: using UnityEngine;
04: using UnityEngine.Tilemaps;
05:
06: public class EnemySpawner : MonoBehaviour
07: {
08:     [SerializeField]
09:     private Tilemap tilemap;
10:     [SerializeField]                                        추가
11:     private GameObject enemySpawnTile;
      (... 생략 ...)
20:     /*[SerializeField]                                     삭제
21:     private int enemyCount = 10;*/
22:
23:     private Vector3 offset = new Vector3(0.5f, 0.5f, 0);
24:     private List<Vector3> possibleTiles = new List<Vector3>();
25:     private MemoryPool enemySpawnTilePool;                  추가
26:     private WaitForSeconds waitTime = new WaitForSeconds(2.0f);
      (... 생략 ...)
38:     private void Awake()
39:     {
40:         enemySpawnTilePool = new MemoryPool(enemySpawnTile);   추가
41:
42:         // Tilemap의 Bounds 재설정
               (맵을 수정했을 때 Bounds가 바뀌지 않는 문제 해결)
43:         tilemap.CompressBounds();
44:         // 타일맵의 모든 타일을 대상으로 적을 배치할 수 있는 타일 계산
45:         CalculatePossibleTiles();
46:
47:         /*// 임의의 타일에 적 10개 생성                     삭제
48:         for ( int i = 0; i < enemyCount; ++ i )
49:         {
50:             int type = Random.Range(0, enemyPrefabs.Length);
51:             int index = Random.Range(0, possibleTiles.Count);
52:             int wayIndex = Random.Range(0, wayPointData.Length);
53:
54:             GameObject clone = Instantiate(
                    enemyPrefabs[type], possibleTiles[index], Quaternion.identity,
                    transform);
55:             clone.GetComponent<EnemyBase>().Initialize(
                    this, parentTransform, gemCollector);
```

```
56:             clone.GetComponent<EnemyFSM>().Setup(
                    target, wayPointData[wayIndex].wayPoints);
57:
58:             Enemies.Add(clone.GetComponent<EntityBase>());
59:         }*/
60:     }
61:
62:     public void SpawnEnemys(int count)
63:     {
64:         Enemies.Clear();
65:         StartCoroutine(nameof(Process), count);
66:     }
67:
68:     private IEnumerator Process(int count)
69:     {
70:         Vector3[] positions = new Vector3[count];
71:         for ( int i = 0; i < count; ++ i )
72:         {
73:             // 적을 배치할 임의의 위치 설정
74:             positions[i] = possibleTiles[Random.Range(0, possibleTiles.Count)];
75:             // 적이 배치될 위치에 타일 생성
76:             enemySpawnTilePool.ActivatePoolItem(positions[i]);
77:         }
78:
79:         yield return waitTime;
80:
81:         // 모든 타일 삭제
82:         enemySpawnTilePool.DeactivateAllPoolItems();
83:         // 적 생성
84:         for ( int i = 0; i < count; ++ i )
85:         {
86:             int type = Random.Range(0, enemyPrefabs.Length);
87:             int wayIndex = Random.Range(0, wayPointData.Length);
88:
89:             GameObject clone = Instantiate(
                    enemyPrefabs[type], positions[i], Quaternion.identity, transform);
90:             clone.GetComponent<EnemyBase>().Initialize(
                    this, parentTransform, gemCollector);
91:             clone.GetComponent<EnemyFSM>().Setup(
                    target, wayPointData[wayIndex].wayPoints);
92:
93:             Enemies.Add(clone.GetComponent<EntityBase>());
94:         }
95:     }
```

추가

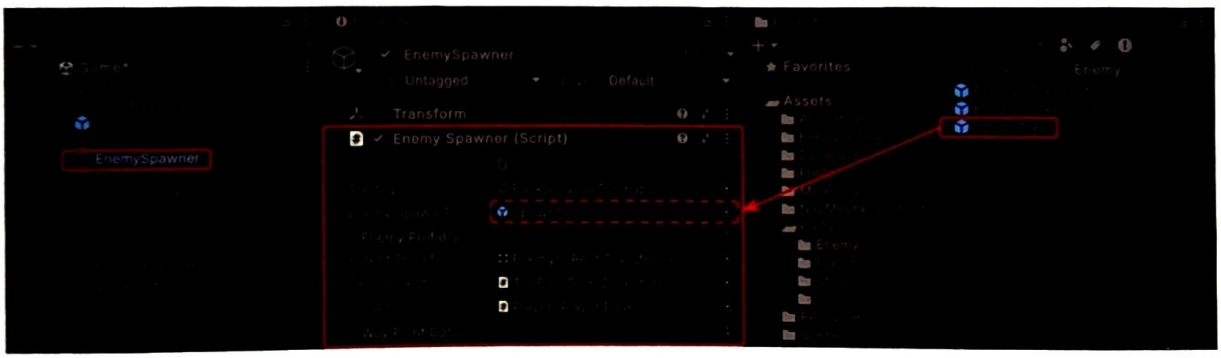
```
        (... 생략 ...)
127:}
```

62~66줄 각 스테이지에서 생성하는 적 숫자 count를 매개변수로 하여 SpawnEnemys() 메서드를 호출합니다.

70~77줄 Process() 코루틴 메서드에서 적을 배치할 임의의 위치를 count개만큼 생성하고 해당 위치에 타일을 생성합니다.

79~94줄 생성한 타일은 waitTime 시간이 지나면 사라지고 타일 위치에 적을 생성합니다.

05 EnemySpawner 오브젝트 Enemy Spawner 컴포넌트의 Enemy Spawn Tile 변수에 SpawnTile 프리팹을 등록합니다.



EnemySpawner 오브젝트 Enemy Spawner 컴포넌트의 Enemy Spawn Tile 변수 설정하기

06 현재 챕터, 스테이지에 따라 정해진 숫자만큼 적을 생성하는 작업은 GameController에서 처리합니다. baseEnemyCount 숫자만큼 적을 생성하도록 GameController.cs 스크립트를 수정합니다.

Assets/Scripts/GameSystem/GameController.cs

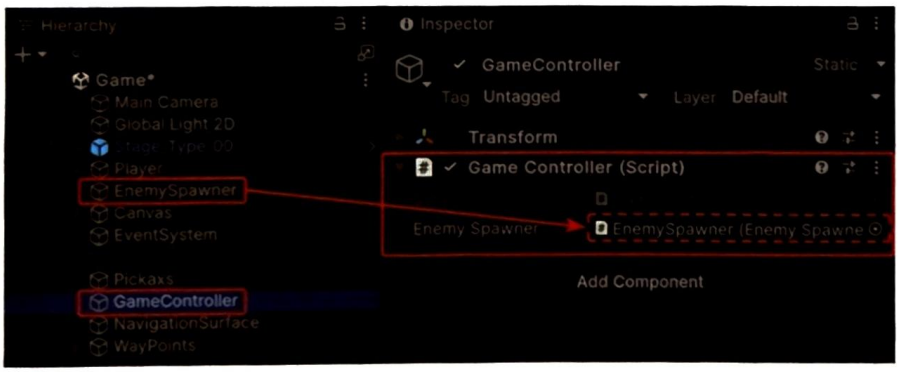
코드 12-03 baseEnemyCount 숫자만큼 적 생성 제어

```
01: using UnityEngine;
02:
03: public class GameController : MonoBehaviour
04: {
05:     [SerializeField]                    추가
06:     private EnemySpawner enemySpawner;
07:
```

```
08:     private int baseEnemyCount = 10;
09:
10:     private void Start()
11:     {
12:         // 스테이지 설정, 스테이지에 등장하는 적 생성
13:         SetupStage();
14:     }
15:
16:     public void SetupStage()
17:     {
18:         // 스테이지에 따라 등장하는 적 숫자 연산/생성
19:         enemySpawner.SpawnEnemys(baseEnemyCount);
20:     }
        (... 생략 ...)
26: }
```

08~20줄 게임 씬을 실행하면 SetupStage() 메서드를 호출해 baseEnemyCount 숫자인 10만큼 스테이지 1에서 등장할 적을 생성합니다.

07 GameController 오브젝트 Game Controller 컴포넌트의 Enemy Spawner 변수에 EnemySpawner 오브젝트를 등록합니다.



GameController 오브젝트 Game Controller 컴포넌트의 Enemy Spawner 변수 설정하기

Do it! 실습 모든 적을 물리치면 다음 스테이지 실행하기

01 현재 스테이지의 적 숫자가 0이면, 즉 Enemies.Count == 0이면 exitEvent 이벤트에 등록된 메서드를 실행하도록 EnemySpawner.cs 스크립트를 수정합니다.

Assets/Scripts/Enemy/EnemySpawner.cs

코드 12-04 적(Enemies) 숫자가 0일 때 exitEvent 이벤트 실행

```
01: using System.Collections;
02: using System.Collections.Generic;
03: using UnityEngine;
04: using UnityEngine.Events;                          추가
05: using UnityEngine.Tilemaps;
06:
07: public class EnemySpawner : MonoBehaviour
08: {
        (... 생략 ...)
25:     private WaitForSeconds waitTime = new WaitForSeconds(2.0f);
26:     public static UnityEvent exitEvent = new UnityEvent();          추가
27:
28:     public static List<EntityBase> Enemies { get; private set; } =
            new List<EntityBase>();
        (... 생략 ...)
108:    public void Deactivate(EntityBase enemy)
109:    {
110:        Enemies.Remove(enemy);
111:        Destroy(enemy.gameObject);
112:
113:        if ( Enemies.Count == 0 )                  추가
114:        {
115:            exitEvent?.Invoke();
116:        }
117:    }
118:}
```

02 현재 스테이지의 모든 적을 물리치면 스테이지를 1 늘리고 다시 적을 생성하도록 GameController.cs 스크립트를 수정합니다.

Assets/Scripts/GameSystem/GameController.cs

코드 12-05 스테이지+1과 적 생성 제어

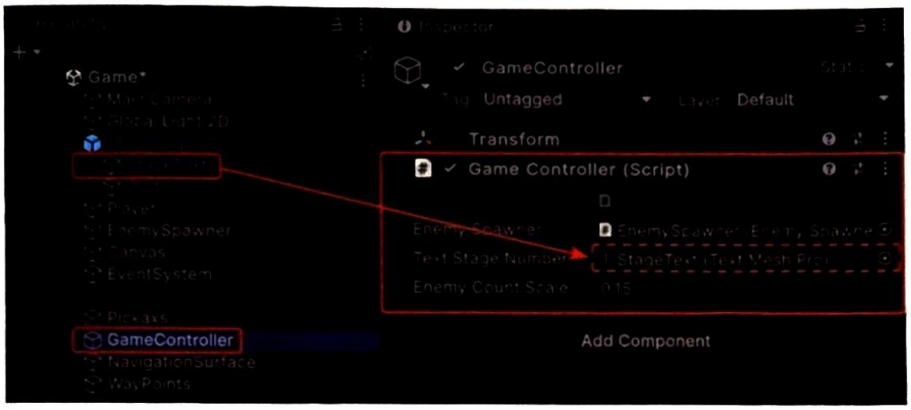
```
01: using UnityEngine;
02: using TMPro;                                       추가
03:
04: public class GameController : MonoBehaviour
05: {
06:     [SerializeField]
```

```
07:     private EnemySpawner enemySpawner;
08:     [SerializeField]                                    추가
09:     private TextMeshPro textStageNumber;
10:     [SerializeField]
11:     private float enemyCountScale = 0.15f;
12:
13:     private int currentStage = 0;                       추가
14:     private int baseEnemyCount = 10;
15:
16:     private void Start()
17:     {
18:         // 현재 스테이지에 남은 적 숫자가 0이면 다음 스테이지로 넘어감   추가
19:         EnemySpawner.exitEvent.AddListener(SetupStage);
20:         // 스테이지 설정, 스테이지에 등장하는 적 생성
21:         SetupStage();
22:     }
23:
24:     public void SetupStage()
25:     {
26:         currentStage ++;                                추가
27:         // 맵에 출력하는 currentStage Text UI 갱신
28:         textStageNumber.text = $"STAGE {currentStage:D2}";
29:         // 스테이지에 따라 등장하는 적 숫자 연산/생성
30:         enemySpawner.SpawnEnemys((int)(baseEnemyCount +
                currentStage * enemyCountScale));           수정
31:     }
        (... 생략 ...)
37: }
```

18~19줄 EnemySpawner 클래스에 선언한 exitEvent 이벤트에 SetupStage() 메서드를 등록했으므로 현재 월드에 있는 적의 숫자가 0이 되어 exitEvent.Invoke() 메서드를 호출하면 SetupStage() 메서드를 함께 호출합니다.

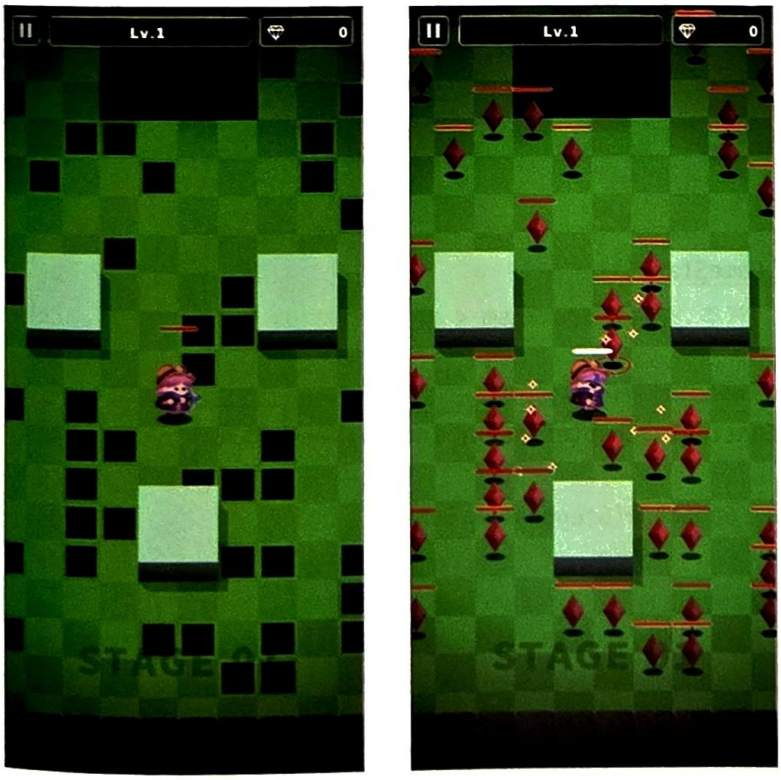
24~31줄 SetupStage() 메서드에서 현재 스테이지 currentStage를 1 늘리고 맵에 출력하는 Text UI 정보를 갱신합니다. 맵에 등장하는 적 숫자는 스테이지가 증가할 때마다 늘어나도록 설정합니다.

03 GameController 오브젝트 Game Controller 컴포넌트의 Text Stage Number 변수에 StageText 오브젝트를 등록합니다.



GameController 오브젝트 Game Controller 컴포넌트의 Text Stage Number 변수 설정하기

04 플레이 버튼 ▶을 눌러 게임을 실행하면 적이 생성될 위치에 타일이 먼저 생성됩니다. 그리고 2초가 지나면 타일이 사라지고 타일 위치에 적이 등장합니다. 현재 스테이지에 있는 모든 적을 물리치면 스테이지 숫자를 1 늘리고 다시 적을 생성합니다.



스테이지+1과 적 생성을 제어하는 결과 화면

Do it! 실습 선택 챕터 저장하고 불러오기 & 챕터 클리어 검사하기

01 Assets 〉 Scripts 〉 Common 폴더에 게임에서 사용하는 상수를 저장하는 Constants.cs 스크립트를 생성합니다.

Assets/Scripts/Common/Constants.cs

코드 12-06 상수 저장

```
01: public static class Constants
02: {
03:     public static readonly string ChapterIndex = "CHAPTERINDEX";
04: }
```

02 〈게임 시작 (-5)〉 버튼을 눌러 게임 씬으로 이동할 때 챕터 인덱스 정보를 저장하도록 LobbySceneController.cs 스크립트를 수정합니다.

Assets/Scripts/LobbyScene/LobbySceneController.cs

코드 12-07 챕터 인덱스 정보 저장

```
01: using UnityEngine;
02:
03: public class LobbySceneController : MonoBehaviour
04: {
        (... 생략 ...)
23:     public void ButtonEvent_GameStart()
24:     {
            (... 생략 ...)
33:         //Logger.Log($"챕터 {index+1} 시작");
34:         PlayerPrefs.SetInt(Constants.ChapterIndex, index);
35:
36:         SceneLoader.Instance.LoadScene(SceneNames.Game);
37:     }
38: }
```

삭제

추가

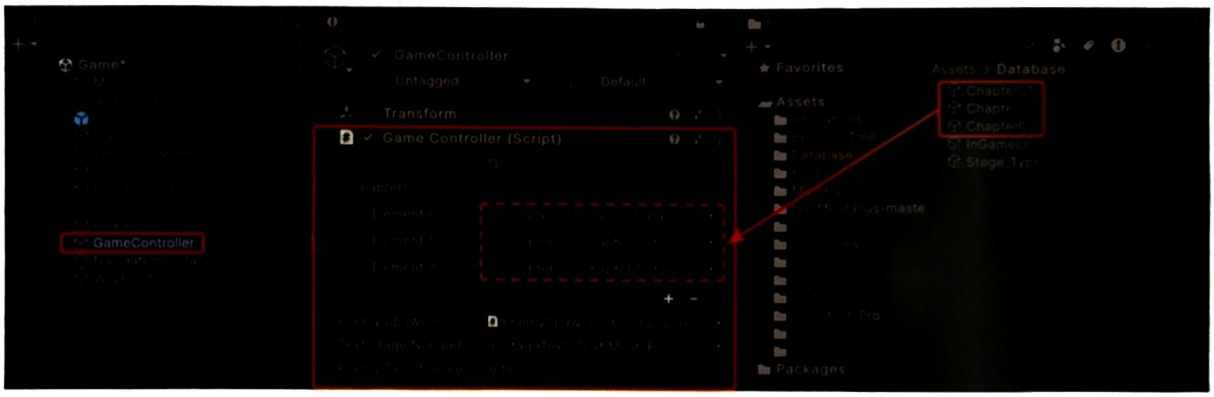
03 현재 챕터에 있는 모든 스테이지를 클리어했는지 검사하고 챕터 클리어 처리를 제어하고자 GameController.cs 스크립트를 수정합니다.

Assets/Scripts/GameSystem/GameController.cs

코드 12-08 챕터 클리어 검사 및 처리

```
    (... 생략 ...)
04: public class GameController : MonoBehaviour
05: {
06:     [SerializeField]                                    추가
07:     private ChapterData[] chapters;
        (... 생략 ...)
15:     private int currentChapter;                         추가
16:     private int maxStage;
17:     private int currentStage = 0;
18:     private int baseEnemyCount = 10;
19:
20:     private void Start()
21:     {
22:         currentChapter = PlayerPrefs.GetInt(Constants.ChapterIndex);    추가
23:         maxStage = chapters[currentChapter].StageDataTable.maxStage;
24:
25:         // 현재 스테이지에 남은 적 숫자가 0이면 다음 스테이지로 넘어가도록 설정
26:         EnemySpawner.exitEvent.AddListener(SetupStage);
27:         // 스테이지 설정, 스테이지에 등장하는 적 생성
28:         SetupStage();
29:     }
30:
31:     public void SetupStage()
32:     {
33:         currentStage ++;
34:
35:         if ( currentStage > maxStage )                  추가
36:         {
37:             Logger.Log("챕터 클리어");
38:             return;
39:         }
40:
41:         // 맵에 출력하는 currentStage Text UI 갱신
42:         textStageNumber.text = $"STAGE {currentStage:D2}";
43:         // 스테이지에 따라 등장하는 적 숫자 연산/생성
44:         enemySpawner.SpawnEnemys((int)(baseEnemyCount +
                currentStage * enemyCountScale));
45:     }
        (... 생략 ...)
51: }
```

04 GameController 오브젝트 Game Controller 컴포넌트의 Chapters 배열 크기를 '3'으로 설정한 후 Chapter01, Chapter02, Chapter03 에셋을 순서대로 등록합니다.



GameController 오브젝트 Game Controller 컴포넌트의 Chapters 배열 변수 설정하기

12-2

JSON 직렬화로 게임 세이브와 로드 구현하기

이 절에서는 게임 데이터를 저장하여 게임이 끝난 후에도 정보를 유지하도록 JSON 형식의 로컬 파일로 게임 세이브/로드 기능을 구현합니다. 플레이어가 죽거나 챕터를 클리어하면 결과 화면을 출력하여 얻은 보석과 경험치를 표시합니다. 이와 함께 게임을 플레이할 때는 하트를 사용하므로 시간 경과에 따라 자동으로 충전되도록 만들어 봅시다.

Do it! 실습 게임 데이터 저장하고 불러오기

유니티에서 게임 재화 데이터를 저장하는 방법은 다음 표처럼 다양하며 각각 장단점이 있습니다. 로비 씬에서 선택한 챕터 정보를 저장하고 게임 씬에서 불러올 때는 PlayerPrefs를 사용하므로 재화 데이터는 JSON 파일로 저장해 봅니다.

게임 재화 데이터를 저장하는 방법과 장단점

저장 방법	보안	데이터 크기	인터넷 사용 여부	추천 용도
PlayerPrefs	X (해킹 쉬움)	소량 (코인, 설정값)	X	캐주얼 게임, 설정값 저장
JSON 파일	X (파일 수정 가능)	중간 (단순한 구조 데이터)	X	로그라이크, 오프라인 RPG
SQLite	O (보안 가능)	대량 (DB 구조 필요 시)	X	인벤토리, 랭킹 저장
Firebase, PlayFab 등	O (보안 강함)	클라우드 (무제한 가능)	O	온라인 게임, 클라우드 백업

01 Assets 〉 Scripts 〉 Database 폴더에 게임 안에서 데이터 저장과 불러오기를 제어하는 Database.cs 스크립트를 생성합니다.

Assets/Scripts/Database/Database.cs

코드 12-09 게임 내 데이터 저장하고 불러오기

```
01: using System.IO;
02: using Newtonsoft.Json;
03: using UnityEngine;
04:
05: public static class Database
06: {
07:     // 플레이어 데이터를 저장하고 불러오는 파일 이름
08:     public static readonly string DBFileName = "Database.dat";
09:     public static readonly int maxChapter = 3;
10:
11:     // 게임에서 사용하는 모든 데이터를 저장하는 클래스
12:     public static DatabaseItem DBItem { get; private set; } = new DatabaseItem();
13:     public static bool IsRead { get; private set; } = false;
14:
15:     /// <summary>
16:     /// 파일에 데이터를 저장한다.
17:     /// </summary>
18:     public static void Write()
19:     {
20:         Logger.Log("Database::Write - Save data in Database");
21:
22:         string path = Path.Combine(Application.persistentDataPath, DBFileName);
23:
24:         // JSON 직렬화
25:         string json = JsonConvert.SerializeObject(DBItem, Formatting.Indented);
26:         // 파일에 데이터 저장
27:         File.WriteAllText(path, json);
28:     }
29:
30:     /// <summary>
31:     /// 파일에서 데이터를 불러온다.
32:     /// </summary>
33:     public static void Read()
34:     {
35:         Logger.Log("Database::Read - Load data in Database");
36:
37:         IsRead = true;
38:
39:         string path = Path.Combine(Application.persistentDataPath, DBFileName);
40:
41:         // 파일이 있다면
42:         if ( File.Exists(path) )
```

```
43:         {
44:             Logger.Log("Database::Read - File Exist in Folder");
45:
46:             // 파일에서 데이터 불러옴
47:             string json = File.ReadAllText(path);
48:             // JSON 역직렬화
49:             DBItem = JsonConvert.DeserializeObject<DatabaseItem>(json);
50:         }
51:         // 파일이 없다면
52:         else
53:         {
54:             Logger.Log("Database::Read - File Not Exist in Folder");
55:             Reset();
56:         }
57:
58:         IsRead = false;
59:     }
60:
61:     public static void Reset()
62:     {
63:         // DBItem이 null이면 메모리 할당
64:         if ( DBItem == null ) DBItem = new DatabaseItem();
65:         // 전체 데이터 초기화
66:         DBItem.Reset();
67:         // 파일에 초기화된 데이터 저장
68:         Write();
69:     }
70: }
71:
72: [System.Serializable]
73: public class DatabaseItem
74: {
75:     public DBItem_Player player;
76:     public DBItem_Goods goods;
77:     public DBItem_Chapter[] chapters;
78:
79:     public DatabaseItem()
80:     {
81:         player = new DBItem_Player();
82:         goods = new DBItem_Goods();
83:         chapters = new DBItem_Chapter[Database.maxChapter];
84:         for ( int i = 0; i < chapters.Length; ++i)
85:         {
86:             chapters[i] = new DBItem_Chapter();
```

```
87:         }
88:     }
89:
90:     public void Reset()
91:     {
92:         player.Reset();
93:         goods.Reset();
94:
95:         for ( int i = 0; i < chapters.Length; ++ i )
96:         {
97:             chapters[i].Reset();
98:         }
99:         // 첫 번째 챕터 풀림 여부를 true로 설정
100:        chapters[0].isUnlock = true;
101:    }
102: }
103:
104: [System.Serializable]
105: public class DBItem_Player
106: {
107:    public int level;
108:    public float experience;
109:
110:    public void Reset()
111:    {
112:        level = 1;
113:        experience = 0f;
114:    }
115: }
116:
117: [System.Serializable]
118: public class DBItem_Goods
119: {
120:    public int heart;
121:    public float heartTimer;     // 하트 충전까지 남은 시간(초)
122:    public string heartLastTime; // 게임 종료 시간
123:    public float gem;
124:
125:    public readonly int maxHeart = 50;                  // 최대 하트 개수
126:    public readonly float heartRefillTime = 20 * 60; // 하트 회복 시간(초): 20분
127:
128:    public void Reset()
129:    {
130:        heart = maxHeart;
```

```
131:         heartTimer = 0f;
132:         heartLastTime = string.Empty;
133:         gem = 0;
134:     }
135: }
136:
137: [System.Serializable]
138: public class DBItem_Chapter
139: {
140:     public bool isUnlock; // 챕터 잠금 해제 여부
141:     public int bestStage; // 현재 챕터에서 도달한 최고 스테이지
142:
143:     public void Reset()
144:     {
145:         isUnlock = false;
146:         bestStage = 1;
147:     }
148: }
```

05줄 Database 클래스는 부모 클래스를 상속받지 않고 다른 클래스에서 자유롭게 접근해 사용할 수 있도록 정적 클래스로 정의합니다.

07~09줄 데이터를 저장하고 불러올 파일 이름인 DBFileName 변수와 현재 게임의 최대 챕터 숫자인 maxChapter 변수를 읽기 전용으로 선언합니다.

11~13줄 게임에서 사용하는 모든 데이터를 저장하는 DBItem 속성과 현재 데이터 불러오기 여부를 저장하는 IsRead 속성을 정의합니다.

18~28줄 DatabaseItem 클래스 인스턴스에 있는 데이터를 파일로 저장하는 Write() 메서드는 Console View에 Write() 메서드 호출 정보를 출력합니다. Application.persistentDataPath 경로에 DBFileName 파일 이름 정보를 path에 저장합니다. DBItem 클래스에 저장한 데이터를 직렬화해 문자열 변수 json에 저장하고 path 파일에 json 데이터를 저장합니다.

33~59줄 파일에 저장된 데이터를 불러와 DatabaseItem 클래스 인스턴스에 저장하는 Read() 메서드는 Console View에 메서드를 호출했다는 텍스트를 출력합니다. 파일에서 데이터를 불러오는 동안 IsRead를 true로 설정합니다. Application.persistentDataPath 경로에 DBFileName 파일 이름 정보를 path에 저장합니다. path 파일이 있는지 검사하고, 파일이 있다면 Console View에 텍스트를 출력합니다. path 파일에서 데이터를 불러와 json 변수에 저장합니다. 그리고 json 데이터를 역직렬화해 DatabaseItem형으로 변

환하고 DBItem 속성에 저장합니다. 파일이 없다면 Console View에 텍스트를 출력하고 Reset() 메서드를 호출합니다. 데이터 불러오기가 끝나면 IsRead를 false로 설정합니다.

61~69줄 Reset() 메서드는 DBItem이 비었으면 메모리를 할당합니다. DBItem.Reset() 메서드를 호출해 player, goods, chapters 정보를 초기화합니다. Write() 메서드를 호출해 파일을 생성하고 초기화한 정보를 저장합니다.

72~102줄 게임 안에서 저장/불러오기를 해야 하는 플레이어 DBItem_Player, 재화 DBItem_Goods, 챕터 정보 DBItem_Chapter를 묶어서 관리하는 DatabaseItem 클래스를 정의합니다.

104~115줄 로비 씬에 출력하는 플레이어 레벨과 경험치 정보를 관리하는 DBItem_Player 클래스를 정의합니다.

117~135줄 플레이어의 재화로 사용하는 하트와 보석 정보를 관리하는 DBItem_Goods 클래스를 정의합니다.

137~148줄 각 챕터의 잠금 해제 여부와 해당 챕터에서 도달한 최고 스테이지 정보를 관리하는 DBItem_Chapter 클래스를 정의합니다.

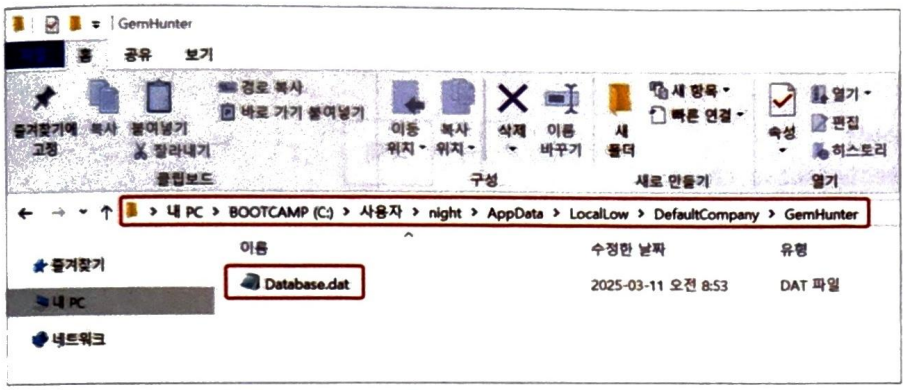
알아 두면 좋아요! Newtonsoft.Json 이름공간이 없다는 오류 메시지가 보이면?

Newtonsoft.Json 패키지가 없어서 일어나는 문제입니다. 먼저 [Window → Package Management → Package Manager] 메뉴로 Package Manager View를 엽니다. 그리고 왼쪽 위 + ▾ 버튼을 클릭하고 [Install package from git URL...] 메뉴를 선택한 후 'com.unity.nuget.newtonsoft-json'을 입력하고 <Install> 버튼을 클릭하여 설치합니다.

패키지 관리자로 Newtonsoft.Json 설치하기

설치가 끝나면 앞서 작성한 Database.cs 스크립트 25줄과 49줄처럼 클래스 내용을 파일 형태로 저장할 수 있도록 직렬화하거나 역직렬화하는 JsonConvert 클래스를 사용할 수 있습니다(Newtonsoft.Json 이름공간 안에 JsonConvert가 있음).

02 여기서 Application.persistentDataPath는 PC라면 'C:/Users/사용자 이름/AppData/LocalLow/회사 이름/프로젝트 이름/' 경로를, 안드로이드라면 'Android/data/com.프로젝트 이름.회사 이름/files/' 경로를 가리킵니다.



파일 저장 위치 Application.persistentDataPath

03 인트로 씬을 시작할 때 Database.Read() 메서드를 호출해 데이터를 불러오고자 IntroSceneController.cs 스크립트를 수정합니다.

Assets/Scripts/IntroScene/IntroSceneController.cs

코드 12-10 인트로 씬을 시작할 때 게임 재화 데이터 불러오기

```
    (... 생략 ...)
05: public class IntroSceneController : MonoBehaviour
06: {
        (... 생략 ...)
12:     private void Awake()        ← 추가
13:     {
14:         Database.Read();
15:     }
        (... 생략 ...)
34: }
```

04 해당 챕터가 풀렸는지를 나타내는 isUnlock과 도달한 최고 스테이지 bestStage는 플레이에 따라 계속 변하는 런타임 데이터입니다. 그러므로 해당 데이터는 Database로 관리하고 ChapterDatabase 구조체는 삭제하도록 ChapterData.cs 스크립트를 수정합니다.

Assets/Scripts/Database/ChapterData.cs

코드 12-11 ChapterDatabase 구조체 삭제

```
01: using UnityEngine;
02:
03: [CreateAssetMenu]
04: public class ChapterData : ScriptableObject
05: {
06:     //[SerializeField]
07:     //private ChapterDatabase chapterDatabase;
08:     [SerializeField]
09:     private ChapterDataTable chapterDataTable;
10:     [SerializeField]
11:     private StageDataTable stageDataTable;
12:
13:     //public ChapterDatabase ChapterDatabase => chapterDatabase;
14:     public ChapterDataTable ChapterDataTable => chapterDataTable;
15:     public StageDataTable StageDataTable => stageDataTable;
16: }
17:
18: /*[System.Serializable]
19: public struct ChapterDatabase
20: {
21:     public bool isUnlock; // 챕터 플림 여부
22:     public int bestStage; // 최고 스테이지
23: }*/
    (... 생략 ...)
```

삭제

삭제

삭제

05 챕터 플림 여부와 도달 스테이지 정보를 Database에서 불러오고자 UIChapterIcon.cs 스크립트를 수정합니다.

Assets/Scripts/LobbyScene/UIChapterIcon.cs

코드 12-12 챕터 플림 여부와 도달 스테이지 정보 설정

```
    (... 생략 ...)
05: public class UIChapterIcon : MonoBehaviour
06: {
        (... 생략 ...)
16:     public void Setup(int index, ChapterData chapterData)
17:     {
18:         lockedIcon.SetActive(!Database.DBItem.chapters[index].isUnlock);
19:
20:         imageChapter.sprite = chapterData.ChapterDataTable.spriteChapter;
```

수정

```
21:         // Debug.. 챕터 이미지가 모두 있을 땐 색상 정보는 필요 없음
22:         imageChapter.color = chapterData.ChapterDataTable.colorChapter;
23:         textChapterName.text =
                $"#{index+1:D2} {chapterData.ChapterDataTable.chapterName}";
24:         textStage.text = $"스테이지 {Database.DBItem.chapters[index].bestStage}/
                {chapterData.StageDataTable.maxStage}";
25:     }
26: }
```

수정

06 챕터 풀림 여부 정보는 Database에서 불러오도록 LobbySceneController.cs 스크립트를 수정합니다.

Assets/Scripts/LobbyScene/LobbySceneController.cs

코드 12-13 챕터 풀림 여부 확인

```
01: using UnityEngine;
02:
03: public class LobbySceneController : MonoBehaviour
04: {
        (... 생략 ...)
23:     public void ButtonEvent_GameStart()
24:     {
25:         int index = swipeUI.CurrentPage;
26:
27:         if ( Database.DBItem.chapters[index].isUnlock == false )
28:         {
29:             Logger.Log("잠긴 챕터입니다.");
30:             return;
31:         }
            (... 생략 ...)
36:     }
37: }
```

수정

Do it! 실습 게임 오버, 챕터 클리어, 보상 구현하기

01 Assets 〉 Scripts 〉 UI 폴더에 보상 아이콘 정보 출력을 제어하는 UIRewardIcon.cs 스크립트를 생성합니다.

Assets/Scripts/UI/UIRewardIcon.cs

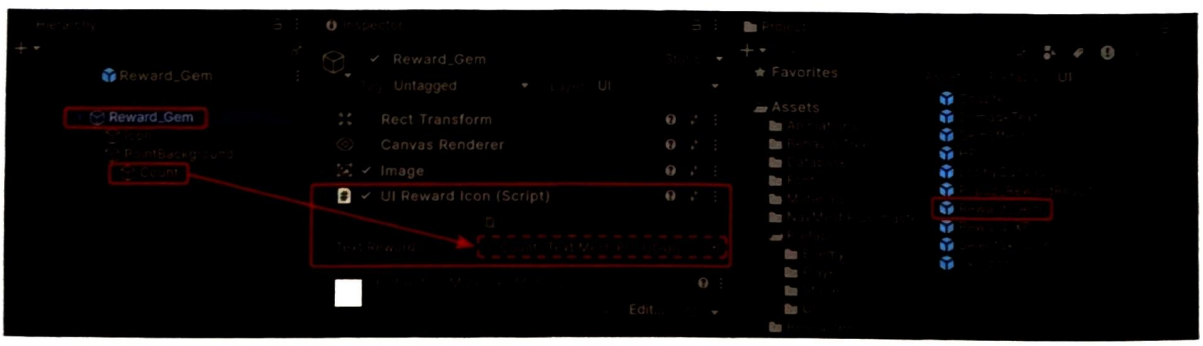
코드 12-14 보상 아이콘 정보 출력 제어

```
01: using UnityEngine;
02: using TMPro;
03:
04: public class UIRewardIcon : MonoBehaviour
05: {
06:     [SerializeField]
07:     private TextMeshProUGUI textReward;
08:
09:     public void SetReward(long reward)
10:     {
11:         textReward.text = reward.ToString();
12:     }
13: }
```

09~12줄 플레이어의 체력이 0이 되어 게임 오버가 되거나 챕터를 클리어했을 때 경험치, 보석 등 보상을 얻습니다. 이때 얻는 보상의 양을 SetReward() 메서드로 UI에 출력합니다.

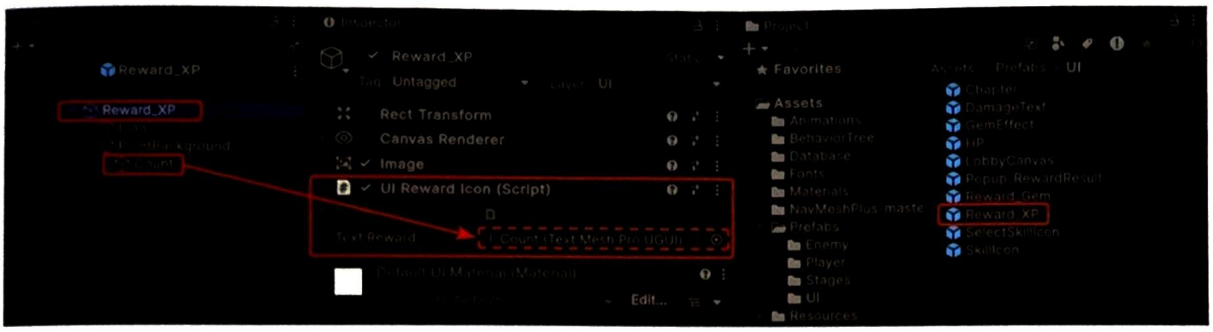
02 Reward_Gem 프리팹에 UI Reward Icon 컴포넌트를 추가하고 Text Reward 변수에 Count 오브젝트를 등록합니다.

Reward_Gem, Reward_XP, Popup_RewardResult 프리팹은 자료실에서 내려받은 패키지 안에 있습니다.



Reward_Gem 프리팹에 UI Reward Icon 컴포넌트 추가하고 설정하기

03 Reward_XP 프리팹에 UI Reward Icon 컴포넌트를 추가하고 Text Reward 변수에 Count 오브젝트를 등록합니다.



Reward_XP 프리팹에 UI Reward Icon 컴포넌트 추가하고 설정하기

04 Assets 〉 Scripts 〉 UI 폴더에 게임 결과 화면 UI를 제어하는 UIRewardResult.cs 스크립트를 생성합니다.

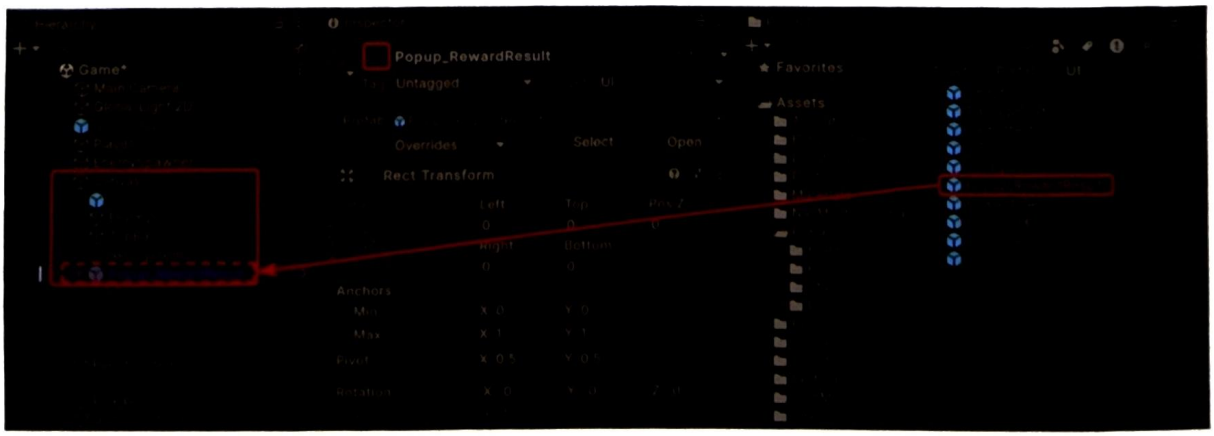
Assets/Scripts/UI/UIRewardResult.cs

코드 12-15 게임 결과 화면 UI 제어

```
01: using UnityEngine;
02: using TMPro;
03:
04: public enum RewardType { GEM, EXP, ITEM }
05:
06: public class UIRewardResult : MonoBehaviour
07: {
08:     [SerializeField]
09:     private GameController gameController;
10:     [SerializeField]
11:     private GameObject panelResult;
12:     [SerializeField]
13:     private GameObject textNewRecord;
14:     [SerializeField]
15:     private TextMeshProUGUI textTheme;
16:     [SerializeField]
17:     private TextMeshProUGUI textChapter;
18:     [SerializeField]
19:     private TextMeshProUGUI textStage;
20:     [SerializeField]
21:     private Transform rewardParent;
22:     [SerializeField]
23:     private UIRewardIcon[] rewards;
24:
```

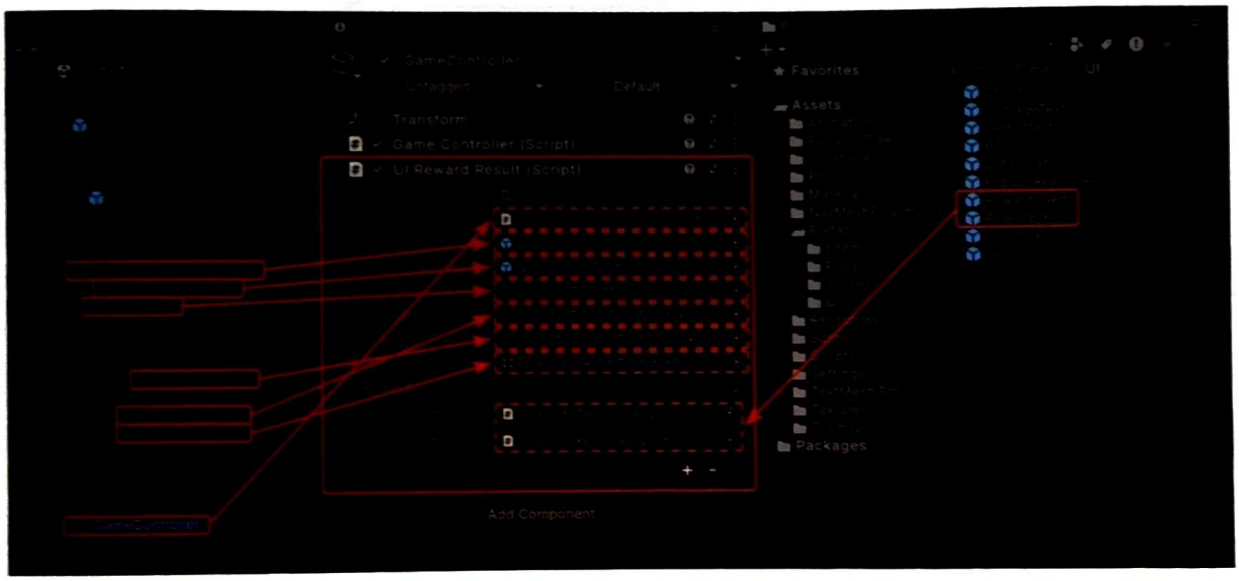
```
25:     public void OnRewardResult(bool isNewRecord, bool isClear,
            int chapter, int stage, (RewardType, long)[] items)
26:     {
27:         panelResult.SetActive(true);
28:         textNewRecord.SetActive(isNewRecord);
29:
30:         if ( isClear == true ) textTheme.text = "챕터 클리어";
31:         else textTheme.text = "게임 오버";
32:
33:         textChapter.text = $"CHAPTER {chapter+1:D2}";
34:         textStage.text = stage.ToString();
35:
36:         UIRewardIcon item;
37:         for ( int i = 0; i < items.Length; ++ i )
38:         {
39:             item = Instantiate(rewards[(int)items[i].Item1], rewardParent);
40:             item.SetReward(items[i].Item2);
41:         }
42:     }
43:
44:     public void ButtonEvent_ReturnLobby()
45:     {
46:         gameController.SetTimeScale(1);
47:         SceneLoader.Instance.LoadScene(SceneNames.Lobby);
48:     }
49: }
```

05 미리 제작한 게임 결과 화면 UI는 다른 UI보다 앞에 출력되도록 설정합니다. Popup_RewardResult 프리팹을 Hierarchy View에 있는 Canvas 오브젝트의 마지막 자식으로 배치합니다. 그리고 게임이 끝날 때만 표시하도록 비활성화합니다.



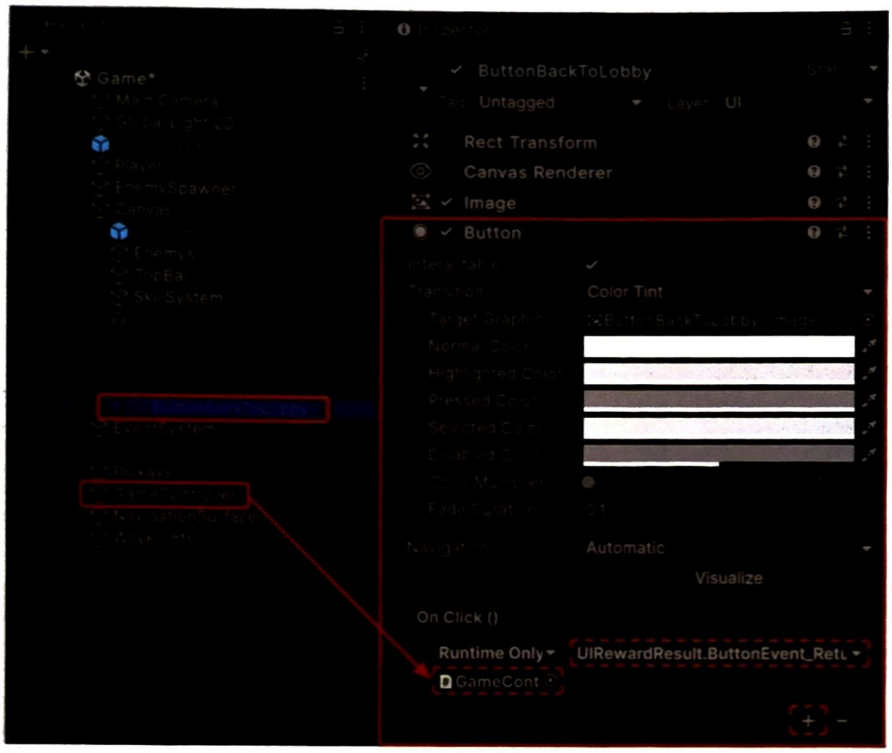
Popup_RewardResult 프리팹을 Hierarchy View로 끌어서 놓기

06 GameController 오브젝트에 UI Reward Result 컴포넌트를 추가하고 Game Controller 변수에 GameController 오브젝트를, Panel Result 변수에 Popup_RewardResult 오브젝트를, Text New Record 변수에 NewRecordText 오브젝트를 등록합니다. 이어서 Text Theme 변수에 Theme 오브젝트를, Text Chapter 변수에 ChapterText 오브젝트를, Text Stage 변수에 StageNumber 오브젝트를, Reward Parent 변수에 Rewards 오브젝트를 등록합니다. Rewards 배열 크기는 '2'로 설정하고 Reward_Gem과 Reward_XP 프리팹을 등록합니다.



GameController 오브젝트에 UI Reward Result 컴포넌트 추가하고 설정하기

07 ButtonBackToLobby 오브젝트의 Button 컴포넌트에서 On Click () 리스트에 요소를 추가하고 GameController 오브젝트의 UI Reward Result 컴포넌트 ButtonEvent_ReturnLobby() 메서드를 등록합니다.



ButtonBackToLobby 오브젝트의 Button 컴포넌트에 On Click () 이벤트 등록하기

08 게임이 끝나거나 챕터를 클리어했을 때 호출하는 **GameOver()**, **GameClear()** 메서드를 정의하고 데이터베이스에 재화 정보를 저장하고자 GameController.cs 스크립트를 수정합니다.

Assets/Scripts/GameSystem/GameController.cs

코드 12-16 GameOver(), GameClear(), 재화 저장

```
    (... 생략 ...)
04: public class GameController : MonoBehaviour
05: {
        (... 생략 ...)
10:     [SerializeField]
11:     private TextMeshPro textStageNumber;
12:     [SerializeField]                                  추가
13:     private UIRewardResult uiRewardResult;
14:     [SerializeField]
15:     private PlayerBase player;
        (... 생략 ...)
34:
35:     public void SetupStage()
36:     {
37:         currentStage ++;
```

```
38:
39:         if ( currentStage > maxStage )
40:         {
41:             //Logger.Log("챕터 클리어");
42:             GameClear();
43:             return;
44:         }
            (... 생략 ...)
50:     }
            (... 생략 ...)
57:     public void GameClear()
58:     {
59:         SetTimeScale(0);
60:
61:         long baseExp = (long)(currentStage * (5 + (currentChapter+1) * 1.2f));
62:         long bonusExp = (long)Mathf.Pow(2, (currentChapter+1)) * 100;
63:         long bonusGem = (currentChapter+1) * 5000;
64:         bool isNewRecord
                = Database.DBItem.chapters[currentChapter].bestStage != maxStage;
65:
66:         // Database 클래스의 DBItem에 데이터 저장
67:         Database.DBItem.player.experience += (baseExp + bonusExp);
68:         Database.DBItem.goods.gem += (player.GEM + bonusGem);
69:         Database.DBItem.chapters[currentChapter].bestStage = maxStage;
70:
71:         if ( currentChapter+1 < Database.DBItem.chapters.Length )
72:             Database.DBItem.chapters[currentChapter+1].isUnlock = true;
73:
74:         Database.Write(); // DBItem에 저장된 데이터를 파일에 저장
75:
76:         // 새 기록 여부, 클리어 여부, 챕터, 스테이지, 보상 정보 전달
               (보상은 원하는 개수만큼 추가 가능)
77:         uiRewardResult.OnRewardResult(isNewRecord, true, currentChapter, maxStage,
78:             new (RewardType, long)[] {
                    (RewardType.GEM, player.GEM), (RewardType.EXP, baseExp),
                    (RewardType.GEM, bonusGem), (RewardType.EXP, bonusExp) });
79:
80:     }
81:
82:     public void GameOver()
83:     {
84:         SetTimeScale(0);
85:
86:         long exp = (long)(currentStage * (5 + (currentChapter+1) * 1.2f));
```

삭제

추가

추가

```
87:         bool isNewRecord
                = Database.DBItem.chapters[currentChapter].bestStage < currentStage;
88:
89:         // Database 클래스의 DBItem에 데이터 저장
90:         Database.DBItem.player.experience += exp;
91:         Database.DBItem.goods.gem += player.GEM;
92:         if ( isNewRecord )
                Database.DBItem.chapters[currentChapter].bestStage = currentStage;
93:
94:         Database.Write(); // DBItem에 저장된 데이터를 파일에 저장
95:
96:         // 새 기록 여부, 챕터, 스테이지, 보상 정보 전달
              (보상은 원하는 개수만큼 추가 가능)
97:         uiRewardResult.OnRewardResult(isNewRecord, false, currentChapter,
              currentStage,
98:             new (RewardType, long)[] {
                  (RewardType.GEM, player.GEM), (RewardType.EXP, exp) });
99:     }
100: }
```

게임을 클리어 했을 때 호출하는 GameClear() 메서드는 SetTimeScale(0) 메서드를 호출해 게임 속도를 0으로 설정하고 게임을 멈춥니다. 게임을 클리어 했을 때는 기본 경험치, 보석과 함께 클리어 보너스 경험치, 보석 재화를 제공합니다. 기본 보석은 스테이지를 플레이하며 수집한 player.GEM 정보를 사용합니다.

61~63줄 기본 경험치 baseExp, 보너스 경험치 bonusExp, 보너스 보석 bonusGem은 수식으로 계산합니다.

64줄 DBItem에 저장된 현재 챕터의 최고 스테이지 정보를 불러와 신기록인지를 판단해 저장합니다.

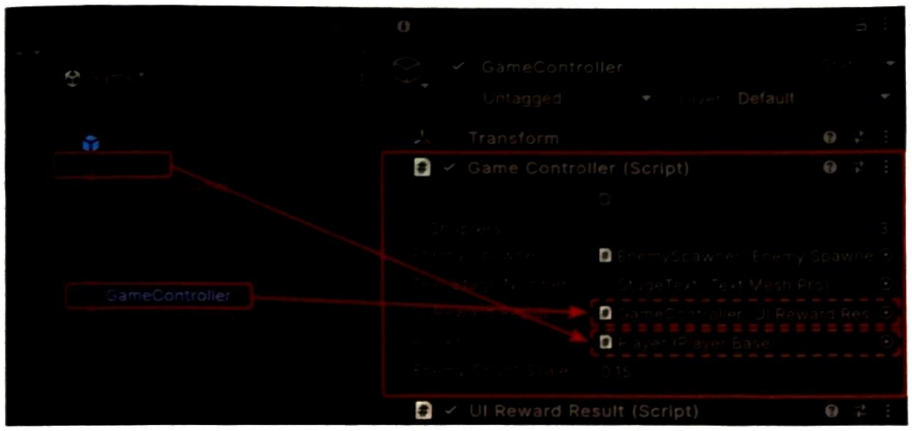
66~69줄 데이터를 저장하고자 DBItem에 경험치, 보석, 최고 스테이지 정보를 갱신합니다.

71~72줄 챕터를 클리어했으므로 다음 챕터가 있으면 isUnlock = true로 잠금을 해제합니다.

74줄 DBItem에 저장된 데이터를 파일로 저장합니다.

76~79줄 챕터, 스테이지, 보상 정보를 게임 결과 화면 UI에 출력하도록 uiRewardResult.OnRewardResult() 메서드를 호출합니다. 지금은 GameClear()에서 4개의 보상, GameOver()에서 2개의 보상을 사용하는데, 원하는 개수만큼 메서드 매개변수로 추가하면 자동으로 적용됩니다.

09 GameController 오브젝트 Game Controller 컴포넌트의 UI Reward Result 변수에는 GameController 오브젝트를, Player 변수에는 Player 오브젝트를 등록합니다.



GameController 오브젝트의 Game Controller 컴포넌트 변수 설정하기

10 플레이어의 체력이 0이 되면 GameController.GameOver() 메서드를 호출하고자 PlayerBase.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerBase.cs

코드 12-17 플레이어 체력이 0이 되면 게임 오버 처리

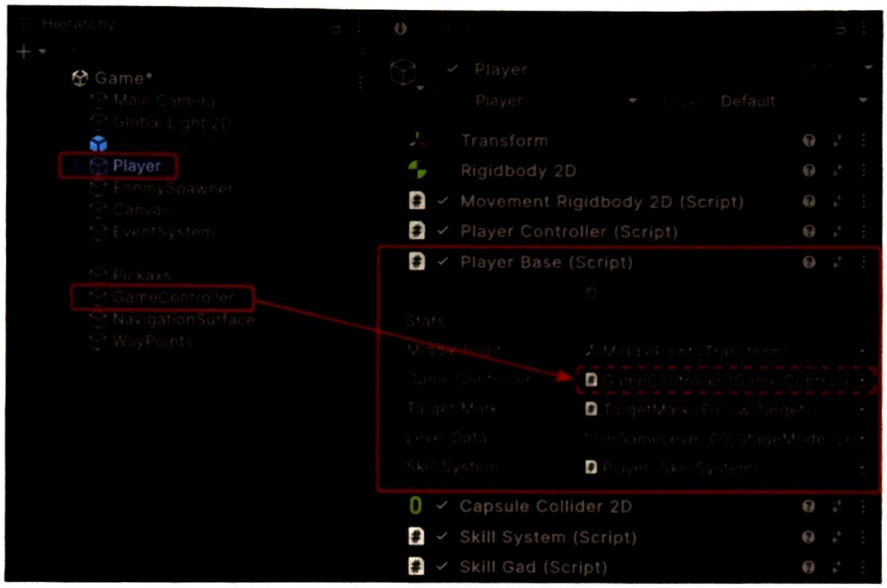
```
01: using UnityEngine;
02:
03: public class PlayerBase : EntityBase
04: {
05:     [SerializeField]
06:     private GameController gameController;
        (... 생략 ...)
42:     protected override void OnDie()
43:     {
44:         //Logger.Log("플레이어 사망");
45:         gameController.GameOver();
46:     }
        (... 생략 ...)
114: }
```

추가

삭제

추가

11 Player 오브젝트 Player Base 컴포넌트의 Game Controller 변수에 GameController 오브젝트를 등록합니다.



Player 오브젝트의 Player Base 컴포넌트 변수 설정하기

Do it! 실습 로비 UI와 게임 데이터 연동하기

01 Assets 〉 Scripts 〉 Common 폴더에 큰 단위 숫자를 직접 출력하는 대신 '숫자+심벌' 형태로 축약해서 출력하는 NotateNumber.cs 스크립트를 생성합니다.

Assets/Scripts/Common/NotateNumber.cs

코드 12-18 숫자+심벌 출력 제어

```
01: public static class NotateNumber
02: {
03:     public static string Transform(long originNumber)
04:     {
05:         // 1,000=1K, 1,000,000=1M
06:         string[] symbol = new string[7]{ "K", "M", "G", "T", "P", "E", "Z" };
07:
08:         // 매개변수로 받은 숫자를 문자열로 변환
09:         string result = originNumber.ToString();
10:
11:         // 숫자+심벌은 최대 4자리까지 표기, 4자리 이하면 심벌 없이 출력
12:         if ( result.Length < 4 )
13:         {
14:             return result;
```

```
        }

        for ( int i = 0; i < symbol.Length; ++ i )
        {
            if ( 4 + 3 * i <= result.Length && result.Length < 4 + 3 * (i + 1) )
            {
                // 3으로 나누므로 나머지(n)는 0, 1, 2
                int n = result.Length % 3;
                // 나머지(n)가 0이면 3으로 설정
                n = n == 0 ? 3 : n;

                // 나머지 개수(n) = 앞자리 개수
                // 앞자리 표현에 사용한 값 바로 뒤의 값을 소수점 이하 숫자로 표현
                result = $"{result.Substring(0, n)}.{result.Substring(n, 1)}";
                // 축약 기호 추가
                result += symbol[i];
                break;
            }
        }

        return result;
    }
}
```

02 Assets 〉 Scripts 〉 LobbyScene 폴더에 Database에 저장된 플레이어와 재화 정보를 바탕으로 Lobby UI를 출력하는 UILobbyTopBar.cs 스크립트를 생성합니다.

Assets/Scripts/LobbyScene/UILobbyTopBar.cs

코드 12-19 플레이어와 재화 UI 출력

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class UILobbyTopBar : MonoBehaviour
{
    [SerializeField]
    private LevelData levelData;
    [SerializeField]
    private TextMeshProUGUI textLevel;
    [SerializeField]
    private Slider fillGaugeEXP;
```

```
    [SerializeField]
    private TextMeshProUGUI textHeart;
    [SerializeField]
    private TextMeshProUGUI textHeartTimer;
    [SerializeField]
    private TextMeshProUGUI textGEMCount;

    private void Awake()
    {
        // 현재는 로비에서 재화를 사용하지 않으므로 로비 씬을 로드할 때 1회만 호출
        // 일반적으로는 Stat.cs처럼 재화에 delegate, event를 설정하고
        // 값이 변경될 때마다 호출하도록 설정해서 사용

        // 레벨, 경험치 계산과 출력
        UpdateLevel();
        // 보유 보석 출력
        textGEMCount.text = NotateNumber.Transform((long)Database.DBItem.goods.gem);
    }

    private void UpdateLevel()
    {
        int level = Database.DBItem.player.level;

        // 경험치가 최대이면 레벨업
        while ( Database.DBItem.player.experience >= levelData.MaxExperience[level-1] )
        {
            if ( level > levelData.MaxExperience.Length )
                level = levelData.MaxExperience.Length;

            Database.DBItem.player.experience -= levelData.MaxExperience[level-1];
            Database.DBItem.player.level ++;
        }
        Database.Write();

        fillGaugeEXP.value =
            Database.DBItem.player.experience / levelData.MaxExperience[level-1];
        textLevel.text = Database.DBItem.player.level.ToString();
    }

    public void UpdateHeart(int current, int max)
    {
        textHeart.text = $"{current}/{max}";
    }

```

```
55:     public void UpdateHeartTimer(string text)
56:     {
57:         textHeartTimer.text = text;
58:     }
59: }
```

26~27줄 Awake() 메서드에서 UpdateLevel() 메서드를 호출해 플레이어 레벨과 경험치를 출력합니다.

28~29줄 플레이어가 지닌 보석 정보 Database.DBItem.goods.gem을 그대로 출력하면 '10,000'으로 출력하고 NotateNumber.Transform() 메서드를 이용하면 '10K'처럼 축약 형태로 출력합니다.

34줄 DBItem에서 불러온 플레이어 레벨 정보를 level 변수에 저장합니다.

37줄 플레이어의 현재 경험치가 현재 레벨의 최대 경험치보다 크거나 같으면 레벨업 합니다.

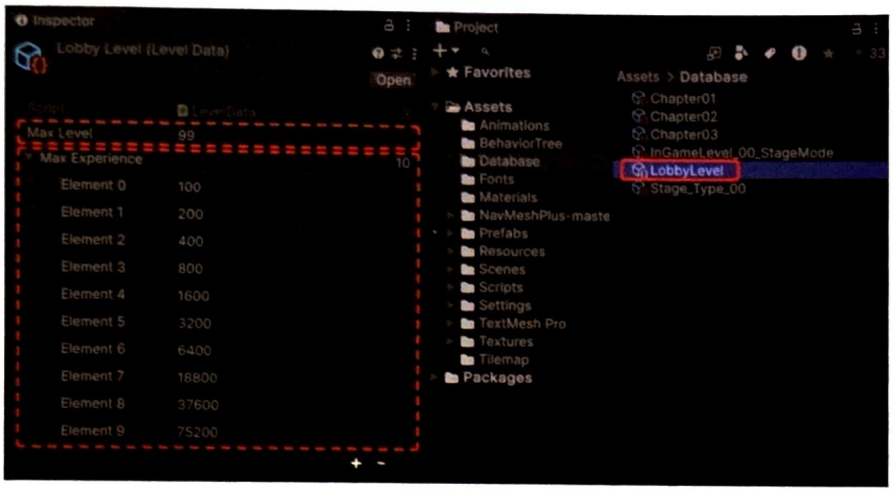
39줄 현재 LevelData에 레벨별 최대 경험치 배열 크기는 10입니다. 즉, 10레벨까지는 테이블에 있는 값을 사용하고 그 이상 레벨업 할 때는 마지막 레벨인 10레벨의 테이블값을 사용하도록 설정합니다.

41~42줄 방금 레벨업에 소모한 경험치만큼 줄이고 현재 레벨을 1 늘립니다.

44줄 레벨과 경험치 정보가 변경되었으므로 파일로 데이터를 저장합니다.

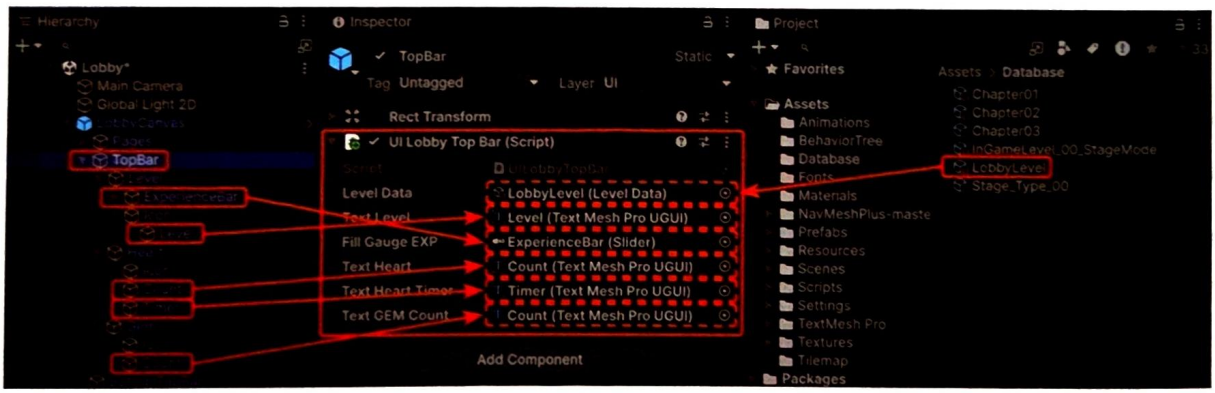
46~47줄 로비 씬 화면에 레벨, 경험치 UI를 출력합니다.

03 Project View의 ＋▾ 버튼을 클릭해 [Level Data] 메뉴로 로비 씬에서 사용하는 플레이어 레벨과 경험치 데이터를 Assets 〉 Database 폴더에 생성하고 이름을 'LobbyLevel'로 변경합니다. Max Level은 '99'로, Max Experience 배열 크기는 '10'으로 설정합니다. 요소는 각각 '100', '200', '400', '800', '1600', '3200', '6400', '18800', '37600', '75200'으로 설정합니다.



로비 씬에서 사용하는 플레이어 레벨과 경험치 데이터 생성하고 설정하기

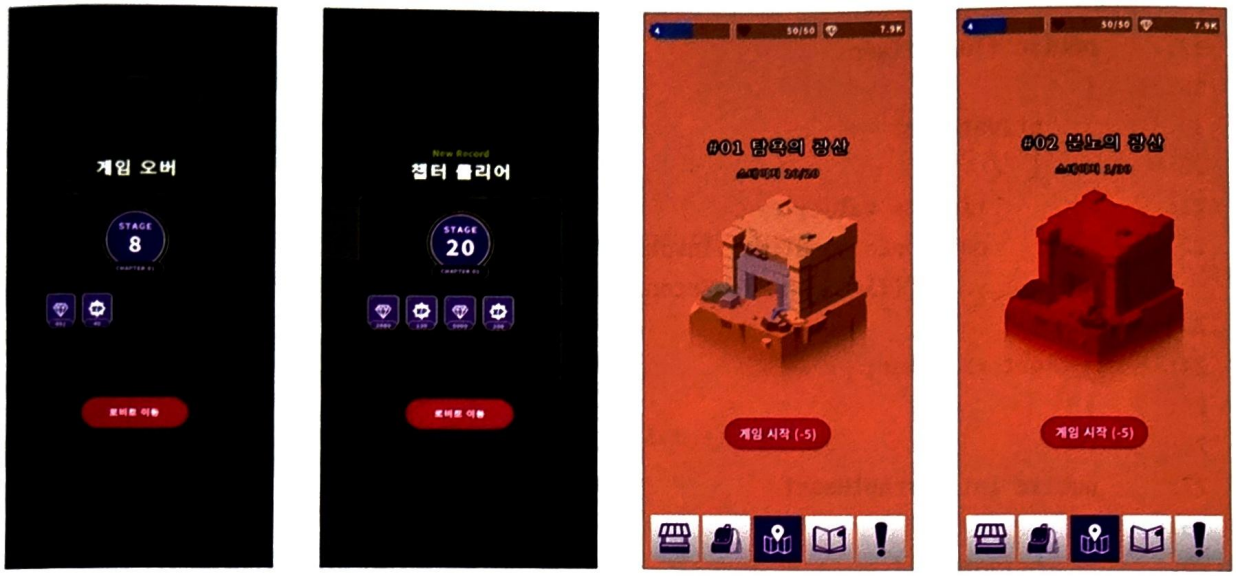
04 로비 씬의 TopBar 오브젝트에 UI Lobby Top Bar 컴포넌트를 추가한 후 Level Data 변수에는 LobbyLevel 에셋을, Text Level 변수에는 Icon 오브젝트의 자식인 Level 오브젝트를, Fill Gauge EXP 변수에는 ExperienceBar 오브젝트를, Text Heart 변수에는 Heart 오브젝트의 자식인 Count 오브젝트를, Text Heart Timer 변수에는 Heart 오브젝트의 자식인 Timer 오브젝트를, Text GEM Count 변수에는 Gem 오브젝트의 자식인 Count 오브젝트를 등록합니다.



TopBar 오브젝트에 UI Lobby Top Bar 컴포넌트 추가하고 설정하기

05 인트로 씬을 활성화하고 플레이 버튼 ▶ 을 눌러 게임을 실행합니다. 게임이 끝나면 도달한 스테이지 정보와 수집한 보석, 경험치 정보를 출력합니다. 챕터를 클리어하면 도달한 스테이지 정보와 수집한 보석, 경험치, 챕터 클리어 보상인 보석, 경험치를 추가로 출력합니다. 또한 처음 도달한 스테이지라면 'New Record' 텍스트를 출력합니다. 로비 씬에서는 도달한 스테이지 정보를 출력하고 챕터를 클리어하면 다음 챕터의 잠금을 풉니다.

● 한 번에 생긴 적을 모두 물리치면 1 스테이지 클리어입니다.



게임이 끝나거나 챕터 클리어 시 보상 표시와 로비 씬에 게임 데이터를 표시한 결과 화면

다른 씬에서 게임 실행 후 오류가 발생한다면 Database.dat 파일을 삭제하고 다시 실행합니다.

Do it! 실습 하트 사용하고 자동 충전하기

01 Assets 〉 Scripts 〉 Common 폴더에 게임을 플레이할 때마다 사라지고 시간이 흐르면 자동 충전되는 하트를 제어하는 HeartSystem.cs 스크립트를 생성합니다.

Assets/Scripts/Common/HeartSystem.cs

코드 12-20 게임 플레이에 필요한 하트 사용과 충전

```
01: using UnityEngine;
02: using UnityEngine.Events;
03: using System;
04:
05: public class HeartSystem : MonoBehaviour
06: {
07:     [SerializeField]
08:     private UnityEvent<int, int> onValueChangedHeart; // 하트 수가 바뀔 때마다 호출
09:     [SerializeField]
10:     private UnityEvent<string> onValueChangedTimer; // 충전 시간이 바뀔 때마다 호출
11:
12:     private int maxHeart;     // 최대 하트 개수
13:     private int currentHeart; // 현재 보유한 하트 개수
14:     private float timer;      // 다음 하트 회복까지 남은 시간(초)
15:     private float refillTime; // 하트 회복 시간(초)
16:
```

```
17:     public float Timer
18:     {
19:         private set
20:         {
21:             timer = value;
22:             onValueChangedTimer.Invoke((CurrentHeart < maxHeart) ?
                    $"{TimeSpan.FromSeconds(timer):mm\\:ss}" : "FULL");
23:         }
24:         get => timer;
25:     }
26:
27:     public int CurrentHeart
28:     {
29:         private set
30:         {
31:             currentHeart = value;
32:             onValueChangedHeart.Invoke(currentHeart, maxHeart);
33:         }
34:         get => currentHeart;
35:     }
36:
37:     private void Awake()
38:     {
39:         maxHeart = Database.DBItem.goods.maxHeart;
40:         refillTime = Database.DBItem.goods.heartRefillTime;
41:
42:         LoadData();
43:     }
44:
45:     private void Update()
46:     {
47:         if ( CurrentHeart < maxHeart )
48:         {
49:             Timer -= Time.deltaTime;
50:
51:             // 충전 시간 도달 시
52:             if ( Timer <= 0f )
53:             {
54:                 CurrentHeart ++;
55:
56:                 // 회복할 하트가 남았으면 Timer를 refillTime인 1200으로 초기화
57:                 if ( CurrentHeart < maxHeart ) Timer = refillTime;
58:
59:                 // 하트, 시간 정보 저장
```

```
60:                 SaveData();
61:             }
62:         }
63:     }
64:
65:     private void OnApplicationPause(bool pause)
66:     {
67:         if ( pause ) SaveData();
68:     }
69:
70:     private void OnApplicationFocus(bool focus)
71:     {
72:         if ( focus ) LoadData();
73:     }
74:
75:     private void OnApplicationQuit()
76:     {
77:         SaveData();
78:     }
79:
80:     public bool UseHeart(int count)
81:     {
82:         if ( CurrentHeart >= count )
83:         {
84:             CurrentHeart -= count;
85:
86:             // 하트 개수가 최대일 때 처음 하트를 소모했으면
87:             // 충전에 필요한 시간(Timer)을 refillTime으로 초기화
88:             if ( CurrentHeart == maxHeart - count ) Timer = refillTime;
89:
90:             SaveData();
91:
92:             return true;
93:         }
94:
95:         return false;
96:     }
97:
98:     private void SaveData()
99:     {
100:        Database.DBItem.goods.heart = CurrentHeart;
101:        Database.DBItem.goods.heartTimer = Timer;
102:        Database.DBItem.goods.heartLastTime = DateTime.UtcNow.ToBinary().ToString();
103:        Database.Write();
```

```
104:     }
105:
106:     private void LoadData()
107:     {
108:         CurrentHeart = Database.DBItem.goods.heart;
109:         string lastTimeString = Database.DBItem.goods.heartLastTime;
110:
111:         // 이전 실행에서 저장한 데이터가 있다면
112:         if ( !string.IsNullOrEmpty(lastTimeString) )
113:         {
114:             DateTime lastTime = DateTime.FromBinary(Convert.ToInt64(lastTimeString));
115:             // 이전 저장 시 남은 timer가 있으면 그 값을 사용, 없으면 refillTime 사용
116:             float savedTimer = Database.DBItem.goods.heartTimer;
117:             // 앱 종료 후 흐른 시간
118:             float elapsed = (float)(DateTime.UtcNow - lastTime).TotalSeconds;
119:             // 이전에 남은 시간을 합쳐 총 경과 시간 계산
120:             float totalElapsed = elapsed + (refillTime - savedTimer);
121:
122:             // 경과 시간 동안 회복할 수 있는 하트 개수 계산
                    (totalElapsed 1200마다 하트 1개씩 회복)
123:             int heartToRecover = Mathf.FloorToInt((float)totalElapsed / refillTime);
124:             CurrentHeart = Mathf.Min(CurrentHeart + heartToRecover, maxHeart);
125:
126:             // 현재 하트 개수가 최대이면 timer에 refillTime 저장
127:             if ( CurrentHeart >= maxHeart ) Timer = refillTime;
128:             // 현재 하트 개수가 최대가 아니면 남은 시간을 계산해서 저장
129:             else Timer = refillTime - (totalElapsed % refillTime);
130:         }
131:         // 처음 실행하거나 데이터가 없다면
132:         else Timer = refillTime;
133:     }
134: }
```

17~25줄 현재 클래스에서만 값을 설정할 수 있는 Timer 속성을 정의합니다. timer값을 설정할 때 onValueChangedTimer에 등록된 메서드를 호출합니다. 현재 하트 개수가 최대 하트보다 작으면 timer 시간을, 현재 하트 개수가 최대이면 "FULL" 텍스트를 매개변수로 전달합니다.

27~35줄 현재 클래스에서만 값을 설정할 수 있는 CurrentHeart 속성을 정의합니다. 현재 하트 개수를 설정할 때 onValueChangedHeart에 등록된 메서드를 호출합니다. 현재 하트 개수와 최대 하트 개수를 매개변수로 전달합니다.

37~43줄 Awake() 메서드에서 DBItem에 선언된 읽기 전용 변숫값을 불러와 최대 하트 개수 maxHeart와 하트 회복 시간 refillTime 변수에 저장하고 LoadData() 메서드를 호출해 하트와 하트 충전 시간 정보를 불러옵니다.

45~63줄 Update() 메서드에서 현재 하트 개수가 최대 하트보다 작으면 하트를 충전해야 하므로 Timer를 Time.deltaTime만큼 줄이고 Timer가 0이 되면 현재 보유 중인 하트를 1개 늘립니다. 아직 회복할 하트가 남았으면 Timer를 refillTime으로 설정합니다. SaveData() 메서드를 호출해 하트와 하트 충전 시간 정보를 저장합니다.

65~68줄 다른 프로그램이나 앱을 활성화해서 게임을 일시 정지했을 때 호출하는 OnApplicationPause() 메서드에서 SaveData() 메서드를 호출해 데이터를 파일에 저장합니다.

70~73줄 다시 게임이 활성화되었을 때 호출되는 OnApplicationFocus() 메서드에서 LoadData() 메서드를 호출해 파일에 저장된 데이터를 불러옵니다.

75~78줄 게임을 종료할 때 호출하는 OnApplicationQuit() 메서드에서 SaveData() 메서드를 호출해 데이터를 파일에 저장합니다.

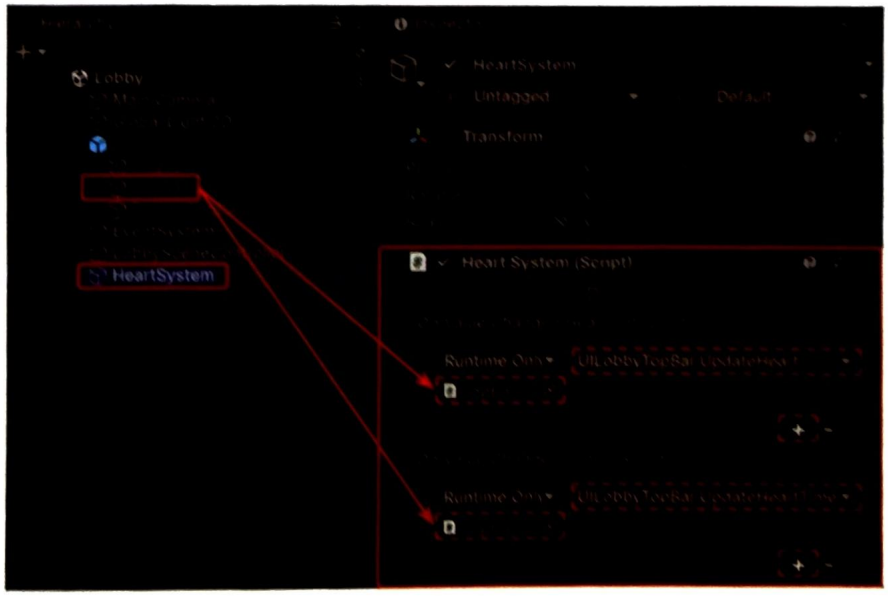
80~96줄 게임 플레이로 하트를 사용할 때 호출하는 UseHeart() 메서드는 가진 하트 개수 CurrentHeart가 count보다 크거나 같으면 하트를 count 개수만큼 줄입니다. 하트가 가득 찼을 때 사용했다면 하트를 충전하는 Timer값을 refillTime으로 설정합니다. SaveData() 메서드를 호출해 하트와 하트 충전 시간 정보를 저장합니다. 하트 개수가 충분해 하트 사용에 성공했다면 true를 반환하고 그렇지 않다면 false를 반환합니다.

98~104줄 SaveData() 메서드는 현재 하트 개수, 다음 하트 충전까지 남은 시간, 앱 종료 후 다시 실행까지 걸린 시간 등을 계산하고자 현재 시간 정보를 DBItem에 저장하고 DBItem 정보를 파일로 저장합니다.

106~133줄 LoadData() 메서드는 DBItem에 저장된 현재 하트 개수를 CurrentHeart 변수에 저장합니다. DBItem에 저장된 마지막 앱 종료 시간을 lastTimeString 변수에 저장합니다. 데이터를 초기화할 때 lastTimeString에 string.Empty를 저장하므로 이 값이 null이 아니면 이전 실행에서 저장한 데이터가 있는 상태입니다. 앱을 종료할 때 저장한 현재 시간 정보 lastTimeString을 DateTime형으로 변경해 lastTime에 저장합니다. DBItem에 저장된 heartTimer값을 savedTimer 변수에 저장합니다.

앱을 종료하고 다시 실행할 때까지 흐른 시간을 초 단위로 변환해 **elapsed** 변수에 저장합니다. 이전에 남은 시간을 합쳐 총 경과 시간을 계산합니다. 경과 시간 동안 회복할 수 있는 하트 개수를 계산합니다. 이 값이 최대 하트를 넘는다면 **maxHeart**까지만 채웁니다. 처음 실행했거나 데이터가 없다면 **Timer** 변수에 **refillTime**을 저장합니다.

02 로비 씬으로 이동하여 [GameObject → Create Empty] 메뉴로 빈 오브젝트를 생성하고 이름을 'HeartSystem'으로 변경한 후 Heart System 컴포넌트를 추가합니다. 하트 개수가 변경될 때 호출하는 On Value Changed Heart 이벤트에 요소를 추가하고 TopBar 오브젝트 UI Lobby Top Bar 컴포넌트의 UpdateHeart() 메서드를 등록합니다. 하트 충전 시간이 변경될 때 호출되는 On Value Changed Timer 이벤트에 요소를 추가하고 TopBar 오브젝트 UI Lobby Top Bar 컴포넌트의 UpdateHeartTimer() 메서드를 등록합니다.



HeartSystem 오브젝트에 Heart System 컴포넌트 추가하고 이벤트 메서드 등록하기

03 게임을 플레이할 때 하트를 소모하고 하트 개수가 부족하면 게임을 플레이할 수 없도록 설정하고자 LobbySceneController.cs 스크립트를 수정합니다.

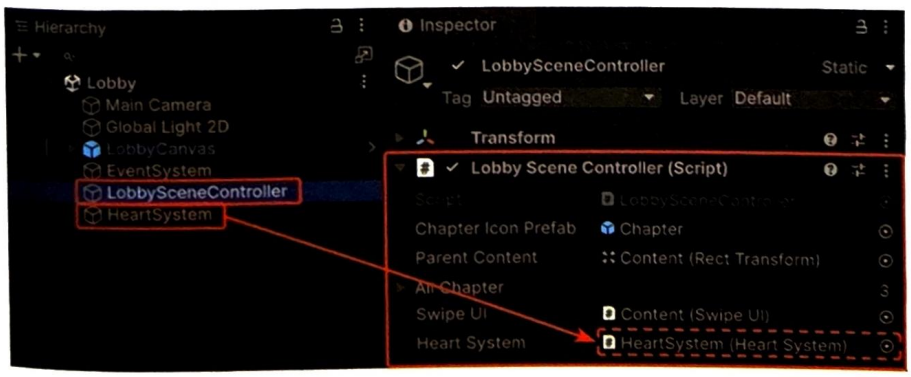
Assets/Scripts/LobbyScene/LobbySceneController.cs

코드 12-21 게임 플레이 시 하트 개수 파악 및 하트 소모

```
01: using UnityEngine;
02:
03: public class LobbySceneController : MonoBehaviour
```

```
04: {
    (... 생략 ...)
13:     [SerializeField]                    추가
14:     private HeartSystem heartSystem;
    (... 생략 ...)
25:     public void ButtonEvent_GameStart()
26:     {
27:         int index = swipeUI.CurrentPage;
            (... 생략 ...)
35:         if ( !heartSystem.UseHeart(5) )          추가
36:         {
37:             Logger.Log("하트가 부족합니다.");
38:             return;
39:         }
40:
41:         PlayerPrefs.SetInt(Constants.ChapterIndex, index);
42:
43:         SceneLoader.Instance.LoadScene(SceneNames.Game);
44:     }
45: }
```

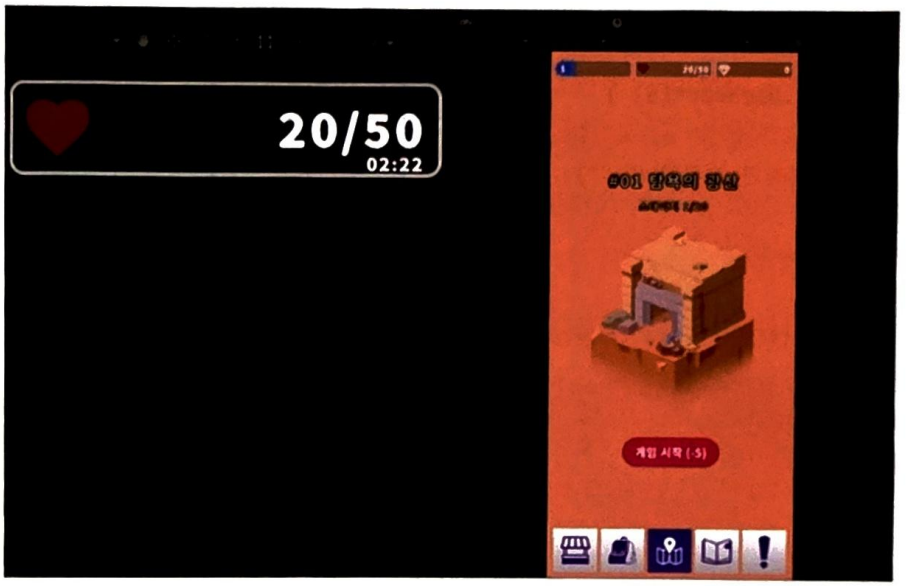
04 LobbySceneController 오브젝트 Lobby Scene Controller 컴포넌트의 Heart System 변수에 HeartSystem 오브젝트를 등록합니다.



LobbySceneController 오브젝트의 Lobby Scene Controller 컴포넌트 변수 설정하기

05 인트로 씬을 활성화하고 플레이 버튼 ▶을 눌러 게임을 실행하면 로비 씬 위에 하트 개수를 출력합니다. 50/50으로 하트가 가득 찼다면 'FULL' 텍스트를 출력하고 그렇지 않다면 다음 하트 충전까지 남은 시간을 출력합니다.

〈게임 시작 (-5)〉 버튼을 누르면 하트 5개를 소모하고 게임 씬을 실행합니다. 다시 로비 씬으로 오면 하트 5개가 사라진 것을 확인할 수 있습니다.



하트 시스템 결과 화면

부록

컨트롤러 추가하여 빌드하고 배포하기

부록에서는 게임을 빌드해 모바일 기기로 배포하는 방법을 알아봅니다. 모바일에서 실행하고자 가상 컨트롤러를 추가하고 모바일 플랫폼 빌드 옵션을 설정한 후 빌드합니다. 빌드가 끝나면 설치 파일을 기기로 전송해 설치하고 실행해 봅시다.

A-1 가상 컨트롤러 추가하기

A-2 모바일용 빌드하기

A-1

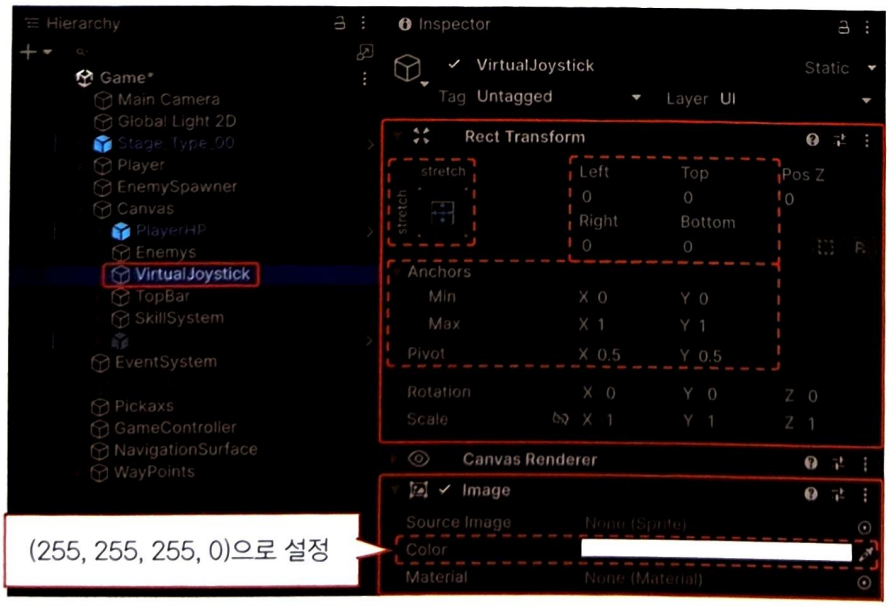
가상 컨트롤러 추가하기

모바일에서 게임을 실행할 때는 별도의 가상 컨트롤러를 추가하여 플레이어 이동을 조작합니다. 물론 PC에서도 마우스를 이용해 똑같은 가상 컨트롤러를 사용할 수 있습니다.

Do it! 실습 모바일용 가상 컨트롤러 추가하기

01 게임 씬에서 [GameObject → UI → Image] 메뉴로 가상 컨트롤러의 터치 제어에 사용하는 Image UI를 생성하고 이름을 'VirtualJoystick'으로 변경합니다. 이 오브젝트는 Canvas의 3번째 자식으로 배치합니다. Rect Transform 컴포넌트의 Anchor Presets은 stretch | stretch로, Rect는 (0, 0, 0, 0)으로 설정합니다. 가상 컨트롤러의 터치를 담당하는 오브젝트이므로 Image 컴포넌트를 비활성화하지는 않지만 화면에 보이지 않도록 Color를 (255, 255, 255, 0)으로 설정하여 투명하게 만듭니다.

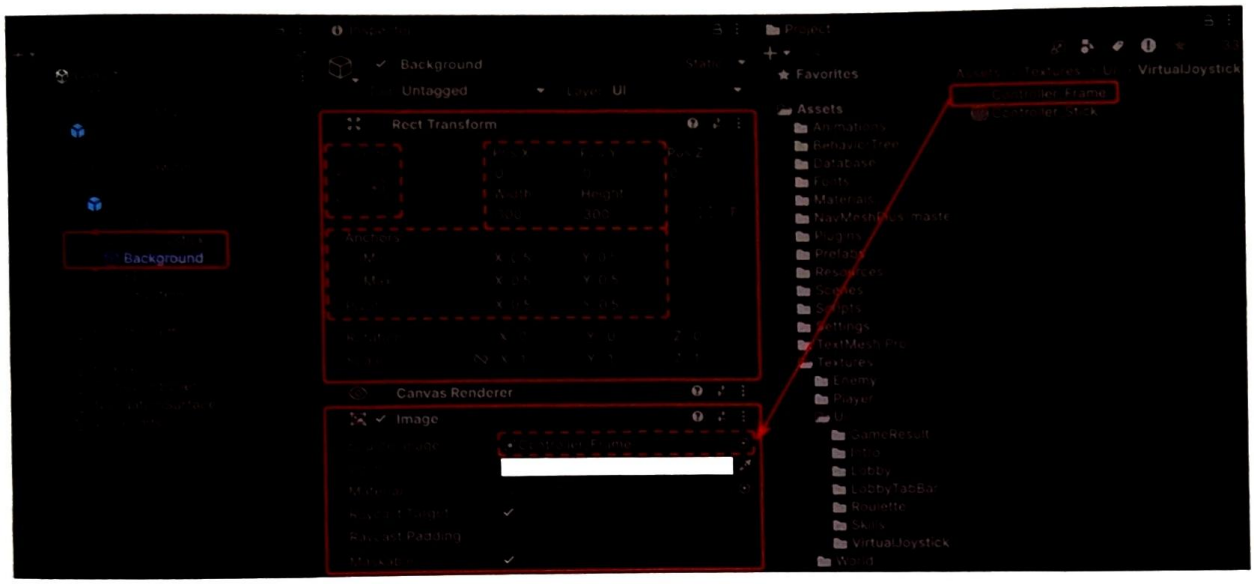
여기서는 플레이어가 화면을 터치한 위치에 가상 컨트롤러가 출력되도록 설정하고자 VirtualJoystick 오브젝트의 크기를 전체 화면으로 설정했습니다. 그러므로 TopBar, SkillSystem, Popup_RewardResult 등 터치로 조작할 수 있는 UI보다 뒤에 배치해야 다른 UI의 터치를 사용할 수 있습니다.



가상 컨트롤러의 터치 제어에 사용하는 Image UI 생성하고 설정하기

02 [GameObject → UI → Image] 메뉴로 가상 컨트롤러의 배경 이미지로 사용할 Image UI를 생성하고 이름을 'Background'로 변경한 후 VirtualJoystick 오브젝트의 자식으로 배치합니다.

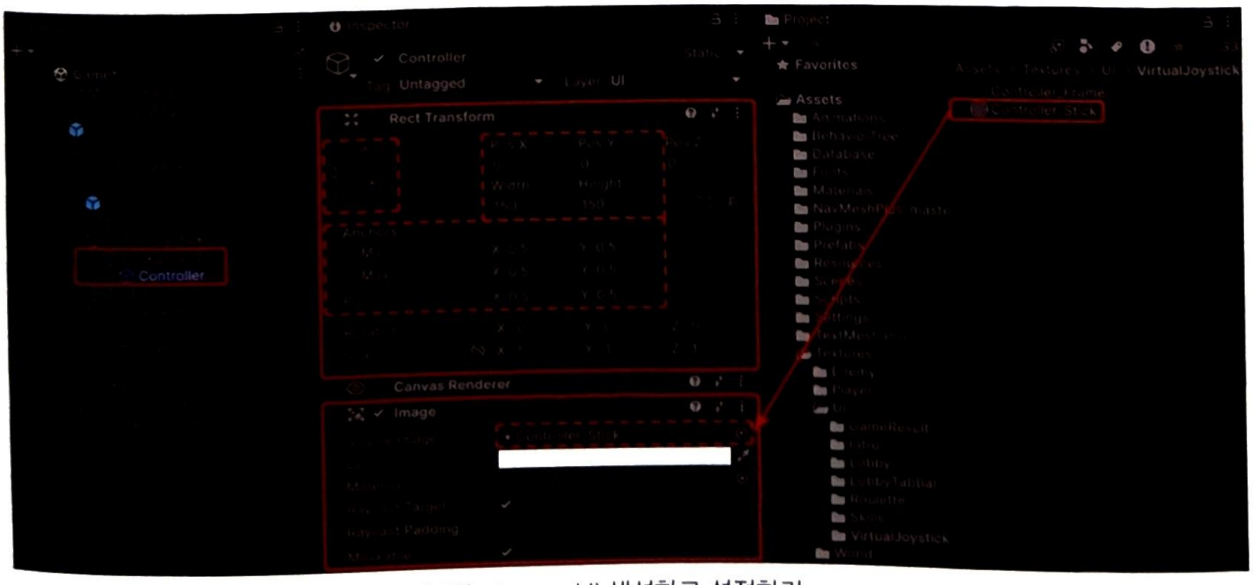
Rect Transform 컴포넌트의 Rect는 (0, 0, 300, 300)으로 설정하고 Image 컴포넌트의 Source Image에 Controller_Frame.png 이미지를 등록합니다.



가상 컨트롤러의 배경 이미지로 사용하는 Image UI 생성하고 설정하기

03 [GameObject → UI → Image] 메뉴로 가상 컨트롤러 이미지로 사용할 Image UI를 생성하고 이름을 'Controller'로 변경한 후 Background 오브젝트의 자식으로 배치합니다.

Rect Transform 컴포넌트의 Rect는 (0, 0, 150, 150)으로 설정하고 Image 컴포넌트의 Source Image에 Controller_Stick.png 이미지를 등록합니다.



가상 컨트롤러의 컨트롤러 이미지로 사용하는 Image UI 생성하고 설정하기

04 유니티는 마우스/터치를 이용해 오브젝트와 상호작용할 수 있도록 **IPointerHandler**, **IDragHandler**, **IDropHandler** 인터페이스를 제공합니다. 해당 인터페이스는 **UnityEngine.EventSystem** 이름공간에 정의됩니다. 이 기능을 사용하려면 UI 오브젝트는 Graphic Raycaster(Canvas 게임오브젝트)와 Event System(EventSystem 게임오브젝트) 컴포넌트가 필요하고, Image/TextMeshPro-Text 등의 UI 컴포넌트 내부 Raycast Target 변수가 true로 설정되어야 합니다. 2D/3D 오브젝트는 Event System(EventSystem 게임오브젝트) 컴포넌트가 필요하고, Main Camera 오브젝트에는 Physics 2D Raycaster/Physics Raycaster 컴포넌트가, 터치할 수 있는 게임오브젝트에는 Collider 2D/Collider 컴포넌트가 필요합니다.

또한 다음 표의 인터페이스를 상속받아 해당하는 메서드를 정의해야 하며 모든 메서드는 **PointerEventData** 매개변수를 전달받습니다.

IPointerHandler, IDragHandler, IDropHandler 인터페이스

인터페이스	메서드	설명
IPointerEnterHandler	void OnPointerEnter()	마우스 포인터가 현재 오브젝트 영역 안으로 들어갈 때 1회 호출합니다.
IPointerExitHandler	void OnPointerExit()	마우스 포인터가 현재 오브젝트 영역을 빠져나갈 때 1회 호출합니다.
IPointerDownHandler	void OnPointerDown()	현재 오브젝트 안에서 클릭하는 순간 1회 호출합니다.
IPointerUpHandler	void OnPointerUp()	현재 오브젝트 안에서 클릭했다가 떼는 순간 1회 호출합니다.
IPointerClickHandler	void OnPointerClick()	현재 오브젝트 안에서 클릭했다가 떼는 순간 1회 호출합니다 (다운&업 할 때 마우스 포인터가 해당 오브젝트 안에 있어야 함).
IBeginDragHandler	void OnBeginDrag()	현재 오브젝트를 끌기 시작할 때 1회 호출합니다.
IDragHandler	void OnDrag()	현재 오브젝트를 끌 때 매 프레임 호출합니다.
IEndDragHandler	void OnEndDrag()	현재 오브젝트 끌기를 종료할 때 1회 호출합니다.
IDropHandler	void OnDrop()	현재 오브젝트 영역 안에 끌어서 놓을 때 1회 호출합니다.

05 Assets 〉 Scripts 〉 Common 폴더에 가상 컨트롤러 터치를 제어하는 VirtualJoystick.cs 스크립트를 생성합니다.

Assets/Scripts/Common/VirtualJoystick.cs

코드 A-01 가상 컨트롤러 터치 제어

```
01: using UnityEngine;
02: using UnityEngine.EventSystems;
03:
04: public class VirtualJoystick : MonoBehaviour, IPointerDownHandler,
        IDragHandler, IPointerUpHandler
05: {
06:     [SerializeField]
07:     private RectTransform rectBackground; // 컨트롤러 배경
08:     [SerializeField]
09:     private RectTransform rectController; // 터치 정보에 따라 위치가 바뀌는 컨트롤러
10:
11:     // 터치 위치를 외부로 보내고자 지역 변수가 아닌 멤버 변수로 선언
12:     private Vector2 touchPosition;
13:
14:     // x, y 방향값을 외부에서 열람할 수 있도록 Get 전용 속성 정의
15:     public float Horizontal => touchPosition.x;
16:     public float Vertical => touchPosition.y;
17:
18:     private void Awake()
19:     {
20:         rectBackground.gameObject.SetActive(false);
21:     }
22:
23:     /// <summary>
24:     /// 터치하는 순간 1회 호출한다.
25:     /// </summary>
26:     public void OnPointerDown(PointerEventData eventData)
27:     {
28:         rectBackground.gameObject.SetActive(true);
29:
30:         // 터치한 위치로 가상 컨트롤러 위치 변경
31:         rectBackground.transform.position = eventData.position;
32:     }
33:
34:     /// <summary>
35:     /// 터치 상태로 드래그할 때 매 프레임을 호출한다.
36:     /// </summary>
37:     public void OnDrag(PointerEventData eventData)
38:     {
39:         touchPosition = Vector2.zero;
40:
41:         // 터치 위치(eventData.position)가 Background 오브젝트(rectBackground)에서
```

```
42:         // 얼마나 떨어졌는지 계산해(rectBackground 중심 위치) touchPosition에 저장
43:         if ( RectTransformUtility.ScreenPointToLocalPointInRectangle(
44:             rectBackground, eventData.position, eventData.pressEventCamera,
                out touchPosition) )
45:         {
46:             // touchPosition값 연산
47:             // touchPosition을 컨트롤러 배경(rectBackground) 크기로 나누고 2를 곱함
48:             touchPosition.x = (touchPosition.x / rectBackground.sizeDelta.x * 2);
49:             touchPosition.y = (touchPosition.y / rectBackground.sizeDelta.y * 2);
50:
51:             // touchPosition값 정규화(-1~1)
52:             // 현재 터치 위치가 가상 컨트롤러 배경(rectBackground) 바깥일 때
53:             // -1~1보다 큰 값이므로 -1~1 사이 값으로 정규화(normalized)
54:             touchPosition = (touchPosition.magnitude > 1) ?
                    touchPosition.normalized : touchPosition;
55:
56:             // 가상 컨트롤러의 컨트롤러(rectController) 이동
57:             rectController.anchoredPosition = new Vector2(
58:                 touchPosition.x * rectBackground.sizeDelta.x * 0.5f,
59:                 touchPosition.y * rectBackground.sizeDelta.y * 0.5f);
60:         }
61:     }
62:
63:     /// <summary>
64:     /// 터치를 종료하는 순간 1회 호출한다.
65:     /// </summary>
66:     public void OnPointerUp(PointerEventData eventData)
67:     {
68:         // 터치 종료 시 이미지 위치를 가운데로 이동
69:         rectController.anchoredPosition = Vector2.zero;
70:         // 터치 종료 시 touchPosition값도 (0, 0)으로 초기화
71:         touchPosition = Vector2.zero;
72:
73:         rectBackground.gameObject.SetActive(false);
74:     }
75: }
```

18~21줄 Awake() 메서드에서 가상 컨트롤러의 배경 오브젝트를 비활성화합니다. 화면을 터치했을 때만 화면에 조이스틱을 출력합니다. 화면에 계속 출력하려면 이 코드는 삭제합니다.

26~32줄 해당 오브젝트를 터치하는 순간 1회 호출되는 `OnPointerDown()` 메서드는 가상 컨트롤러의 배경 오브젝트를 활성화하고, 가상 컨트롤러의 배경 오브젝트 위치를 방금 터치한 위치인 `eventData.position`으로 설정합니다. 이때 가상 컨트롤러 위치를 고정이 아닌 플레이어가 터치한 위치에 출력하도록 설정합니다.

37~61줄 해당 오브젝트를 터치한 상태에서 끌 때 프레임마다 호출하는 `OnDrag()` 메서드는 43줄의 `RectTransformUtility.ScreenPointToLocalPointInRectangle()` 메서드를 호출해서 화면 터치 위치가 `rectBackground` 기준으로 얼마나 떨어져 있는지를 계산한 후 `touchPosition` 변수에 저장(방향+크기)합니다.

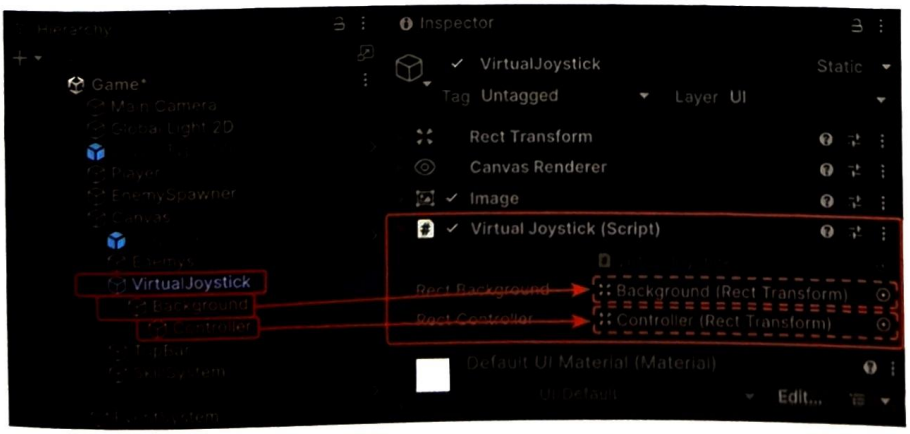
48~49줄 이동 방향으로 사용하고자 -n~n으로 정규화합니다. 지금은 가상 컨트롤러 배경 오브젝트를 벗어나지 않으면 -1.0~1.0, 벗어나면 그 이상의 값이 나옵니다.

54줄 가상 컨트롤러 배경 이미지 범위를 벗어나면 -1.0~1.0보다 더 큰 값이 나오므로 방향으로 사용하고자 -1.0~1.0 사이 값으로 정규화합니다.

56~59줄 플레이어에게 보이고자 실제 가상 컨트롤러의 컨트롤러 이미지 이동을 제어합니다.

66~74줄 해당 오브젝트 터치가 끝나는 순간 1회 호출하는 `OnPointerUp()` 메서드는 가상 컨트롤러 위치를 배경 오브젝트 가운데로 이동합니다. 해당 값을 사용해 움직이는 오브젝트의 이동 방향인 `touchPosition`도 (0, 0)으로 설정합니다. 그리고 가상 컨트롤러의 배경 오브젝트를 비활성화합니다.

06 VirtualJoystick 오브젝트에 Virtual Joystick 컴포넌트를 추가한 후 Rect Background 변수에는 Background 오브젝트를, Rect Controller 변수에는 Controller 오브젝트를 등록합니다.



VirtualJoystick 오브젝트에 Virtual Joystick 컴포넌트 추가하고 설정하기

07 게임을 실행하는 플랫폼에 따라 가상 컨트롤러를 활성화/비활성화하고 가상 컨트롤러로 입력값을 받아 플레이어를 이동할 수 있도록 PlayerController.cs 스크립트를 수정합니다.

Assets/Scripts/Player/PlayerController.cs

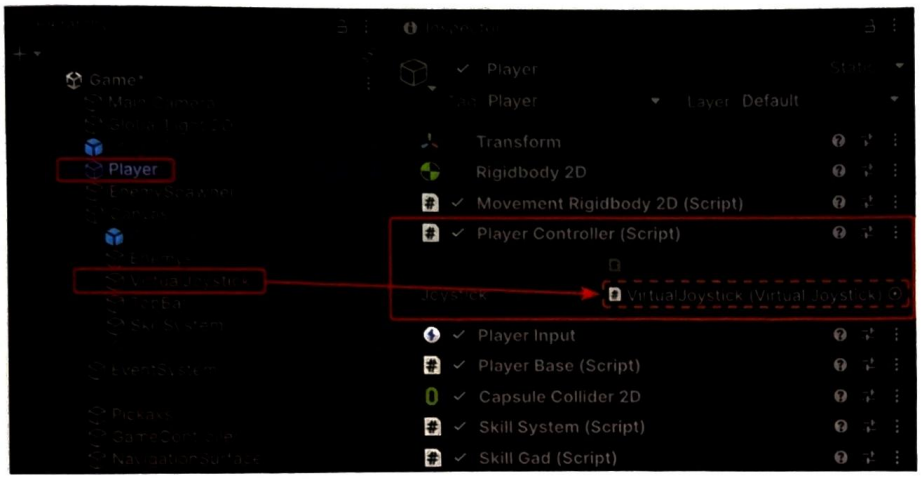
코드 A-02 플랫폼에 따라 플레이어 캐릭터 제어

```
    (... 생략 ...)
04: public class PlayerController : MonoBehaviour
05: {
06:     [SerializeField]                          ← 추가
07:     private VirtualJoystick joystick;
        (... 생략 ...)
14:     private void Awake()
15:     {
            (... 생략 ...)
20:         #if UNITY_EDITOR                      ← 추가
21:         joystick.gameObject.SetActive(false);
22:         #elif UNITY_ANDROID
23:         joystick.gameObject.SetActive(true);
24:         #endif
25:     }
26:
27:     private void Update()
28:     {
29:         #if UNITY_ANDROID && !UNITY_EDITOR    ← 추가
30:         moveInput = new Vector2(joystick.Horizontal, joystick.Vertical);
31:         #endif
            (... 생략 ...)
45:     }
        (... 생략 ...)
54: }
```

20~24줄 유니티 에디터에서 플레이 한다면 가상 컨트롤러를 볼 수 없도록 비활성화하고 안드로이드에서 플레이 한다면 가상 컨트롤러를 활성화합니다.

29~31줄 안드로이드에서 플레이 한다면 가상 컨트롤러의 입력값을 받아서 **moveInput**에 저장합니다. 20~24줄과 29줄, 31줄을 주석 처리하면 유니티 에디터에서도 마우스를 누르는 방법으로 가상 컨트롤러를 사용할 수 있습니다.

08 Player 오브젝트 Player Controller 컴포넌트의 Joystick 변수에 VirtualJoystick 오브젝트를 등록합니다.



Player 오브젝트의 Player Controller 컴포넌트 변수 설정하기

A-2

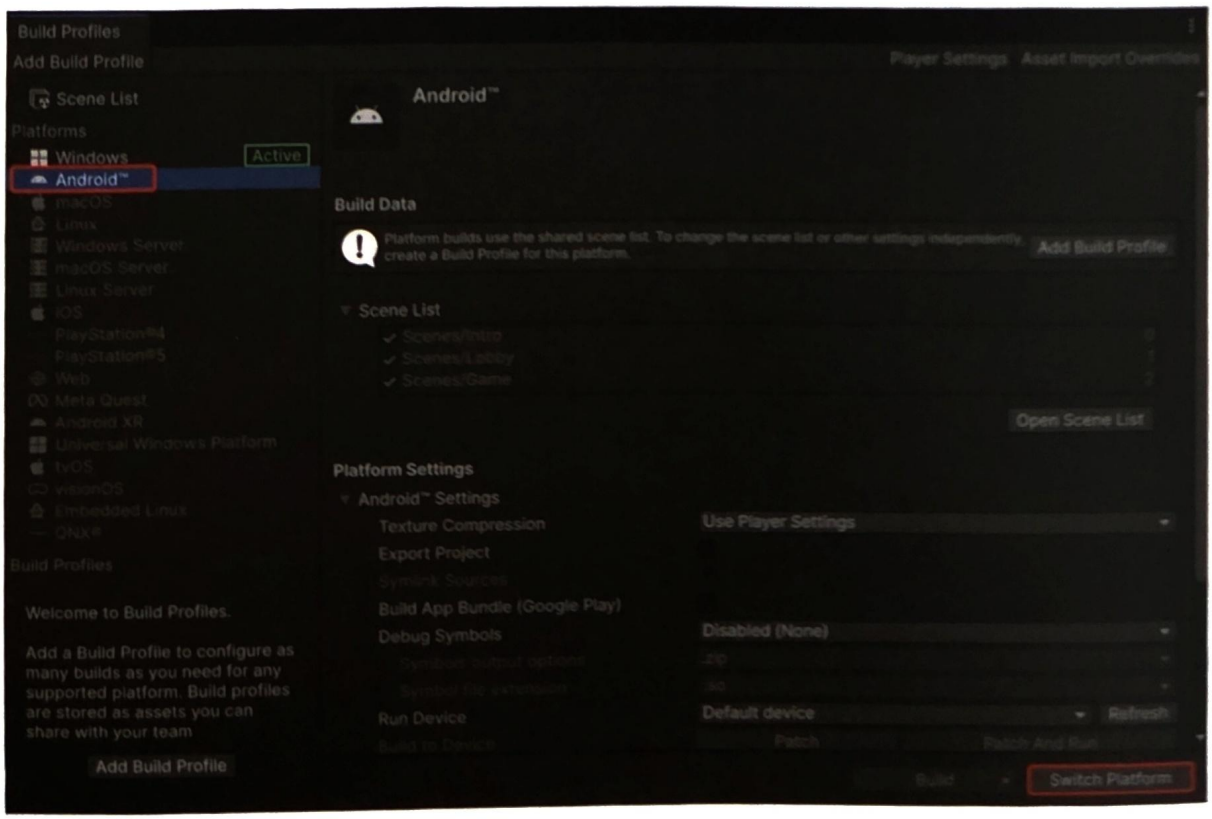
모바일용 빌드하기

이번에는 안드로이드용으로 빌드하여 기기에 설치하고 게임을 실행해 가상 컨트롤러로 조작하며 플레이해 봅시다.

Do it! 실습 안드로이드용으로 빌드하기

01 가상 컨트롤러를 테스트하고자 안드로이드에서 빌드를 진행합니다. [File → Build Profiles] 메뉴로 Build Profiles View를 열고 Android 플랫폼을 선택한 후 오른쪽 아래 〈Switch Platform〉 버튼을 눌러 플랫폼을 변경합니다.

- 유니티 6.4부터는 빌드 프로파일이 보이지 않습니다. 이럴 때는 [Edit → Project Settings...] 메뉴 [Editor] 탭 맨 아래에 있는 Hide Classic Platforms를 false로 설정하면 기본 빌드 프로파일을 사용할 수 있습니다.
- Android 플랫폼을 설치하지 않았으면 〈Install with Unity Hub〉 버튼을 눌러 유니티 허브에서 해당 플랫폼부터 설치하세요.



빌드 플랫폼을 안드로이드로 변경하기

02 유니티는 모바일에서 30FPS를 기본으로 설정하는데, 정적인 씬이라면 30FPS를, 동적인 씬이라면 테스트를 거쳐 30~60FPS 정도를 선택합니다. 여기서는 게임 프레임을 60FPS로 설정하고자 IntroSceneController.cs 스크립트를 수정합니다.

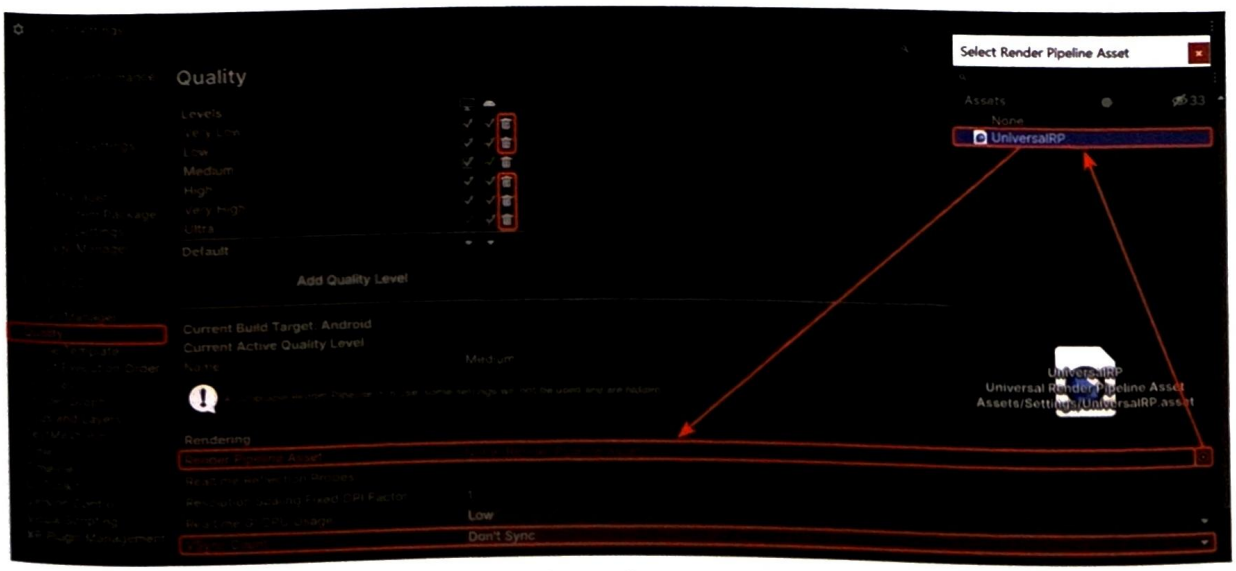
Assets/Scripts/IntroScene/IntroSceneController.cs

코드 A-03 게임 프레임을 60으로 설정

```
    (... 생략 ...)
05: public class IntroSceneController : MonoBehaviour
06: {
        (... 생략 ...)
12:     private void Awake()
13:     {
14:         Application.targetFrameRate = 60;
15:
16:         Database.Read();
17:     }
        (... 생략 ...)
36: }
```

추가

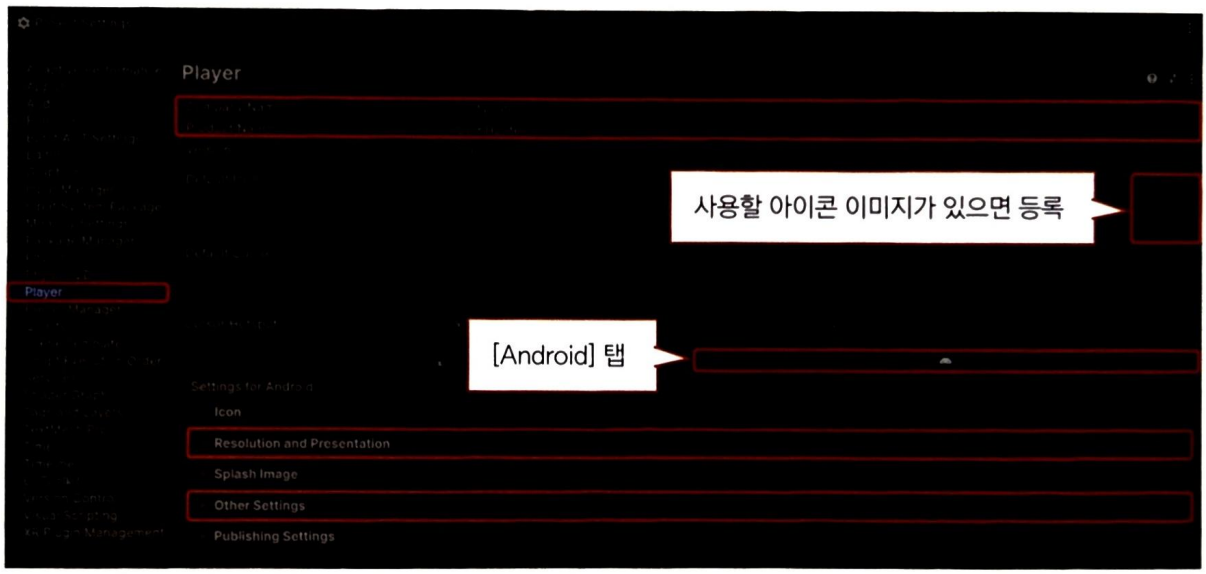
03 [Edit → Project Settings...] 메뉴로 Project Settings View를 열고 [Quality] 탭을 활성화합니다. 안드로이드 등 모바일에서 사용할 Medium만 제외하고 모든 Levels를 삭제합니다. Render Pipeline Asset에 UniversalRP 에셋을 등록하고 VSync Count를 [Don't Sync]로 설정합니다.



[Edit → Project Settings] 메뉴의 [Quality] 탭에서 프로젝트 품질 설정하기

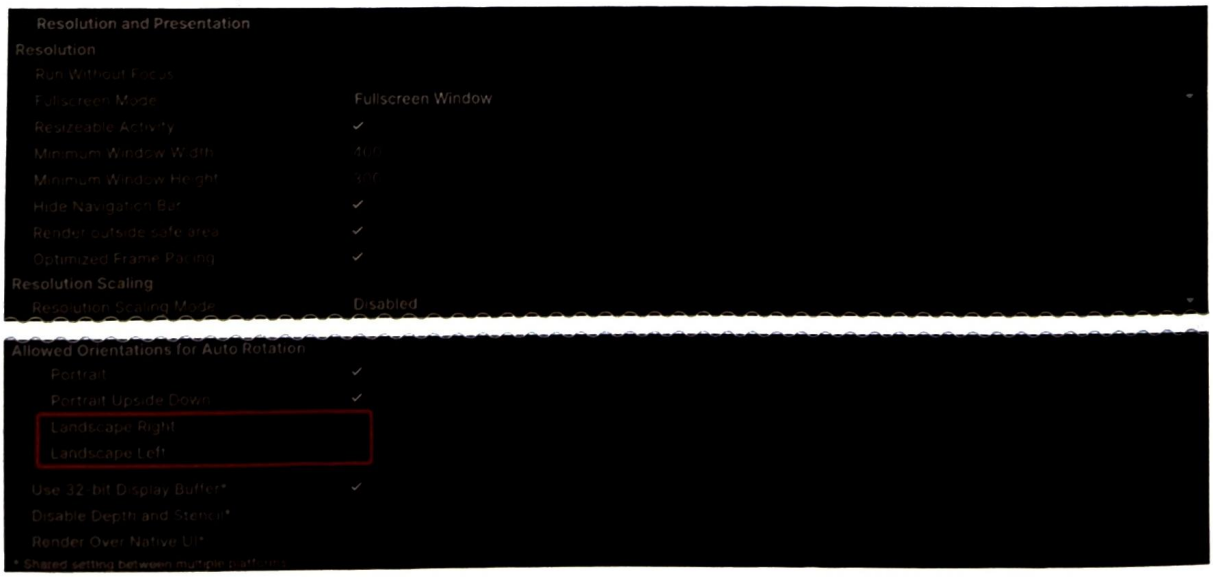
04 [Player] 탭을 활성화하고 회사 이름Company Name과 프로젝트 이름Project Name을 설정합니다. 이 이름은 데이터 저장 경로나 광고, 로그 등에서 식별자로 사용합니다. 또한 안드로이드와 iOS 등의 플랫폼으로 빌드할 때 앱 번들 ID로 사용하기도 합니다. 앱 아이콘을 변경하려면 Default Icon에 사용할 이미지를 등록합니다.

여기서는 안드로이드로 빌드하므로 [Android] 탭 옵션을 추가로 설정합니다.



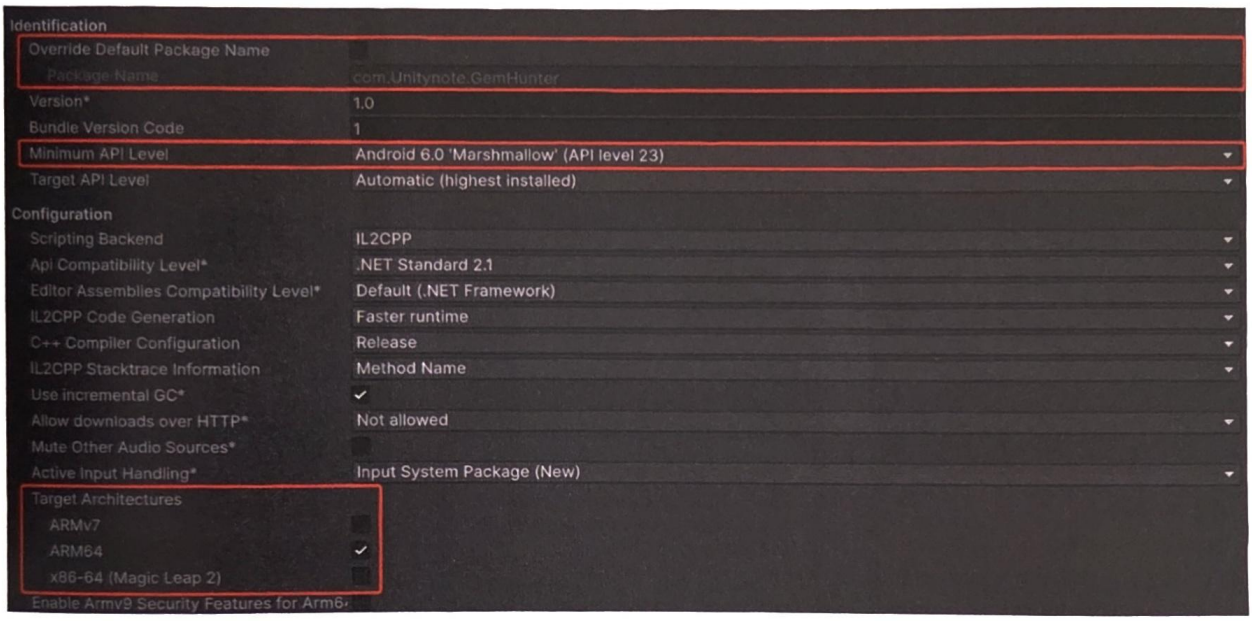
[Player] 탭에서 프로젝트 빌드 설정하기

05 GemHunter는 세로가 긴 1440x3088 해상도의 게임이므로 기기를 세로로 세웠을 때만 화면이 정상으로 보이도록 해야 합니다. Resolution and Presentation 섹션을 클릭하여 옵션을 표시하고 Landscape Right와 Landscape Left 모두 비활성화합니다.



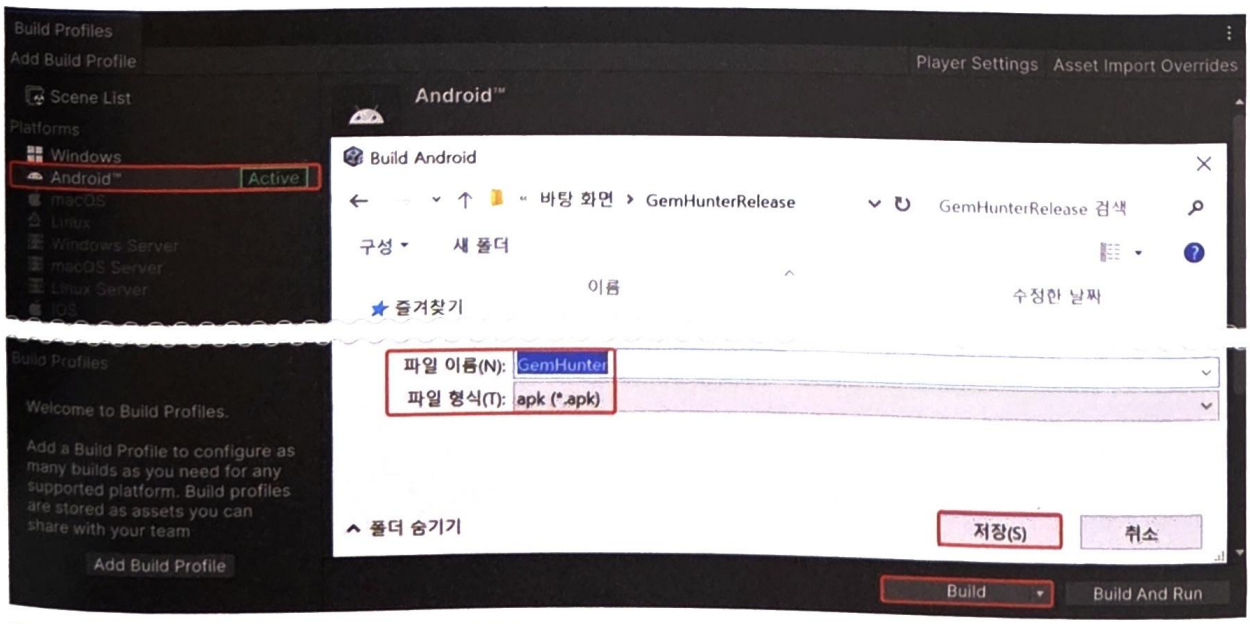
[Player → Android] 탭의 Resolution and Presentation 섹션 설정하기

06 Other Settings 섹션에서 Override Default Package Name 옵션을 비활성화하여 패키지 이름을 'com.CompanyName.ProductName'으로 입력합니다. 여기서는 Minimum API Level을 최소 버전 그대로 두었지만 이 값은 게임에 사용한 여러 기능에 따라 변경해야 합니다. Target Architectures에서 ARMv7은 32비트를, ARM64는 64비트를 뜻하는데, 오래된 기종을 지원하려면 ARMv7도 함께 체크합니다.



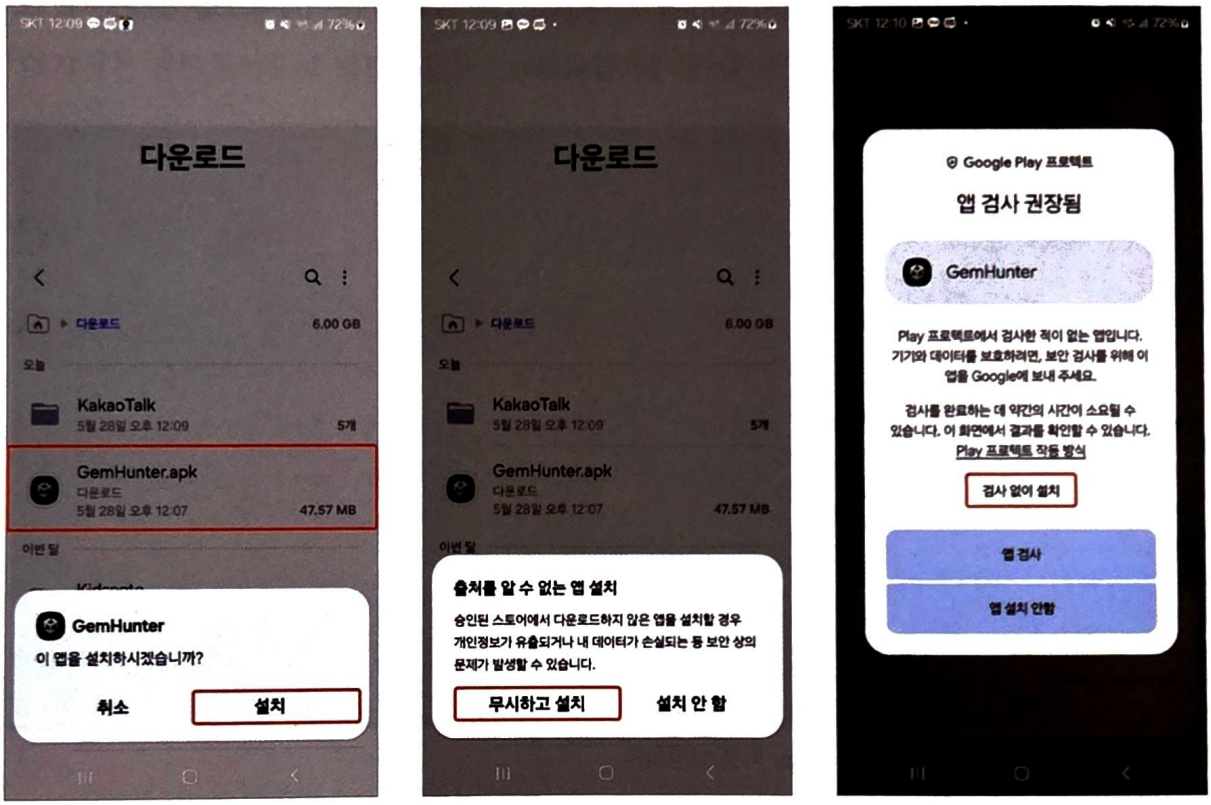
[Player → Android] 탭의 Other Settings 섹션 설정하기

07 [File → Build Profiles] 메뉴로 Build Profiles View를 다시 열고 [Android] 탭을 선택한 다음, 〈Build〉 버튼을 눌러 안드로이드 설치 파일(*.apk)을 생성합니다.



Build Profiles View에서 〈Build〉 버튼으로 빌드하기

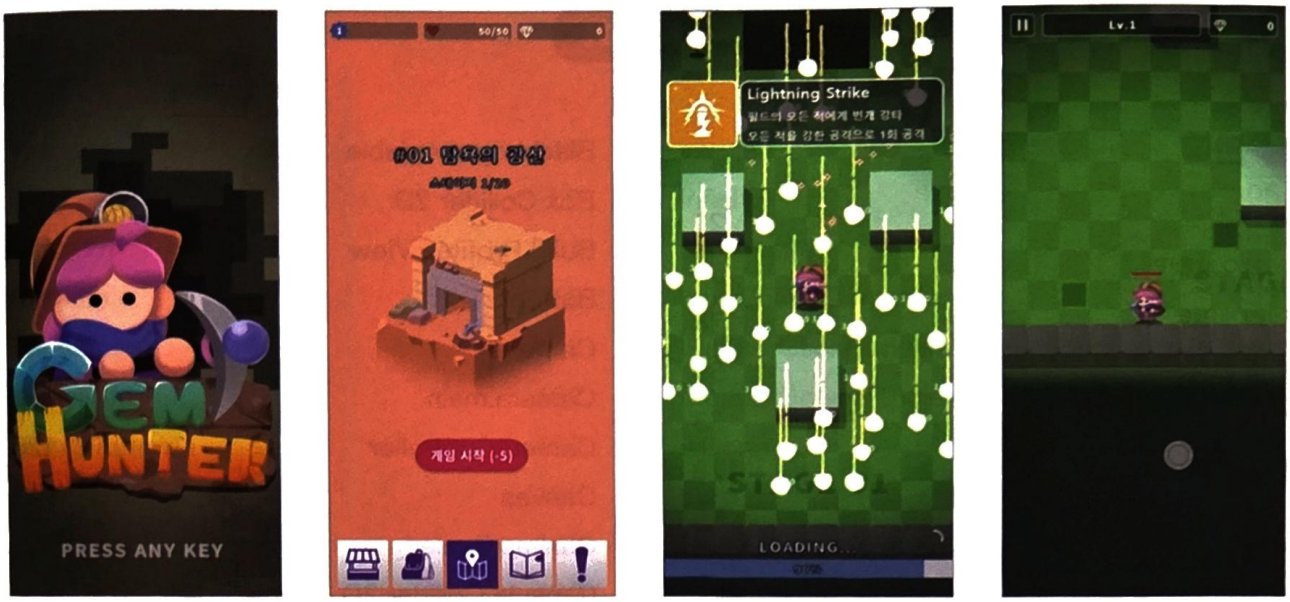
08 생성한 설치 파일은 안드로이드 기기로 옮겨 앱을 설치합니다. 여기서 만든 *.apk 파일은 구글스토어에 정식으로 등록한 파일이 아니므로 출처를 알 수 없는 앱으로 분류됩니다. 〈무시하고 설치〉, 〈검사 없이 설치〉를 차례대로 클릭해 앱 설치를 진행합니다.



안드로이드 기기에 모바일 게임 설치하고 실행하기

위 화면은 갤럭시 S23 울트라 기준으로, 기기에 따라 설치 과정이나 메뉴 이름은 다를 수 있습니다.

09 설치한 모바일 게임을 실행하면 유니티 에디터와 똑같은 타이틀 화면을 출력합니다. 화면을 터치하면 로비 씬으로 이동하고 〈게임 시작 (-5)〉 버튼을 눌러 게임을 플레이합니다. 화면을 터치하면 가상 컨트롤러를 출력하는데, 이를 터치한 상태에서 원하는 쪽으로 움직이면 해당 방향으로 플레이어를 이동할 수 있습니다.



모바일 기기에서 게임을 실행하고 가상 컨트롤러를 출력한 결과 화면